CHARLES PETZOLD · MICROSOFT PRESS, 2000.

Code

Skriveni jezik računalnog hardvera i softvera

Kako računalo radi iznutra: od Morseova koda, struje i releja, preko logičkih vrata, memorije i mikroprocesora, do operacijskih sustava, programskih jezika i grafičkih sučelja.

Pojednostavljena verzija ključnog znanja iz knjige

```
0000h:  LOD A,[1005h]
        ADD A,[1001h]
        STO [1005h],A
        ...
        JNZ 0000h

  1 0 1 1   0 1 1 0  =  B6h
```

Sažetak na hrvatskom jeziku

25 poglavlja · uvod · završni dio s pojmovnikom

listopad 2026.

Sadržaj

Uvod 3

1. Najbolji prijatelji: što je kod 5

2. Kodovi i kombinacije: koliko kodova daju dva znaka 9

3. Brailleovo pismo i binarni kodovi 12

4. Anatomija svjetiljke: osnove elektriciteta 15

5. Pogled iza ugla: žice, zajednički vodič i uzemljenje 20

6. Telegraf i relej 23

7. Naših deset znamenki: pozicijski brojevni sustav 26

8. Alternative broju deset: oktalni, četvorni i binarni sustav 29

9. Bit po bit: informacija kao izbor 34

10. Logika i prekidači: Booleova algebra 39

11. Logička vrata 44

12. Binarni stroj za zbrajanje 50

13. A oduzimanje? Komplementi i negativni brojevi 55

14. Povratna veza i bistabili (flip-flopovi) 60

15. Bajtovi i heksadecimalni sustav 66

16. Sklapanje memorije 74

17. Automatizacija: od zbrajala do računala 83

18. Od abakusa do čipova 98

19. Dva klasična mikroprocesora 110

20. ASCII i skup znakova 125

21. Sabirnica: kako se dijelovi računala povezuju 134

22. Operacijski sustav 145

23. Nepomični i pomični zarez 156

24. Jezici visoke i niske razine 165

25. Grafička revolucija 176

Završni dio 189

Uvod

O čemu je knjiga

Knjiga odgovara na jedno pitanje: **kako računalo zapravo radi?** Ne objašnjava kako se koristi neki program, nego od čega je računalo napravljeno i zašto radi to što radi.

Put je postupan. Kreće od dvoje djece koja svjetiljkama šalju poruke, uvodi ideju **koda** i pokazuje da su za svaku informaciju dovoljna **dva znaka**. Zatim objašnjava struju, telegraf i **relej**, pa od releja gradi **logička vrata**, **zbrajalo**, **memoriju** i na kraju jednostavno **računalo**. Drugi dio knjige prelazi na stvarne mikroprocesore (Intel 8080, Motorola 6800), tekst (ASCII), sabirnicu i periferiju, operacijski sustav, brojeve s pomičnim zarezom, programske jezike i grafička sučelja.

Glavna poruka: u računalu nema ničeg čarobnog. Sve se svodi na **jednostavne prekidače** složene na pametan način i na **dogovore** o tome što pojedini nizovi bitova znače.

Kome je namijenjena

- Svakome tko želi razumjeti računalo "ispod haube", bez obzira na predznanje.
- Studentima informatike, elektrotehnike i srodnih područja kao temelj za kolegije iz arhitekture računala, digitalne logike, operacijskih sustava i programiranja.
- Programerima koji žele znati što se događa ispod programskog jezika.

Nije potrebno predznanje iz matematike osim osnovne aritmetike. Svi pojmovi objašnjeni su od početka.

Što ćeš znati nakon čitanja

- Raditi s **binarnim** i **heksadecimalnim** brojevima i pretvarati ih.
- Objasniti kako se od releja ili tranzistora grade **logička vrata**, a od vrata **zbrajalo** i **memorija**.
- Objasniti kako procesor **dohvaća i izvršava instrukcije**, što su **registri, zastavice, stog, potprogrami i prekidi**.
- Pročitati jednostavan **asemblerski** program i prevesti ga u strojni kod.
- Objasniti kako se spremaju **tekst** (ASCII, Unicode), **razlomci** (pomični zarez), **slike** i **zvuk**.
- Objasniti ulogu **sabirnice**, **operacijskog sustava**, **datotečnog sustava** i **programskih jezika**.
- Razumjeti kako se razvijala računalna tehnologija i zašto su važne ideje poput **von Neumannove arhitekture**.

Kako je knjiga pojednostavljena

Ova skripta nije prijevod knjige, nego sažetak njezina znanja vlastitim riječima.

Zadržano:

- sve definicije, principi i postupci,
- ključni primjeri (uglavnom zamijenjeni vlastitim primjerima s istim principom, i računski provjereni),
- tablice (Morseov kod, Braille, ASCII, instrukcije 8080, formati brojeva...),
- formule i sheme sklopova (pojednostavljene, u tekstualnom obliku),
- česte pogreške i praktična primjena.

Izbačeno ili jako skraćeno:

- priče, anegdote i motivacijski dijelovi,
- životopisi (sveden na jednu rečenicu ili redak tablice),

- ponavljanja i usputna razmatranja,
- citati i književni ulomci.

Gdje knjiga nešto ne objašnjava detaljno, to je navedeno rečenicom **"U knjizi nije detaljno objašnjeno."** Na nekoliko mjesta gdje knjiga sadrži očitu pogrešku (u kodu ili formuli), dan je ispravan oblik s kratkom napomenom.

Napomena o vremenu: knjiga je napisana 1999. Brojke poput "današnjih" brzina procesora, veličina diskova i cijena odnose se na to vrijeme.

Struktura knjige

Cjelina	Poglavlja	Sadržaj
Kodovi	1 – 3	Morseov kod, broj kombinacija, Brailleovo pismo
Struja i releji	4 – 6	elektricitet, strujni krug, uzemljenje, telegraf, relej
Brojevi i informacija	7 – 9	pozicijski sustavi, binarni brojevi, bit, paritet, UPC
Logika i sklopovi	10 – 14	Booleova algebra, logička vrata, zbrajalo, oduzimanje, bistabili
Od memorije do računala	15 – 17	bajt i heks, RAM, automatizacija i prvo računalo
Stvarni procesori	18 – 19	povijest, tranzistor, čipovi, Intel 8080 i Motorola 6800
Sustav oko procesora	20 – 22	ASCII, sabirnica, memorija, zaslon, tipkovnica, disk, operacijski sustav
Softver	23 – 24	nepomični i pomični zarez, programski jezici
Suvremeno računalo	25	grafička sučelja, slike, zvuk, video, Internet

Najvažnija poglavlja za razumijevanje rada računala su **8 – 17** (binarni brojevi, logika, zbrajalo, bistabili, memorija, automatizacija) te **19** (stvarni procesor). Poglavlje 18 je pretežno povijesno.

Kako koristiti skriptu

Svako poglavlje ima isti raspored:

1. **Što ćeš naučiti**: kratki popis ciljeva.
2. **Objašnjenje** s primjerima, analogijama, tablicama i shemama.
3. **Ključne definicije**: tablica pojmova.
4. **Ključno za zapamtiti**: 5 – 10 najvažnijih tvrdnji.
5. **Najčešće zabune**: pogrešna shvaćanja i ispravak.
6. **Brza provjera razumijevanja**: pitanja bez odgovora, za samoprovjeru.

Poglavlja se nadovezuju jedno na drugo, pa ih je najbolje čitati redom. Na kraju je **završni dio** s najvažnijim lekcijama, vezama među poglavljima, praktičnim savjetima i pojmovnikom.

1. Najbolji prijatelji: što je kod

Što ćeš naučiti u ovom poglavlju

- Što je **kod** i zašto je kod svaki sustav kojim prenosimo informaciju, a ne samo tajno pismo.
- Kako radi **Morseov kod** i zašto je bolji od "brojenja bljeskova".
- Zašto su za prijenos bilo kakve informacije dovoljna **dva različita znaka**.

Problem: kako razgovarati bez glasa

Knjiga počinje jednostavnom situacijom. Dvoje djece živi preko puta, prozori im gledaju jedan u drugi, a navečer ne smiju razgovarati. Imaju samo **džepne svjetiljke**. Kako svjetlom poslati rečenicu?

Prva ideja je "crtati" slova svjetlom po zraku. To ne radi: pokreti su neprecizni i druga strana ih ne može pouzdano pročitati.

Druga ideja je **brojiti bljeskove**: A je 1 bljesak, B su 2 bljeska, i tako dalje do Z, koje je 26 bljeskova. Između slova kratka stanka, između riječi dulja.

To radi, ali je jako sporo. Primjer: riječ BOK traži 2 + 15 + 11 = **28 bljeskova**. Duža rečenica traži stotine bljeskova. Osim toga, za interpunkcijske znakove nemaš dogovor.

Problem je u tome što se bljeskovi razlikuju **samo po broju**. Ako uvedeš i drugu vrstu bljeska, dobivaš puno više mogućnosti s manje bljeskova. Upravo to radi Morseov kod.

Morseov kod

Morseov kod koristi **dvije vrste signala**:

- **točku**: kratak signal (kratko upali i ugasi svjetiljku);
- **crtu**: dug signal (svjetlo duže upaljeno).

Svako slovo je kratak niz točaka i crta. Riječ BOK u Morseovom kodu je `-...` `---` `-.-`, ukupno samo **10 signala** umjesto 28.

Slovo	Kod	Slovo	Kod	Slovo	Kod
A	`.-`	J	`.---`	S	`...`
B	`-...`	K	`-.-`	T	`-`
C	`-.-.`	L	`.-..`	U	`..-`
D	`-..`	M	`--`	V	`...-`
E	`.`	N	`-.`	W	`.--`
F	`..-.`	O	`---`	X	`-..-`
G	`--.`	P	`.--.`	Y	`-.--`
H	`....`	Q	`--.-`	Z	`--..`
I	`..`	R	`.-.`		

Brojevi imaju po pet znakova i slijede jasan uzorak:

Znamenka	Kod	Znamenka	Kod
1	`.----`	6	`-....`

Znamenka	Kod	Znamenka	Kod
2	`..---`	7	`--...`
3	`...--`	8	`---..`
4	`....-`	9	`----.`
5	`.....`	0	`-----`

Interpunkcijski znakovi (točka, zarez, upitnik i drugi) imaju pet do sedam znakova. Postoje i kodovi za neka slova s naglaskom iz europskih jezika.

Vrijeme je dio koda

Točka i crta razlikuju se samo po **trajanju**. Zato su važna i trajanja stanki. Sve se mjeri u "jedinicama", a jedinica je trajanje jedne točke:

Element	Trajanje
Točka	1 jedinica
Crta	oko 3 jedinice
Stanka između točaka i crta unutar istog slova	1 jedinica
Stanka između slova	oko 3 jedinice (kao jedna crta)
Stanka između riječi	dulja; u knjizi oko dvije crte

Trajanja nisu fiksna u sekundama. Ovise o brzini pošiljatelja. Brzi pošiljatelj može imati crtu kraću od točke sporog pošiljatelja. Primatelj se zato prilagodi nakon nekoliko slova: gleda koji su signali kraći, a koji duži.

Analogija: ritam u glazbi. Četvrtinka nije određen broj sekundi; važan je omjer prema drugim notama.

Česta slova imaju kratke kodove

Na prvi pogled raspored kodova izgleda nasumično. Nije posve. Najčešća slova u engleskom (E i T) imaju najkraće kodove: jednu točku i jednu crtu. Rijetka slova (Q, Z) imaju duge kodove. Tako prosječna poruka traje kraće.

Ograničenja i zanimljivosti

- Morseov kod **ne razlikuje velika i mala slova**.
- **SOS** (`... --- ...`) nije kratica. To je samo niz koji se lako pamti i šalje bez stanki između slova.
- Morseov kod može se i **izgovarati**: točka je "di" (ili "dit"), crta je "da". Tako se govor svodi na samo dva zvuka.
- Brzina slanja svjetiljkom s malo vježbe je oko 5 do 10 riječi u minuti. Govor je oko 100 riječi u minuti.

Što je kod

U knjizi riječ **kod** znači: **sustav kojim prenosimo informaciju između ljudi i strojeva**. Kod omogućuje komunikaciju.

Kod **ne mora biti tajan**. Većina kodova je javna, jer ih svi moraju razumjeti. Primjeri kodova koje svakodnevno koristimo:

Kod	Što predstavlja	Zašto postoji
Govor	misli zvukovima	najprirodniji način komunikacije
Pismo	govor znakovima na papiru	govor se ne može spremiti na papir
Znakovni jezik	slova i riječi pokretima ruku	za ljude koji ne čuju ili ne govore

Kod	Što predstavlja	Zašto postoji
Brailleovo pismo	slova izbočenim točkama	za ljude koji ne vide
Stenografija	govor brzim znakovima	za brzo bilježenje govora
Morseov kod	slova kratkim i dugim signalima	za tihu komunikaciju na daljinu, i u mraku

Svaki kod postoji jer u nekoj situaciji radi bolje od drugih. Govor se ne može spremiti na papir, pa imamo pismo. Pismo se ne može pročitati u mraku preko ulice, pa imamo Morseov kod.

I sama tablica Morseovog koda je kod: točke i crte na papiru predstavljaju kratke i duge bljeskove.

Veza s računalima

Računala također koriste kodove: za brojeve, tekst, zvuk, slike i video. Računalo ne može izravno koristiti ljudske kodove, jer nema oči, uši i usta. Zato sve vrste informacija mora pretvoriti u **svoje** kodove. Ostatak knjige objašnjava upravo te kodove i strojeve koji s njima rade.

Najvažnija ideja poglavlja

Knjiga ističe riječ **dva**. Dvije vrste bljeskova, dva zvuka, dvije bilo kakve različite stvari: uz dobre kombinacije dovoljne su za prijenos **svih vrsta informacija**.

To je prva naznaka **binarnog** načina razmišljanja, na kojem počiva cijelo računalo.

Ključne definicije

Pojam	Značenje
Kod	sustav znakova i pravila za prijenos informacije između ljudi i strojeva
Morseov kod	kod u kojem je svaki znak niz kratkih (točka) i dugih (crta) signala
Točka i crta	dva osnovna signala Morseovog koda; crta traje oko tri točke
Jedinica (u Morseovom kodu)	trajanje jedne točke; sve ostalo mjeri se u njoj

Ključno za zapamtiti

1. Kod je bilo koji sustav za prijenos informacije. Nije nužno tajan.
2. Govor, pismo, znakovni jezik, Brailleovo pismo i Morseov kod su kodovi; svaki služi u situaciji gdje drugi ne rade.
3. Kod s jednom vrstom signala (samo broj bljeskova) je spor. S dvije vrste signala dobivamo puno kraće kodove.
4. Morseov kod koristi točku i crtu; razlika među njima je samo trajanje.
5. Stanke su dio koda: odvajaju znakove, slova i riječi.
6. Česta slova imaju kratke kodove, pa je prosječna poruka kraća.
7. Dvije različite stvari su dovoljne za prijenos svake informacije.

Najčešće zabune

- **"Kod znači tajnu šifru."** U ovoj knjizi kod je bilo koji dogovoreni sustav znakova; većina kodova je javna.
- **"Točka i crta su različiti znakovi na papiru."** U prijenosu su to samo kraći i duži signal; na papiru ih crtamo radi preglednosti.

- **"Trajanje točke je fiksno."** Ovisi o pošiljatelju; važan je omjer prema crti i stankama.

Brza provjera razumijevanja

1. Zašto je sustav "A = 1 bljesak, B = 2 bljeska..." sporiji od Morseovog koda?
2. Koliko traje crta, a koliko stanka između slova, ako točka traje pola sekunde?
3. Zašto E i T imaju najkraće kodove?
4. Navedi tri koda iz svakodnevnog života i za svaki reci zašto postoji.
5. Što je zajedničko Morseovom kodu i izgovorenim "di" i "da"?

2. Kodovi i kombinacije: koliko kodova daju dva znaka

Što ćeš naučiti u ovom poglavlju

- Zašto je Morseov kod lakše **slati** nego **primati** i kako to riješiti.
- Koliko različitih kodova možeš napraviti od točaka i crta zadane duljine.
- Formulu **broj kodova = 2^n** i zašto se potencije broja 2 stalno pojavljuju u računalima.
- Što znače riječi **binarno** i **kombinatorika**.

Problem dekodiranja

Tablica Morseovog koda ide u jednom smjeru: od slova prema točkama i crtama. Za slanje je to dovoljno: nađeš slovo i pošalješ njegov kod.

Za primanje treba suprotni smjer: od niza točaka i crta prema slovu. Ako primiš `-.--`, morao bi pregledati cijelu tablicu dok ne nađeš Y. Točke i crte nemaju "abecedni red", pa ih ne možeš složiti kao rječnik.

Rješenje: složi kodove po **broju znakova** (točaka i crta) u kodu. Kad primiš kod, prebrojiš znakove i gledaš samo malu tablicu te duljine.

Kodovi složeni po duljini

Duljina koda	Kodovi i slova	Broj kodova
1	`.` E, `-` T	2
2	`..` I, `.-` A, `-.` N, `--` M	4
3	`...` S, `..-` U, `.-.` R, `.--` W, `-..` D, `-.-` K, `--.` G, `---` O	8
4	H, V, F, Ü, L, Ä, P, J, B, X, C, Y, Z, Q, Ö, Š (CH)	16

Kodovi duljine 1 do 4 daju ukupno 2 + 4 + 8 + 16 = **30 kodova**. Latinica ima 26 slova, pa četiri koda duljine 4 služe za slova s naglaskom.

Zašto se broj kodova udvostručuje

Uzmi sve kodove duljine 2. Svakom dodaj na kraj **točku**, pa svakom dodaj na kraj **crtu**. Dobio si sve kodove duljine 3, i ima ih točno dvaput više. Isto vrijedi za svaki sljedeći korak.

Zato:

Duljina n	Broj kodova	Zapis potencijom
1	2	2^1
2	2 × 2 = 4	2^2
3	2 × 2 × 2 = 8	2^3
4	2 × 2 × 2 × 2 = 16	2^4
5	32	2^5
6	64	2^6
7	128	2^7
8	256	2^8
9	512	2^9

Duljina n	Broj kodova	Zapis potencijom
10	1024	2^{10}

Formula

> **broj kodova = 2^n**

- **2** je broj različitih znakova od kojih gradimo kod (točka i crta).
- **n** je duljina koda (koliko znakova ima u nizu).
- **2^n** ("dva na n-tu") znači 2 pomnoženo sa samim sobom n puta.

Primjer: koliko ima Morseovih kodova od točno 5 znakova? $2^5 = 32$. To je dovoljno za 10 znamenaka i 16 interpunkcijskih znakova koliko ih je definirano s pet znakova. Neki od tih 32 kodova koriste se i za slova s naglaskom.

Ako dopustimo kodove duljine od 1 do 6, dobivamo 2 + 4 + 8 + 16 + 32 + 64 = **126** mogućih kodova. Morseovom kodu toliko ne treba, pa mnogi dugi kodovi nemaju značenje. Takav kod zovemo **nedefiniran**. Ako primiš nedefiniran kod, znaš da je netko pogriješio.

Formula je korisna jer ne moraš ispisati sve kodove da bi znao koliko ih ima: samo množiš 2 sa samim sobom.

Stablo za dekodiranje

Još brži način dekodiranja je **stablo**. Kreneš od početka. Za svaku točku ideš jednom granom (npr. gore), za svaku crtu drugom (dolje). Gdje stane kod, tamo piše slovo.

```
početak
├── .  E
│   ├── .  I
│   │   ├── .  S ── (.) H  (-) V
│   │   └── -  U ── (.) F  (-) Ü
│   └── -  A
│       ├── .  R ── (.) L  (-) Ä
│       └── -  W ── (.) P  (-) J
└── -  T
    ├── .  N
    │   ├── .  D ── (.) B  (-) X
    │   └── -  K ── (.) C  (-) Y
    └── -  M
        ├── .  G ── (.) Z  (-) Q
        └── -  O ── (.) Ö  (-) Š
```

Primjer: `.-.` → kreni s točkom (E), zatim crta (A), zatim točka (R). Slovo je **R**.

Ovakvo stablo vjerojatno je trebalo i za samo **stvaranje** Morseovog koda. Osigurava dvije stvari:

1. dva slova nikad nemaju isti kod;
2. iskorištene su sve kratke kombinacije prije nego što se uzmu duže.

Binarno i kombinatorika

Morseov kod je **binarni** kod. "Binarno" znači "od dva dijela": kod se sastoji od samo dvije vrste znakova.

Analogija: novčić. Može pasti samo na glavu ili pismo. Ako baciš tri novčića, broj mogućih ishoda je 2 × 2 × 2 = 8. Isto je s Morseovim kodom duljine 3.

Binarni objekti i binarni kodovi uvijek se opisuju **potencijama broja 2**.

Ono što smo radili (brojanje mogućih kombinacija) dio je matematike koji se zove **kombinatorika**. Obično se koristi u vjerojatnosti i statistici (na koliko načina mogu pasti kocke), ali pomaže i razumjeti kako se kodovi grade i rastavljaju.

Ključne definicije

Pojam	Značenje
Binarni kod	kod sastavljen od samo dvije vrste znakova
Potencija 2^n	2 pomnoženo sa sobom n puta; broj binarnih kodova duljine n
Nedefiniran kod	kombinacija koja ne znači ništa; znak greške u prijenosu
Stablo za dekodiranje	dijagram u kojem svaki znak koda bira granu, a na kraju puta je značenje
Kombinatorika	grana matematike koja broji moguće kombinacije

Ključno za zapamtiti

1. Kod je lako slati, a teže primati, ako tablica ide samo u jednom smjeru.
2. Kodove se isplati složiti po duljini ili u stablo.
3. Svaki dodatni znak u kodu **udvostručuje** broj mogućih kodova.
4. Formula: broj kodova duljine n od dva znaka je 2^n.
5. Kodovi duljine 1 do 6 daju ukupno 126 Morseovih kodova; mnogi su nedefinirani.
6. "Binarno" znači "od dvije vrste"; binarni kodovi uvijek vode na potencije broja 2.

Najčešće zabune

- **"Kodova duljine 4 ima 4 × 2 = 8."** Ne: 2 × 2 × 2 × 2 = 16. Množi se 2 sa sobom, a ne duljina s 2.
- **"2^n je ukupan broj kodova do duljine n."** Ne: 2^n je broj kodova **točno** duljine n. Ukupan broj do duljine n je zbroj svih manjih potencija.

Brza provjera razumijevanja

1. Zašto se kod "točke i crte" ne može složiti abecedno?
2. Koliko ima binarnih kodova duljine 7? A duljine 8?
3. Kako iz svih kodova duljine 3 dobiješ sve kodove duljine 4?
4. Dekodiraj stablom niz `--.-`.
5. Kad bi kod imao tri vrste znakova (točka, crta, dvostruka crta), koliko bi bilo kodova duljine 2?

3. Brailleovo pismo i binarni kodovi

Što ćeš naučiti u ovom poglavlju

- Kako je građeno Brailleovo pismo i zašto daje točno **64** koda.
- Kako jedan kod može imati **više značenja**, ovisno o kontekstu.
- Što su **kodovi pomaka** (shift) i **izlazni kodovi** (escape), dva trika koja se kasnije stalno pojavljuju u računalima.

Ukratko o nastanku

Louis Braille, slijepi francuski mladić iz 19. stoljeća, kao tinejdžer je razvio pismo izbočenih točaka. Polazio je od vojnog sustava "noćnog pisanja" koji je bio previše složen jer je kodirao glasove, a ne slova. Braille je napravio sustav koji se lako **čita prstima** i lako **piše** šiljkom na papiru. Ta druga osobina bila je velika prednost: učenik je mogao sam bilježiti i čitati svoje bilješke.

Biografski detalji nisu važni za razumijevanje koda, pa ih preskačemo.

Brailleova ćelija

Svaki znak zauzima **ćeliju** od 2 stupca × 3 retka. Mjesta su numerirana ovako:

```
1 ● ● 4
2 ● ● 5
3 ● ● 6
```

Svaka od šest točaka je **izbočena ili ravna**. To su dvije mogućnosti po točki, dakle točke su **binarne**.

Primijenimo formulu iz poglavlja 2: 6 točaka, svaka s 2 stanja:

> broj kodova = 2^6 = 2 × 2 × 2 × 2 × 2 × 2 = **64**

Brailleovo pismo ima točno 64 moguća koda, ni više ni manje. (Među njima je i ćelija bez ijedne izbočene točke, koja služi kao razmak.)

Analogija: šest prekidača za svjetlo u nizu. Svaki je upaljen ili ugašen. Broj različitih rasporeda je 64.

Slova

Slova su složena u tri "retka" po deset, i ti retci imaju jasan uzorak:

Slova	Koje točke se koriste
a – j	samo gornje četiri: 1, 2, 4, 5
k – t	isto kao a – j, plus točka 3
u, v, x, y, z	isto kao a – e, plus točke 3 i 6

Osnovnih deset uzoraka (a – j):

Slovo	Točke	Slovo	Točke
a	1	f	1, 2, 4

Slovo	Točke	Slovo	Točke
b	1, 2	g	1, 2, 4, 5
c	1, 4	h	1, 2, 5
d	1, 4, 5	i	2, 4
e	1, 5	j	2, 4, 5

Primjer: k je a + točka 3, dakle točke 1 i 3. u je a + točke 3 i 6, dakle točke 1, 3 i 6.

Slovo **w** nema mjesto u ovom uzorku jer ga klasični francuski nije koristio. Dodano je kasnije (točke 2, 4, 5, 6).

Slova unutar riječi odvojena su malim razmakom, a riječi praznom ćelijom.

Brailleovo pismo 2. razine: skraćenice

Dosad smo potrošili 26 kodova za slova i 1 za razmak. Od 64 kombinacije ostaje mnogo neiskorištenih. Današnje Brailleovo pismo za engleski (tzv. **Grade 2**, druga razina) koristi ih za **skraćenice**, da bi se uštedjelo papira i ubrzalo čitanje.

Ideja je da **isti kod ima različito značenje ovisno o tome gdje stoji**:

- Kod slova koji stoji **sam** (kao zasebna riječ) znači cijelu čestu riječ. Na primjer, samo "b" znači *but*, samo "c" znači *can*.
- Kodovi a – j s dodanom **točkom 6** znače česte skupine slova (npr. *ch*, *sh*, *th*).
- Kodovi a – j **spušteni** na točke 2, 3, 5, 6 služe kao interpunkcija (zarez, točka-zarez, dvotočka, točka...) ili kao skraćenice, ovisno o kontekstu.
- Preostale kombinacije točaka 3, 4, 5 i 6 služe za još skraćenica i interpunkcije.

Rezultat: u Brailleovom pismu 2. razine **nijedan od 64 koda nije neiskorišten**.

Brojevi: kod koji mijenja značenje sljedećih kodova

Brailleovo pismo nema posebne kodove za znamenke. Umjesto toga, ima **oznaku broja** (točke 3, 4, 5, 6). Nakon nje se kodovi slova a – j čitaju kao znamenke:

Kod slova	a	b	c	d	e	f	g	h	i	j
Iza oznake broja znači	1	2	3	4	5	6	7	8	9	0

Primjer: oznaka broja, zatim b, e, f znači **256**.

Oznaka broja vrijedi za **sve** kodove koji slijede, dok je ne poništi **oznaka slova** (točke 5 i 6). Tek tada se a – j opet čitaju kao slova.

Velika slova: kod koji mijenja samo sljedeći znak

Oznaka velikog slova je samo točka 6. Ona kaže da je **sljedeće slovo** (samo jedno) veliko.

Tu je važna razlika u dosegu:

Vrsta posebnog koda	Primjer	Na što djeluje
Kod pomaka (shift, "precedence code")	oznaka broja	na **sve** sljedeće kodove, dok se ne poništi
Izlazni kod (escape)	oznaka velikog slova	samo na **jedan** sljedeći kod

Analogija za kod pomaka: tipka Caps Lock na tipkovnici. Kad je uključiš, sva slova su velika dok je ne isključiš.

Analogija za izlazni kod: tipka Shift koju pritisneš za jedno slovo. Djeluje samo na taj jedan znak.

Riječ "escape" (bijeg) znači da taj kod omogućuje "pobjeći" iz uobičajenog tumačenja niza kodova u neko drugo tumačenje.

Zašto je ovo važno za računala

Šest binarnih točaka daje 64 koda i nijedan više. Kad treba prikazati više od 64 stvari, Brailleovo pismo ne povećava ćeliju, nego daje kodovima **značenje ovisno o kontekstu**. Računala koriste iste trikove: kodove pomaka i izlazne kodove srest ćemo kod prikaza teksta (poglavlje 20).

Ključne definicije

Pojam	Značenje
Brailleova ćelija	6 mjesta (2 × 3) za izbočene točke; jedna ćelija je jedan kod
Binarna točka	točka koja ima samo dva stanja: izbočena ili ravna
Brailleovo pismo 2. razine	proširenje sa skraćenicama za česte riječi i skupine slova
Kod pomaka (shift)	kod koji mijenja značenje svih sljedećih kodova dok se ne poništi
Izlazni kod (escape)	kod koji mijenja značenje samo sljedećeg koda

Ključno za zapamtiti

1. Brailleova ćelija ima 6 binarnih točaka, dakle $2^6 = 64$ moguća koda.
2. Slova slijede uzorak: a – j koriste gornje četiri točke, k – t dodaju točku 3, u – z dodaju točke 3 i 6.
3. Isti kod može značiti slovo, riječ, skupinu slova ili interpunkciju, ovisno o kontekstu.
4. Oznaka broja pretvara a – j u znamenke 1 – 0 dok je ne poništi oznaka slova.
5. Oznaka velikog slova djeluje samo na sljedeće slovo.
6. Kod pomaka djeluje na sve što slijedi; izlazni kod samo na sljedeći znak.
7. Ograničen broj kodova proširuje se kontekstom, a ne većom ćelijom.

Najčešće zabune

- **"Brailleovo pismo ima posebne kodove za brojeve."** Nema; brojevi su kodovi slova a – j iza oznake broja.
- **"Oznaka velikog slova djeluje na cijelu riječ."** U ovom opisu djeluje samo na jedno slovo; za trajnu promjenu služi kod pomaka.
- **"Ako postoje 64 koda, može se prikazati najviše 64 stvari."** Kontekstom (pomak, izlaz) isti kodovi mogu značiti puno više stvari.

Brza provjera razumijevanja

1. Zašto Brailleovo pismo ima točno 64 koda?
2. Koje točke ima slovo m, ako znaš da je c točke 1 i 4?
3. Kako bi zapisao broj 405 u Brailleovom pismu?
4. Koja je razlika između oznake broja i oznake velikog slova po dosegu djelovanja?
5. Kojoj tipki na tipkovnici odgovara kod pomaka, a kojoj izlazni kod?

4. Anatomija svjetiljke: osnove elektriciteta

Što ćeš naučiti u ovom poglavlju

- Što je **strujni krug** i zašto mora biti zatvoren.
- Što su **elektroni**, **naboj** i **električna struja**.
- Kako baterija pokreće struju i što znači **serijski** i **paralelni** spoj baterija.
- Četiri osnovne veličine: **napon** (volt), **struja** (amper), **otpor** (om) i **snaga** (vat).
- **Ohmov zakon** i kako s njim računati.
- Zašto je prekidač **binaran** i zašto je to važno za računala.

Za razumijevanje računala ne treba puno elektrotehnike. Treba samo ovih nekoliko pojmova.

Najjednostavniji strujni krug

Rastavi svjetiljku i ostaje: dvije baterije, žarulja, prekidač i nekoliko metalnih dijelova. Najjednostavnija "svjetiljka" su baterije, žarulja i dvije žice. Kad spojiš krajeve žica, žarulja svijetli.

Prva važna stvar: **strujni krug je krug**. Žarulja svijetli samo ako postoji neprekinut put: od baterije kroz žicu do žarulje, kroz žarulju, natrag žicom do baterije. Bilo kakav prekid na tom putu i žarulja se gasi. **Prekidač** služi upravo tome: namjerno prekida i ponovno spaja krug.

Analogija: trkaća staza. Automobili mogu kružiti samo ako je staza zatvorena. Ako negdje nedostaje komad staze, sve staje.

Odakle struja: elektroni

Atom

Sva tvar je građena od **atoma**. Atom ima tri vrste čestica:

Čestica	Gdje je	Naboj
Proton	u jezgri	pozitivan (+)
Neutron	u jezgri	nema naboja
Elektron	kruži oko jezgre	negativan (−)

Pojednostavljena slika atoma je mali Sunčev sustav: jezgra je "Sunce", elektroni su "planeti". Stvarni atom ne izgleda tako, ali za razumijevanje je model dovoljan.

Broj protona u jezgri određuje o kojem je **elementu** riječ; taj broj zove se **atomski broj**. Atomi se spajaju u **molekule** (npr. voda je H_2O: dva atoma vodika i jedan kisika). Molekula često ima posve drukčija svojstva od atoma od kojih je građena.

Naboj

Znakovi + i − kod naboja ne znače zbrajanje i oduzimanje. Znače samo da su protoni i elektroni u nekom smislu **suprotni**. Pravilo je:

- **suprotni naboji se privlače**, a
- **isti naboji se odbijaju**.

Atom je najstabilniji kad ima jednako protona i elektrona. Elektrone je ipak moguće "otrgnuti" s atoma. Kad se negdje skupi višak elektrona, a negdje manjak, priroda to pokušava izjednačiti. Tako nastaje elektricitet.

Primjeri:

- **Statički elektricitet.** Hodaš po tepihu i potplati skupljaju elektrone. Kad dotakneš kvaku, višak elektrona preskoči i osjetiš iskru.
- **Munja.** Dno oblaka skupi višak elektrona, a vrh izgubi elektrone. Neravnoteža se izjednači naglim skokom ogromnog broja elektrona.

Struja u žici

U svjetiljci elektroni ne skaču odjednom kao munja. Kreću se mirno i neprekidno: atom preda elektron susjednom atomu, uzme elektron od drugog susjeda, i tako dalje kroz cijeli krug. **Električna struja je kretanje elektrona od atoma do atoma.**

Svi elektroni su jednaki. Elektron iz baterije ne razlikuje se od elektrona u bakrenoj žici, pa se mogu slobodno miješati.

Baterija

U bateriji se odvijaju **kemijske reakcije**. Tvari su odabrane tako da:

- na **negativnom polu** (−, zove se i *anoda*) reakcija stvara **višak elektrona**,
- na **pozitivnom polu** (+, *katoda*) reakcija **traži elektrone**.

Tako se kemijska energija pretvara u električnu.

Reakcija može teći samo ako postoji put kojim elektroni idu od negativnog pola, kroz vanjski krug, do pozitivnog pola. Ako baterija nije spojena, gotovo ništa se ne događa. Kad su kemikalije potrošene, baterija je prazna.

Zato se može reći i obrnuto: nije samo baterija ta koja "daje" struju krugu; i krug omogućuje bateriji da odvija svoju reakciju.

Serijski i paralelni spoj

Spoj	Kako	Ukupni napon	Učinak
Serijski	plus jedne baterije na minus druge, u nizu	zbroj: 1,5 V + 1,5 V = **3 V**	žarulja jače svijetli
Paralelni	plusevi zajedno, minusi zajedno	isti: **1,5 V**	žarulja svijetli slabije, ali baterije traju dvostruko dulje
Krivo okrenuta baterija u nizu	plus na plus		ne radi: elektroni ne mogu doći do pozitivnih polova

Džepne svjetiljke obično imaju dvije baterije u seriji, dakle 3 volta.

Vodiči, otpor i izolatori

Struja teče kroz neke tvari puno lakše nego kroz druge.

- **Vodiči** lako provode struju. Najbolji su **bakar, srebro i zlato**. Njihovi atomi imaju jedan elektron u vanjskoj ljusci koji lako otpuštaju. Bakar je najčešći jer je dovoljno dobar i jeftin.
- **Otpor** je suprotno od vodljivosti: koliko tvar ometa prolazak elektrona. Dijelovi napravljeni da imaju određeni otpor zovu se **otpornici**.
- **Izolatori** imaju vrlo velik otpor i gotovo ne vode struju: guma, plastika, tkanina, drvo, suhi zrak. Zato su žice obložene plastikom.

Važno: gotovo **sve** vodi struju ako je napon dovoljno velik. Zato munja prolazi kroz zrak.

Dvije činjenice o žicama:

Žica	Otpor
duža	veći
deblja	manji

Deblja žica ima manji otpor jer kroz nju istovremeno može prolaziti više elektrona. **Analogija:** širi autoput propušta više automobila.

Četiri veličine

Napon (volt, V)

Napon je mogućnost (potencijal) da se obavi rad. Napon postoji i kad baterija nije ni na što spojena, kao što i lopta na visokoj polici ima "mogućnost" pasti, iako trenutno stoji. Što je polica viša, to je potencijal veći. Baterija od 1,5 V ima određeni potencijal; dvije u seriji imaju dvostruko veći.

Napon je najteži od ovih pojmova jer opisuje **mogućnost**, a ne nešto što se događa.

Struja (amper, A)

Struja je količina elektrona koja u sekundi prođe kroz neku točku kruga. Jedan amper je oko $6{,}24 \times 10^{18}$ elektrona u sekundi (6 240 000 000 000 000 000). Manja jedinica je **miliamper** (mA): tisućinka ampera.

Otpor (om, Ω)

Otpor je sklonost tvari da ometa protok elektrona. Mjeri se u omima.

Analogija vode u cijevima

Elektricitet	Voda
Napon	tlak vode
Struja	količina vode koja proteče
Otpor	uskost cijevi

Veći tlak, više vode. Uža cijev, manje vode. Analogija pomaže, ali nije savršena: elektricitet se na kraju ne ponaša posve kao voda.

Ohmov zakon

> **I = E / R**

Simbol	Značenje	Jedinica
I	struja	amper (A)
E	napon (od *electromotive force*, elektromotorna sila)	volt (V)
R	otpor	om (Ω)

Riječima: **struja je napon podijeljen otporom**. Veći napon, veća struja. Veći otpor, manja struja.

Tri tipična slučaja

1. **Baterija ni na što spojena.** Između polova je zrak s ogromnim otporom. I = 1,5 V / ogroman broj ≈ **0**. Nema struje.
2. **Kratki spoj.** Polovi spojeni samo komadom bakrene žice, gotovo bez otpora. I = 1,5 V / vrlo mali broj = **ogromna struja**. U stvarnosti je struja ograničena veličinom baterije, napon padne, a žica se zagrije, može i usijati ili se rastaliti. Električna energija pretvara se u toplinu.
3. **Normalan krug.** Otpor ni prevelik ni premalen. Na shemama se otpornik crta cik-cak linijom.

Primjer: džepna svjetiljka

Žarulja u svjetiljci ima tanku nit od **volframa** u vakuumu. Kad kroz nit teče struja, ona se zagrije i svijetli; u zraku bi izgorjela, ali u vakuumu ne može.

- Napon: 3 V (dvije baterije u seriji)
- Otpor žarulje: oko 4 Ω
- Struja: I = 3 V / 4 Ω = **0,75 A** = 750 mA

(Napomena iz knjige: hladna žarulja izmjerena ommetrom pokaže puno manji otpor, jer otpor volframa raste s temperaturom.)

Snaga

> **P = E × I**

Simbol	Značenje	Jedinica
P	snaga (*power*)	vat (W)
E	napon	volt (V)
I	struja	amper (A)

Primjer 1: svjetiljka: P = 3 V × 0,75 A = **2,25 W**.

Primjer 2 (iz knjige, američka mreža od 120 V): žarulja od 100 W. Struja I = P / E = 100 / 120 ≈ 0,83 A. Otpor R = E / I = 120 / 0,83 ≈ 144 Ω.

Primjer 3 (dodatak za Hrvatsku, nije iz knjige): u kućnoj mreži od 230 V žarulja od 100 W vuče I = 100 / 230 ≈ 0,43 A i ima otpor R = 230 / 0,43 ≈ 529 Ω.

Prekidač: najvažniji dio

Prekidač određuje teče li struja ili ne.

Stanje	Drugi naziv	Struja
Uključen	**zatvoren**	teče
Isključen	**otvoren**	ne teče

Pazi: riječi su suprotne onome kako ih koristimo za vrata. Zatvorena vrata ne propuštaju ništa, a **zatvoren prekidač propušta struju**. Zamisli prekidač kao pokretni most: kad je most "zatvoren" (spušten), put je spojen i promet prolazi.

Prekidač je ili zatvoren ili otvoren. Struja ili teče ili ne teče. Žarulja ili svijetli ili ne. **Nema ničeg između.** Krug s prekidačem je dakle **binaran**, baš kao Morseov i Brailleov kod. Ta sličnost je temelj cijelog ostatka knjige.

Ključne definicije

Pojam	Značenje
Strujni krug	zatvoren put kojim elektroni idu od negativnog do pozitivnog pola izvora
Električna struja	kretanje elektrona kroz vodič; mjeri se u amperima
Napon	potencijal (mogućnost) za obavljanje rada; mjeri se u voltima
Otpor	koliko tvar ometa protok elektrona; mjeri se u omima
Snaga	koliko se energije troši u sekundi; P = E × I, u vatima
Vodič / izolator	tvar koja lako / jedva provodi struju
Kratki spoj	polovi izvora spojeni gotovo bez otpora; vrlo velika struja i zagrijavanje
Serijski / paralelni spoj	baterije u nizu (naponi se zbrajaju) / usporedno (napon isti, traju dulje)

Ključno za zapamtiti

1. Struja teče samo kroz **zatvoren** krug.
2. Struja je kretanje elektrona od atoma do atoma.
3. Baterija kemijskom reakcijom stvara višak elektrona na minus polu i manjak na plus polu.
4. Serijski spoj zbraja napone; paralelni čuva napon i produljuje trajanje.
5. Duža žica ima veći otpor, deblja manji.
6. Ohmov zakon: I = E / R. Snaga: P = E × I.
7. Kratki spoj daje vrlo veliku struju i toplinu.
8. Zatvoren prekidač propušta struju, otvoren ne.
9. Krug s prekidačem je binaran: struja teče ili ne teče.

Najčešće zabune

- **"Zatvoren prekidač znači da je isključen."** Obrnuto: zatvoren znači spojen, struja teče.
- **"Deblja žica treba više elektrona da se 'napuni', pa ima veći otpor."** Ne: deblja žica ima **manji** otpor.
- **"+ znači da proton ima nešto više od elektrona."** Znakovi samo označavaju suprotne naboje.
- **"Napon teče kroz žicu."** Teče struja; napon je razlika potencijala koja je pokreće.

Brza provjera razumijevanja

1. Zašto žarulja ne svijetli ako je jedna baterija u nizu okrenuta naopako?
2. Kolika je struja kroz otpor od 6 Ω spojen na 3 V? Kolika je snaga?
3. Što se događa u kratkom spoju i zašto?
4. Koja je razlika između napona i struje, objašnjena na primjeru vode?
5. Zašto je krug s prekidačem "binaran"?

5. Pogled iza ugla: žice, zajednički vodič i uzemljenje

Što ćeš naučiti u ovom poglavlju

- Kako se svjetiljka pretvara u **telegraf** kad se žarulja i prekidač razdvoje dugim žicama.
- Što je **zajednički vodič** (common) i kako štedi žice.
- Kako **Zemlja** može poslužiti kao vodič (**uzemljenje**) i što znači oznaka **V**.
- Zašto duge žice ne mogu raditi zauvijek: **otpor žice** raste s duljinom.

Svjetiljka na daljinu

Svjetiljka iz poglavlja 4 ima bateriju, prekidač i žarulju u istom kućištu. Ništa ne brani da ih razdvojimo: baterija i prekidač ostanu kod tebe, a žarulja je u susjednoj kući, povezana s dvije dugačke žice. Kad zatvoriš prekidač, žarulja kod susjeda zasvijetli.

Tako možeš slati Morseov kod i **izvan vidnog polja**, iza ugla, bez obzira na položaj prozora.

Na shemama se prekidač crta ovako:

```
otvoren (isključen):   ──•/  •──
zatvoren (uključen):   ──•────•──
```

Dvosmjerna veza

Za razgovor u oba smjera trebaju dva neovisna kruga: jedan s tvojim prekidačem i susjedovom žaruljom, drugi s njegovim prekidačem i tvojom žaruljom. Svaki krug ima dvije žice, dakle između kuća idu **četiri žice**. Krugovi su potpuno odvojeni, pa oboje možete slati istovremeno.

Zajednički vodič (common)

Dva kruga mogu **dijeliti jednu žicu**. Spoje se negativni polovi obiju baterija i taj spoj ide jednom žicom do druge kuće. Sad između kuća idu **tri žice** umjesto četiri, 25 % manje.

Zašto to radi:

- Kad šalješ samo ti, struja teče kroz tvoj krug; drugi dio nema zatvoren put pa njime ne teče ništa.
- Kad šalje samo susjed, teče kroz njegov krug.
- Kad šaljete oboje, oba kruga rade; zajednička žica vodi struju u oba smjera koji se poništavaju, pa njome ne teče ništa.

Svaki krug i dalje radi **neovisno**, iako dijele dio puta. Takav dijeljeni dio zove se **zajednički vodič** (engl. *common*).

Analogija: dvije kuće dijele isti prilazni put do ceste. Svaka ima svoja vrata i svoj automobil, ali dio puta je zajednički.

Zemlja kao vodič: uzemljenje

Zajednička žica može se zamijeniti nečim posve drukčijim: **samom Zemljom**.

Zemlja nije posebno dobar vodič, ali za vodiče vrijedi pravilo: **što deblji, to bolji**. A Zemlja je ogromna. Da bi je iskoristio, treba dobar i velik dodir: npr. bakrena šipka zabijena duboko u vlažno tlo ili metalna cijev za vodu koja ide u zemlju.

Takav spoj sa Zemljom zove se **uzemljenje** (engl. *ground*, u britanskom engleskom *earth*). Na shemama ima ovaj simbol:



S uzemljenjem jednosmjerni telegraf treba samo **jednu žicu**: kod tebe je minus pol baterije uzemljen, kod susjeda je žarulja uzemljena, a između vas je jedna žica. Elektroni dolaze iz zemlje kod susjeda, prolaze žaruljom, žicom, tvojim prekidačem i ulaze u plus pol baterije; s minus pola baterije odlaze u zemlju.

Elektroni naravno ne "putuju kroz zemlju" od kuće do kuće. Bolja slika je: **Zemlja je ogroman spremnik elektrona**, kao što je ocean spremnik kapi vode. Iz njega se elektroni mogu uzeti bilo gdje i u njega vratiti bilo gdje.

Dvosmjerni telegraf s uzemljenjem treba samo **dvije žice** između kuća, a radi isto kao verzija s tri žice.

Ograničenje: Zemlja ima određeni otpor, pa uzemljenje ne radi s baterijama od 1,5 V i malim žaruljama. Treba veći napon (npr. 100 V) i veće žarulje.

Oznaka V i nulti potencijal

Kad je minus pol baterije uzemljen, na shemi se baterija više ne crta. Umjesto nje stoji samo slovo **V** (napon), i podrazumijeva se da je druga strana izvora spojena na zemlju.

- **V** se može zamisliti kao "usisavač elektrona": vuče elektrone iz zemlje kroz krug, i pritom se obavlja rad (npr. svijetli žarulja).
- **Uzemljenje** je točka **nultog potencijala**. Tamo nema napona. U analogiji iz poglavlja 4, to je lopta koja već leži na podu: nema kamo pasti.

Krug s V i uzemljenjem ne izgleda kao krug, ali **jest** krug. Kad bi umjesto V nacrtao bateriju s uzemljenim minusom i spojio sve oznake uzemljenja jednom žicom, dobio bi običan zatvoren krug.

Granica: otpor dugih žica

Žicama smo riješili dva problema svjetiljke: ne treba nam vidna linija i nismo ograničeni dometom snopa svjetla. Ali bakar ipak ima otpor, i on raste s duljinom žice.

Debljina žice mjeri se u američkom sustavu **AWG** (American Wire Gauge): **manji broj znači deblju žicu** i manji otpor.

Primjer iz knjige

Situacija	Otpor	Struja kroz žarulju od 4 Ω na 3 V
Kratke žice (bez otpora)	4 Ω	3 / 4 = 0,75 A
Tanka žica (oko 10 Ω na 1000 stopa), duga 1 milju (oko 1,6 km), tamo i natrag	više od 100 Ω	manje od 3 / 100 = 0,03 A, žarulja ne svijetli

Moguća rješenja:

1. **Deblja žica.** Ima manji otpor (npr. oko 5 Ω po milji), ali je skupa.
2. **Veći napon i žarulje većeg otpora.** Žarulja od 100 W za 120 V ima oko 144 Ω, pa otpor žice postaje relativno mali dio ukupnog otpora i manje smeta.

S takvim rješenjima prvi telegrafski sustavi u 19. stoljeću dosezali su najviše nekoliko stotina milja. Za veće udaljenosti trebao je novi izum. On je tema sljedećeg poglavlja i, kako ćemo vidjeti, od njega se može sagraditi i računalo.

Ključne definicije

Pojam	Značenje
Zajednički vodič (common)	dio kruga koji dijele dva ili više krugova
Uzemljenje (ground, earth)	električni spoj sa Zemljom; Zemlja služi kao golemi spremnik elektrona
Nulti potencijal	točka bez napona; uzemljenje
V (na shemi)	izvor napona čija je druga strana uzemljena
AWG	američka mjera debljine žice; manji broj = deblja žica = manji otpor

Ključno za zapamtiti

1. Prekidač i žarulja mogu biti daleko jedan od drugoga; tako nastaje telegraf.
2. Dvosmjerna veza sa zasebnim krugovima treba četiri žice; zajednički vodič svodi to na tri.
3. Uzemljenje zamjenjuje jednu žicu Zemljom; dvosmjerna veza treba tada samo dvije žice.
4. Zemlja je kao ocean elektrona: iz nje se uzimaju i u nju vraćaju.
5. Uzemljenje radi samo s dovoljno visokim naponom jer i Zemlja ima otpor.
6. Oznaka V znači izvor napona čija je druga strana uzemljena; krug je i dalje zatvoren.
7. Otpor žice raste s duljinom, pa telegraf na veliku udaljenost ne radi bez dodatnog izuma.

Najčešće zabune

- **"Uzemljenje znači da krug nije zatvoren."** Krug je zatvoren kroz Zemlju.
- **"Elektroni putuju kroz zemlju od jedne kuće do druge."** Zemlja je spremnik; elektroni se uzimaju i vraćaju lokalno.
- **"Veći AWG broj znači deblju žicu."** Obrnuto.
- Riječ *ground* u engleskom znači i fizičko uzemljenje i zajednički vodič na shemi. U ovom poglavlju znači fizički spoj sa Zemljom.

Brza provjera razumijevanja

1. Koliko žica treba za dvosmjerni telegraf bez zajedničkog vodiča, sa zajedničkim vodičem i s uzemljenjem?
2. Zašto zajednička žica ne smeta kad oboje šaljete istovremeno?
3. Zašto uzemljenje ne radi s baterijom od 1,5 V?
4. Što predstavlja slovo V na shemi?
5. Ako se otpor žice udvostruči, što se događa sa strujom (uz isti napon)?

6. Telegraf i relej

Što ćeš naučiti u ovom poglavlju

- Što je **elektromagnet** i kako je omogućio telegraf.
- Kako radi telegrafski **ključ** i **zvučnik** (sounder).
- Što je **relej** i zašto je to najvažniji uređaj u prvom dijelu knjige.

Kratki kontekst

Početkom 19. stoljeća komunikacija je mogla biti **trenutna** (dovikivanje, signalizacija zastavicama) ili na **veliku udaljenost** (pismo), ali ne oboje istovremeno. Telegraf je to promijenio. Samuel Morse ga je razvijao od 1830-ih, a 1844. linija između Washingtona i Baltimorea prenijela je prvu javnu poruku. Biografski detalji nisu važni za razumijevanje gradiva.

Ideja telegrafa je jednostavna: napraviš nešto na jednom kraju žice, i zbog toga se nešto dogodi na drugom kraju. Upravo to smo napravili sa svjetiljkom u poglavlju 5. Morse ipak nije mogao koristiti žarulju, jer praktična žarulja tada još nije postojala. Koristio je **elektromagnet**.

Elektromagnet

Uzmi željeznu šipku i omotaj je stotinama namotaja tanke žice. Kad kroz žicu teče struja, šipka postaje **magnet** i privlači željezo i čelik. Kad struja prestane, šipka prestaje biti magnet.

Tanka i duga žica ima dovoljno otpora da elektromagnet ne bude kratki spoj.

Analogija: magnet koji možeš paliti i gasiti prekidačem. Kao dizalica na otpadu koja podigne auto i pusti ga kad se isključi struja.

Elektromagnet je temelj telegrafa: prekidač na jednom kraju uzrokuje **pokret** na drugom kraju.

Dijelovi telegrafa

Dio	Što je zapravo	Što radi
Ključ (key)	prekidač oblikovan za brzo tipkanje	kratki pritisak je točka, dugi je crta
Prijemnik	elektromagnet koji privlači metalnu polugu	u prvim verzijama poluga je pomicala olovku i crtala točke i crte po traci papira
Zvučnik (sounder)	elektromagnet s pomičnom polugom	kad struja počne teći, poluga udari dolje ("klik"); kad prestane, odskoči gore ("klak")

Operateri su brzo shvatili da kod mogu čitati **po zvuku**: brzi "klik-klak" je točka, sporiji "klik... klak" je crta. Zato je traka s olovkom napuštena.

Ključ, zvučnik, baterija i žice spajaju se isto kao telegraf sa žaruljom iz poglavlja 5. Uz uzemljenje treba samo jedna žica; baterija s uzemljenim minusom crta se kao V. Za dvosmjernu vezu treba još jedan ključ i zvučnik.

Telegraf je zanimljiv i zato što koristi **binarni kod**. Kasniji načini komunikacije (telefon, radio, televizija) napustili su binarne kodove, a oni su se vratili tek s računalima, CD-ima i digitalnom televizijom.

Morseov telegraf bio je uspješan i zato što je **otporan na loše uvjete**: ako je postojala žica između ključa i zvučnika, uglavnom je radio.

Problem udaljenosti

Kao u poglavlju 5, velika smetnja je **otpor dugih žica**. Neke telegrafske linije koristile su i 300 V i radile na 300 milja, ali žica se nije mogla produljivati beskonačno.

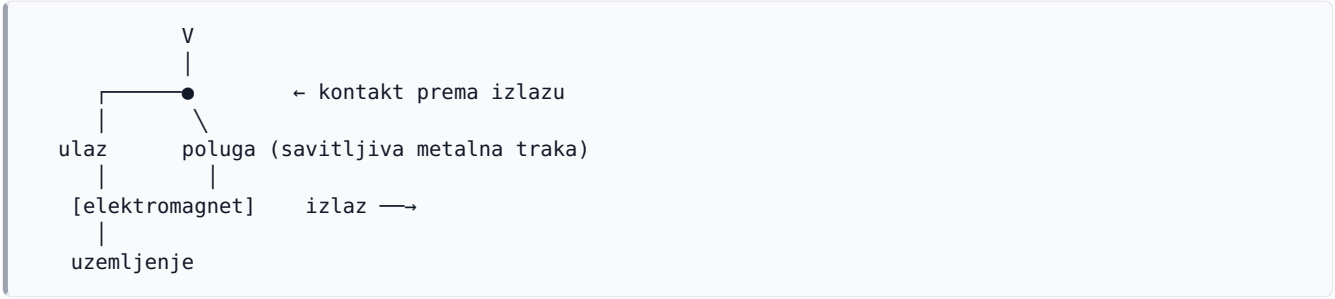
Očito rješenje: svakih nekoliko stotina milja postaviti čovjeka sa zvučnikom i ključem, koji primi poruku i **ponovno je pošalje** dalje. Operater gleda kako se poluga zvučnika miče gore-dolje, i svojim prstima miče ključ potpuno jednako.

Ako zvučnik i ključ rade isto, zašto ih ne spojiti **mehanički**, tako da poluga zvučnika sama pritišće ključ? Tada čovjek više nije potreban. Taj uređaj postoji: zove se **relej** (ili *ponavljač*, engl. *repeater*).

Relej

Relej je prekidač kojim ne upravlja ruka, nego električna struja.

Građa:



Kako radi:

1. Na **ulaz** stiže slaba struja (npr. s udaljene telegrafske stanice).
2. Ta struja prolazi kroz **elektromagnet** i on privuče savitljivu metalnu polugu.
3. Poluga je dio **prekidača** u drugom krugu. Kad je privučena, spoji izvor napona **V** s **izlazom**.
4. Na izlaz teče **nova, jaka struja** iz lokalnog izvora.
5. Kad ulazna struja prestane, poluga se vrati, a izlazna struja prestane.

Ulaz i izlaz su **dva odvojena kruga**. Ulazni krug samo "pritisne prekidač" izlaznog kruga.

Zato relej **pojačava** signal: slaba ulazna struja stvara jaku izlaznu struju jednakog oblika (točke i crte ostaju iste). Lanac releja svakih nekoliko stotina milja omogućuje telegraf preko cijelog kontinenta.

Cijeli lanac: ključ na tvojoj stanici → slaba struja dugom žicom → relej na releju stanici → jaka struja iz lokalne baterije → sljedeća duga žica → zvučnik kod prijatelja.

Analogija: štafeta. Trkač je umoran nakon nekoliko kilometara, ali preda palicu svježem trkaču. Palica (poruka) ostaje ista; snaga (struja) se obnavlja.

Zašto je relej važan

Relej je prekidač kojim upravlja struja. To znači da **izlaz jednog releja može upravljati drugim relejem**. Od takvih uređaja može se sagraditi velik dio računala, i to je plan sljedećih poglavlja. Prije toga treba naučiti brojati, i to ne samo u desetičnom sustavu.

Ključne definicije

Pojam	Značenje
Elektromagnet	željezo omotano žicom; magnet je samo dok žicom teče struja

Pojam	Značenje
Telegrafski ključ	prekidač za brzo slanje točaka i crta
Zvučnik (sounder)	elektromagnet s polugom koja klikom pokazuje primljeni signal
Relej (ponavljač)	prekidač kojim upravlja struja kroz elektromagnet; slab ulaz pokreće jak izlaz
Pojačanje	slab signal na ulazu daje jak signal istog oblika na izlazu

Ključno za zapamtiti

1. Elektromagnet je magnet koji se pali i gasi strujom.
2. Telegraf: prekidač (ključ) na jednom kraju, elektromagnet (zvučnik) na drugom.
3. Telegraf je koristio binarni kod (točke i crte).
4. Otpor dugih žica ograničava domet; rješenje je periodično obnavljanje signala.
5. Relej je prekidač kojim upravlja struja, a ne ruka.
6. Relej ima dva odvojena kruga: ulazni (elektromagnet) i izlazni (prekidač s vlastitim izvorom V).
7. Relej pojačava: slaba ulazna struja daje jaku izlaznu struju istog oblika.
8. Budući da izlaz releja može upravljati drugim relejem, od releja se može graditi računalo.

Najčešće zabune

- **"Relej prosljeđuje istu struju dalje."** Ne: ulazna struja samo pokreće prekidač; izlazna struja dolazi iz drugog, lokalnog izvora.
- **"Zvučnik je zvučnik kao na radiju."** Telegrafski zvučnik je mehanički: poluga udara i proizvodi klik.

Brza provjera razumijevanja

1. Što se dogodi s elektromagnetom kad se prekine struja?
2. Kako operater razlikuje točku od crte slušajući zvučnik?
3. Zašto telegrafska linija ne može biti beskonačno duga bez releja?
4. Opiši korak po korak što se događa u releju kad na ulaz stigne struja.
5. Zašto je važno da izlaz releja može upravljati drugim relejem?

7. Naših deset znamenki: pozicijski brojevni sustav

Što ćeš naučiti u ovom poglavlju

- Zašto su brojevi također **kod**, i zašto broj deset nije ničim poseban.
- Kako su radili stari brojevni sustavi (rimski brojevi) i zašto su loši za računanje.
- Tri ključne osobine našeg sustava: **pozicijski zapis**, **nema znaka za deset** i **nula**.
- Kako se broj rastavlja na **potencije broja 10**, uključujući decimale.

Brojevi su kod

Riječ "mačka" u drugim jezicima zvuči drukčije, i lako prihvaćamo da je jezik kod. Za brojeve to manje osjećamo, jer gotovo cijeli svijet piše iste znamenke: 1, 2, 3…

Ali brojevi su **najapstraktniji kod** koji svakodnevno koristimo. Znamenka **3** može značiti tri jabuke, treći kanal, rezultat utakmice ili tri šalice brašna. Upravo zato što smo navikli na taj zapis, teško prihvaćamo da se ista količina (tri jabuke) može zapisati i drugim znakovima, npr. kao **11**. To je tema ovog i sljedećeg poglavlja.

Deset nije poseban broj

Većina civilizacija broji u desetkama (ponekad u peticama) zato što ljudi imaju **deset prstiju**. Engleska riječ *digit* znači i "znamenka" i "prst". Da imamo osam ili dvanaest prstiju, brojili bismo drukčije.

Desetični (decimalni) sustav je dakle **proizvoljan**. Ipak, potencijama broja 10 dajemo posebna imena:

Potencija	Vrijednost	Ime
10^1	10	deset
10^2	100	sto
10^3	1 000	tisuća
10^6	1 000 000	milijun
10^9	1 000 000 000	milijarda (engl. *billion*)

Pazi: engleski *billion* je hrvatska **milijarda** (10^9), a ne bilijun.

Od crtica do rimskih brojeva

Brojevi su vjerojatno nastali za brojanje imovine i trgovinu. Prvo su se crtali predmeti, zatim jedan predmet i **crtice** za količinu. Kad crtica ima previše, treba bolji sustav.

Od starih sustava u upotrebi su ostali samo **rimski brojevi**:

Znak	I	V	X	L	C	D	M
Vrijednost	1	5	10	50	100	500	1000

Primjer: XXVII = 10 + 10 + 5 + 1 + 1 = 27.

Rimske brojeve je relativno lako **zbrajati**: spojiš znakove oba broja i pojednostaviš ih (pet I postaje V, dva V postaju X i tako dalje). Zato su se u Europi dugo koristili u knjigovodstvu. Ali **množenje i dijeljenje** su vrlo teški. Slično vrijedi i za druge stare sustave.

Hindsko-arapski sustav

Sustav koji danas koristimo potječe iz Indije, a u Europu su ga donijeli arapski matematičari. Važnu ulogu imao je perzijski matematičar al-Hvarizmi, oko 825. godine; iz njegova imena dolazi riječ **algoritam**.

Od starijih sustava razlikuje se po tri stvari:

1. **Pozicijski je.** Vrijednost znamenke ovisi o njezinom **mjestu** u broju. U 100 i u 1 000 000 postoji samo jedna jedinica, ali milijun je puno veći.
2. **Nema poseban znak za deset.** Deset se piše kao 1 na drugom mjestu i 0 na prvom.
3. **Ima nulu.** Nula je jedan od najvažnijih izuma u matematici. Omogućuje razlikovati 25, 205 i 250, i olakšava množenje i dijeljenje.

Kako se čita broj

Broj 4825 izgovaramo "četiri tisuće osamsto dvadeset pet", što znači:

> $4825 = 4 \times 1000 + 8 \times 100 + 2 \times 10 + 5 \times 1$

Ili potencijama broja 10:

> $4825 = 4 \times 10^3 + 8 \times 10^2 + 2 \times 10^1 + 5 \times 10^0$

Svaki broj na nultu potenciju je 1, pa je $10^0 = 1$.

Svako mjesto u broju odgovara jednoj potenciji broja 10:

Mjesto (zdesna)	7.	6.	5.	4.	3.	2.	1.
Vrijednost mjesta	10^6	10^5	10^4	10^3	10^2	10^1	10^0
Ime	milijuni	stotine tisuća	desetci tisuća	tisuće	stotine	desetice	jedinice

Posebni znak za deset ne treba jer 1 jednostavno stavimo na drugo mjesto, a 0 drži prazno mjesto.

Decimale

Isti obrazac nastavlja se desno od decimalne točke, s **negativnim** potencijama:

> $42\,705{,}684 = 4 \times 10^4 + 2 \times 10^3 + 7 \times 10^2 + 0 \times 10^1 + 5 \times 10^0 + 6 \times 10^{-1} + 8 \times 10^{-2} + 4 \times 10^{-3}$

gdje je $10^{-1} = 0{,}1$, $10^{-2} = 0{,}01$ i $10^{-3} = 0{,}001$. Eksponenti idu prema nuli, a zatim postaju negativni.

Zašto je pozicijski sustav dobar za računanje

Ako znaš da je 3 + 4 = 7, znaš i da je 30 + 40 = 70 i 3000 + 4000 = 7000. Zbrajanje brojeva bilo koje duljine svodi se na **zbrajanje parova jednoznamenkastih brojeva**, mjesto po mjesto, uz prijenos. Zato smo u školi učili **tablicu zbrajanja** (0 + 0 do 9 + 9, najveći rezultat 18).

Isto vrijedi za množenje: svodi se na **tablicu množenja** jednoznamenkastih brojeva (do 9 × 9 = 81) i zbrajanje.

Najvažnije je ovo: pozicijski sustav **jednako dobro radi i s osnovicom koja nije deset**. To je tema sljedećeg poglavlja. (Knjiga to šaljivo uvodi crtanim likovima koji imaju po četiri prsta na ruci, pa im prirodno odgovara sustav s osnovicom osam.)

Ključne definicije

Pojam	Značenje
Desetični (decimalni) sustav	brojevni sustav s osnovicom 10 i znamenkama 0 – 9
Pozicijski zapis	vrijednost znamenke ovisi o njezinom mjestu u broju
Osnovica (baza)	broj čije potencije daju vrijednosti mjesta; kod nas 10
Potencija	10^n je 10 pomnoženo sa sobom n puta; $10^0 = 1$; $10^{-n} = 1 / 10^n$
Nula	znamenka koja drži prazno mjesto i omogućuje pozicijski zapis

Ključno za zapamtiti

1. Brojevi su apstraktan kod; ista količina može se zapisati na više načina.
2. Osnovica 10 dolazi od deset prstiju; nema matematičkog razloga za nju.
3. Rimski brojevi se lako zbrajaju, ali teško množe i dijele.
4. Naš sustav je pozicijski, nema znak za deset i ima nulu.
5. Svaki broj je zbroj znamenki pomnoženih potencijama osnovice: $4825 = 4 \cdot 10^3 + 8 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0$.
6. Desno od decimalne točke potencije su negativne: 10^{-1}, 10^{-2}, ...
7. Zbrajanje i množenje svode se na tablice za jednoznamenkaste brojeve.
8. Pozicijski zapis radi s bilo kojom osnovicom.

Najčešće zabune

- **"10^0 je 0."** Ne: svaki broj na nultu potenciju je 1.
- **"Billion je bilijun."** Engleski *billion* je milijarda (10^9).
- **"Nula je samo 'ništa'."** U pozicijskom sustavu nula ima važnu ulogu: drži mjesto.

Brza provjera razumijevanja

1. Zapiši 3907 kao zbroj znamenki pomnoženih potencijama broja 10.
2. Zašto u našem sustavu ne treba poseban znak za broj deset?
3. Koliko je MMXXVI?
4. Što znači 6×10^{-2} i koliko je to?
5. Zašto je zbrajanje u pozicijskom sustavu jednostavnije nego u rimskom?

8. Alternative broju deset: oktalni, četvorni i binarni sustav

Što ćeš naučiti u ovom poglavlju

- Kako se broji u sustavima s osnovicom **8**, **4** i **2**.
- Kako se broj iz bilo kojeg od tih sustava pretvori u desetični i obrnuto.
- Kako se **zbraja i množi u binarnom sustavu**.
- Zašto je binarni sustav most između **aritmetike i elektriciteta**, i odakle riječ **bit**.

Ista količina, drugi zapis

Zapis "10" znači deset samo zato što imamo deset prstiju. Kad bismo brojili na manje prstiju, "10" bi značilo osam, četiri ili samo dva predmeta. Pravilo je uvijek isto:

> U bilo kojem pozicijskom sustavu, **10** znači "jedna osnovica i nula jedinica".

Kad ponestane jednoznamenkastih brojeva, sljedeći broj je uvijek 10.

Da se brojevi iz različitih sustava ne pobrkaju, osnovicu pišemo kao indeks: 10_{10} (deset), 10_{8} (osam), 10_{2} (dva). Broj 10_{8} čita se "jedan nula", a ne "deset".

Oktalni sustav (osnovica 8)

Ima znamenke **0 – 7**. Znamenke 8 i 9 ne postoje, kao što u desetičnom sustavu ne postoji jedna znamenka za deset.

Brojanje: 0, 1, 2, 3, 4, 5, 6, 7, **10**, 11, ..., 17, **20**, ..., 77, **100**, ...

Isto kao desetično, samo se preskače svaki broj koji sadrži 8 ili 9.

Mjesta u oktalnom broju su potencije broja 8:

Mjesto (zdesna)	5.	4.	3.	2.	1.
Vrijednost	$8^4 = 4096$	$8^3 = 512$	$8^2 = 64$	$8^1 = 8$	$8^0 = 1$

Primjer pretvorbe u desetični:

> $247_{8} = 2 \times 64 + 4 \times 8 + 7 \times 1 = 128 + 32 + 7 = \mathbf{167_{10}}$

Lijepi "okrugli" oktalni brojevi (100_{8}, 200_{8}, 400_{8}, 1000_{8}) desetično su 64, 128, 256 i 512, dakle **potencije broja 2**. To je prvi nagovještaj da je oktalni sustav koristan za rad s binarnim kodovima.

Računanje u oktalnom sustavu

Zbraja se i množi isto kao desetično, samo s drugim tablicama za jednoznamenkaste brojeve:

- $5_{8} + 7_{8} = 14_{8}$ (jer je 5 + 7 = 12 desetično, a to je jedna osmica i četiri jedinice);
- $3_{8} \times 3_{8} = 11_{8}$ (9 desetično);
- $4_{8} \times 6_{8} = 30_{8}$ (24 desetično).

Prijenos ("pamtim jedan") ide kad zbroj dosegne **8**, a ne 10.

Četvorni sustav (osnovica 4)

Znamenke **0 – 3**, mjesta su potencije broja 4 (1, 4, 16, 64, 256, 1024...).

Brojanje: 0, 1, 2, 3, 10, 11, 12, 13, 20, ..., 33, 100, ...

Primjer: $1302_4 = 1 \times 64 + 3 \times 16 + 0 \times 4 + 2 \times 1 = 64 + 48 + 0 + 2 = \mathbf{114_{10}}$.

Ovaj sustav je samo međukorak prema najvažnijem.

Binarni sustav (osnovica 2)

Ima samo dvije znamenke: **0 i 1**. Ime dolazi od latinskog za "dva po dva".

Brojanje vrlo brzo prelazi na više znamenki:

Desetično	Binarno	Desetično	Binarno
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Nule s lijeve strane (0011 umjesto 11) ne mijenjaju vrijednost; pišu se radi preglednosti.

Binarni brojevi nisu "veliki", nego **dugi**: za istu vrijednost trebaju puno više znamenki.

Uzorak u stupcima

Pogledaj tablicu stupac po stupac:

- krajnja desna znamenka izmjenjuje se: 0, 1, 0, 1...
- sljedeća: dvije nule, dvije jedinice...
- sljedeća: četiri nule, četiri jedinice...
- sljedeća: osam nula, osam jedinica.

Drugim riječima, kad god neka znamenka prijeđe iz **1 u 0**, sljedeća znamenka lijevo se **promijeni**. To pravilo je temelj brojila u poglavlju 14.

Sljedećih 16 brojeva (16 – 31) dobiješ tako da ispred prvih 16 staviš 1: 10000, 10001, ..., 11111.

Krajnja desna znamenka zove se **najmanje značajna**, a krajnja lijeva **najznačajnija**.

Mjesta su potencije broja 2

Mjesto (zdesna)	8.	7.	6.	5.	4.	3.	2.	1.
Vrijednost	128	64	32	16	8	4	2	1
Potencija	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Zato je svaki binarni broj oblika **1 i iza njega samo nule** potencija broja 2, a broj nula je eksponent: $1000_2 = 2^3 = 8$.

Ista potencija broja 2 u različitim sustavima:

Potencija	Desetično	Oktalno	Četvorno	Binarno
2^0	1	1	1	1
2^3	8	10	20	1000
2^6	64	100	1000	1000000
2^8	256	400	10000	100000000
2^{10}	1024	2000	100000	10000000000

Binarno u desetično

Pomnoži svaku znamenku s vrijednošću njezinog mjesta i zbroji. Budući da su znamenke samo 0 i 1, **zbrojiš vrijednosti mjesta na kojima je 1**.

Primjer: 10110110_2

Mjesto	128	64	32	16	8	4	2	1
Znamenka	1	0	1	1	0	1	1	0
Doprinos	128	0	32	16	0	4	2	0

Zbroj: 128 + 32 + 16 + 4 + 2 = $\mathbf{182_{10}}$.

Desetično u binarno (metoda iz knjige, za brojeve 0 – 255)

Ideš od najveće vrijednosti mjesta (128) prema najmanjoj (1):

1. Ako je broj **veći ili jednak** vrijednosti mjesta, upiši **1** i oduzmi tu vrijednost od broja.
2. Ako je **manji**, upiši **0** i ne oduzimaj ništa.
3. Ono što ostane, nastavi s idućim mjestom.

Primjer: 182

Mjesto	128	64	32	16	8	4	2	1
Ostatak prije	182	54	54	22	6	6	2	0
Znamenka	1	0	1	1	0	1	1	0

Rezultat: $\mathbf{10110110_2}$. Provjera: jednak je broju iz prethodnog primjera.

Računanje u binarnom sustavu

Najljepše u binarnom sustavu je koliko su male tablice.

Tablica zbrajanja

+	0	1
0	0	1
1	1	**10**

$1 + 1 = 10_2$: pišeš 0, prenosiš 1. Ako se pojavi i prijenos, $1 + 1 + 1 = 11_2$: pišeš 1, prenosiš 1.

Primjer: $1011_2 + 0110_2$ (11 + 6)

```
  1 0 1 1
+ 0 1 1 0
---------
1 0 0 0 1
```

Stupac po stupac zdesna: 1 + 0 = 1; 1 + 1 = 10 (pišem 0, prenosim 1); 0 + 1 + 1 = 10 (pišem 0, prenosim 1); 1 + 0 + 1 = 10 (pišem 0, prenosim 1); prijenos 1 ide na novo mjesto. Rezultat 10001_2 = **17**. Točno: 11 + 6 = 17.

Tablica množenja

×	0	1
0	0	0
1	0	1

Ovo slijedi iz dva pravila koja vrijede u svakom sustavu: sve puta 0 je 0, sve puta 1 ostaje isto. Zato se binarno množenje svodi na **pomicanje i zbrajanje**: za svaku jedinicu u drugom broju prepišeš prvi broj, pomaknut ulijevo.

Primjer: $1011_2 \times 110_2$ (11 × 6)

```
    1 0 1 1
  ×   1 1 0
  ---------
    0 0 0 0      (0 × 1011)
  1 0 1 1        (1 × 1011, pomaknuto za 1)
1 0 1 1          (1 × 1011, pomaknuto za 2)
-------------
1 0 0 0 0 1 0
```

1000010_2 = 64 + 2 = **66**. Točno: 11 × 6 = 66.

Čitljivost dugih binarnih brojeva

Desetične brojeve grupiramo po tri znamenke (12 000 000). Binarne brojeve obično grupiramo **po četiri** znamenke, razmakom ili crticom: 1011 0111 0001. Kasnije (poglavlje 15) upoznat ćemo kraći zapis.

Most između brojeva i elektriciteta

Binarni sustav je najjednostavniji mogući: samo 0 i 1. A upravo zato se savršeno slaže s napravama iz prethodnih poglavlja:

Naprava	Znamenka 1	Znamenka 0
Žica	struja teče	struja ne teče
Prekidač	zatvoren (uključen)	otvoren (isključen)
Žarulja	svijetli	ne svijetli
Relej	zatvoren (privučen)	u mirovanju

Svaka od tih stvari može predstavljati **jednu binarnu znamenku**. To je veza između aritmetike i električnih krugova, i na njoj se gradi računalo.

Riječ bit

Oko 1948. američki matematičar John Tukey predložio je kratku riječ za "binarnu znamenku" (*binary digit*): **bit**.

Ključne definicije

Pojam	Značenje
Oktalni sustav	osnovica 8, znamenke 0 – 7
Četvorni sustav	osnovica 4, znamenke 0 – 3
Binarni sustav	osnovica 2, znamenke 0 i 1
Najmanje / najznačajnija znamenka	krajnja desna / krajnja lijeva znamenka broja
Bit	binarna znamenka (*binary digit*)

Ključno za zapamtiti

1. U svakom pozicijskom sustavu "10" znači jednu osnovicu.
2. Vrijednosti mjesta su potencije osnovice: 1, 8, 64... (oktalno), 1, 2, 4, 8... (binarno).
3. Binarno u desetično: zbroji vrijednosti mjesta na kojima stoji 1.
4. Desetično u binarno: od najveće vrijednosti mjesta, oduzimaj gdje može (1) i preskoči gdje ne može (0).
5. Binarno zbrajanje: 1 + 1 = 10, uz prijenos; binarno množenje je pomicanje i zbrajanje.
6. Kad znamenka prijeđe iz 1 u 0, sljedeća lijevo se mijenja.
7. Binarni broj 1 s n nula je 2^n.
8. Žica, prekidač, žarulja i relej mogu predstavljati jedan bit.

Najčešće zabune

- **"101_2 je sto jedan."** To je pet; čitaj "jedan nula jedan".
- **"U binarnom 1 + 1 = 2."** Znamenka 2 ne postoji; rezultat je 10_2.
- **"Binarni brojevi su veći od desetičnih."** Nisu veći, samo su duži.
- **"Prijenos u oktalnom ide na 10."** Ide kad zbroj dosegne osnovicu, dakle 8.

Brza provjera razumijevanja

1. Pretvori 11010_2 u desetični sustav.
2. Pretvori 100_{10} u binarni sustav metodom oduzimanja.
3. Koliko je 777_8 desetično? Koji je sljedeći oktalni broj?
4. Zbroji binarno 0111 + 0101 i provjeri desetično.
5. Zašto je binarni sustav prirodan za prekidače i releje?

9. Bit po bit: informacija kao izbor

Što ćeš naučiti u ovom poglavlju

- Zašto je **bit** najmanja moguća količina informacije.
- Kako odrediti **koliko bitova treba** za zadani broj mogućnosti (potencije broja 2 i logaritam).
- Što su **šum** i **zalihost** (redundancija).
- Da značenje bitova uvijek ovisi o **dogovoru** (kodu) i kontekstu.
- Kako su bitovi skriveni u stvarnim predmetima: kodiranje filma i **crtični kod (UPC)** s ugrađenom provjerom grešaka.

Bit kao "da ili ne"

Ponekad trebaš poslati samo odgovor **da ili ne**. Za to ne treba rečenica ni riječ. Dovoljan je jedan znak koji ima dva stanja: zastavica izvješena ili ne, natpis "otvoreno" ili "zatvoreno", svjetlo u prozoru upaljeno ili ugašeno. Knjiga to pokazuje primjerom pjesme u kojoj vrpca na drvetu znači "da", a njezin izostanak "ne".

Taj jedan izbor između dvije mogućnosti je **bit**: 0 ili 1.

Binarni sustav nije poseban po broju prstiju, nego po tome što je **najjednostavniji mogući**. Ako iz njega makneš znamenku 1, ostaje samo 0, a s jednom znamenkom ne možeš ništa reći.

U računalnom dobu bit je postao **osnovna građevna jedinica informacije**. Bit nosi vrlo **malo** informacije, najmanje moguće; manje od bita znači nikakvu informaciju. Ali "malo" ne znači "nevažno": jedan bit može nositi vrlo važnu odluku. Složenije informacije prenose se s **više** bitova.

Informacija je izbor među mogućnostima

Najvažnija ideja poglavlja:

> **Informacija predstavlja izbor između dvije ili više mogućnosti.**

Kad govoriš, svaka riječ je izbor iz svih riječi u rječniku. Kad bi sve riječi bile numerirane, mogli bismo umjesto riječi slati brojeve (naravno, uz isti numerirani rječnik na obje strane).

Obrnuto vrijedi isto: **sve što se može svesti na izbor među mogućnostima može se izraziti bitovima.** Ono što se ne može tako svesti (osjećaji, iskustva) ne može se ni kodirati bitovima.

Koliko bitova treba

Jedan, dva, tri bita

Broj bitova	Kombinacije	Broj mogućnosti
1	0, 1	2
2	00, 01, 10, 11	4
3	000, 001, ..., 111	8

Primjer iz knjige (svjetiljke u tornju): ako treba javiti samo "napad dolazi" ili "nema napada", dovoljna je jedna svjetiljka (jedan bit). Ako treba javiti "nema napada", "dolaze kopnom" ili "dolaze morem", trebaju **dvije** svjetiljke. Dva bita daju čak četiri mogućnosti; ovdje se koriste tri.

Primjer (dva kritičara): svaki daje palac gore (1) ili dolje (0). Dva bita opisuju sve četiri kombinacije njihovih ocjena, npr. 10 znači "prvi je volio film, drugi nije". Ako dodaš i svoje mišljenje, treći bit, mogućnosti je osam.

Dobro je kod bitova to što **znaš da su pokrivene sve mogućnosti**: s tri bita ima ih točno osam, ni više ni manje.

Opće pravilo

> **broj kodova = 2^(broj bitova)**

Svaki dodatni bit **udvostručuje** broj mogućnosti. Ista je formula kao kod Morseovog koda u poglavlju 2.

Bitova	1	2	3	4	5	6	7	8	9	10
Kodova	2	4	8	16	32	64	128	256	512	1024

Analogno je i s desetičnim znamenkama: troznamenkasti pozivni broj daje $10^3 = 1000$ kombinacija (000 – 999).

Unatrag: logaritam

Ako znaš **koliko mogućnosti** trebaš, a pitaš **koliko bitova**, koristiš **logaritam po bazi 2**. Logaritam je suprotno od potencije:

> $2^7 = 128$ je isto što i $\log_2 128 = 7$

Pravilo u praksi: uzmi prvu potenciju broja 2 koja je **jednaka ili veća** od broja mogućnosti.

Broj mogućnosti	Koliko bitova	Zašto
7 (npr. ocjene od 1 do 4 zvjezdice s polovicama)	3	$2^3 = 8 \geq 7$; jedan kod ostaje neiskorišten
13 (ocjene od F do A+)	4	$2^3 = 8$ je premalo, $2^4 = 16$ dovoljno; 3 koda neiskorištena
24 (standardne osjetljivosti filma)	5	$2^4 = 16$ premalo, $2^5 = 32$ dovoljno
200	8	$\log_2 200 \approx 7{,}64$, pa treba 8 bitova

Neiskorišten kod je **nedefiniran**. Ako se pojavi, znaš da je nastala greška.

Značenje bitova je stvar dogovora

Bit sam po sebi ne znači ništa. Je li 1 "palac gore" ili "palac dolje" posve je **proizvoljno**; važno je samo da svi koji koriste kod znaju dogovor. Isto vrijedi za skupine bitova: za ocjene s tri bita može se odabrati redoslijed kodova koji ima neki smisao (npr. veći broj = bolja ocjena), ali jednako ispravan je i posve nasumičan raspored, ako ga svi znaju.

Značenje bitova uvijek se razumije **iz konteksta**. Vrpca na drvetu nešto znači samo osobi koja zna dogovor; za sve ostale je komad tkanine.

Šum i zalihost

U teoriji komunikacije:

- **Šum** je sve što ometa komunikaciju: krčanje na telefonskoj liniji, mrak i udaljenost kod svjetiljki.
- **Zalihost** (redundancija) je dodatna informacija koja pomaže da poruka prođe unatoč šumu.

Govor je jako zalihan: ne trebaš čuti svaki slog da bi razumio rečenicu. U primjeru sa svjetiljkama, korištenje samo tri od četiri moguće kombinacije, uz kodove "jedna svjetiljka" i "dvije svjetiljke" (a ne "lijeva" i "desna"), čini poruku otpornijom: promatrač ne mora znati **koja** je svjetiljka upaljena, samo **koliko** ih je.

Bitovi u stvarnim predmetima

Kodiranje filma (DX)

Na kolutu fotografskog filma od 35 mm postoji polje od 12 kvadratića: metalni (srebrni) kvadratić je bit 1 (vodi struju), obojeni crni je bit 0 (izolator). Kontakti u fotoaparatu "čitaju" te bitove. Pet bitova kodira osjetljivost filma (jednu od 24 standardne vrijednosti, npr. 100, 200, 400), a ostali bitovi broj snimaka i druge podatke. Jeftiniji fotoaparati čitaju samo dva bita i razlikuju četiri najčešće osjetljivosti.

Crtični kod (UPC)

Crtični kod na proizvodima (Universal Product Code) izgleda kao crte različitih širina, ali je zapravo **niz bitova**. Čitač gleda tanku vodoravnu "krišku" koda: crna crta je niz jedinica, bijeli razmak niz nula. Najtanja crta je 1 bit; deblje crte i razmaci su 2, 3 ili 4 bita.

Cijeli UPC je **95 bitova**:

Dio	Bitova	Sadržaj
Lijevi zaštitni uzorak	3	uvijek `101`; čitač iz njega određuje širinu jednog bita
Lijevih 6 znamenki	6 × 7	svaka znamenka 7 bitova
Srednji zaštitni uzorak	5	uvijek `01010`
Desnih 6 znamenki	6 × 7	svaka znamenka 7 bitova
Desni zaštitni uzorak	3	uvijek `101`

Kodovi znamenki s **lijeve strane**:

Znamenka	Kod	Znamenka	Kod
0	0001101	5	0110001
1	0011001	6	0101111
2	0010011	7	0111011
3	0111101	8	0110111
4	0100011	9	0001011

Kodovi s **desne strane** su **komplementi** lijevih: svaka 0 postaje 1 i obrnuto (npr. 0 desno je 1110010).

Ugrađene provjere grešaka

UPC je pun zalihosti, da kod ne bi bio krivo pročitan ni lako krivotvoren:

1. **Zaštitni uzorci** moraju biti na pravom mjestu, inače čitač odbija kod.
2. **Početak i kraj kodova:** lijevi kodovi počinju s 0 i završavaju s 1; desni obrnuto.
3. **Paritet.** Skupina bitova ima **parni paritet** ako sadrži paran broj jedinica, a **neparni** ako sadrži neparan. Svi lijevi kodovi imaju neparan paritet, a svi desni paran. Svaka znamenka ujedno ima točno dvije skupine jedinica, dakle dvije crne crte.
4. **Kontrolna znamenka** na kraju (vidi niže).

Zbog pariteta čitač zna i u kojem **smjeru** čita: ako prve znamenke imaju paran paritet, kod se čita zdesna nalijevo. Tako se proizvod može provući kroz čitač u oba smjera.

Što kodira 12 znamenki

Znamenke	Značenje
1.	vrsta koda (0 je običan proizvod; postoje posebne vrijednosti, npr. za robu koja se važe i za kupone)

Znamenke	Značenje
2. – 6.	proizvođač
7. – 11.	proizvod tog proizvođača
12.	kontrolna znamenka

UPC **ne sadrži cijenu**. Cijena se dohvaća iz računala trgovine. Ispod crta su otisnute iste znamenke, za slučaj da ih blagajnik treba upisati ručno.

Kontrolna znamenka

Označi prvih 11 znamenki slovima A, B, C, ..., K. Izračunaj:

> S = 3 × (A + C + E + G + I + K) + (B + D + F + H + J)

Kontrolna znamenka je razlika između S i **sljedećeg višekratnika broja 10** (ako je S već višekratnik, znamenka je 0).

Primjer (izmišljen kod 0 12345 67890):

- znamenke na neparnim mjestima: 0, 2, 4, 6, 8, 0 → zbroj 20 → × 3 = 60
- znamenke na parnim mjestima: 1, 3, 5, 7, 9 → zbroj 25
- S = 60 + 25 = 85; sljedeći višekratnik od 10 je 90
- kontrolna znamenka = 90 − 85 = **5**

Ako čitač izračuna drugu kontrolnu znamenku od otisnute, odbija kod.

Cijena sigurnosti: za 4 bita potrebna za jednu znamenku UPC troši 7, a s praznim rubovima cijeli kod zauzima više od 10 bitova po korisnoj znamenki.

Morseov kod i Brailleovo pismo kao bitovi

Morseov kod također se može zapisati bitovima. Uz pojednostavljenje da crta traje dvije točke: točka je `1`, crta `11`, stanka unutar slova `0`, stanka između slova `00`. Tako je E = `100`, A = `101100`, T = `1100`. Morseov kod je tada niz bitova nalik na presjek crtičnog koda.

Brailleovo pismo je još jednostavnije: to je **6-bitni kod**. Izbočena točka je 1, ravna 0, a točke 1 – 6 su bitovi s lijeva na desno. Slovo a (samo točka 1) je `100000`, slovo c (točke 1 i 4) je `100100`.

Zaključak poglavlja

Bitovi mogu predstavljati riječi, slike, zvuk, glazbu, filmove, šifre proizvoda, osjetljivost filma, ocjene i namjere. Ali u temelju **bitovi su brojevi**. Da bi nešto prikazao bitovima, prebroji mogućnosti; to određuje koliko bitova treba, pa svakoj mogućnosti dodijeliš broj.

Bitovi imaju ulogu i u **logici**, gdje se pitamo je li tvrdnja istinita ili lažna. I istina i laž mogu biti 1 i 0. To je tema sljedećeg poglavlja.

Ključne definicije

Pojam	Značenje
Bit	najmanja jedinica informacije; izbor između dvije mogućnosti (0 ili 1)
Logaritam po bazi 2	koliko puta treba pomnožiti 2 da se dobije broj; $\log_2 128 = 7$
Šum	sve što ometa prijenos poruke

Pojam	Značenje
Zalihost (redundancija)	dodatna informacija koja pomaže da poruka preživi šum
Paritet	je li broj jedinica u skupini bitova paran ili neparan
Kontrolna znamenka	znamenka izračunata iz ostalih, za otkrivanje grešaka
UPC	crtični kod: 95 bitova, 12 znamenki, sa zaštitnim uzorcima i provjerama

Ključno za zapamtiti

1. Informacija je izbor između mogućnosti; bit je izbor između dvije.
2. n bitova daje 2^n mogućnosti; svaki dodatni bit udvostručuje broj mogućnosti.
3. Broj potrebnih bitova je najmanji n za koji je $2^n \geq$ broj mogućnosti ($\log_2$ zaokružen naviše).
4. Značenje bitova je dogovor; mora ga znati i pošiljatelj i primatelj.
5. Neiskorišteni (nedefinirani) kodovi otkrivaju greške.
6. Zalihost štiti poruku od šuma.
7. UPC koristi paritet, zaštitne uzorke i kontrolnu znamenku; ne sadrži cijenu.
8. Morseov kod i Brailleovo pismo mogu se zapisati kao nizovi bitova.
9. U temelju su bitovi brojevi.

Najčešće zabune

- **"Jedan bit je nevažna informacija."** Bit je najmanja količina informacije, ali može nositi vrlo važnu odluku.
- **"1 uvijek znači 'da'."** Značenje 1 i 0 je proizvoljan dogovor.
- **"Crtični kod sadrži cijenu."** Ne; sadrži šifru proizvođača i proizvoda.
- **"Za 200 mogućnosti treba 7 bitova jer je $\log_2 200 \approx 7{,}6$."** Treba 8; zaokružuje se naviše.

Brza provjera razumijevanja

1. Koliko bitova treba da se kodira 50 različitih stanja? Koliko kodova ostaje neiskorišteno?
2. Što je zalihost i zašto je govor otporan na šum?
3. Izračunaj kontrolnu znamenku za UPC čijih je prvih 11 znamenki 0 36000 29145.
4. Kako čitač UPC-a zna čita li kod slijeva ili zdesna?
5. Zapiši slovo b iz Brailleovog pisma (točke 1 i 2) kao 6-bitni broj.

10. Logika i prekidači: Booleova algebra

Što ćeš naučiti u ovom poglavlju

- Što je **Booleova algebra** i po čemu se razlikuje od obične algebre.
- Tri osnovne operacije: **I (AND)**, **ILI (OR)** i **NE (NOT)**, i njihove tablice.
- Kako se uvjeti iz svakodnevnog govora zapisuju kao Booleov izraz i provjeravaju s 0 i 1 (**Booleov test**).
- Najvažnije otkriće poglavlja: **prekidači u seriji rade I, a prekidači u paraleli rade ILI**. Tako se logika može izvesti strujnim krugom.

Od filozofije do matematike

Logika je stoljećima bila dio filozofije: proučavala je kako iz istinitih pretpostavki izvesti zaključak. Osnovni oblik je **silogizam**: dvije premise i zaključak. Primjer: "Svi sisavci dišu plućima. Dupin je sisavac. Dakle, dupin diše plućima."

Matematičari su dugo pokušavali logiku zapisati simbolima. Prvi koji je u tome uspio bio je engleski matematičar **George Boole** (19. stoljeće). Njegova algebra danas se zove **Booleova algebra**.

Podsjetnik: obična algebra

U običnoj algebri slova predstavljaju **brojeve**, a operacije su zbrajanje i množenje. Vrijede pravila:

Pravilo	Zbrajanje	Množenje
Komutativnost (redoslijed nije bitan)	A + B = B + A	A × B = B × A
Asocijativnost (grupiranje nije bitno)	A + (B + C) = (A + B) + C	A × (B × C) = (A × B) × C
Distributivnost		A × (B + C) = (A × B) + (A × C)

Oduzimanje i dijeljenje nisu komutativni.

Booleova algebra s razredima

Booleova genijalna ideja bila je odvojiti algebru od brojeva. U njegovoj algebri slova predstavljaju **razrede** (danas kažemo **skupove**): skupine stvari s nekim svojstvom.

Primjer (knjiga koristi mačke; ovdje koristimo automobile na parkiralištu):

- **C** = crveni automobili, **P** = plavi, **O** = ostale boje
- **E** = električni automobili
- **N** = novi automobili

Operacije znače nešto drugo nego u aritmetici:

Zapis	Ime	Značenje	Primjer
A + B	**unija**	sve što je u A **ili** u B	C + P: svi crveni i svi plavi automobili
A × B	**presjek**	sve što je **i** u A **i** u B	E × N: novi električni automobili
1	**univerzum**	sve o čemu govorimo	svi automobili na parkiralištu
0	**prazan razred**	ništa	
1 − A	**komplement**	sve osim A	1 − C: svi automobili koji nisu crveni

Presjek se često piše i bez znaka: E × N = EN, kao dva pridjeva ("novi električni").

Pravila

Komutativnost, asocijativnost i distributivnost množenja preko zbrajanja vrijede i u Booleovoj algebri. Ali vrijede i neka pravila koja u običnoj algebri **ne vrijede**:

Pravilo	Značenje	U običnoj algebri
C + (E × N) = (C + E) × (C + N)	zbrajanje je distributivno preko množenja	ne vrijedi
1 + A = 1	sve plus bilo što je i dalje sve	ne vrijedi
A × A = A	presjek skupa sa samim sobom je isti skup	ne vrijedi (osim za 0 i 1)
A + A = A	unija skupa sa samim sobom je isti skup	ne vrijedi
A + (1 − A) = 1	nešto ili njegova suprotnost čine sve	
A × (1 − A) = 0	ništa nije istovremeno nešto i njegova suprotnost (**zakon proturječja**)	

Neka pravila izgledaju isto kao u aritmetici: 1 × A = A, 0 × A = 0, 0 + A = A.

Boole je izraz **$X^2 = X$** (dakle X × X = X) smatrao ključnom razlikom svoje algebre od obične.

Silogizam algebrom

Booleova algebra može provjeriti silogizam. Neka je L = ljudi, S = smrtna bića, K = razred koji sadrži samo Sokrata.

1. "Svi ljudi su smrtni" znači: presjek ljudi i smrtnih bića su svi ljudi: **L × S = L**.
2. "Sokrat je čovjek" znači: **K × L = K**.
3. Uvrsti prvu jednadžbu u drugu: K × (L × S) = K.
4. Asocijativnost: (K × L) × S = K.
5. Budući da je K × L = K: **K × S = K**.

Rezultat znači: Sokrat je u razredu smrtnih bića, dakle smrtan je.

Od razreda do istine: I, ILI, NE

Uniju i presjek možemo izgovoriti običnim riječima:

Booleov znak	Riječ
+ (unija)	**ILI** (OR)
× (presjek)	**I** (AND)
1 − (komplement)	**NE** (NOT)

Booleov test

U drugom obliku Booleove algebre slova više ne predstavljaju razrede, nego **brojeve, ali samo 0 i 1**:

- **1** znači "da, istina, ova stvar zadovoljava uvjet";
- **0** znači "ne, laž, ne zadovoljava".

Tada operacije imaju ove tablice:

AND	0	1
0	0	0

AND	0	1
1	0	1

Rezultat je 1 samo ako su **oba** operanda 1. To je isto kao obično množenje.

OR	0	1
0	0	1
1	1	1

Rezultat je 1 ako je **barem jedan** operand 1. Gotovo kao zbrajanje, osim što je 1 + 1 = 1.

NOT	rezultat
0	1
1	0

NOT okreće vrijednost. U Booleovom zapisu to je 1 − A. Druga oznaka je crta iznad slova: Ā.

U nekim tekstovima koriste se i znakovi ∧ za AND i ∨ za OR.

Primjer: kupnja automobila

Kupac kaže: "Želim **novi** automobil koji je **crven ili plav**, **ili** bilo koji **električni** automobil koji **nije crven**."

Booleov izraz:

> (N × (C + P)) + (E × (1 − C))

ili riječima: (N AND (C OR P)) OR (E AND NOT C).

Prodavač dovodi automobile i za svaki u izraz uvrštava 0 i 1:

Automobil	N	C	P	E	Izraz	Rezultat
Novi, plavi, benzinski	1	0	1	0	(1 × (0 + 1)) + (0 × 1) = 1 + 0	**1**: odgovara
Rabljeni, crveni, električni	0	1	0	1	(0 × 1) + (1 × 0) = 0 + 0	**0**: ne odgovara
Rabljeni, sivi, električni	0	0	0	1	(0 × 0) + (1 × 1) = 0 + 1	**1**: odgovara

Zagrade su važne: one određuju što se s čim povezuje, isto kao u običnoj algebri. (Knjiga isti postupak pokazuje na izboru mačke u trgovini kućnim ljubimcima.)

Najvažnije otkriće: prekidači rade logiku

Spoji bateriju i žarulju s **dva prekidača**.

Serijski spoj: AND

Prekidači su jedan iza drugoga. Struja mora proći kroz oba.

Lijevi prekidač	Desni prekidač	Žarulja
otvoren (0)	otvoren (0)	ne svijetli (0)
otvoren (0)	zatvoren (1)	ne svijetli (0)
zatvoren (1)	otvoren (0)	ne svijetli (0)
zatvoren (1)	zatvoren (1)	**svijetli (1)**

Žarulja odgovara na pitanje "jesu li **oba** prekidača zatvorena?". Tablica je **ista kao tablica AND**.

Analogija: dvoja vrata jedna iza drugih. Proći možeš samo ako su otvorena i prva i druga.

Paralelni spoj: OR

Prekidači su jedan pored drugoga. Struja može proći kroz bilo koji.

Lijevi prekidač	Desni prekidač	Žarulja
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

Žarulja odgovara na pitanje "je li **barem jedan** prekidač zatvoren?". Tablica je **ista kao tablica OR**.

Analogija: dva ulaza u zgradu jedan pored drugog. Dovoljno je da je jedan otvoren.

U oba slučaja nije bitno koji je prekidač lijevi, a koji desni, jer su AND i OR komutativni.

Cijeli izraz kao krug

Svaki × u izrazu postaje **serijski** spoj, a svaki + **paralelni** spoj. NOT znači prekidač koji je zatvoren kad svojstvo **ne vrijedi** (npr. prekidač "nije crven").

Izraz za automobil postaje mreža od sedam prekidača:



Za svaki automobil zatvoriš prekidače čija svojstva on ima. Ako postoji zatvoren put, žarulja svijetli: automobil odgovara uvjetima.

Zašto je to važno

Boole nikad nije vidio svoju algebru izvedenu prekidačima. Nitko u 19. stoljeću nije povezao Booleove I i ILI s prekidačima u seriji i paraleli, iako su i telegraf (1844.) i Booleova knjiga (1854.) već postojali. Ni Charles Babbage, koji je projektirao mehaničke računske strojeve, nije shvatio da bi stroj za računanje bilo bolje graditi od **telegrafskih releja** nego od zupčanika.

Upravo to je sljedeći korak: releji su prekidači kojima upravlja struja, pa izlaz jedne logičke operacije može upravljati sljedećom.

Ključne definicije

Pojam	Značenje
Booleova algebra	algebra u kojoj varijable predstavljaju razrede ili vrijednosti istine (0 i 1)
Unija / presjek	sve što je u A ili B / sve što je i u A i u B
AND (I)	1 samo ako su oba operanda 1
OR (ILI)	1 ako je barem jedan operand 1
NOT (NE)	okreće 0 u 1 i 1 u 0
Booleov test	izraz u koji se uvrštavaju 0 i 1 da se provjeri zadovoljava li nešto uvjete
Serijski spoj prekidača	prekidači u nizu; ostvaruje AND

Pojam	Značenje
Paralelni spoj prekidača	prekidači usporedno; ostvaruje OR

Ključno za zapamtiti

1. Booleova algebra koristi + i ×, ali oni znače ILI i I, a ne zbrajanje i množenje.
2. 1 znači "sve" ili "istina", 0 znači "ništa" ili "laž".
3. Neka pravila vrijede samo u Booleovoj algebri: A × A = A, A + A = A, 1 + A = 1.
4. AND je 1 samo kad su oba ulaza 1; OR je 1 kad je barem jedan ulaz 1; NOT okreće vrijednost.
5. Uvjeti iz govora mogu se zapisati Booleovim izrazom i provjeriti uvrštavanjem 0 i 1.
6. Prekidači u seriji su AND; prekidači u paraleli su OR.
7. Svaki Booleov izraz može se izvesti mrežom prekidača.
8. Kad prekidače zamijene releji, logika se može nizati: to je put do računala.

Najčešće zabune

- **"U Booleovoj algebri 1 + 1 = 2."** Ne: 1 OR 1 = 1.
- **"Serijski spoj je OR jer struja ide redom kroz oba."** Ne: serijski je AND, jer moraju biti zatvorena oba.
- **"Zagrade nisu bitne."** Jesu: (A AND B) OR C nije isto što i A AND (B OR C).
- **"ILI u logici znači 'jedno ili drugo, ali ne oba'."** Booleov OR uključuje i slučaj kad su oba 1.

Brza provjera razumijevanja

1. Ispiši tablice AND, OR i NOT napamet.
2. Izračunaj (1 + 0) × (1 − 1) + 1 × 1.
3. Zapiši Booleovim izrazom: "stan s balkonom koji je u centru ili ima garažu, ili bilo koji stan koji nije u prizemlju".
4. Nacrtaj prekidače za izraz A × (B + C).
5. Zašto je zbrajanje distributivno preko množenja u Booleovoj algebri, a u običnoj nije? Provjeri na primjeru s 0 i 1.

11. Logička vrata

Što ćeš naučiti u ovom poglavlju

- Kako je Claude Shannon povezao Booleovu algebru i električne sklopove.
- Kako se od **releja** grade **logička vrata**: AND, OR, NAND, NOR, invertor i međuspremnik (buffer).
- Simbole i tablice svih vrata.
- Što je **dekoder** (2 na 4 linije) i pravila spajanja vrata.
- **De Morganove zakone**.

Shannon: logika kao sklop

Sklop prekidača iz prošlog poglavlja zove se i **mreža** (danas ta riječ uglavnom znači povezana računala). U 19. stoljeću nitko nije uočio da se Booleovi izrazi mogu izravno izvesti prekidačima. To je sustavno pokazao **Claude Shannon** 1938., u magistarskom radu o analizi relejnih i prekidačkih sklopova. (Deset godina kasnije Shannon je objavio i prvi rad u kojem se pojavljuje riječ *bit*.)

Ljudi su i prije znali da serijski prekidači traže da oba budu zatvorena, a paralelni da je dovoljan jedan. Novost je bila da inženjeri mogu koristiti **cijelu Booleovu algebru** za projektiranje sklopova. Posebno: **ako pojednostaviš Booleov izraz, pojednostavio si i sklop**.

Primjer pojednostavljenja

Izraz iz poglavlja 10:

> (N × (C + P)) + (E × (1 − C))

Pretpostavimo da kupac želi da i druga skupina bude nova: (N × (C + P)) + (N × E × (1 − C)). Varijabla N se ponavlja. Distributivnim zakonom (izlučivanjem) dobivamo:

> N × ((C + P) + (E × (1 − C)))

Izraz izgleda zamršenije, ali ima jedan N manje, dakle **jedan prekidač manje**.

Najmanje bitova

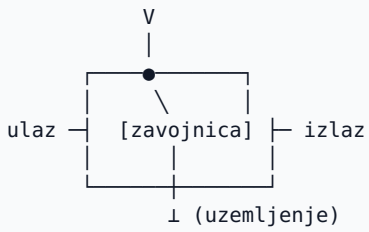
Još bolje: svaki prekidač je jedan bit. Ako automobil opisuju svojstva "nov ili ne" i "električan ili ne" i boja od četiri moguće (crvena, plava, siva, bijela), dovoljno je **4 bita**: 1 za "nov", 1 za "električan" i **2 za boju**, jer dva bita daju četiri kombinacije.

Prekidači su **ulazni uređaj** (informacija koja upravlja sklopom), a žarulja **izlazni uređaj**.

Relej kao prekidač kojim upravlja struja

Relej iz poglavlja 6 u telegrafiji je pojačavao slab signal. Ovdje nas zanima druga stvar: **relej je prekidač kojim upravlja električna struja, a ne prst**. Zato izlaz jednog releja može upravljati drugim relejem.

Pojednostavljeni crtež releja:



- **Ulaz**: kad na njemu ima napona (1), elektromagnet privuče kontakt; kaže se da je relej **okinut**.
- **Izlaz**: kad je relej okinut, izlaz je spojen na V, dakle ima napon (1).

Svi V u crtežima mogu se spojiti na isti izvor, a sva uzemljenja međusobno. Uzemljenje ovdje ne mora biti stvarni spoj sa Zemljom; znači samo **zajednički vodič**.

Dvopoložajni relej (*double-throw*) ima dva kontakta. Gornji je spojen kad relej **nije** okinut, a donji kad **jest**. Tako relej ima dva izlaza suprotnih vrijednosti. To se koristi za invertor, NOR i NAND.

Osnovna vrata

Logička vrata su kombinacije releja koje izvode jednu jednostavnu logičku operaciju. U ostatku knjige se releji više ne crtaju: crtaju se samo simboli vrata.

Odsutnost napona je **0**, prisutnost napona je **1**.

AND (I)

Dva releja **u seriji**: izlaz prvog daje napon kontaktu drugog. Izlaz je 1 samo ako su **oba** releja okinuta.



A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

AND može imati i **više ulaza**: tri releja u seriji daju 3-ulazni AND, koji je 1 samo kad su sva tri ulaza 1.

OR (ILI)

Dva releja **u paraleli**: izlazi oba releja spojeni su zajedno. Dovoljan je jedan okinut relej.

Simbol je sličan AND-u, ali je ulazna strana **zakrivljena** (pomoć za pamćenje: zaobljena kao "O" u OR).

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

OR s više ulaza je 1 ako je **bilo koji** ulaz 1, a 0 samo ako su **svi** 0.

Invertor (NOT)

Jedan relej spojen preko **gornjeg** kontakta: izlaz ima napon kad relej **nije** okinut.

Simbol: trokut s malim **kružićem** na izlazu. Kružić u logičkim simbolima uvijek znači **invertiranje**.

Ulaz	Izlaz
0	1
1	0

Invertor strogo uzevši nije "vrata" (vrata imaju dva ili više ulaza), ali je jednako koristan.

NOR (NE-ILI)

Dva releja u seriji, ali preko **gornjih** kontakata. Žarulja svijetli samo kad **nijedan** relej nije okinut. To je suprotno od OR.

Simbol: OR s kružićem na izlazu. NOR je isto što i OR iza kojeg slijedi invertor.

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

NAND (NE-I)

Dva releja u paraleli, preko **gornjih** kontakata. Žarulja se gasi samo kad su **oba** releja okinuta. To je suprotno od AND.

Simbol: AND s kružićem na izlazu.

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Međuspremnik (buffer)

Običan relej preko donjeg kontakta. Izlaz je **jednak** ulazu. Simbol je trokut bez kružića.

Čemu služi nešto što "ništa ne radi"?

1. **Pojačanje** slabog signala (kao u telegrafiji).
2. **Kašnjenje**: relej treba djelić sekunde da se okine, pa međuspremnik malo zakasni signal.

Pregled svih vrata

Vrata	Izlaz je 1 kad...	Od releja
AND	su svi ulazi 1	serija, donji kontakti
OR	je barem jedan ulaz 1	paralela, donji kontakti
NAND	nisu svi ulazi 1	paralela, gornji kontakti
NOR	su svi ulazi 0	serija, gornji kontakti
NOT (invertor)	je ulaz 0	jedan relej, gornji kontakt

Vrata	Izlaz je 1 kad...	Od releja
Buffer	je ulaz 1	jedan relej, donji kontakt

Analogija: zamisli vrata AND kao sef s dvije brave: otvara se samo kad su oba ključa okrenuta. Vrata OR su kuća s dva ulaza: dovoljno je da je jedan otvoren. NAND i NOR su njihova "suprotna" verzija.

Spajanje vrata

Signali iz prekidača

Iz jednog prekidača i invertora dobivaš **dva** suprotna signala. Ako je prekidač zatvoren za "nov", onda invertor na njegovom izlazu daje signal "nije nov". Kad je jedan 1, drugi je 0.

Dekoder 2 na 4 linije

Dva prekidača (dva bita) mogu kodirati četiri boje automobila. Treba sklop koji od ta dva bita napravi **četiri zasebna signala**, od kojih je u svakom trenutku **točno jedan** 1.

Prekidač 1	Prekidač 2	Crvena	Plava	Siva	Bijela
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Svaki izlaz je jedna vrata AND sa po dva ulaza, uz invertore gdje treba:

- Crvena = (NOT s1) AND (NOT s2)
- Plava = (NOT s1) AND s2
- Siva = s1 AND (NOT s2)
- Bijela = s1 AND s2

Ukupno **4 vrata AND i 2 invertora**. Takav sklop zove se **dekoder 2 na 4 linije** (*2-Line-to-4-Line Decoder*). Na isti način grade se dekoderi 3 na 8, 4 na 16 i tako dalje. Dekoder će se kasnije koristiti za adresiranje memorije (poglavlje 16).

Pravila spajanja

1. Izlaz jednih vrata (ili invertora) može biti ulaz **jednih ili više** drugih vrata.
2. Izlazi dvaju ili više vrata **nikad se ne spajaju izravno** jedan s drugim.

(Izlazi releja u OR vratima spojeni su unutar samih vrata; to je dio njihove građe.)

Cijeli izraz od vrata

Svaki **×** u Booleovom izrazu postaje vrata **AND**, svaki **+** vrata **OR**, a svaki **1 −** invertor. Ulazi su signali iz prekidača i dekodera. Izlaz posljednjih vrata pali žarulju.

To je puno releja: dva u svakim vratima AND i OR, jedan u svakom invertoru. Ali to je cijena, i u nastavku knjige ih treba još puno više.

Invertirani ulazi i De Morganovi zakoni

Kružić može stajati i na **ulazu** vrata: tada se taj ulaz invertira prije ulaska u vrata.

Dvije važne jednakosti:

Sklop	Isto što i	Zašto
AND s oba ulaza invertirana	**NOR**	izlaz je 1 samo kad su oba ulaza 0
OR s oba ulaza invertirana	**NAND**	izlaz je 0 samo kad su oba ulaza 1

To su **De Morganovi zakoni** (Augustus De Morgan, Booleov suvremenik):

> NOT A AND NOT B = NOT (A OR B)
> NOT A OR NOT B = NOT (A AND B)

U Booleovom zapisu s crtom iznad: $\bar{A} \times \bar{B} = (A + B)$ s crtom iznad cijelog izraza; $\bar{A} + \bar{B} = (A \times B)$ s crtom iznad.

Primjer iz života: "Nisam gladan i nisam žedan" znači isto što i "nije istina da sam gladan ili žedan".

De Morganovi zakoni važan su alat za pojednostavljenje izraza, a time i sklopova. Knjiga se ipak ne bavi pretjeranim pojednostavljivanjem; važnije je da sklop radi.

Ključne definicije

Pojam	Značenje
Logička vrata	sklop (od releja, kasnije tranzistora) koji izvodi jednu logičku operaciju
Okinut relej	relej čiji je elektromagnet privukao kontakt
Dvopoložajni relej	relej s dva kontakta suprotnih stanja
Invertor	sklop koji okreće 0 u 1 i 1 u 0
Buffer (međuspremnik)	izlaz jednak ulazu; služi za pojačanje i kašnjenje
Dekoder 2 na 4	od 2 ulazna bita daje 4 izlaza, od kojih je točno jedan 1
De Morganovi zakoni	NOT A AND NOT B = NOT(A OR B); NOT A OR NOT B = NOT(A AND B)

Ključno za zapamtiti

1. Shannon je pokazao da se Booleova algebra može koristiti za projektiranje i pojednostavljenje sklopova.
2. Pojednostavljen izraz znači manje prekidača ili vrata.
3. Relej je prekidač kojim upravlja struja; njegov izlaz može upravljati drugim relejem.
4. AND = serija, OR = paralela; s gornjim kontaktima dobivaju se NOR i NAND.
5. Kružić u simbolu znači invertiranje.
6. Invertor okreće vrijednost; buffer je ne mijenja, ali pojačava i kasni.
7. Dekoder pretvara n bitova u 2^n linija, od kojih je točno jedna 1.
8. Izlaz može ići na više ulaza, ali se izlazi nikad ne spajaju međusobno.
9. De Morgan: AND s invertiranim ulazima je NOR; OR s invertiranim ulazima je NAND.

Najčešće zabune

- **"NAND je AND s invertiranim ulazima."** Ne: NAND je AND s invertiranim **izlazom**. AND s invertiranim ulazima je NOR.
- **"Buffer je beskoristan."** Pojačava signal i dodaje kašnjenje; oboje je kasnije važno.
- **"Uzemljenje u ovim crtežima mora biti spojeno na Zemlju."** Ovdje znači samo zajednički vodič.

Brza provjera razumijevanja

1. Napiši tablice za AND, OR, NAND i NOR napamet.
2. Kako se od dvopoložajnog releja dobije invertor?
3. Koliko vrata AND i invertora treba dekoder 2 na 4? Što je na izlazima kad su oba ulaza 1?
4. Dokaži tablicom da je OR s invertiranim ulazima isto što i NAND.
5. Zašto se izlazi dvaju vrata ne smiju izravno spojiti?

12. Binarni stroj za zbrajanje

Što ćeš naučiti u ovom poglavlju

- Kako se binarno zbrajanje rastavlja na **bit zbroja** i **bit prijenosa**.
- Kako se od vrata OR, NAND i AND gradi vrata **XOR** (isključivo ILI).
- Što su **poluzbrajalo** (half adder) i **potpuno zbrajalo** (full adder).
- Kako se od osam potpunih zbrajala dobiva **8-bitno zbrajalo**, i kako se zbrajala nižu za veće brojeve.
- Što je **valoviti prijenos** (ripple carry) i zašto je spor.

Zašto baš zbrajanje

Zbrajanje je najosnovnija aritmetička operacija. Kad imamo stroj koji zbraja, na zbrajanju se može izgraditi oduzimanje, množenje, dijeljenje i sve ostalo što računala rade. Knjiga to ističe gotovo provokativno: zbrajanje je u biti **jedino** što računala rade.

Stroj iz ovog poglavlja gradi se samo od prekidača, žarulja, žica, baterije i releja složenih u logička vrata. Sve te stvari postojale su prije više od 120 godina. Stroj radi samo s binarnim brojevima: ulaz su **dva reda po osam prekidača** (gore = 1, dolje = 0), a izlaz je **devet žarulja** (svijetli = 1). Devet zato što zbroj dvaju 8-bitnih brojeva može imati 9 bitova: najveći je 255 + 255 = 510 = $1\,1111\,1110_2$.

Rastavljanje zbrajanja dvaju bitova

Tablica binarnog zbrajanja s rezultatom zapisanim kao dva bita:

A	B	Rezultat
0	0	00
0	1	01
1	0	01
1	1	10

Rezultat ima dva dijela:

- **bit zbroja** (desni bit): ono što pišeš u taj stupac;
- **bit prijenosa** (lijevi bit): ono što "pamtiš" i prenosiš u sljedeći stupac.

A	B	Zbroj	Prijenos
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Prijenos je AND

Usporedi stupac "Prijenos" s tablicom AND: **identični su**. Prijenos je 1 samo kad su oba bita 1. Dakle, vrata AND računaju prijenos.

Zbroj je XOR

Za bit zbroja treba vrata koja daju 1 kad je **točno jedan** ulaz 1. Nijedna od dosadašnjih vrata to ne radi, ali dvoje je blizu:

- **OR** je dobar, osim u slučaju 1, 1 (daje 1, a treba 0);
- **NAND** je dobar, osim u slučaju 0, 0 (daje 1, a treba 0).

Traženi rezultat je 1 samo kad su **i** OR **i** NAND jednaki 1. Zato oba izlaza vodimo u vrata AND:

A	B	OR	NAND	AND(OR, NAND)
0	0	0	1	**0**
0	1	1	1	**1**
1	0	1	1	**1**
1	1	1	0	**0**

To su vrata **XOR** (*exclusive OR*, isključivo ILI): izlaz je 1 ako je A **ili** B jednako 1, **ali ne oba**. Simbol je kao OR, s dodatnom zakrivljenom crtom na ulaznoj strani.

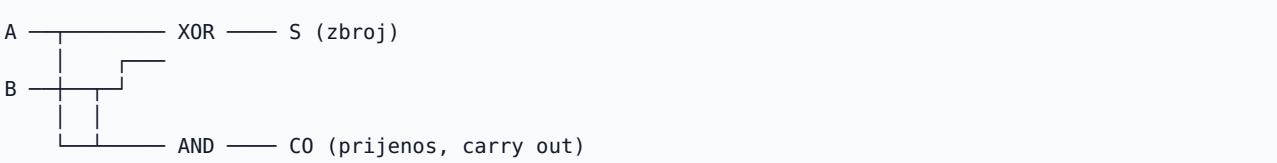
XOR	0	1
0	0	1
1	1	0

XOR je posljednja vrata koja knjiga detaljno opisuje. Postoje i vrata **koincidencije** (ekvivalencije), koja su suprotna XOR-u: izlaz je 1 kad su ulazi **jednaki**. Simbol im je XOR s kružićem na izlazu.

Analogija za XOR: svjetlo na stubištu s dva prekidača (dolje i gore). Promjena bilo kojeg prekidača mijenja stanje svjetla; ako su oba u istom položaju, svjetlo je ugašeno.

Poluzbrajalo (half adder)

Vrata XOR i vrata AND spojena na iste ulaze A i B zajedno zbrajaju dva bita:



Taj sklop crta se kao kutija s ulazima A i B i izlazima S i CO, i zove se **poluzbrajalo**.

"Polu" zato što radi samo pola posla: zbraja dva bita, ali **ne zna dodati prijenos iz prethodnog stupca**. Primjer:

```
  1 1 1
+ 0 1 1
-------
1 0 1 0
```

U krajnjem desnom stupcu zbrajaju se samo dva bita (1 + 1). Ali u svakom sljedećem stupcu zbrajaju se **tri** bita: dva iz brojeva i prijenos iz prethodnog stupca.

Potpuno zbrajalo (full adder)

Za zbrajanje **tri** bita (A, B i ulazni prijenos CI, *carry in*) trebaju **dva poluzbrajala i jedna vrata OR**:

1. Prvo poluzbrajalo zbraja A i B.

2. Drugo poluzbrajalo zbraja dobiveni zbroj i ulazni prijenos. Njegov izlaz S je konačni bit zbroja.
3. Prijenosi iz oba poluzbrajala idu u vrata OR, čiji je izlaz konačni prijenos CO.

Zašto je dovoljan OR, a ne treće poluzbrajalo? Prijenosi dvaju poluzbrajala **nikad nisu oba 1** istovremeno. Kad ulazi nikad nisu oba 1, OR i XOR daju isti rezultat.

Tablica potpunog zbrajala:

A	B	CI	S	CO
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Jednostavno pravilo: S je 1 kad je **neparan broj** ulaza 1; CO je 1 kad su **barem dva** ulaza 1.

Koliko releja

Dio	Releja
AND, OR, NAND	po 2
XOR (OR + NAND + AND)	6
Poluzbrajalo (XOR + AND)	8
Potpuno zbrajalo (2 poluzbrajala + OR)	18
8-bitno zbrajalo (8 potpunih)	**144**

8-bitno zbrajalo

Osam potpunih zbrajala spaja se u lanac:

1. **Prvi** (krajnji desni) par prekidača ide u potpuno zbrajalo čiji je ulazni prijenos CI spojen na **uzemljenje**, dakle 0. U prvom stupcu nema prijenosa.
2. Izlaz S svakog zbrajala pali svoju žarulju.
3. Izlazni prijenos CO svakog zbrajala spaja se na ulazni prijenos CI **sljedećeg** zbrajala lijevo.
4. CO **posljednjeg** zbrajala pali devetu žarulju.

Oznake bitova

Bitovi višebitnog broja označavaju se indeksima od 0 nadalje:

- A_0, B_0, S_0 su **najmanje značajni** (krajnji desni) bitovi;
- A_7, B_7, S_7 su **najznačajniji** (krajnji lijevi).

Indeks je ujedno eksponent potencije broja 2 za to mjesto:

Bit	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
Vrijednost	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Primjer: 0011 0110	0	0	1	1	0	1	1	0

$0011\ 0110_2 = 32 + 16 + 4 + 2 = 54$.

Cijelo 8-bitno zbrajalo crta se kao jedna kutija s ulazima $A_7 \ldots A_0$ i $B_7 \ldots B_0$, ulaznim prijenosom CI, izlazima $S_7 \ldots S_0$ i izlaznim prijenosom CO. Na crtežima se skupina od osam žica često crta kao jedna debela strelica s brojem 8.

Primjer zbrajanja

0101 1100 (92) + 0110 1011 (107):

```
  0 1 0 1 1 1 0 0
+ 0 1 1 0 1 0 1 1
-----------------
0 1 1 0 0 0 1 1 1
```

Rezultat $0\ 1100\ 0111_2 = 128 + 64 + 4 + 2 + 1 =$ **199**. Provjera: 92 + 107 = 199. Deveta žarulja (prijenos) ostaje ugašena jer zbroj stane u 8 bitova.

Ulančavanje za veće brojeve

Dva 8-bitna zbrajala daju **16-bitno**: CO desnog (koje zbraja nižih 8 bitova) spaja se na CI lijevog (koje zbraja viših 8 bitova). Na isti način može se graditi zbrajalo bilo koje širine. Takvo nizanje zove se **kaskadiranje**.

Zbrajaju li računala stvarno tako

U osnovi da, ali s dvije razlike:

1. **Brzina.** U opisanom zbrajalu svaki stupac mora čekati prijenos iz prethodnog. Ukupno vrijeme je broj bitova pomnožen s vremenom jednog potpunog zbrajala. To se zove **valoviti prijenos** (*ripple carry*), jer prijenos "putuje" kao val. Brža zbrajala imaju dodatne sklopove za **predviđanje prijenosa** (*look-ahead carry*).
2. **Materijal.** Računala više ne koriste releje. Prva digitalna računala (od 1930-ih) koristila su releje, zatim elektronske cijevi, a današnja **tranzistore**. Tranzistori rade isto što i releji, ali su puno brži, manji, tiši, troše manje energije i jeftiniji su. 8-bitno zbrajalo i dalje treba oko 144 tranzistora (više uz predviđanje prijenosa), ali stane u mikroskopski prostor.

Ključne definicije

Pojam	Značenje
Bit zbroja	bit koji se upisuje u stupac; za dva bita to je XOR
Bit prijenosa (carry)	bit koji se prenosi u sljedeći stupac; za dva bita to je AND
XOR	izlaz 1 kad je točno jedan od dva ulaza 1
Poluzbrajalo	XOR + AND; zbraja dva bita, daje zbroj i prijenos
Potpuno zbrajalo	dva poluzbrajala + OR; zbraja tri bita (A, B, ulazni prijenos)
Valoviti prijenos	prijenos putuje stupac po stupac; spor za duge brojeve
Kaskadiranje	spajanje CO jednog zbrajala na CI sljedećeg

Ključno za zapamtiti

1. Zbroj dvaju bitova ima bit zbroja (XOR) i bit prijenosa (AND).
2. XOR = (A OR B) AND (A NAND B).
3. Poluzbrajalo ne zna dodati ulazni prijenos; potpuno zbrajalo zna.
4. Potpuno zbrajalo = 2 poluzbrajala + OR; prijenosi poluzbrajala nikad nisu oba 1.
5. 8-bitno zbrajalo je lanac 8 potpunih zbrajala; prvi CI je 0, posljednji CO je deveti bit.

6. Bit s indeksom 0 je najmanje značajan; indeks je eksponent potencije broja 2.
7. Zbrajala se kaskadiraju za brojeve bilo koje širine.
8. Valoviti prijenos je spor; predviđanje prijenosa ga ubrzava.
9. Releje su zamijenili tranzistori; logika je ista.

Najčešće zabune

- **"Poluzbrajalo je dovoljno za višebitne brojeve."** Nije: od drugog stupca nadalje treba zbrojiti tri bita.
- **"XOR i OR su isto."** Razlikuju se samo kad su oba ulaza 1: OR daje 1, XOR daje 0.
- **"Zbroj dvaju 8-bitnih brojeva uvijek stane u 8 bitova."** Može trebati 9 bitova; deveti je izlazni prijenos.

Brza provjera razumijevanja

1. Izgradi XOR od OR, NAND i AND i provjeri tablicom.
2. Koliki su S i CO potpunog zbrajala za A = 1, B = 0, CI = 1?
3. Zbroji binarno 1011 0011 + 0110 1110. Svijetli li deveta žarulja?
4. Zašto je u potpunom zbrajalu dovoljan OR umjesto trećeg poluzbrajala?
5. Što usporava zbrajalo s valovitim prijenosom i kako se to rješava?

13. A oduzimanje? Komplementi i negativni brojevi

Što ćeš naučiti u ovom poglavlju

- Zašto je oduzimanje "s posuđivanjem" nezgodno za sklopove.
- Kako se oduzima **bez posuđivanja**, pomoću **komplementa devetke** i **komplementa jedinice**.
- Kako se zbrajalo iz poglavlja 12 nadogradi u **zbrajalo-oduzimalo** s nekoliko vrata XOR.
- Što su **preljev** (overflow) i **podljev** (underflow).
- Kako se bez znaka minus zapisuju negativni brojevi: **komplement desetke** i **komplement dvojke**.
- Razliku između brojeva **s predznakom** i **bez predznaka**.

Problem: posuđivanje

Zbrajanje ide uredno zdesna nalijevo, a prijenos uvijek ide u sljedeći stupac. Oduzimanje koristi **posuđivanje**: ako je znamenka gore manja od znamenke dolje, posudiš 1 iz stupca lijevo, a ako je i tamo 0, posuđuješ još dalje. To je nezgrapan mehanizam "naprijed-natrag" i teško ga je izvesti logičkim vratima.

Zato se koristi trik kojim se oduzimanje svodi na **zbrajanje**, bez ijednog posuđivanja.

Nazivi: **umanjenik** − **umanjitelj** = **razlika**.

Oduzimanje pomoću komplementa devetke

Komplement devetke broja je ono što dobiješ kad ga oduzmeš od broja sastavljenog samo od devetki (iste duljine). Za troznamenkasti broj oduzimaš od 999.

Važno: računanje komplementa devetke **nikad ne traži posuđivanje**, jer je svaka znamenka 9 veća ili jednaka bilo kojoj znamenki.

Primjer: 532 − 289

Korak	Račun	Rezultat
1. Komplement devetke umanjitelja	999 − 289	710
2. Dodaj ga umanjeniku	532 + 710	1242
3. Dodaj 1	1242 + 1	1243
4. Oduzmi 1000 (makni vodeću jedinicu)	1243 − 1000	**243**

Provjera: 532 − 289 = 243.

Zašto radi: dodali smo i oduzeli 1000, što ne mijenja rezultat:

> 532 − 289 = 532 + (999 − 289) + 1 − 1000

Jedno oduzimanje s posuđivanjem zamijenili smo zbrajanjima i oduzimanjima koja ne trebaju posuđivanje.

Kad je umanjitelj veći

Primjer: 289 − 532

1. 999 − 532 = 467
2. 289 + 467 = 756

3. Sad bi trebalo dodati 1 i oduzeti 1000, ali 757 − 1000 traži posuđivanje. Umjesto toga oduzmi 999 (koje smo dodali): 756 − 999. Rezultat je negativan, pa okreneš: 999 − 756 = 243, i zapamtiš da je rezultat **−243**.

Isto u binarnom: komplement jedinice

U binarnom sustavu umjesto niza devetki koristi se **niz jedinica**. Komplement prema 1111 1111 zove se **komplement jedinice** (*ones' complement*).

Najljepše: za komplement jedinice ne treba oduzimati. Svaka 0 postaje 1, a svaka 1 postaje 0. Komplement jedinice je **invertiranje svih bitova** (zove se i *negacija* ili *inverz*). To je upravo posao invertora iz poglavlja 11.

Primjer: 1100 1000 (200) − 0101 1010 (90)

Korak	Račun	Rezultat
1. Komplement jedinice umanjitelja	invertiraj 0101 1010	1010 0101
2. Dodaj umanjeniku	1100 1000 + 1010 0101	1 0110 1101
3. Dodaj 1	1 0110 1101 + 1	1 0110 1110
4. Oduzmi 1 0000 0000 (256)	makni deveti bit	**0110 1110**

$0110\ 1110_2$ = 64 + 32 + 8 + 4 + 2 = **110**. Provjera: 200 − 90 = 110.

Ako je umanjitelj veći, kao i kod devetki, rezultat se na kraju još jednom invertira i zna se da je negativan.

Zbrajalo-oduzimalo

Iz ovoga slijedi kako 8-bitno zbrajalo naučiti oduzimati. Na upravljačku ploču dodaje se jedan prekidač: **SUB** (0 = zbrajanje, 1 = oduzimanje).

Invertiranje po potrebi: XOR

Osam invertora bi **uvijek** invertiralo ulaz B. Treba invertirati **samo pri oduzimanju**. Za to služi XOR:

Signal Invert	Ulaz	XOR izlaz
0	x	x (nepromijenjen)
1	x	NOT x (invertiran)

Osam vrata XOR, svaka s jednim bitom B i zajedničkim signalom Invert, čine kutiju **"komplement jedinice"**. Kad je Invert = 0, prolazi B; kad je Invert = 1, prolazi invertirani B.

Primjer: ulaz 0110 0001 uz Invert = 0 daje 0110 0001; uz Invert = 1 daje 1001 1110.

Spoj

Signal **SUB** ide na tri mjesta:

1. na ulaz **Invert** kutije komplementa jedinice (B se invertira pri oduzimanju);
2. na **ulazni prijenos CI** zbrajala (pri oduzimanju dodaje se 1, to je korak 3 gore);
3. u vrata **XOR** zajedno s izlaznim prijenosom CO; izlaz tih vrata pali lampicu **preljev/podljev**.

Pri zbrajanju (SUB = 0) sklop radi kao prije.

Preljev i podljev

- **Preljev** (overflow): zbroj je veći od 255, dakle ne stane u 8 bitova. Pri zbrajanju to pokazuje CO = 1.
- **Podljev** (underflow): rezultat oduzimanja je negativan (umanjitelj je bio veći). Pri ispravnom oduzimanju CO je normalno 1 (to je onaj 1 0000 0000 koji se "oduzima" u koraku 4). Ako je CO = 0, rezultat je negativan.

Zato lampica = SUB XOR CO:

SUB	CO	Lampica	Značenje
0	0	0	zbroj u redu
0	1	1	preljev: zbroj > 255
1	1	0	razlika u redu
1	0	1	podljev: razlika negativna

Ovaj stroj ne prikazuje negativne brojeve; samo javlja da je rezultat izvan raspona.

Negativni brojevi bez znaka minus

Binarni brojevi trebaju **sve** izraziti nulama i jedinicama, pa i predznak. Jedna ideja je dodatni bit za predznak (1 = negativan), ali postoji bolje rješenje koje se uz to može izravno zbrajati. Uvjet je da se **unaprijed odluči koliko znamenki** brojevi imaju.

Komplement desetke (desetično)

Pretpostavimo da su svi brojevi koji nas zanimaju između −500 i 499. To je točno 1000 brojeva, a toliko ima i troznamenkastih kombinacija 000 – 999. Dogovor:

Zapis	Znači
000 – 499	0 do 499
500	−500
501	−499
…	…
998	−2
999	−1

Svaki zapis koji počinje s 5, 6, 7, 8 ili 9 je **negativan broj**. Brojevi čine **krug**: iza 999 (−1) dolazi 000 (0), jer 999 + 1 = 1000, a s tri znamenke to je 000.

To je **komplement desetke**: komplement devetke plus 1. Za −x računaš 999 − x + 1.

Primjer: −317 = 999 − 317 + 1 = **683**.

Sad je **oduzimanje samo zbrajanje** negativnog broja. Stanje računa je 420. Plaćaš 150, dakle zbrajaš −150 = 850: 420 + 850 = 1270; zanemari znamenku koja ne stane: **270**. Zatim plaćaš 400: −400 = 600; 270 + 600 = **870**, a to je −130. Stvarno: 270 − 400 = −130.

Komplement dvojke (binarno)

Isti sustav u binarnom zove se **komplement dvojke**. Za 8-bitne brojeve:

Binarno	Desetično
1000 0000	−128
1000 0001	−127
…	…
1111 1110	−2
1111 1111	−1
0000 0000	0
0000 0001	1

Binarno	Desetično
…	…
0111 1111	127

- Raspon je **−128 do 127**.
- Krajnji lijevi bit zove se **bit predznaka**: 1 za negativne, 0 za nenegativne brojeve.

Kako se dobije −x: invertiraj sve bitove i dodaj 1 (komplement jedinice plus 1). Isti postupak radi i unatrag.

Primjer: 45 = 0010 1101. Invertiraj: 1101 0010. Dodaj 1: **1101 0011** = −45. Provjera unatrag: invertiraj 1101 0011 → 0010 1100, dodaj 1 → 0010 1101 = 45.

Zbrajanje pozitivnih i negativnih

U komplementu dvojke pozitivni i negativni brojevi zbrajaju se **običnim zbrajalom**, bez posebnih pravila. Bit koji ispadne preko 8 bitova se zanemaruje.

Primjer: −45 + 100:

```
  1 1 0 1 0 0 1 1    (−45)
+ 0 1 1 0 0 1 0 0    (100)
-----------------
1 0 0 1 1 0 1 1 1
```

Zanemari deveti bit: 0011 0111 = **55**. Točno.

Preljev u brojevima s predznakom

Rezultat mora ostati između −128 i 127. Ako ne ostane, rezultat je kriv:

- 100 + 100 = 200, ali 0110 0100 + 0110 0100 = 1100 1000, što se čita kao **−56**.
- (−100) + (−100) = −200, ali 1001 1100 + 1001 1100 = 1 0011 1000; bez devetog bita 0011 1000 = **+56**.

Pravilo: zbroj je neispravan ako oba pribrojnika imaju **isti** bit predznaka, a rezultat ima **drukčiji**. (Zbrajanje brojeva suprotnih predznaka nikad ne može prelijevati.)

S predznakom ili bez predznaka

Isti 8-bitni uzorak može značiti dva različita broja:

Vrsta	Raspon	1011 0110 znači
bez predznaka (*unsigned*)	0 do 255	182
s predznakom (*signed*, komplement dvojke)	−128 do 127	−74

Bitovi sami ništa ne govore o tome kako ih čitati. To mora znati onaj tko ih koristi. Ovo je opet ideja iz poglavlja 9: značenje bitova je dogovor.

Ključne definicije

Pojam	Značenje
Komplement devetke	broj oduzet od niza devetki iste duljine
Komplement jedinice	svi bitovi invertirani
Komplement desetke / dvojke	komplement devetke / jedinice plus 1; način zapisa negativnih brojeva
Bit predznaka	najznačajniji bit u komplementu dvojke; 1 znači negativan

Pojam	Značenje
Preljev (overflow)	rezultat prevelik za zadani broj bitova
Podljev (underflow)	rezultat premalen (negativan kad negativni nisu dopušteni)
S predznakom / bez predznaka	ista bitova tumače se kao −128..127 / 0..255

Ključno za zapamtiti

1. Oduzimanje se svodi na zbrajanje: A − B = A + (komplement B) + 1 − 2^n.
2. Komplement jedinice je invertiranje bitova; ne treba posuđivanje.
3. Zbrajalo postaje zbrajalo-oduzimalo dodavanjem 8 vrata XOR, a signal SUB ide i na CI.
4. Lampica preljev/podljev = SUB XOR CO.
5. Komplement dvojke: −x = invertiraj bitove i dodaj 1.
6. 8-bitni brojevi s predznakom idu od −128 do 127; bit predznaka je krajnji lijevi.
7. U komplementu dvojke pozitivni i negativni brojevi zbrajaju se običnim zbrajalom.
8. Preljev s predznakom: pribrojnici istog predznaka, rezultat drugog.
9. Isti bitovi mogu biti broj s predznakom ili bez njega; to se mora znati unaprijed.

Najčešće zabune

- **"Komplement dvojke je samo invertiranje."** Invertiranje je komplement jedinice; za komplement dvojke treba dodati 1.
- **"1111 1111 je −127 jer je bit predznaka 1 i ostatak 127."** U komplementu dvojke 1111 1111 je −1.
- **"CO = 1 pri oduzimanju znači grešku."** Pri oduzimanju je CO = 1 normalno; greška (podljev) je CO = 0.
- **"Raspon s predznakom je −127 do 127."** U komplementu dvojke je −128 do 127.

Brza provjera razumijevanja

1. Izračunaj 703 − 458 metodom komplementa devetke.
2. Zapiši −20 kao 8-bitni broj u komplementu dvojke.
3. Zbroji u komplementu dvojke 0101 0000 (80) i 1110 0010 (−30). Provjeri rezultat.
4. Je li 0111 0000 + 0011 0000 ispravan zbroj brojeva s predznakom? Zašto?
5. Koliko je 1111 0000 bez predznaka, a koliko s predznakom?

14. Povratna veza i bistabili (flip-flopovi)

Što ćeš naučiti u ovom poglavlju

- Što je **povratna veza**: izlaz sklopa vraća se na njegov ulaz.
- Kako radi **oscilator** (sat) i što su **ciklus**, **period** i **frekvencija** (herc).
- Kako bistabil **R-S** "pamti" koji je ulaz zadnji bio aktivan.
- Kako **D-zasun** (D latch) pamti jedan bit u odabranom trenutku: to je **1 bit memorije**.
- Kako se zasun koristi za **zbrajalo koje akumulira** zbroj.
- Razliku između okidanja **razinom** i **bridom**, i kako bistabil okidan bridom daje **djelitelj frekvencije** i **brojilo**.

Ovo poglavlje uvodi najvažniji novi pojam nakon logičkih vrata: **sklop koji pamti**. Bez pamćenja nema brojanja, nema memorije i nema računala.

Povratna veza: zujalica i oscilator

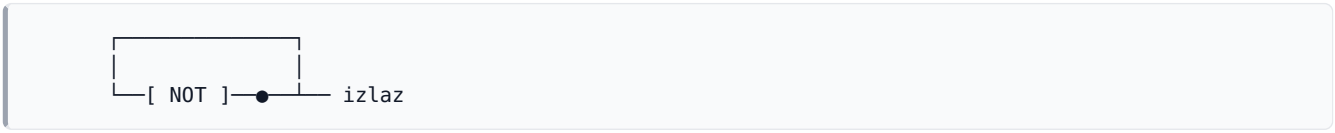
Do sada su sklopovi imali odvojen ulaz i izlaz. Što ako izlaz releja spojimo natrag na njegov vlastiti elektromagnet?

Zamisli relej spojen preko **gornjeg** kontakta (kao invertor), ali tako da struja za elektromagnet prolazi baš kroz taj kontakt:

1. Struja teče kroz kontakt i elektromagnet, pa elektromagnet privuče kontakt.
2. Kad se kontakt pomakne, krug se prekine i elektromagnet izgubi magnetizam.
3. Kontakt se vrati, krug se opet zatvori, i sve kreće ispočetka.

Kontakt neprestano titra naprijed-natrag. Ako pritom stvara zvuk, to je **zujalica**; s čekićem i metalnim zvonom dobiva se električno zvono.

U jeziku vrata, to je **invertor čiji je izlaz spojen na vlastiti ulaz**:

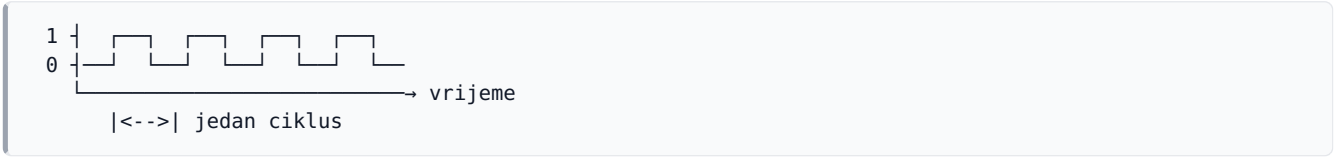


Izgleda kao proturječje: izlaz invertora mora biti suprotan ulazu, a ovdje mu je jednak. Rješenje je u tome što relej treba **malo vremena** da promijeni stanje. Izlaz zato stalno "kasni" za ulazom i neprestano se mijenja: 0, 1, 0, 1...

Takav sklop zove se **oscilator**. Za razliku od svih dosadašnjih sklopova, oscilatoru ne treba čovjek da mijenja prekidače: radi sam od sebe.

Sat, ciklus, period, frekvencija

Izlaz oscilatora crta se kao **vremenski dijagram**: vodoravna os je vrijeme, okomita je vrijednost (0 ili 1).



Budući da se mijenja pravilno, oscilator se zove i **sat** (*clock*): brojanjem ciklusa može se mjeriti vrijeme. Svako računalo ima neku vrstu oscilatora koji usklađuje rad svih dijelova.

Pojam	Značenje
Ciklus	od jedne promjene izlaza do trenutka kad se izlaz vrati u početno stanje
Period (T)	trajanje jednog ciklusa, u sekundama
Frekvencija (f)	broj ciklusa u sekundi
Herc (Hz)	jedinica frekvencije; 1 Hz = 1 ciklus u sekundi (naziv po Heinrichu Hertzu)

> **f = 1 / T**

Primjer: ako jedan ciklus traje 0,25 s, frekvencija je 1 / 0,25 = **4 Hz**, dakle četiri ciklusa u sekundi.

Brzina relejnog oscilatora ovisi o građi releja: velik, težak relej titra sporo, a mali i lagan brzo.

Bistabil R-S

Spoji dvoja vrata **NOR** tako da izlaz svakih bude ulaz drugih (unakrsno). Svaka vrata imaju još po jedan slobodan ulaz.

Podsjetnik: izlaz NOR-a je 1 samo kad su **oba** ulaza 0.

Ponašanje (žarulja je na izlazu desnih vrata):

1. Oba prekidača otvorena, žarulja ugašena.
2. Zatvoriš **gornji** prekidač: žarulja se **pali**.
3. **Otvoriš** gornji prekidač: žarulja **ostaje upaljena**.
4. Zatvoriš **donji** prekidač: žarulja se **gasi**.
5. Otvoriš donji prekidač: žarulja **ostaje ugašena**.

Neobično: kad su oba prekidača otvorena, žarulja je **nekad upaljena, a nekad ugašena**. Izlaz ne ovisi samo o trenutnim ulazima, nego i o tome **što se dogodilo prije**. Sklop ima **dva stabilna stanja**, pa se zove **bistabil** (engl. *flip-flop*).

Analogija: klackalica. Ima dva stabilna položaja i nikad ne ostaje u sredini. Pogledom na klackalicu znaš koja je strana zadnja pritisnuta.

Bistabil pamti informaciju. Ovaj pamti koji je prekidač zadnji bio zatvoren. Bez pamćenja ne bi se moglo ni brojati: da ne pamtiš do kojeg si broja došao, ne bi znao koji je sljedeći.

Oznake i tablica

Ulazi se zovu **S** (*set*, postavi: Q postaje 1) i **R** (*reset*, poništi: Q postaje 0). Izlazi su **Q** i **$\bar{Q}$** ("Q crta"), uvijek suprotni.

S	R	Q	$\bar{Q}$	Značenje
1	0	1	0	postavljen
0	1	0	1	poništen
0	0	Q	$\bar{Q}$	**pamti** prethodno stanje
1	1	—	—	**zabranjeno**

Kad su S i R oba 1, oba izlaza postanu 0, a to krši pravilo da je $\bar{Q}$ suprotan od Q. Zato se ta kombinacija izbjegava.

Ovakva tablica zove se **funkcijska tablica**, **logička tablica** ili **tablica istinitosti**.

D-zasun: zapamti bit u određenom trenutku

Bistabil R-S pamti koji je ulaz zadnji bio 1. Puno korisniji je sklop koji pamti **vrijednost jednog signala u odabranom trenutku**. Treba mu dva ulaza:

- **Data (D)**: podatak, 0 ili 1;
- **Clock (Clk)**: signal "zapamti sada". (Knjiga ga najprije zove "drži taj bit".)

Željeno ponašanje:

D	Clk	Q
0	1	0
1	1	1
X	0	Q (nepromijenjen)

X znači "nije bitno" (*don't care*): kad je Clk = 0, vrijednost D ne utječe na izlaz.

Kako se gradi

1. Ispred ulaza R-S bistabila stave se dvoja vrata **AND**. Svaka imaju Clk kao jedan ulaz. Dok je Clk = 0, oba AND-a daju 0, pa bistabil samo pamti.
2. Kad je Clk = 1, AND-ovi propuštaju signale, i sklop radi kao obični R-S bistabil.
3. Zabranjeno stanje (S = R = 1) i besmisleno stanje (S = R = 0, koje već postiže Clk = 0) izbjegavaju se tako da **S dobije D, a R dobije invertirani D**. Uvijek je točno jedan od njih 1.

Rezultat: dok je Clk = 1, Q prati D. Kad Clk padne na 0, Q zadrži **zadnju** vrijednost D, kako god se D poslije mijenjao.

Ovo je **D-bistabil okidan razinom** (*level-triggered D-type flip-flop*), koji se zove i **zasun** (*latch*): "zasune" jedan bit i drži ga. To je **1 bit memorije**. Ukupno: dvoja NOR, dvoja AND i jedan invertor.

8-bitni zasun

Osam zasuna sa zajedničkim ulazom Clk čine **8-bitni zasun**: ulazi D_0 – D_7, izlazi Q_0 – Q_7. Dok je Clk = 0, izlazi drže spremljenu vrijednost; kad Clk postane 1, na izlaze se prepisuje 8-bitna vrijednost s ulaza.

Primjena: zbrajanje više brojeva

Zbrajalo iz poglavlja 12 zbraja samo dva broja; međurezultat bi trebalo ručno prepisati na prekidače. Zasunom se međurezultat **sprema** i koristi izravno.

Prva verzija: izlaz zbrajala ide i na žarulje i na ulaze zasuna. Prekidač **Save** (spremi) je Clk zasuna. Uz to treba **selektor 2 na 1**: sklop koji prekidačem bira odakle ulaz B zbrajala dobiva podatke, s drugog reda prekidača ili iz zasuna.

Selektor 2 na 1 (za jedan bit): dvoja AND, jedna OR i invertor.

Select	A	B	Izlaz
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

Kad je Select = 0, izlaz je A; kad je 1, izlaz je B. Osam takvih sklopova sa zajedničkim Select čini 8-bitni selektor.

(Ova verzija ne rješava izlazni prijenos: ako zbroj prijeđe 255, prijenos se gubi. Knjiga se tome vraća u poglavlju 17.)

Brisanje (Clear) i akumulator

Zasunu se dodaje ulaz **Clear** (obriši): kad je 1, Q postaje 0 bez obzira na ostale ulaze. Ostvaruje se još jednim vratima OR.

Zašto ne obrisati upisom D = 0? Zato što ponekad ne upravljamo izravno onim što dolazi na D, npr. kad je D spojen na izlaz zbrajala.

Bolja verzija zbrajala: izlaz zbrajala ide na D zasuna, a izlaz zasuna vraća se na ulaz B zbrajala. Prekidač **Add** je Clk zasuna, a prekidač **Clear** briše zasun.

Postupak:

1. **Clear**: zasun je 0, žarulje ugašene, B zbrajala je 0.
2. Upiši prvi broj na prekidače, pritisni **Add**: broj se pojavi na žaruljama.
3. Upiši sljedeći broj, pritisni **Add**: zbroj je na žaruljama.
4. Ponavljaj za koliko god brojeva.

Ovaj sklop zbraja dugačak niz brojeva s jednim redom prekidača. Zasun u kojem se skuplja zbroj zove se **akumulator** (o tome više u poglavlju 17).

Okidanje razinom i okidanje bridom

D-zasun okidan **razinom** sprema D dok je Clk na **razini** 1. Za sve vrijeme dok je Clk = 1, svaka promjena D vidi se na izlazu.

Bistabil **okidan bridom** (*edge-triggered*) mijenja izlaz **samo u trenutku kad Clk prijeđe iz 0 u 1**. Ni kad je Clk = 0 ni kad je Clk = 1 promjene D ne utječu na izlaz.

- Prijelaz 0 → 1 je **pozitivni brid** (rastući brid); u tablicama se označava strelicom ↑.
- Prijelaz 1 → 0 je **negativni brid**.

D	Clk	Q	$\bar{Q}$
0	↑	0	1
1	↑	1	0
X	0	Q	$\bar{Q}$

Na simbolu se okidanje bridom označava malim **kutom (>)** uz ulaz Clk.

Kako se gradi

Od **dva stupnja** R-S bistabila (s vratima AND na ulazu):

1. Prvi stupanj dobiva **invertirani** Clk: sprema D dok je Clk = 0.
2. Drugi stupanj dobiva izlaz prvog i **neinvertirani** Clk: preuzima ga kad Clk postane 1.

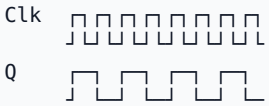
U trenutku prijelaza 0 → 1 prvi stupanj prestaje primati D, a drugi preuzima ono što je prvi zapamtio. Rezultat je da se sprema vrijednost D **upravo u trenutku brida**.

Analogija: fotoaparat. Okidanje razinom je kao video dok držiš gumb; okidanje bridom je kao fotografija u trenutku pritiska.

Djelitelj frekvencije

Spoji izlaz oscilatora na Clk bistabila okidanog bridom, a izlaz **$\bar{Q}$ natrag na D**. Opet povratna veza.

Što se događa: pri svakom pozitivnom bridu Q preuzme vrijednost D, a D je uvijek suprotan od Q. Dakle, **pri svakom pozitivnom bridu sata Q se okrene**.



Q mijenja stanje upola rjeđe od sata: ako sat ima 20 Hz, Q ima **10 Hz**. Takav sklop zove se **djelitelj frekvencije**.

(Napomena iz knjige: u stvarnosti releji bistabila moraju biti dovoljno brzi da prate oscilator; pretpostavlja se da je relej oscilatora puno sporiji.)

Brojilo s valovitim prijenosom

Izlaz jednog djelitelja može biti sat sljedećeg. Lanac djelitelja daje signale sve niže frekvencije: Q_1 upola sporiji od sata, Q_2 upola sporiji od Q_1, Q_3 upola sporiji od Q_2...

Ako izlaze pročitaš kao binarni broj (Q_3 Q_2 Q_1 ...), oni redom prolaze kroz **0000, 0001, 0010, ..., 1111**. To je isti uzorak kao u poglavlju 8: svaki stupac mijenja se dvaput rjeđe od stupca desno od sebe.

Sklop broji u binarnom sustavu. Pri svakom pozitivnom bridu sata vrijednost se **povećava za 1** (*increment*).

Osam bistabila daje **8-bitno brojilo** (0 – 255, zatim opet 0). Zove se **brojilo s valovitim prijenosom** (*ripple counter*), jer izlaz svakog bistabila je sat sljedećeg, pa promjene "putuju" kroz stupnjeve s malim kašnjenjem. Naprednija **sinkrona** brojila mijenjaju sve izlaze istovremeno.

Mjerenje frekvencije oscilatora

Spoji oscilator na brojilo i osam žarulja na izlaze. Kad su sve žarulje ugašene, pokreni štopericu; kad se sve opet ugase, brojilo je prošlo 256 ciklusa.

Primjer: ako je trebalo 32 sekunde, frekvencija je 256 / 32 = **8 Hz**.

Bistabil s Preset i Clear

Najpotpuniji bistabil u ovom poglavlju je **D-bistabil okidan bridom s ulazima Preset i Clear**:

- **Preset = 1**: Q postaje 1, bez obzira na D i Clk;
- **Clear = 1**: Q postaje 0, bez obzira na D i Clk;
- oba ne smiju biti 1 istovremeno (kao S i R);
- kad su oba 0, radi kao obični bistabil okidan bridom.

Pre	Clr	D	Clk	Q	$\bar{Q}$
1	0	X	X	1	0
0	1	X	X	0	1
0	0	0	↑	0	1
0	0	1	↑	1	0
0	0	X	0	Q	$\bar{Q}$

S relejima smo sada naučili **zbrajati, oduzimati i brojati**, opremom starijom od sto godina.

Ključne definicije

Pojam	Značenje
Povratna veza	izlaz sklopa spojen natrag na njegov ulaz
Oscilator (sat)	sklop čiji se izlaz sam pravilno mijenja između 0 i 1
Period / frekvencija	trajanje ciklusa / broj ciklusa u sekundi (Hz); f = 1/T
Bistabil (flip-flop)	sklop s dva stabilna stanja; pamti jedan bit
R-S bistabil	dvoja NOR unakrsno; S postavlja, R poništava; S = R = 1 zabranjeno
D-zasun (okidan razinom)	sprema D dok je Clk = 1; nakon toga drži vrijednost
Okidan bridom	sprema D samo u trenutku prijelaza Clk iz 0 u 1 (↑)
Selektor 2 na 1	bira jedan od dva ulaza prema signalu Select
Djelitelj frekvencije	bistabil s $\bar{Q}$ spojenim na D; izlaz upola sporiji od sata
Brojilo s valovitim prijenosom	lanac djelitelja; izlazi broje binarno
Preset / Clear	ulazi koji izravno postavljaju Q na 1 / 0

Ključno za zapamtiti

1. Povratna veza omogućuje sklopove koji titraju (oscilator) i koji pamte (bistabil).
2. Oscilator je invertor s izlazom na vlastitom ulazu; radi zbog kašnjenja releja.
3. Frekvencija = 1 / period; mjeri se u hercima.
4. Bistabil R-S ima dva stabilna stanja i pamti koji je ulaz zadnji bio aktivan.
5. D-zasun je 1 bit memorije: kad je Clk = 1 upisuje D, kad je 0 pamti.
6. Zasun + zbrajalo daju akumulator koji zbraja niz brojeva.
7. Bistabil okidan bridom reagira samo na prijelaz 0 → 1.
8. Bistabil s $\bar{Q}$ na D dijeli frekvenciju s 2; lanac takvih je binarno brojilo.
9. Preset i Clear izravno postavljaju izlaz, neovisno o satu.

Najčešće zabune

- **"Izlaz sklopa ovisi samo o trenutnim ulazima."** Kod bistabila ovisi i o prošlosti; zato pamti.
- **"S = R = 1 je samo još jedno stanje."** Zabranjeno je: oba izlaza postanu 0.
- **"Razina i brid su isto."** Zasun okidan razinom propušta promjene D cijelo vrijeme dok je Clk = 1; bistabil okidan bridom samo u trenutku prijelaza.
- **"Brojilo s valovitim prijenosom mijenja sve bitove istovremeno."** Ne: promjene putuju bistabil po bistabil.

Brza provjera razumijevanja

1. Oscilator ima period 0,02 s. Kolika mu je frekvencija?
2. Napiši tablicu R-S bistabila i objasni zašto je S = R = 1 zabranjeno.
3. Zašto se kod D-zasuna na R dovodi invertirani D?
4. Ako sat ima 64 Hz, koliku frekvenciju ima izlaz trećeg bistabila u brojilu?
5. Opiši korak po korak kako bi s akumulatorom zbrojio 12, 30 i 7.

15. Bajtovi i heksadecimalni sustav

Što ćeš naučiti u ovom poglavlju

- Što je **bajt** i zašto ima baš 8 bitova.
- Koliko različitih vrijednosti stane u jedan i dva bajta.
- Što je **nibl** (pola bajta).
- Zašto je oktalni zapis nespretan za bajtove.
- Kako radi **heksadecimalni** (šesnaestinski) sustav: znamenke 0–9 i A–F.
- Kako pretvarati brojeve između binarnog, heksadecimalnog i dekadskog zapisa.
- Kako zbrajati heksadecimalne brojeve i kako prepoznati negativan broj u heksadecimalnom zapisu.

Ovo je kratko, ali vrlo praktično poglavlje. Heksadecimalni zapis koristit ćemo u svim daljnjim poglavljima: za adrese memorije, sadržaj memorije i strojne instrukcije.

Bajt: skupina od 8 bitova

Sklopovi iz prethodnih poglavlja (8-bitno zbrajalo, 8-bitni zasun, brojilo) ne rade s pojedinačnim bitovima nego sa **skupinama bitova** koje putuju zajedno. Kaže se da je **put podataka** (*data path*) širok 8 bitova.

> **Bajt** (*byte*) je skupina od **8 bitova** koja se obrađuje kao cjelina.

Analogija: bit je jedno slovo, a bajt je riječ od točno osam slova. Računalo rijetko gleda pojedinačno slovo; uglavnom čita cijele riječi.

Kratka povijest, samo bitno:

- Riječ *byte* nastala je u IBM-u oko 1956. godine. Dolazi od engleske riječi *bite* (zalogaj), a piše se s "y" da se ne bi brkala s riječju *bit*.
- Isprva je značila broj bitova u nekom putu podataka, kakav god on bio.
- Od IBM-ova sustava **System/360** (sredina 1960-ih) bajt znači **točno 8 bitova**, i tako je ostalo.

Koliko stane u jedan bajt

Osam bitova daje 2^8 = **256** različitih kombinacija, od `00000000` do `11111111`. Isti bajt možemo tumačiti na više načina:

Tumačenje	Raspon	Napomena
Pozitivan cijeli broj (bez predznaka)	0 do 255	`11111111` = 255
Broj s predznakom (komplement dvojke)	−128 do 127	najlijeviji bit je predznak (poglavlje 13)
Kod za 256 različitih stvari	256 oznaka	npr. znakovi, nijanse, naredbe

Bajt sam po sebi ne "zna" što predstavlja. Značenje mu daje **dogovor** o tome kako ga čitamo.

Zašto baš 8 bitova

Knjiga navodi nekoliko razloga zašto je 8 bitova "prava mjera":

1. **Dekadske znamenke.** Jedna znamenka 0–9 traži 4 bita. U bajt stanu točno **dvije** znamenke. Takav zapis zove se **BCD** (*binary-coded decimal*); detaljnije u poglavlju 23.

2. **Tekst.** Mala i velika slova engleske abecede (26 + 26), znamenke (10) i interpunkcija zajedno imaju manje od 128 znakova. Većina pisama na svijetu stane u manje od 256 znakova. Dakle, jedan znak može biti jedan bajt (poglavlje 20).
3. **Slike.** Crno-bijela fotografija može koristiti **256 nijansi sive**, od crne (0) do bijele (255), po jedan bajt za svaku točku.
4. **Veći brojevi.** Kad bajt nije dovoljan, uzmu se dva. Dva bajta (16 bitova) daju 2^{16} = **65.536** različitih vrijednosti.

Broj bajtova	Broj bitova	Broj različitih vrijednosti
1	8	256
2	16	65.536
3	24	16.777.216
4	32	4.294.967.296

Nibl: pola bajta

Polovica bajta, dakle **4 bita**, zove se **nibl** (*nibble*, ponekad *nybble*). Ime je šala: *nibble* znači "mali zalogaj", a bajt je "zalogaj".

Jedan nibl ima 2^4 = 16 mogućih vrijednosti (0–15). Ta činjenica bit će ključna za heksadecimalni sustav.

Problem: kako kratko zapisati bajt

Bajt zapisan binarno, npr. `10110110`, dug je i lako se pogriješi pri prepisivanju. Treba nam kraći zapis iz kojeg se bitovi lako vrate.

Dekadski zapis (182) je kratak, ali iz njega se bitovi **ne vide**; treba računati. Želimo sustav u kojem jedna znamenka odgovara **točno određenoj skupini bitova**.

Pokušaj s oktalnim sustavom

U poglavlju 8 vidjeli smo da jedna oktalna znamenka odgovara skupini od **3 bita**. Bajt se grupira s desna:

```
10 110 110
 2   6   6    →  266₈
```

Raspon bajta u oktalnom zapisu je od `000` do `377`. Do ovdje sve radi.

Problem nastaje kod **višebajtnih** vrijednosti. Osam nije djeljivo s tri, pa granice skupina od 3 bita ne padaju na granice bajtova. Primjer za 16-bitni broj `10110011 11000101`:

```
Cijeli 16-bitni broj, skupine po 3 s desna:
 1 011 001 111 000 101
 1   3   1   7   0   5     →  131705₈

Svaki bajt zasebno:
 10 110 011    11 000 101
  2   6   3     3   0   5  →  263₈ i 305₈
```

Zapis "263 305" **nije** isto što i 131705. Isti bitovi daju različite oktalne zapise ovisno o tome gledamo li ih kao jedan broj ili kao dva bajta. To je nespretno i izvor pogrešaka.

Heksadecimalni sustav

Rješenje je grupirati bitove po **4**. Osam je djeljivo s četiri, pa svaki bajt ima točno **dva** takva dijela, a granice se uvijek poklapaju.

Skupina od 4 bita ima 16 mogućih vrijednosti, pa nam treba **brojevni sustav s bazom 16**. Zove se **heksadecimalni** (grč. *hexa* = šest, lat. *decem* = deset), skraćeno **heks**.

Šesnaest znamenki

Treba nam 16 različitih znamenki, a imamo samo deset (0–9). Dogovor je da se za vrijednosti 10–15 koriste **prva slova abecede**:

Dekadski	Binarno (nibl)	Heks
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Ovu tablicu vrijedi naučiti napamet. Ona je cijeli "rječnik" za pretvorbu između binarnog i heksadecimalnog zapisa.

Analogija: heks znamenka je kao kratica. Umjesto da kažeš "jedan-jedan-nula-jedan", kažeš samo "D", a svi znaju što to znači.

Oznaka heksadecimalnog broja

Broj `26` može biti dekadski ili heksadecimalni (a heksadecimalni 26 je 38 dekadski). Da ne bude zabune, knjiga heksadecimalne brojeve piše sa sufiksom **h**:

- `26h` je heksadecimalni broj (vrijednost 38),
- `26` je dekadski broj,
- `B6h`, `FFh`, `7Bh` su također heks brojevi.

Slova (A–F) u broju jasno pokazuju da je heks, ali sufiks se piše uvijek, radi dosljednosti.

Brojenje u heksu

Broji se isto kao u dekadskom, samo se prijenos događa tek nakon F:

```
0, 1, 2, ..., 9, A, B, C, D, E, F, 10h, 11h, ..., 19h, 1Ah, ..., 1Fh, 20h, ...
..., FEh, FFh, 100h, ...
```

- `10h` je 16 (jedna šesnaestica i nula jedinica).
- `FFh` je najveća vrijednost bajta: 255.

- `100h` je 256.

Pretvorba između binarnog i heksadecimalnog zapisa

Ovo je najlakša pretvorba i zato se heks toliko koristi.

Iz binarnog u heks

1. Podijeli bitove u skupine po 4, **s desna**. Ako lijevoj skupini nedostaje bitova, dopuni je nulama.
2. Svaku skupinu zamijeni heks znamenkom iz tablice.

Primjer 1: bajt `10110110` (u poglavlju 8 izračunali smo da je to 182).

```
1011 0110
  B    6    →  B6h
```

Primjer 2: 16-bitni broj `1101011100101010`.

```
1101 0111 0010 1010
  D    7    2    A    →  D72Ah
```

Primjer 3: broj `110101` (samo 6 bitova).

```
0011 0101      (dopunjeno nulama s lijeva)
  3    5    →  35h
```

Iz heksa u binarni

Obrnuto: svaku heks znamenku zamijeni s 4 bita.

```
9Fh   →  1001 1111
4E2Ch →  0100 1110 0010 1100
```

Rješenje problema iz oktalnog primjera

Uzmimo isti 16-bitni broj kao gore: `10110011 11000101`.

```
Cijeli broj:      1011 0011 1100 0101  →  B3C5h
Bajt po bajt:     1011 0011 = B3h,  1100 0101 = C5h  →  B3h C5h
```

Oba načina daju **isto**: B3C5h. U heksu se višebajtni broj dobije jednostavnim spajanjem heks zapisa njegovih bajtova. To je glavni razlog zašto se heks koristi umjesto oktalnog.

Zapis istog 16-bitnog broja	Kao jedan broj	Kao dva bajta
Oktalni	131705_8	263_8 305_8 (ne poklapa se)
Heksadecimalni	B3C5h	B3h C5h (poklapa se)

Mjesne vrijednosti u heksadecimalnom sustavu

Heks je pozicijski sustav kao i dekadski (poglavlje 7). Svako mjesto vrijedi **16 puta više** od mjesta desno od njega.

Mjesto (s desna)	Potencija	Vrijednost
1.	16^0	1

Mjesto (s desna)	Potencija	Vrijednost
2.	16^1	16
3.	16^2	256
4.	16^3	4.096
5.	16^4	65.536

Opća formula za četveroznamenkasti heks broj *abcd*h:

> **vrijednost = a × 4096 + b × 256 + c × 16 + d**

pri čemu se slova A–F zamjenjuju vrijednostima 10–15.

Pretvorba iz heksa u dekadski

Svaku znamenku pomnoži s vrijednošću njezina mjesta i zbroji.

Primjer: 2C7Eh

Znamenka	Vrijednost znamenke	Mjesna vrijednost	Umnožak
2	2	4096	8192
C	12	256	3072
7	7	16	112
E	14	1	14
		Ukupno	**11.390**

Za jedan bajt (dvije znamenke) dovoljno je: *gornja znamenka × 16 + donja znamenka*.

- B6h = 11 × 16 + 6 = 176 + 6 = **182**
- 7Fh = 7 × 16 + 15 = 112 + 15 = **127**
- 80h = 8 × 16 + 0 = **128**

Pomoćna tablica za gornju znamenku bajta:

Gornja znamenka	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Vrijednost	0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240

Pretvorba iz dekadskog u heks

Metoda 1: dijeljenje mjesnim vrijednostima

Dijeli redom s 4096, 256 i 16. Količnik je znamenka, a ostatak ide u sljedeći korak.

Primjer: 50.000

Korak	Račun	Znamenka	Ostatak
1	50.000 ÷ 4096 = 12	C	50.000 − 49.152 = 848
2	848 ÷ 256 = 3	3	848 − 768 = 80
3	80 ÷ 16 = 5	5	80 − 80 = 0
4	ostatak	0	

Rezultat: **C350h**.

Metoda 2: prvo bajtovi, pa niblovi

1. Podijeli broj s **256**. Količnik je gornji bajt, ostatak donji bajt.
2. Svaki bajt podijeli sa **16**. Količnik je gornja znamenka, ostatak donja.

Primjer: opet 50.000

```
50.000 ÷ 256 = 195, ostatak 80
195 ÷ 16 = 12, ostatak 3   →  C3h  (gornji bajt)
 80 ÷ 16 =  5, ostatak 0   →  50h  (donji bajt)
Rezultat: C350h
```

Provjera: C350h = 12 × 4096 + 3 × 256 + 5 × 16 + 0 = 49.152 + 768 + 80 = 50.000. ✓

Ova druga metoda prirodno slijedi bajtnu građu broja. Zato je praktičnija kad radiš s memorijom.

Zbrajanje heksadecimalnih brojeva

Zbraja se kao u dekadskom sustavu, znamenka po znamenka s desna, s jednom razlikom:

> Ako je zbroj znamenki (s prijenosom) **16 ili više**, zapiši zbroj **− 16** i prenesi **1** na sljedeće mjesto.

Primjer: 3A7Fh + 1C95h

Mjesto	Račun (dekadski)	Zapiši	Prijenos
jedinice	F + 5 = 15 + 5 = 20	20 − 16 = 4	1
šesnaestice	7 + 9 + 1 = 17	17 − 16 = 1	1
256-ice	A + C + 1 = 10 + 12 + 1 = 23	23 − 16 = 7	1
4096-ice	3 + 1 + 1 = 5	5	0

```
  1 1 1       (prijenosi)
  3 A 7 F
+ 1 C 9 5
---------
  5 7 1 4     →  5714h
```

Provjera u dekadskom: 3A7Fh = 14.975, 1C95h = 7.317, zbroj = 22.292, a 5714h = 20.480 + 1.792 + 16 + 4 = 22.292. ✓

Korisna pravila za brzo zbrajanje:

- F + 1 = 10h (kao 9 + 1 = 10 u dekadskom).
- F + F = 1Eh (15 + 15 = 30 = 16 + 14).
- 8 + 8 = 10h.

Oduzimanje bi se moglo raditi izravno, s posuđivanjem od 16. U računalu se ipak radi kao u poglavlju 13: zbrajanjem komplementa.

Brojevi s predznakom u heksu

U poglavlju 13 vidjeli smo da je kod brojeva s predznakom **najlijeviji bit** predznak: 0 za pozitivan, 1 za negativan.

U heks zapisu najlijeviji bit pripada **prvoj** heks znamenki. Znamenke 0–7 počinju bitom 0, a 8–F bitom 1. Zato:

> Ako se bajt (ili višebajtni broj) tumači kao broj s predznakom, onda je **negativan ako mu je prva heks znamenka 8, 9, A, B, C, D, E ili F**.

Bajt	Binarno	Bez predznaka	S predznakom
00h	0000 0000	0	0
7Fh	0111 1111	127	127 (najveći pozitivan)
80h	1000 0000	128	−128 (najmanji negativan)
C8h	1100 1000	200	−56
FFh	1111 1111	255	−1

Pravilo za čitanje s predznakom: ako je prva znamenka 8–F, vrijednost s predznakom = vrijednost bez predznaka − 256. Na primjer C8h: 200 − 256 = −56.

Isto vrijedi za 16-bitne brojeve: `8000h` do `FFFFh` su negativni (−32.768 do −1), a `0000h` do `7FFFh` pozitivni (0 do 32.767).

Isti bajt, tri značenja

Uzmimo bajt **99h** = `1001 1001`. Što on znači?

Tumačenje	Vrijednost
Broj bez predznaka	9 × 16 + 9 = **153**
Broj s predznakom (komplement dvojke)	153 − 256 = **−103**
BCD (dvije dekadske znamenke po 4 bita)	**99**

Bitovi su isti, a značenja različita. Iz samog bajta ne može se zaključiti koje je tumačenje točno. To se mora znati iz **konteksta**: tko je bajt zapisao i s kojom namjerom.

Analogija: riječ "kosa" može značiti dlake na glavi, alat za košnju ili nagnutu plohu. Slova su ista; značenje određuje rečenica u kojoj stoji.

Gdje ćemo koristiti heks

Od ovog poglavlja dalje knjiga gotovo sve brojeve u vezi s računalom piše heksadecimalno:

- **adrese memorije** (npr. 0000h–FFFFh za 64 K memorije, poglavlje 16),
- **sadržaj memorije** (bajtovi kao dvoznamenkasti heks brojevi),
- **strojne instrukcije** (operacijski kodovi poput 10h, 11h, poglavlja 17 i 19),
- **kodove znakova** (ASCII, poglavlje 20).

Ključne definicije

Pojam	Značenje
Bajt	skupina od 8 bitova; 256 mogućih vrijednosti
Put podataka	broj bitova koji sklop obrađuje istovremeno
Nibl	pola bajta, 4 bita; 16 mogućih vrijednosti
Heksadecimalni sustav	pozicijski sustav s bazom 16; znamenke 0–9 i A–F
Sufiks h	oznaka heksadecimalnog broja (npr. B6h)
Mjesne vrijednosti u heksu	1, 16, 256, 4096, 65.536

Pojam	Značenje
BCD	zapis dekadske znamenke u 4 bita; dvije znamenke po bajtu

Ključno za zapamtiti

1. Bajt je 8 bitova; može biti 0–255, −128 do 127 ili 256 različitih oznaka.
2. Dva bajta daju 65.536 vrijednosti.
3. Nibl je 4 bita i odgovara točno jednoj heks znamenki.
4. Bajt se zapisuje s točno dvije heks znamenke: od 00h do FFh.
5. Oktalni zapis nije prikladan za bajtove jer se skupine od 3 bita ne poklapaju s granicama bajtova.
6. U heksu se višebajtni broj dobije spajanjem heks zapisa bajtova.
7. Pretvorba binarno ↔ heks radi se nibl po nibl, pomoću tablice od 16 redaka.
8. Heks ↔ dekadski: množenje ili dijeljenje s 16, 256, 4096...
9. Kod zbrajanja u heksu prijenos nastaje kad je zbroj 16 ili više.
10. Broj s predznakom je negativan ako mu je prva heks znamenka 8–F.

Najčešće zabune

- **"10h je deset."** Nije: 10h je **16**. Deset je Ah.
- **"Bajt sam po sebi ima jedno značenje."** 99h može biti 153, −103 ili BCD 99. Značenje određuje kontekst.
- **"Binarni broj se dijeli na skupine od 4 s lijeva."** Dijeli se **s desna**; nedostajuće bitove dopuni nulama s lijeva.
- **"Heks je samo drugačiji način pisanja dekadskih brojeva."** Heks je stvoren da bi se **bitovi vidjeli**. Svaka znamenka je točno 4 bita, a kod dekadskog zapisa toga nema.
- **"FFh je uvijek 255."** Samo kad se čita bez predznaka. S predznakom je FFh = −1.

Brza provjera razumijevanja

1. Pretvori `1110001101011111` u heks, a zatim u dekadski broj.
2. Pretvori 1000 (dekadski) u heks na oba načina opisana u poglavlju.
3. Izračunaj A9C4h + 5B7Dh i provjeri rezultat u dekadskom sustavu.
4. Zašto oktalni zapis 16-bitnog broja nije jednak spojenim oktalnim zapisima njegovih dvaju bajtova?
5. Koje su tri moguće vrijednosti bajta 85h, ovisno o tumačenju?

16. Sklapanje memorije

Što ćeš naučiti u ovom poglavlju

- Kako se D-zasun koristi kao **1 bit memorije** (ulazi *Data In*, *Write*, izlaz *Data Out*).
- Kako **selektor 8 na 1** bira jedan od osam izlaza za čitanje.
- Kako **dekoder 3 na 8** usmjerava signal pisanja u točno jedan zasun.
- Što je **adresa** i što je **RAM** (memorija s nasumičnim pristupom).
- Kako se manja RAM polja spajaju u veća i formulu **broj vrijednosti = 2^(broj adresnih linija)**.
- Što su **kilobajt, megabajt, gigabajt** i zašto je kilobajt 1024, a ne 1000 bajtova.
- Kako izgleda memorija od **64 KB** s upravljačkom pločom.
- Zašto je RAM **nepostojana** (*volatile*) memorija.

Što je memorija

Memorija čuva podatak između dva trenutka: trenutka kad ga **spremamo** i trenutka kad ga **dohvaćamo**. U knjizi se ti parovi riječi koriste kao sinonimi:

Spremanje	Dohvat
pisati (*write*)	čitati (*read*)
spremiti (*save*)	dohvatiti (*retrieve*)
pohraniti (*store*)	pristupiti (*access*)

Papir je memorija za tekst, magnetska vrpca za glazbu i film. U poglavlju 14 vidjeli smo da i **bistabil** pamti: jedan bistabil sprema 1 bit. Ako znamo spremiti 1 bit, znamo spremiti i 2, 3 ili milijun bitova. Ovo poglavlje pokazuje kako.

1 bit memorije: D-zasun s novim imenima

Polazimo od D-bistabila okidanog razinom iz poglavlja 14 (invertor, dvoja vrata AND, dvoja vrata NOR). Podsjetnik na tablicu:

D	Clk	Q	$\bar{Q}$
0	1	0	1
1	1	1	0
X	0	Q	$\bar{Q}$ (zadnja vrijednost)

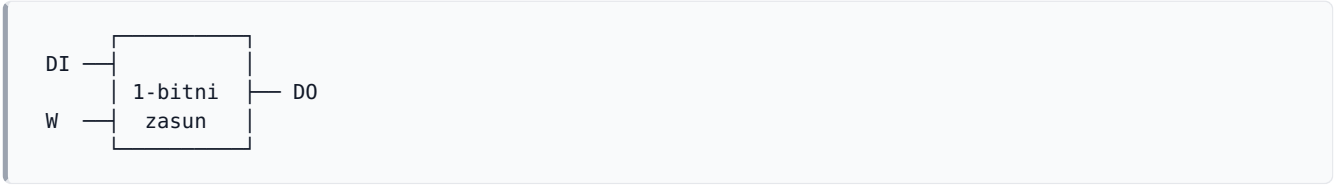
X znači "nije važno". Kad je Clk = 0, izlaz drži zadnju spremljenu vrijednost.

U ovom poglavlju sklop koristimo **samo za pamćenje**, pa mu dajemo imena koja to opisuju:

Staro ime	Novo ime	Uloga
D (Data)	**DI** (*Data In*)	podatak koji želimo spremiti
Clk (Clock)	**W** (*Write*)	naredba "upiši sada"
Q	**DO** (*Data Out*)	spremljeni podatak

Kako se piše: W je normalno 0, pa promjene na DI ne utječu na izlaz. Kad želimo spremiti podatak, postavimo W na **1, pa natrag na 0**. Vrijednost koja je u tom trenutku bila na DI ostaje na DO.

Takav sklop zove se **zasun** (*latch*) jer "zakvači" podatak. Crtamo ga kao kućicu:

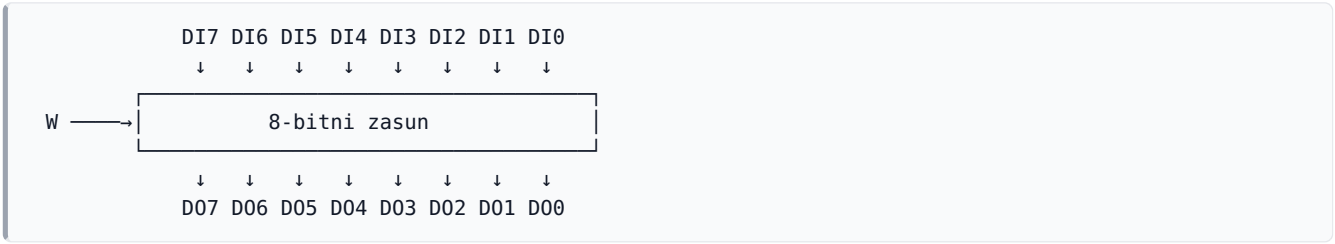


Analogija: zasun je ploča za bilješke s poklopcem. Dok je poklopac zatvoren (W = 0), ništa ne možeš upisati, ali bilješku vidiš. Otvoriš ga (W = 1), napišeš, zatvoriš (W = 0), i zapis ostaje.

Dva načina spajanja osam zasuna

Način 1: 8-bitni zasun (jedna 8-bitna vrijednost)

Osam 1-bitnih zasuna s **zajedničkim W**. Svaki ima svoj DI i svoj DO. Kad W ode na 1 i natrag na 0, svih osam bitova spremi se istovremeno.



Ovo već znamo iz poglavlja 14. Sprema **jednu** vrijednost od 8 bitova.

Način 2: osam zasebnih 1-bitnih vrijednosti

Sada želimo nešto drukčije: spremiti **osam odvojenih bitova** u različitim trenucima, a imamo samo **jedan** ulaz *Data In* i **jedan** izlaz *Data Out* (npr. samo jednu žaruljicu).

Za to trebaju dvije stvari:

1. **Na izlazu:** sklop koji bira čiji DO ide na žaruljicu.
2. **Na ulazu:** sklop koji signal W šalje samo u **jedan** zasun.

Za izbor jednog od 8 predmeta trebaju **3 prekidača**, jer 3 bita daju $2^3 = 8$ kombinacija (000 do 111).

Izlazna strana: selektor 8 na 1

Selektor podataka 8 na 1 (*8-to-1 data selector*) ima:

- 8 podatkovnih ulaza: D0 do D7,
- 3 ulaza za izbor: S0, S1, S2,
- 1 izlaz.

Ulazi za izbor određuju **koji od osam podatkovnih ulaza** se pojavljuje na izlazu.

S2	S1	S0	Izlaz
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4

S2	S1	S0	Izlaz
1	0	1	D5
1	1	0	D6
1	1	1	D7

Pravilo je jednostavno: S2 S1 S0 čitani kao binarni broj daju **redni broj** odabranog ulaza. Za 101 izlaz je D5, jer je $101_2 = 5$.

Analogija: selektor je birač kanala na televizoru. Osam programa stalno stiže, a gumbi određuju koji gledaš.

U poglavlju 14 vidjeli smo jednostavniju verziju, **selektor 2 na 1**. Ovaj radi isto, samo s više ulaza.

Građa selektora

Selektor 8 na 1 sastoji se od:

- **3 invertora** (daju S̄0, S̄1, S̄2),
- **8 vrata AND s 4 ulaza** (po jedna za svaki D),
- **1 vrata OR s 8 ulaza** (skupljaju izlaze svih vrata AND).

Svaka vrata AND dobivaju jedan D ulaz i jedinstvenu kombinaciju S signala ili njihovih negacija. Primjer za D5 (S = 101):

```
D5 ─┐
S2 ─┤
S̄1 ─┤ AND ──→ (u vrata OR)
S0 ─┘
```

Kad je S2 = 1, S1 = 0, S0 = 1, samo su ta vrata AND "otvorena" (sva tri izborna ulaza su im 1). Sva ostala vrata AND daju 0. Izlaz ovih vrata je 1 ako je D5 = 1, a 0 ako je D5 = 0. Vrata OR zato daju upravo vrijednost D5.

Ulazna strana: dekoder 3 na 8

Na ulaznoj strani svi DI ulazi zasuna mogu se spojiti zajedno: svi vide isti podatak. Ali **W signali ne smiju biti spojeni**, jer bi tada svi zasuni upisivali istovremeno. Jedan W mora stići u točno jedan zasun.

Za to služi **dekoder 3 na 8**. Radi suprotno od selektora:

- ima 1 podatkovni ulaz i 3 izborna ulaza (S0, S1, S2),
- ima **8 izlaza** (O0 do O7),
- svi izlazi su 0, **osim jednog**: onoga koji je odabran signalima S. Na njemu je vrijednost podatkovnog ulaza.

S2	S1	S0	O7	O6	O5	O4	O3	O2	O1	O0
0	0	0	0	0	0	0	0	0	0	Pod.
0	0	1	0	0	0	0	0	0	Pod.	0
0	1	0	0	0	0	0	0	Pod.	0	0
0	1	1	0	0	0	0	Pod.	0	0	0
1	0	0	0	0	0	Pod.	0	0	0	0
1	0	1	0	0	Pod.	0	0	0	0	0
1	1	0	0	Pod.	0	0	0	0	0	0
1	1	1	Pod.	0	0	0	0	0	0	0

("Pod." = vrijednost podatkovnog ulaza.)

Građa: 3 invertora i 8 vrata AND s 4 ulaza. Svaka vrata AND dobivaju podatkovni ulaz i svoju kombinaciju S signala. Za S = 101 otvorena su samo vrata za O5 (S0, S̄1, S2), pa samo O5 može postati 1.

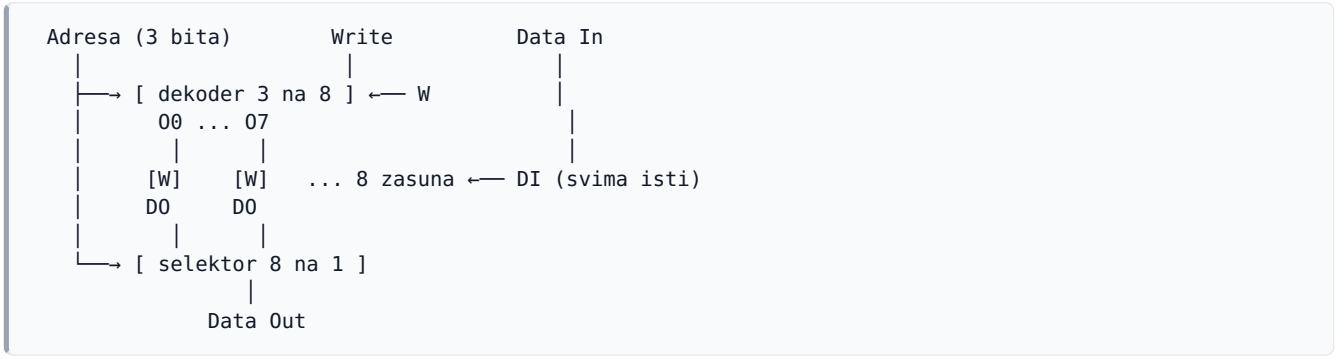
Analogija: dekoder je skretničar na kolodvoru. Jedan vlak (signal W) dolazi, a skretnice (S) ga usmjeravaju na točno jedan od osam kolosijeka.

Jednostavniji dekoder (2 na 4) vidjeli smo u poglavlju 11.

Sklop	Ulazi	Izlazi	Što radi
Selektor 8 na 1	8 podatkovnih + 3 izborna	1	**skuplja**: bira jedan od 8 ulaza
Dekoder 3 na 8	1 podatkovni + 3 izborna	8	**raspodjeljuje**: šalje ulaz na jedan od 8 izlaza

Sve zajedno: adresa i RAM

Sada spajamo dijelove:



Ključno: **ista tri signala** idu i u dekoder i u selektor. Ta tri signala zovu se **adresa**.

- Na **ulaznoj** strani adresa određuje koji zasun prima signal W (kamo se piše).
- Na **izlaznoj** strani adresa određuje koji se zasun vidi na *Data Out* (odakle se čita).

Analogija (iz knjige): adresa je kao broj poštanskog pretinca. Broj određuje u koji pretinac se ubacuje pošta i iz kojeg se vadi.

Zašto se zove RAM

Ovaj sklop zove se **memorija za čitanje i pisanje** (*read/write memory*), a češće **memorija s nasumičnim pristupom**, **RAM** (*random access memory*).

Dio imena	Zašto
Memorija	čuva podatke
Čitanje/pisanje	u svaki zasun može se upisati nova vrijednost i pročitati spremljena
Nasumičan pristup	svakom zasunu pristupa se **izravno**, samo promjenom adrese

Suprotnost je **slijedna** (sekvencijalna) memorija: da bi pročitao vrijednost na adresi 101, moraš prvo proći kroz adresu 100. Kod RAM-a to nije potrebno.

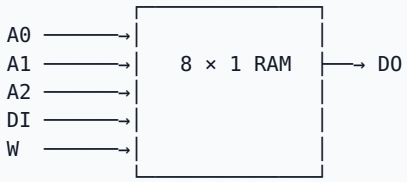
Analogija: RAM je kao knjiga s kazalom (otvoriš točno stranicu koju trebaš), a slijedna memorija kao vrpca (moraš premotati sve prije).

Oznaka RAM polja

Konkretna izvedba RAM-a zove se **RAM polje** (*RAM array*). Opisuje se oznakom **broj vrijednosti × broj bitova po vrijednosti**:

- **8 × 1** ("osam puta jedan"): 8 vrijednosti, svaka 1 bit.

- Ukupan broj bitova = umnožak: 8 × 1 = 8 bitova.



Primjer čitanja i pisanja u 8 × 1 RAM:

1. Želimo upisati 1 na adresu 6: postavimo A2 A1 A0 = **110**, DI = 1, pa W na 1 i natrag na 0.
2. Želimo upisati 0 na adresu 3: A = **011**, DI = 0, W: 1 → 0.
3. Želimo pročitati adresu 6: postavimo A = 110 i gledamo DO. Pokazuje **1**. W ostaje 0.

Spajanje RAM polja u veća

Širenje vrijednosti: 8 × 1 + 8 × 1 = 8 × 2

Dva polja 8 × 1 dijele **iste adresne linije i isti W**, a svako ima svoj DI i DO. Kad se upiše adresa, oba upisuju istovremeno, svako svoj bit. Rezultat je **8 × 2 RAM**: 8 vrijednosti, svaka od 2 bita.

Povećanje broja vrijednosti: 8 × 1 + 8 × 1 = 16 × 1

Druga mogućnost: dodamo **dekoder 1 na 2** na ulaz i **selektor 2 na 1** na izlaz. Novi izborni signal bira **koje od dva polja** se koristi.

Taj novi signal je zapravo **četvrta adresna linija** (A3). Rezultat je **16 × 1 RAM**: 16 vrijednosti, svaka 1 bit.

Način spajanja	Što se dijeli	Rezultat
Paralelno	adresa i W zajednički	više bitova po vrijednosti (8 × 2)
Preko dekodera i selektora	jedna nova adresna linija	više vrijednosti (16 × 1)

Formula za broj vrijednosti

Broj vrijednosti u RAM polju ovisi samo o broju adresnih linija:

> **broj vrijednosti = 2^(broj adresnih linija)**

Adresne linije	Broj vrijednosti
0 (obični zasun)	1
1	2
2	4
3	8
4	16
10	1024
16	65.536

Svaka nova adresna linija **udvostručuje** memoriju.

Primjer: polje **1024 × 8**

- 10 adresnih linija (2^{10} = 1024),

- 8 linija DI i 8 linija DO,
- ukupno 1024 × 8 = **8192 bita**, odnosno **1024 bajta**.

Analogija: pošta s 1024 poštanska pretinca. Svaki ima svoj broj i u svakom je jedan bajt.

Kilobajt, megabajt, gigabajt

Zašto kilobajt nije 1000 bajtova

1024 bajta zove se **kilobajt**. Ovdje nastaje zabuna:

- Prefiks *kilo* (grč. *khilioi*, tisuću) u metričkom sustavu znači **1000**: kilogram = 1000 grama, kilometar = 1000 metara.
- U memoriji kilobajt znači **1024** bajta.

Razlog: metrički sustav temelji se na potencijama broja **10**, a memorija na potencijama broja **2** (jer adresne linije udvostručuju memoriju). Nijedna cjelobrojna potencija broja 10 nije jednaka potenciji broja 2, ali neke su **blizu**:

> $2^{10} = 1024 \approx 10^3 = 1000$

To je samo slučajna sličnost brojeva, ali omogućuje praktičan naziv.

- Kratice: **K** ili **KB** (1K, 1 KB).
- **Ispravno je reći:** 1 KB je 1024 bajta, dakle **više** od 1000.

Tablica jedinica

Jedinica	Kratica	Točno (bajtova)	Potencija	Približno
kilobajt	KB	1.024	2^{10}	10^3 (tisuća)
megabajt	MB	1.048.576	2^{20}	10^6 (milijun)
gigabajt	GB	1.073.741.824	2^{30}	10^9 (milijarda)
terabajt	TB	1.099.511.627.776	2^{40}	10^{12} (bilijun)
petabajt	PB	1.125.899.906.842.624	2^{50}	10^{15}
eksabajt	EB	1.152.921.504.606.846.976	2^{60}	10^{18}

Podrijetlo naziva: grč. *megas* = velik, *gigas* = div, *teras* = čudovište.

Svaka sljedeća jedinica je **1024 puta** veća od prethodne: 1 MB = 1024 KB, 1 GB = 1024 MB.

Udvostručavanje memorije

Svaka dodatna adresna linija udvostručuje memoriju:

Memorija	Bajtova	Adresnih linija	Raspon adresa (heks)
1 KB	1.024	10	000h – 3FFh
2 KB	2.048	11	000h – 7FFh
4 KB	4.096	12	000h – FFFh
16 KB	16.384	14	0000h – 3FFFh
32 KB	32.768	15	0000h – 7FFFh
64 KB	65.536	16	0000h – FFFFh
1 MB	1.048.576	20	00000h – FFFFFh

Kako ljudi govore

- "Imam 64K" znači 65.536 bajtova; "32 mega" znači 33.554.432 bajta; "giga" znači 1.073.741.824 bajta.
- Kad knjiga nastaje (1999.), kućna računala imaju obično **32 MB ili 64 MB**, ponekad 128 MB RAM-a.

Bitovi ili bajtovi

Ponekad se govori o **kilobitima** i **megabitima** (*bits*, ne *bytes*). To je rijetko i uglavnom kod **prijenosa podataka** žicom: "kilobita u sekundi", "megabita u sekundi". Na primjer, modem od 56K prenosi **56 kilobita** u sekundi, a ne 56 kilobajta.

> **bitovi = bajtovi × 8**

Memorija od 64 KB

Pretpostavimo da smo složili RAM polje **64K × 8**:



- **16 adresnih linija**: adresa je točno 2 bajta, od **0000h do FFFFh**.
- 8 podatkovnih ulaza i 8 izlaza: svaka adresa čuva jedan bajt.
- Zašto 64 KB? Jer je 65.536 = 2^{16} "okrugao" broj u binarnom svijetu. Oko 1980. godine to je bila uobičajena količina memorije u osobnim računalima.

Koliko releja? Naš D-zasun treba otprilike 9 releja po bitu. 64K × 8 = 524.288 bitova, pa je to oko 4,7 milijuna, **gotovo 5 milijuna releja**. Zato ga nitko ne bi gradio od releja, ali princip je isti.

Upravljačka ploča

Za ručni rad s memorijom zamislimo **upravljačku ploču**:

Element	Broj	Uloga
Adresni prekidači A0–A15	16	biraju adresu
Podatkovni prekidači D0–D7	8	određuju bajt koji se upisuje
Prekidač *Write*	1	upisuje bajt na adresu
Žaruljice	8	pokazuju bajt spremljen na odabranoj adresi
Prekidač *Takeover* (preuzimanje)	1	daje upravljanje ploči ili drugim sklopovima

Prekidač *Takeover*:

- **0:** ostali prekidači na ploči ne rade ništa. Adresu, podatke i *Write* daju **drugi sklopovi** (vanjski signali).
- **1:** ploča ima **isključivo** upravljanje memorijom.

Izvedba: **25 selektora 2 na 1** (16 za adresu + 8 za podatke + 1 za *Write*). Signal *Takeover* je njihov izborni ulaz. Izlaz memorije (DO) ide na žaruljice i, po potrebi, dalje u druge sklopove.

Rad s pločom (Takeover = 1):

1. Postavi 16 adresnih prekidača na željenu adresu (bilo koju od 65.536).
2. Žaruljice pokazuju bajt koji je trenutno na toj adresi.
3. Za upis: postavi 8 podatkovnih prekidača i uključi pa isključi *Write*.

Važno: ostavili smo mogućnost da **neki drugi sklop** koristi memoriju, čita vrijednosti iz nje i upisuje nove. To je temelj sljedećeg poglavlja.

Nepostojanost: RAM traži struju

Od poglavlja 11 releje ne crtamo pojedinačno, pa je lako zaboraviti da je **svaki relej spojen na izvor struje**. Kad je relej uključen, struja teče kroz elektromagnet i drži kontakt.

Ako se memoriji isključi napajanje, svi elektromagneti izgube magnetizam, svi kontakti se vrate u početni položaj i **sav sadržaj nestaje**.

> RAM je **nepostojana** (*volatile*) memorija: sadržaj čuva samo dok ima napajanja.

Analogija: RAM je kao crtež na zamagljenom staklu. Postoji samo dok je staklo zamagljeno; čim se uvjeti promijene, nestane.

Ključne definicije

Pojam	Značenje
1-bitni zasun	D-bistabil s ulazima DI i W i izlazom DO; pamti 1 bit
Selektor 8 na 1	prosljeđuje jedan od 8 ulaza na izlaz prema 3 izborna signala
Dekoder 3 na 8	šalje ulaz na jedan od 8 izlaza; ostali su 0
Adresa	skup signala koji bira mjesto u memoriji za čitanje ili pisanje
RAM	memorija s nasumičnim pristupom: svakoj adresi pristupa se izravno
RAM polje m × n	m vrijednosti po n bitova; ukupno m · n bitova
Kilobajt (KB)	1024 bajta = 2^{10}
Megabajt (MB)	1024 KB = 2^{20} bajtova
Gigabajt (GB)	1024 MB = 2^{30} bajtova
Takeover	prekidač koji upravljanje memorijom daje ploči ili drugim sklopovima
Nepostojana memorija	gubi sadržaj kad nestane napajanja

Ključno za zapamtiti

1. D-zasun je 1 bit memorije: upis je W = 1 pa 0, a DO pokazuje spremljeni bit.
2. Selektor čita iz jednog od više zasuna; dekoder upisuje u jedan od više zasuna.
3. Isti adresni signali vode i selektor i dekoder.
4. RAM omogućuje izravan pristup bilo kojoj adresi, bez prolaska kroz ostale.
5. Broj vrijednosti = 2^(broj adresnih linija); svaka nova linija udvostručuje memoriju.
6. Polje m × n ima m · n bitova.
7. 1 KB = 1024 bajta, 1 MB = 1024 KB, 1 GB = 1024 MB.
8. Brzine prijenosa obično se mjere u bitovima u sekundi, a memorija u bajtovima.
9. 64 KB memorije ima 16-bitnu adresu: 0000h–FFFFh.
10. RAM je nepostojan: bez struje gubi sadržaj.

Najčešće zabune

- **"1 KB = 1000 bajtova."** U memoriji je 1 KB = **1024** bajta.
- **"Modem od 56K prenosi 56 kilobajta u sekundi."** Prenosi 56 **kilobita**; kilobajta je 8 puta manje.
- **"Selektor i dekoder su isto."** Selektor bira **jedan od mnogo ulaza**; dekoder šalje **jedan ulaz** na jedan od mnogo izlaza.
- **"Za čitanje treba aktivirati W."** Za čitanje je dovoljno postaviti adresu. W služi samo za pisanje.
- **"Spajanjem dva polja 8 × 1 uvijek dobiješ 16 × 1."** Ovisi o načinu: sa zajedničkom adresom dobiješ 8 × 2, a s dodatnom adresnom linijom 16 × 1.
- **"RAM pamti i nakon isključenja."** Ne: RAM je nepostojan.

Brza provjera razumijevanja

1. Koliko adresnih linija treba RAM polje od 4096 vrijednosti? Koliko bitova ima polje 4096 × 8?
2. Na selektoru 8 na 1 izborni ulazi su S2 S1 S0 = 011. Koji ulaz je na izlazu i koja su vrata AND "otvorena"?
3. Zašto se svi DI ulazi zasuna smiju spojiti zajedno, a W ulazi ne smiju?
4. Koji je raspon adresa (u heksu) za memoriju od 16 KB?
5. Što radi prekidač *Takeover* i zašto je potreban?

17. Automatizacija: od zbrajala do računala

Što ćeš naučiti u ovom poglavlju

- Kako se zbrajalo s akumulatorom poveže s **RAM-om** i **brojilom** da samo zbraja niz brojeva.
- Što su **instrukcije** i **operacijski kodovi** (Load, Store, Add, Subtract, Halt...).
- Kako se oduzima, i kako se zbrajaju **višebajtni** brojevi pomoću **zastavice prijenosa**.
- Zašto instrukcija treba sadržavati **adresu** i kako se odvijaju **dohvat** i **izvršavanje** instrukcije.
- Kako kod i podaci mogu dijeliti **istu memoriju**.
- Što radi instrukcija **skoka** (Jump) i zašto je **uvjetni skok** ono što od stroja čini računalo.
- Kako stroj **množi** pomoću petlje.
- Osnovne pojmove: **procesor (CPU), ALU, akumulator, programsko brojilo, hardver, softver, strojni jezik, asembler, mnemonik, bug**.

Ovo je najvažnije poglavlje prve polovice knjige. Na njegovu kraju imamo stroj koji se s pravom zove **računalo**. Gradimo ga u koracima, a svaki korak rješava problem prethodnog.

Korak 1: automatsko zbrajalo

Polazimo od zbrajala s akumulatorom iz poglavlja 14: 8-bitno zbrajalo, 8-bitni zasun i prekidači. Brojeve unosimo ručno, jedan po jedan, a zasun čuva tekući zbroj.

Od sada pretpostavljamo da su svi zasuni **okidani bridom** (spremaju vrijednost u trenutku prijelaza signala sata iz 0 u 1). S njima je lakše raditi.

> **Akumulator** je zasun koji čuva međurezultat: najprije jedan broj, a zatim taj broj uvećan ili umanjen za drugi broj.

Problem ručnog zbrajanja: ako zbrajaš 100 brojeva i pogriješiš u jednom, moraš sve ponoviti.

Rješenje: brojeve prvo upiši u **RAM** (poglavlje 16). Pogrešku u memoriji lako je ispraviti. Zatim stroj neka sam čita brojeve jedan za drugim.

Građa automatskog zbrajala



- **Brojilo** (poglavlje 14) daje adrese RAM-u: 0000h, 0001h, 0002h...
- **Izlaz RAM-a** ide na jedan ulaz zbrajala (umjesto prekidača).
- **Izlaz zasuna** ide na drugi ulaz zbrajala i na žaruljice.
- **Oscilator** daje signal sata i brojilu i zasunu.
- Prekidač **Clear** briše zasun i brojilo.

Kako se koristi

1. Uključi **Clear**: zasun = 00h, brojilo = 0000h.

2. Uključi **Takeover** na upravljačkoj ploči RAM-a i upiši brojeve od adrese 0000h. Za 100 brojeva to su adrese 0000h do 0063h. Sve ostale adrese postavi na 00h.
3. Isključi **Takeover** (ploča više ne upravlja memorijom) i isključi **Clear**.
4. Gledaj žaruljice.

Što se događa

Na adresi 0000h je prvi broj, a zasun je 00h. Zbrajalo daje prvi broj + 0.

Pri svakom prijelazu sata iz 0 u 1 istovremeno se dogode **dvije stvari**:

- zasun spremi zbroj iz zbrajala,
- brojilo se poveća za 1 i adresira sljedeći broj.

Prijelaz sata	Zasun nakon prijelaza	Brojilo nakon prijelaza
1.	prvi broj	0001h
2.	prvi + drugi	0002h
3.	prvi + drugi + treći	0003h

Pretpostavka: oscilator je **dovoljno spor** da releji stignu proraditi i da se na izlazu zbrajala pojavi točan zbroj prije sljedećeg prijelaza.

Problemi automatskog zbrajala

Problem	Objašnjenje
Ne može se zaustaviti	Nakon zadnjeg broja zbraja nule; kad brojilo dođe do FFFFh, prelazi na 0000h (kao brojač kilometara) i ponovno zbraja sve
Samo 8 bitova	I brojevi i zbroj ograničeni su na 255
Ne oduzima	Osim ako se koristi komplement dvojke; tada je raspon −128 do 127
Samo jedan zbroj	Ne može zbrojiti 50 parova i dati 50 rezultata
Rezultat samo na žaruljicama	Rezultat se ne sprema natrag u memoriju

Udvostručenje svih dijelova (16-bitno zbrajalo, zasun, RAM) riješilo bi problem veličine, ali je skupo. Knjiga redom rješava ostale probleme.

Korak 2: instrukcije Load, Add, Store, Halt

Želimo više zbrojeva i da se rezultati **spremaju u RAM**. Zato:

- žaruljice na zasunu nisu potrebne (rezultat ćemo gledati preko upravljačke ploče RAM-a),
- **izlaz zasuna spaja se na ulaz podataka (DI) RAM-a**, da bi se zbroj mogao upisati,
- potrebno je upravljati signalom **Write** RAM-a.

Primjer zadatka

Zbroji tri broja, zatim dva broja. Podaci u memoriji (vlastiti primjer):

Adresa	Sadržaj	Napomena
0000h	15h	
0001h	2Ch	
0002h	07h	
0003h	—	ovdje ide prvi zbroj

Adresa	Sadržaj	Napomena
0004h	40h	
0005h	3Ah	
0006h	—	ovdje ide drugi zbroj

(Prazno mjesto ne znači da je memorija prazna. Memorija **uvijek** sadrži nešto, makar slučajne vrijednosti, ali ništa korisno.)

Stroj sada mora raditi **četiri različite stvari**:

Operacija	Značenje
Load (učitaj)	prenesi bajt iz memorije u akumulator
Add (zbroji)	dodaj bajt iz memorije sadržaju akumulatora
Store (spremi)	upiši sadržaj akumulatora u memoriju
Halt (zaustavi)	zaustavi stroj

Za naš primjer redoslijed je: Load 0000h, Add 0001h, Add 0002h, Store 0003h, Load 0004h, Add 0005h, Store 0006h, Halt.

Operacijski kodovi

Stroj ne razumije riječi. Svaku operaciju treba označiti **brojem**. Ti brojevi mogu biti bilo koji, dogovorom. Knjiga bira:

Operacija	Kod
Load	10h
Store	11h
Add	20h
Halt	FFh

> **Instrukcijski kod** ili **operacijski kod** (*opcode*) je broj koji "naređuje" sklopovima da izvedu određenu operaciju.

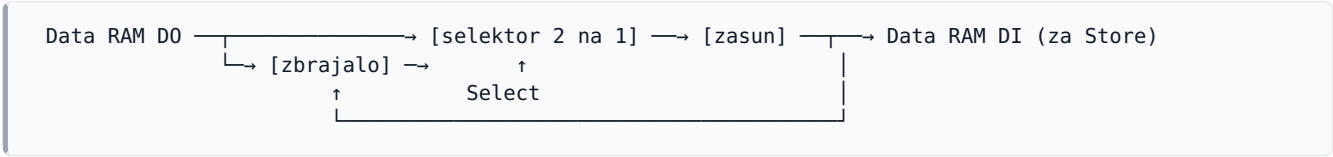
Najjednostavnije (ali ne i najjeftinije) rješenje: kodove držimo u **drugom RAM polju**, nazvanom **Code**, a brojeve u prvom, nazvanom **Data**. Isto brojilo adresira oba polja istovremeno. Kod na adresi *n* kaže što učiniti s podatkom na adresi *n*.

Adresa	Code	Data	Rezultat rada
0000h	10h (Load)	15h	akumulator = 15h
0001h	20h (Add)	2Ch	akumulator = 41h
0002h	20h (Add)	07h	akumulator = 48h
0003h	11h (Store)	—	upisuje 48h
0004h	10h (Load)	40h	akumulator = 40h
0005h	20h (Add)	3Ah	akumulator = 7Ah
0006h	11h (Store)	—	upisuje 7Ah
0007h	FFh (Halt)		stop

Code RAM se puni samo preko upravljačke ploče; u Data RAM stroj može i sam pisati (Store).

Što se mijenja u sklopu

Za **Load** u zasun mora ići **izlaz RAM-a**, a za **Add izlaz zbrajala**. Zato se ispred zasuna stavlja **selektor 2 na 1**.



Upravljački signali

Sve što upravlja dijelovima stroja zove se **upravljački signali** (*control signals*):

- Clock i Clear brojila,
- Clock i Clear zasuna,
- Write za Data RAM,
- Select za selektor 2 na 1.

Ovi signali ovise o operacijskom kodu iz Code RAM-a. Na primjer:

- Select mora biti 0 (izlaz RAM-a) kad je kod **Load**,
- Write mora biti 1 **samo** kad je kod **Store**.

Knjiga kaže da se upravljački signali dobivaju raznim kombinacijama logičkih vrata, ali ih ne crta. Točna izvedba upravljačke logike: **U knjizi nije detaljno objašnjeno.**

Korak 3: oduzimanje (Subtract)

Uz malo dodatnog hardvera stroj može i oduzimati. Dodajemo kod:

Operacija	Kod
Load	10h
Store	11h
Add	20h
Subtract	**21h**
Halt	FFh

Kodovi Add (20h = 0010 000**0**) i Subtract (21h = 0010 000**1**) razlikuju se samo u **najnižem bitu**, koji se zove **C_o**. Taj bit radi dvije stvari:

1. upravlja **invertorom** na izlazu Data RAM-a (kao u poglavlju 13): ako je $C_o = 1$, broj se invertira prije zbrajala,
2. postavlja **ulazni prijenos (CI)** zbrajala na C_o.

Invertiranje i dodavanje 1 daje komplement dvojke, pa se oduzimanje svodi na zbrajanje.

Primjer (vlastiti): 5Bh + 34h − 29h

Korak	Instrukcija	Račun	Akumulator
1	Load 5Bh		5Bh
2	Add 34h	5Bh + 34h	8Fh
3	Subtract 29h	29h se invertira u D6h; 8Fh + D6h + 1 = 166h	66h (prijenos se zanemaruje)
4	Store		66h u memoriju

Provjera: 91 + 52 − 41 = 102 = 66h. ✓

Korak 4: višebajtni brojevi i zastavica prijenosa

8-bitni podaci su ozbiljno ograničenje. Do sada je jedino rješenje bilo udvostručiti sve dijelove. Postoji jeftinije rješenje: 16-bitni broj zbrojiti **u dva koraka**, bajt po bajt.

- **Niži bajt** (*low-order byte*): desni bajt broja.
- **Viši bajt** (*high-order byte*): lijevi bajt broja.

Primjer bez prijenosa: 3512h + 2347h

```
Niži bajtovi:  12h + 47h = 59h
Viši bajtovi:  35h + 23h = 58h
Rezultat:      5859h
```

Ovo radi samo ako zbrajanje nižih bajtova **ne daje prijenos**.

Primjer s prijenosom: 4A9Ch + 17E5h

```
Niži bajtovi:  9Ch + E5h = 181h  →  zapiši 81h, prijenos 1
Viši bajtovi:  4Ah + 17h + 1 = 62h
Rezultat:      6281h
```

Provjera: 19.100 + 6.117 = 25.217 = 6281h. ✓

Prijenos iz prvog zbrajanja mora se **sačuvati** i dodati u drugom. Čuva se u 1-bitnom zasunu koji se zove **zasun prijenosa** (*Carry latch*). Njegova vrijednost zove se **zastavica prijenosa** (*carry flag*).

Nove instrukcije

Operacija	Kod	Što radi
Load	10h	učitaj u akumulator
Store	11h	spremi akumulator
Add	20h	zbroji (ulazni prijenos = 0)
Subtract	21h	oduzmi (ulazni prijenos = 1)
Add with Carry	**22h**	zbroji + vrijednost zasuna prijenosa
Subtract with Borrow	**23h**	oduzmi uz vrijednost zasuna prijenosa kao ulazni prijenos
Halt	FFh	zaustavi

Pravila rada:

- Broj iz RAM-a **invertira se** za Subtract i Subtract with Borrow.
- **Izlazni prijenos** zbrajala ide u zasun prijenosa. Zasun prijenosa sprema vrijednost samo pri Add, Subtract, Add with Carry i Subtract with Borrow.
- **Ulazni prijenos** zbrajala je 1 ako se izvodi Subtract, ili ako je zasun prijenosa = 1 i izvodi se Add with Carry ili Subtract with Borrow.

Postupak za 16-bitno zbrajanje:

1. Niže bajtove zbroji s **Add**. Prijenos se spremi u zasun prijenosa.
2. Više bajtove zbroji s **Add with Carry**. Ako je bilo prijenosa, doda se 1; ako nije, doda se 0.

Add with Carry koristi se **uvijek** za više bajtove, bez obzira je li prijenos stvarno nastao. Tako program radi za sve brojeve.

Za oduzimanje višebajtnih brojeva: niže bajtove oduzmi sa Subtract, a više sa Subtract with Borrow. Kod običnog Subtracta izlazni prijenos 1 je normalan i obično se zanemaruje; kod višebajtnog oduzimanja on se čuva i koristi u sljedećem koraku.

Ponavljanjem Add with Carry mogu se zbrajati brojevi od **16, 24, 32, 40 i više bitova**. Za 32-bitni broj: jedan Add i tri Add with Carry.

Preostali problemi

- Brojevi nisu u uzastopnim adresama. 16-bitni broj dijeli se po memoriji (npr. niži bajt na 0000h, viši na 0003h), jer svaka instrukcija u Code RAM-u odgovara podatku na **istoj adresi** u Data RAM-u.
- Rezultat jednom spremljen (Store) **ne može se ponovno učitati**, jer brojilo uvijek ide naprijed. Ako rezultat treba kasnije, mora se ponovno izračunati.

Korak 5: instrukcija sadrži adresu

Ključna promjena: svaka instrukcija (osim Halt) sada zauzima **3 bajta**:

Bajt	Sadržaj
1.	operacijski kod
2.	viši bajt 16-bitne adrese
3.	niži bajt 16-bitne adrese

Adresa kaže **gdje u Data RAM-u** je podatak:

- za **Load**: odakle se učitava,
- za **Add, Subtract, Add with Carry, Subtract with Borrow**: gdje je broj koji se dodaje ili oduzima,
- za **Store**: kamo se sprema sadržaj akumulatora.

Primjer: zbroji bajt na 0000h i bajt na 0001h, rezultat na 0002h.

Adresa (Code)	Bajtovi	Značenje
0000h	10h 00h 00h	Load bajt s adrese 0000h u akumulator
0003h	20h 00h 01h	Add bajt s adrese 0001h
0006h	11h 00h 02h	Store akumulator na adresu 0002h
0009h	FFh	Halt

Sada podaci mogu biti **bilo gdje** u 64 KB. 16-bitni brojevi mogu biti zapisani prirodno, s višim bajtom na nižoj adresi. Primjer: 4A9Ch + 17E5h s podacima od 4000h:

Adresa (Data)	Sadržaj
4000h	4Ah (viši bajt 1. broja)
4001h	9Ch (niži bajt 1. broja)
4002h	17h (viši bajt 2. broja)
4003h	E5h (niži bajt 2. broja)
4004h	viši bajt rezultata
4005h	niži bajt rezultata

Program:

Adresa (Code)	Bajtovi	Značenje
0000h	10h 40h 01h	Load bajt s 4001h

Adresa (Code)	Bajtovi	Značenje
0003h	20h 40h 03h	Add bajt s 4003h
0006h	11h 40h 05h	Store na 4005h
0009h	10h 40h 00h	Load bajt s 4000h
000Ch	22h 40h 02h	Add with Carry bajt s 4002h
000Fh	11h 40h 04h	Store na 4004h
0012h	FFh	Halt

Niži bajtovi zbrajaju se prvi, a viši s Add with Carry. Nakon izvođenja na 4004h–4005h je 62h 81h. Bez Halta, neki kasniji izračun mogao bi koristiti i ulazne brojeve i rezultat, jednostavno navođenjem njihovih adresa.

Dohvat i izvršavanje

Izlaz Code RAM-a sada ide u **tri 8-bitna zasuna**:

- 1. zasun: operacijski kod,
- 1. zasun: viši bajt adrese,
- 1. zasun: niži bajt adrese.

Izlazi 2. i 3. zasuna zajedno čine 16-bitnu adresu za Data RAM.

Pojam	Značenje
Dohvat instrukcije (*instruction fetch*)	čitanje instrukcije iz memorije; ovdje 3 bajta, jedan po jedan, dakle 3 ciklusa sata
Izvršavanje (*execute*)	izvođenje radnje koju instrukcija opisuje; ovdje 1 ciklus
Instrukcijski ciklus	dohvat + izvršavanje; ovdje ukupno 4 ciklusa sata

Stroj nije "živ" i ništa ne "razumije". Svaki kod samo pokreće određenu kombinaciju upravljačkih signala.

TANSTAAFL

Stroj je sada svestraniji, ali i **sporiji**: s istim oscilatorom zbraja **četiri puta sporije** od prvog automatskog zbrajala (4 ciklusa po instrukciji umjesto 1).

> **TANSTAAFL** (*There Ain't No Such Thing As A Free Lunch*, "nema besplatnog ručka"): kad stroj poboljšaš u jednom pogledu, obično nešto drugo trpi.

Korak 6: kod i podaci u istoj memoriji

Dva RAM polja od 64 KB bila bi najveći dio stroja. Sada kad instrukcija sadrži adresu podatka, **dva polja više nisu potrebna**. Kod i podaci mogu biti u **istom** RAM-u.

Dodaje se **selektor 2 na 1** za adresu RAM-a:

- pri **dohvatu** instrukcije adresa dolazi iz **brojila**,
- pri **izvršavanju** adresa dolazi iz **zasuna** s adresom iz instrukcije.

Ako sve stane u adrese 0000h–03FFh, za početak je dovoljan i 1 KB memorije.

Primjer (vlastiti): zbroji dva broja i oduzmi treći; podaci od 0010h.

Adresa	Bajtovi	Značenje
0000h	10h 00h 10h	Load bajt s 0010h
0003h	20h 00h 11h	Add bajt s 0011h

Adresa	Bajtovi	Značenje
0006h	21h 00h 12h	Subtract bajt s 0012h
0009h	11h 00h 13h	Store na 0013h
000Ch	FFh	Halt
…		
0010h	5Bh	
0011h	34h	
0012h	29h	
0013h	—	rezultat (bit će 66h)

Instrukcije počinju na 0000h jer brojilo nakon poništavanja kreće od 0000h. Podaci mogu biti bilo gdje osim u prvih 13 bajtova (tamo su instrukcije).

Korak 7: skok (Jump)

Problem

Želimo rezultatu dodati još dva broja. Nove instrukcije stavljamo od 0020h, a nove podatke od 0030h:

Adresa	Bajtovi	Značenje
0020h	10h 00h 13h	Load rezultat s 0013h
0023h	20h 00h 30h	Add bajt s 0030h
0026h	20h 00h 31h	Add bajt s 0031h
0029h	11h 00h 32h	Store na 0032h
002Ch	FFh	Halt
0030h	12h	
0031h	05h	
0032h	—	konačni rezultat

Kako stroj od 0000h može doći do 0020h? Ne možemo samo ukloniti Halt na 000Ch:

- što god stavimo umjesto njega, stroj će čitati kao **operacijski kod**,
- isto vrijedi za svaki treći bajt nakon toga (000Fh, 0012h, 0015h…), pa i za **podatke** na 0010h–0013h,
- ako neki od tih bajtova slučajno bude 11h (Store), stroj bi mogao **prebrisati** važan sadržaj,
- čak i bez toga, stroj bi nakon 001Eh čitao od 0021h, a ne od 0020h, gdje počinje sljedeća prava instrukcija.

Rješenje: instrukcija Jump

Operacija	Kod
Jump	**30h**

Normalno stroj čita memoriju **redom**. Instrukcija **Jump** mijenja taj red: sljedeća instrukcija čita se s adrese navedene u instrukciji. Zove se i **Branch** (grananje) ili **Goto** ("idi na").

Na 000Ch umjesto Halta stavimo:

```
000Ch:  30h 00h 20h     ; Jump na instrukciju na adresi 0020h
```

Sada stroj izvede Load, Add, Subtract, Store, skoči na 0020h i nastavi s Load, Add, Add, Store, Halt. Rezultat: 66h + 12h + 05h = **7Dh** na adresi 0032h.

Kako se izvodi skok

Jump mora **upisati novu vrijednost u 16-bitno brojilo**. Brojilo je građeno od D-bistabila okidanih bridom koji imaju ulaze **Preset** i **Clear** (poglavlje 14). Normalno su oba 0. Preset = 1 postavlja Q na 1, a Clear = 1 postavlja Q na 0.

Za svaki bit brojila dodaje se mali sklop sa signalom **Set It** ("postavi") i bitom adrese **A**:

Signal	Izraz
Preset	Set It **I** A
Clear	(Set It **I** **NE** A) **ILI** Reset

- Kad je Set It = 0: Preset je 0, a Clear je 0 osim ako je Reset = 1. Brojilo normalno broji.
- Kad je Set It = 1: ako je A = 1, Preset = 1, a Clear = 0; ako je A = 0, Preset = 0, a Clear = 1. Dakle **Q postaje A**.

Takav sklop treba za **svaki** od 16 bitova brojila. Nakon upisa brojilo nastavlja brojiti od nove vrijednosti. Set It smije biti 1 **samo** kad je kod 30h i kad je adresa već spremljena u zasune.

Korak 8: uvjetni skok i množenje

Skok koji se događa **uvijek** koristan je, ali još korisniji je skok koji se događa **samo ponekad**: **uvjetni skok**.

Zadatak: množenje

Kako pomnožiti dva 8-bitna broja? Primjer (vlastiti): **C8h × 0Ah** (200 × 10). Umnožak dva 8-bitna broja je 16-bitan, pa sve brojeve zapisujemo kao 16-bitne:

Adresa	Sadržaj	Značenje
2000h	00h C8h	broj koji se zbraja (200)
2002h	00h 0Ah	koliko puta se zbraja (10)
2004h	00h 00h	ovdje se gradi umnožak

Množenje je ponovljeno zbrajanje: C8h × 0Ah = C8h zbrojen 10 puta. Šest instrukcija koje **jednom** dodaju C8h 16-bitnom umnošku:

```
0000h:  10h 20h 05h   Load bajt s 2005h (niži bajt umnoška)
0003h:  20h 20h 01h   Add bajt s 2001h (niži bajt broja)
0006h:  11h 20h 05h   Store na 2005h
0009h:  10h 20h 04h   Load bajt s 2004h (viši bajt umnoška)
000Ch:  22h 20h 00h   Add with Carry bajt s 2000h
000Fh:  11h 20h 04h   Store na 2004h
```

Kako ih ponoviti 10 puta?

Mogućnost	Problem
Upisati ih još 9 puta	Broj upisa ovisi o jednom od brojeva; nepraktično
Staviti Halt i 10 puta pritisnuti Reset	Isto: ručni posao proporcionalan broju
Staviti Jump na 0000h	Ponavlja **zauvijek**, nema zaustavljanja

Treba skok koji se ponavlja **samo onoliko puta koliko treba**.

Zastavica nule

Dodaje se još jedan 1-bitni zasun: **zasun nule** (*Zero latch*). Na njegov ulaz spojena su **vrata NOR s 8 ulaza** na izlazu zbrajala. NOR daje 1 samo ako su **svi** ulazi 0.

> **Zastavica nule** (*Zero flag*) = 1 ako je rezultat zbrajala **jednak nuli**, a 0 ako nije.

Pazi: izgleda "naopako". Zastavica je **1** kada je rezultat **0**.

Kao i zasun prijenosa, zasun nule sprema vrijednost samo pri Add, Subtract, Add with Carry i Subtract with Borrow. **Store i Load ne mijenjaju zastavice.**

Instrukcije uvjetnog skoka

Operacija	Kod	Skače ako je
Load	10h	
Store	11h	
Add	20h	
Subtract	21h	
Add with Carry	22h	
Subtract with Borrow	23h	
Jump	30h	uvijek
Jump If Zero	**31h**	zastavica nule = 1
Jump If Carry	**32h**	zastavica prijenosa = 1
Jump If Not Zero	**33h**	zastavica nule = 0
Jump If Not Carry	**34h**	zastavica prijenosa = 0
Halt	FFh	

Izvedba je samo dodatak upravljanju običnim skokom: za Jump If Not Zero signal Set It aktivira se samo ako je zastavica nule 0.

Cijeli program za množenje

Iza šest instrukcija dodajemo smanjenje brojača ponavljanja i uvjetni skok:

```
0012h:  10h 20h 03h   Load bajt s 2003h (brojač ponavljanja)
0015h:  20h 00h 1Eh   Add bajt s 001Eh (tamo je FFh)
0018h:  11h 20h 03h   Store na 2003h
001Bh:  33h 00h 00h   Jump If Not Zero na 0000h
001Eh:  FFh           Halt
```

Trik: na adresi 001Eh je instrukcija Halt, dakle bajt **FFh**. Ali FFh je i ispravan broj: u komplementu dvojke to je **−1**. Dodavanje FFh brojaču isto je što i oduzimanje 1.

Praćenje izvođenja

Prolaz	Umnožak nakon prolaza	Brojač: računanje	Zastavica nule	Što slijedi
1.	00C8h (200)	0Ah + FFh = 09h	0	skok na 0000h
2.	0190h (400)	09h + FFh = 08h	0	skok
3.	0258h (600)	08h + FFh = 07h	0	skok
…	…	…	…	…
10.	07D0h (2000)	01h + FFh = 00h	**1**	**nema skoka**, Halt

U 2. prolazu: niži bajtovi C8h + C8h = 190h, zapiše se 90h i prijenos 1; viši bajtovi 00h + 00h + 1 = 01h. Zato je umnožak 0190h.

Na kraju je na 2004h–2005h vrijednost **07D0h = 2000 = 200 × 10**. ✓

Ovo je računalo

Knjiga tvrdi: s uvjetnim skokom sastavili smo stroj koji se s pravom zove **računalo**. Primitivno, ali računalo.

> **Uvjetni skok omogućuje kontrolirano ponavljanje (petlju). Upravo to razlikuje računalo od kalkulatora.**

Ako stroj može množiti, na sličan način može i **dijeliti**. Nije ograničen na 8 bitova: može zbrajati, oduzimati, množiti i dijeliti 16-, 24-, 32-bitne i veće brojeve. Ako to može, može računati i **kvadratne korijene, logaritme i trigonometrijske funkcije**. Sve što stroj treba jest hardver za zbrajanje i oduzimanje i uvjetni skok; "ostalo se može napraviti softverom".

Digitalno i analogno

- **Digitalno** računalo radi s **diskretnim** brojevima: vrijednosti su odvojene, određene (0, 1, 2...).
- **Analogna** računala radila su s **kontinuiranim** vrijednostima koje se mijenjaju bez skokova. Danas su gotovo izumrla.

Analogija: digitalno je kao stube (stojiš na određenoj stepenici), analogno kao rampa (možeš stati bilo gdje).

Četiri glavna dijela računala

Dio	U našem stroju
Procesor	sve osim memorije i ploče
Memorija	RAM polje od 64 KB
Ulazni uređaj	prekidači na upravljačkoj ploči
Izlazni uređaj	žaruljice na upravljačkoj ploči

Dijelovi procesora

Pojam	Značenje	U našem stroju
Procesor, CPU (*central processing unit*)	središnja jedinica za obradu; ponekad se kolokvijalno zove "mozak", ali knjiga taj izraz izbjegava	sve osim RAM-a i ploče
Mikroprocesor	procesor koji je, zbog tehnologije iz poglavlja 18, vrlo malen	naš relejni procesor to nije
8-bitni procesor	akumulator i većina putova podataka su 8-bitni	da; samo je adresa 16-bitna
Akumulator	zasun koji drži broj unutar procesora	8-bitni zasun
ALU (*Arithmetic Logic Unit*)	aritmetičko-logička jedinica	invertor + 8-bitno zbrajalo; radi samo aritmetiku (zbrajanje, oduzimanje)
Programsko brojilo (*Program Counter*)	drži adresu sljedeće instrukcije	16-bitno brojilo

Adresa je 16-bitna jer bi s 8 bitova memorija bila ograničena na 256 bajtova umjesto 65.536. U složenijim računalima ALU izvodi i **logičke** operacije (AND, OR, XOR).

Hardver, softver, program

Pojam	Značenje
Hardver	fizički dijelovi: releji, žice, prekidači, žaruljice
Softver	instrukcije i brojevi u memoriji; "mekan" jer se mijenja mnogo lakše od hardvera
Program (računalni program)	niz instrukcija; u praksi gotovo sinonim za softver

Pojam	Značenje
Programiranje	pisanje programa (npr. smišljanje niza instrukcija za množenje)
Kod / podaci	instrukcije / brojevi s kojima instrukcije rade
Kodiranje, koderi	razgovorni izrazi za programiranje i programere; neki više vole naziv *softverski inženjer*

Granica između koda i podataka nije uvijek oštra: u programu za množenje bajt FFh bio je i instrukcija Halt i broj −1.

Strojni jezik i mnemonici

Strojni kod ili **strojni jezik** su operacijski kodovi koje procesor razumije (10h, 11h...). "Jezik" jer je, poput ljudskog jezika, nešto što stroj "razumije" i na što reagira.

Dugačke nazive zamjenjujemo **mnemonicima**: kratkim oznakama od 2–3 velika slova.

Operacija	Kod	Mnemonik
Load	10h	LOD
Store	11h	STO
Add	20h	ADD
Subtract	21h	SUB
Add with Carry	22h	ADC
Subtract with Borrow	23h	SBB
Jump	30h	JMP
Jump If Zero	31h	JZ
Jump If Carry	32h	JC
Jump If Not Zero	33h	JNZ
Jump If Not Carry	34h	JNC
Halt	FFh	HLT

Pravila zapisa

```
LOD A,[2003h]      ; učitaj u A bajt s adrese 2003h
ADD A,[001Eh]      ; dodaj A-u bajt s adrese 001Eh
STO [2003h],A      ; spremi A na adresu 2003h
JNZ 0000h          ; skoči na 0000h ako zastavica nule nije 1
```

- Ono iza mnemonika su **argumenti**.
- **Odredište je lijevo, izvor desno.** A znači akumulator.
- **Uglate zagrade** znače "**sadržaj** memorije na toj adresi", a ne sam broj adrese.
- Kod JNZ nema zagrada: skače se **na adresu** 0000h, ne na vrijednost spremljenu na njoj.
- Adresa instrukcije piše se lijevo s dvotočkom: `0000h: LOD A,[2005h]`.
- Podaci: `2000h: 00h, C8h` znači 00h na 2000h i C8h na 2001h.

Cijeli program za množenje:

```
0000h:  LOD A,[2005h]
        ADD A,[2001h]
        STO [2005h],A

        LOD A,[2004h]
        ADC A,[2000h]
        STO [2004h],A

        LOD A,[2003h]
        ADD A,[001Eh]
        STO [2003h],A

        JNZ 0000h

001Eh:  HLT

2000h:  00h, C8h
2002h:  00h, 0Ah
2004h:  00h, 00h
```

Prazni retci (*white space*) služe samo čitljivosti.

Oznake i komentari

Brojčane adrese u programu su nespretne: ako podatke premjestiš, moraš mijenjati mnoge naredbe. Bolje je koristiti **oznake** (*labels*): riječi koje označavaju mjesta u memoriji. **Komentari** se pišu iza točke sa zarezom i služe ljudima.

```
BEGIN:   LOD A,[RESULT + 1]
         ADD A,[NUM1 + 1]      ; zbroji niže bajtove
         STO [RESULT + 1],A

         LOD A,[RESULT]
         ADC A,[NUM1]          ; zbroji više bajtove s prijenosom
         STO [RESULT],A

         LOD A,[NUM2 + 1]
         ADD A,[NEG1]          ; smanji brojač za 1
         STO [NUM2 + 1],A

         JNZ BEGIN

NEG1:    HLT

NUM1:    00h, C8h
NUM2:    00h, 0Ah
RESULT:  00h, 00h
```

- NUM1, NUM2 i RESULT označavaju mjesta gdje su spremljena po 2 bajta.
- `NUM1 + 1` znači "drugi bajt" od NUM1.
- Oznaka **NEG1** (*negative one*) stoji na instrukciji HLT, jer je taj bajt i broj −1.

Asembler

Ovakav zapis zove se **asemblerski jezik** (*assembly language*). To je kompromis između golih brojeva strojnog koda i dugih opisnih rečenica, uz simboličke oznake adresa.

> Svaka naredba asemblerskog jezika odgovara **točno određenim bajtovima** strojnog koda. To su dva načina gledanja na istu stvar.

Postupak rada:

1. Napiši program na papiru u asemblerskom jeziku.
2. Kad je spreman, **ručno asembliraj** (*hand assemble*): svaku naredbu pretvori u bajtove strojnog koda.
3. Prekidačima upiši strojni kod u RAM.
4. **Pokreni program** (*run*): pusti stroj da izvodi instrukcije.

Bugovi

Bug je pogreška u programu. U strojnom kodu lako se pogriješi:

- Ako upišeš 11h (Store) umjesto 10h (Load), stroj ne samo da neće učitati broj, nego će ga **prebrisati** onim što je u akumulatoru.
- **Jump** na mjesto gdje nema ispravne instrukcije, ili **Store** koji prebriše instrukcije, mogu izazvati **nepredvidive** posljedice.

I naš program za množenje ima bug. Ako se pokrene **dvaput**, drugi put je brojač na 2003h već 00h. Dodavanje FFh daje FFh, a ne 0, pa petlja traje **256 puta** umjesto 10. Rezultat se ne zbraja ispočetka, nego se na već izračunati umnožak doda još 256 × C8h (ograničeno na 16 bitova).

Algoritmi

Softver za složenije zadatke može biti vrlo složen. Postupci kojima programeri rješavaju određene probleme zovu se **algoritmi**. Do sada smo radili samo s cijelim brojevima; decimalni razlomci dolaze u poglavlju 23.

Povijesna napomena

Sav hardver za ovaj stroj postojao je prije više od sto godina, ali stroj tada vjerojatno ne bi bio sagrađen:

- mnogi pojmovi iz njegova dizajna nisu bili očiti dok se sredinom 1930-ih nisu počela graditi relejna računala, a shvaćeni su tek oko 1945.,
- dotad su se gradila računala koja **interno koriste dekadske** brojeve, a ne binarne,
- programi nisu uvijek bili spremljeni u memoriji, nego ponekad na **papirnatoj vrpci**,
- memorija je bila skupa i glomazna; 64 KB od pet milijuna releja bilo bi apsurdno i tada i danas.

Releje su zamijenili elektronički uređaji (vakuumske cijevi, tranzistori), a procesor i memorija danas stanu na dlan. To je tema sljedećeg poglavlja.

Ključne definicije

Pojam	Značenje
Akumulator	zasun u procesoru koji drži međurezultat
Operacijski kod (opcode)	broj koji označava instrukciju (npr. 10h = Load)
Instrukcija	naredba procesoru: kod + eventualno adresa
Upravljački signali	signali koji upravljaju brojilom, zasunima, RAM-om i selektorima
Zastavica prijenosa	1-bitni zasun s izlaznim prijenosom zadnje aritmetičke operacije
Zastavica nule	1 ako je rezultat zadnje aritmetičke operacije 0
Dohvat / izvršavanje	čitanje instrukcije iz memorije / izvođenje njezine radnje
Skok (Jump, Branch, Goto)	promjena adrese sljedeće instrukcije
Uvjetni skok	skok samo ako je zastavica u određenom stanju
Petlja	kontrolirano ponavljanje dijela programa
CPU	središnja jedinica za obradu (procesor)

Pojam	Značenje
ALU	aritmetičko-logička jedinica
Programsko brojilo	registar s adresom sljedeće instrukcije
Strojni jezik	brojčani kodovi instrukcija koje procesor izvodi
Asemblerski jezik	zapis strojnog koda mnemonicima i oznakama
Bug	pogreška u programu

Ključno za zapamtiti

1. Brojilo adresira memoriju, a akumulator skuplja rezultat: to je automatsko zbrajalo.
2. Operacijski kodovi govore stroju **što** napraviti; adresa u instrukciji govori **s čim**.
3. Oduzimanje = invertiranje broja + ulazni prijenos 1; Add i Subtract razlikuju se u jednom bitu koda.
4. Višebajtni brojevi zbrajaju se bajt po bajt: niži s Add, viši s Add with Carry.
5. Instrukcija od 3 bajta (kod + 16-bitna adresa) omogućuje da podaci budu bilo gdje i da kod i podaci dijele istu memoriju.
6. Instrukcijski ciklus = dohvat + izvršavanje; svestranost se plaća brzinom (TANSTAAFL).
7. Jump upisuje novu adresu u programsko brojilo preko ulaza Preset i Clear.
8. Zastavice prijenosa i nule mijenjaju samo aritmetičke instrukcije, ne Load i Store.
9. Uvjetni skok omogućuje petlje; to razlikuje računalo od kalkulatora.
10. Asemblerski jezik i strojni kod su isto, zapisano na dva načina.

Najčešće zabune

- **"Zastavica nule je 1 kad rezultat nije nula."** Obrnuto: zastavica nule je 1 kad **je** rezultat nula.
- **"Store mijenja zastavice."** Ne: samo Add, Subtract, Add with Carry i Subtract with Borrow.
- **"[1003h] i 1003h znače isto."** [1003h] je **sadržaj** memorije na toj adresi; 1003h je sama adresa.
- **"Halt se može jednostavno obrisati da program nastavi dalje."** Stroj bi sljedeće bajtove (pa i podatke) čitao kao instrukcije; treba Jump.
- **"Add with Carry treba samo ako je bilo prijenosa."** Koristi se uvijek za više bajtove; ako prijenosa nije bilo, dodaje se 0.
- **"Procesor razumije instrukcije."** Kod samo aktivira kombinaciju upravljačkih signala; stroj ništa ne "razumije".

Brza provjera razumijevanja

1. Zašto prvo automatsko zbrajalo ne može samo stati, i što se dogodi kad brojilo dođe do FFFFh?
2. Napiši korake (Load/Add/Subtract/Store) i izračunaj rezultat za 7Ah + 26h − 1Fh. Kako se izvodi oduzimanje u sklopu?
3. Zbroji 16-bitne brojeve 58F0h i 2A35h bajt po bajt. Koju vrijednost ima zastavica prijenosa nakon zbrajanja nižih bajtova?
4. Zašto je za program množenja ključna instrukcija Jump If Not Zero, a ne obični Jump?
5. Prevedi u bajtove strojnog koda: `LOD A,[3010h]`, `SUB A,[3011h]`, `STO [3012h],A`, `HLT`.

18. Od abakusa do čipova

Što ćeš naučiti u ovom poglavlju

- Glavne korake u razvoju računskih strojeva, od abakusa do prvih računala (samo ono što je tehnički važno).
- Zašto je **prijenos** središnji problem svakog stroja za zbrajanje.
- Što je **von Neumannova arhitektura** (koncept pohranjenog programa) i što je **von Neumannovo usko grlo**.
- Kako su releje zamijenile **elektronske cijevi**, a njih **tranzistori**; što je **poluvodič**.
- Što je **integrirani krug (čip)**, **Mooreov zakon** i razine integracije (SSI, MSI, LSI, VLSI...).
- Kako se čitaju podaci o čipovima obitelji **TTL** (serija 7400): napon napajanja, logičke razine, **vrijeme propagacije**.
- Razliku između obitelji **TTL** i **CMOS**.
- Tri osnovna svojstva mikroprocesora: **širina podataka, brzina sata, adresabilna memorija**.

Poglavlje je velikim dijelom povijesno. Životopisi su ovdje svedeni na najkraće; zadržane su ideje i brojke koje objašnjavaju kako je tehnologija napredovala.

Rani pomagači računanju

Pomagalo / ideja	Tko, kada	Ključna ideja
Kamenčići, zrna, računske ploče	razne kulture	brojanje predmetima
Abakus (okvir s kuglicama)	u Kinu oko 1200.	položaj kuglica predstavlja znamenke
Logaritmi	John Napier (1550.–1617.)	množenje se svodi na zbrajanje
Logaritamsko računalo (šiber)	Gunter, Oughtred (17. st.)	logaritamska ljestvica; zadnji proizveden 1976., ugasio ga je džepni kalkulator
Napierovi štapići	Napier	pomagalo za množenje od trakica s brojevima
Prvi mehanički kalkulator	Wilhelm Schickard, oko 1620.	automatizirani Napierovi štapići
Mehanički kalkulatori	Pascal, Leibniz (17. st.)	kotačići, zupčanici i poluge

Logaritmi: princip

> **Logaritam umnoška jednak je zbroju logaritama.**

Postupak množenja s tablicom logaritama:

1. U tablici pronađi logaritme oba broja.
2. Zbroji ih.
3. Obrnutim korištenjem tablice pronađi broj čiji je to logaritam. To je umnožak.

Tablice logaritama stoljećima su izrađivali vrhunski matematičari.

Prijenos: središnji problem

U 8-bitnom zbrajalu i u računalu iz poglavlja 17 prijenos je stalno stvarao dodatni posao. Kod strojeva za zbrajanje prijenos nije sitnica nego **glavni problem**: stroj koji radi sve osim prijenosa daleko je od gotovog.

Primjer iz povijesti: Pascalov mehanizam prijenosa **onemogućavao je oduzimanje**. Za oduzimanje se morao dodavati komplement devetke (poglavlje 13). Mehanički kalkulatori koje su ljudi stvarno mogli koristiti pojavili su se tek krajem 19. stoljeća.

Jacquardov tkalački stan

Joseph Marie Jacquard (oko 1801.) napravio je tkalački stan kojim upravljaju **metalne kartice s rupama** (slično kao kod pianole). Rupe određuju uzorak tkanja. Važno za računala: ideja da se stroju **zadaju naredbe bušenim karticama**.

"Računalo" je bio čovjek

Do 1940-ih riječ *computer* značila je **osobu** koja za plaću računa brojeve, npr. za tablice logaritama ili trigonometrijske tablice za pomorsku navigaciju. Pogreške su mogle nastati u svakom koraku, od računanja do slaganja slova za tisak.

Charles Babbage: strojevi 19. stoljeća

Charles Babbage (1791.–1871.), britanski matematičar, htio je ukloniti pogreške iz matematičkih tablica.

Tablice se tada nisu računale izravno za svaki redak. Izračunale su se vrijednosti za odabrane brojeve, a međuvrijednosti su se dobivale **interpolacijom** pomoću jednostavnih računa zvanih **diferencije** (*differences*).

Stroj	Što je	Sudbina
Diferencijski stroj (*Difference Engine*), od oko 1820.	veliki mehanički stroj za zbrajanje koji automatizira izradu tablica, sve do slaganja slova za tisak; višeznamenkasti dekadski brojevi na zupčanicima s 10 položaja; negativni brojevi u komplementu desetke	nikad dovršen; Babbage odustaje 1833. Diferencijski stroj su 1853. sagradili Georg i Edvard Scheutz
Analitički stroj (*Analytical Engine*)	najbliže računalu što je 19. st. dalo	radio na njemu do smrti; sagrađeni samo modeli i dijelovi

Analitički stroj imao je:

- **skladište** (*store*): odgovara današnjoj **memoriji**,
- **mlin** (*mill*): **aritmetička jedinica**; množenje ponovljenim zbrajanjem, dijeljenje ponovljenim oduzimanjem,
- programiranje **karticama** preuzetim od Jacquardova stana.

Ada Byron, grofica od Lovelacea (1815.–1852.), u bilješkama uz prijevod članka o stroju usporedila je da analitički stroj "tka algebarske uzorke" kao što Jacquardov stan tka cvijeće i lišće. Babbage je vjerojatno **prvi shvatio važnost uvjetnog skoka**. Ada Byron opisala je **ciklus**: skup operacija koji se ponavlja više puta, dvaput ili neodređeno mnogo puta; ciklusi mogu biti i ugniježđeni (ciklus ciklusa).

Babbageovi strojevi bili su zaboravljeni do 1930-ih. Tada je tehnologija već nadmašila sve što je on napravio; ostala je njegova vizija automatizacije.

Hollerith i bušene kartice

Popis stanovništva u SAD-u 1880. obrađivao se oko **7 godina**. Bojeći se da će popis 1890. trajati više od deset godina, ured za popis odabrao je strojeve **Hermana Holleritha** (1860.–1929.).

Kartica

- Kartica s **24 stupca po 12 položaja = 288 položaja**.

- Popisivač je bušio četvrtaste rupe na mjestima koja opisuju osobu.

Kartica **nije** spremala 288 bitova, jer kodiranje nije bilo čisto binarno:

Podatak	Hollerithov način	Čisto binarni način
Spol	2 položaja (jedan za muški, jedan za ženski)	1 položaj (rupa ili ne)
Dob	2 rupe: prva bira raspon od 5 godina (0–4, 5–9, 10–14…), druga jedan od 5 položaja unutar raspona; ukupno 28 položaja	7 položaja za dob 0–127

Zašto ne binarno:

- pretvaranje dobi u binarni broj bilo je previše za popisivače,
- čisto binarna kartica mogla bi imati gotovo sve rupe izbušene, pa bi bila **krhka**.

Strojevi

- **Tabulator** (stroj za prebrojavanje): operater pritisne ploču s **288 opružnih iglica**. Gdje je rupa, iglica prođe, dotakne posudicu sa **živom** i zatvori strujni krug. Elektromagnet poveća dekadsko brojilo.
- **Sorter** (razvrstavač): isti pritisak, ali elektromagneti otvaraju poklopac jednog od **26 pretinaca**.

Rezultat: obrađeno više od **62 milijuna kartica**, dvostruko više podataka nego 1880., u otprilike **trećini vremena**.

Od Holleritha do IBM-a

Godina	Događaj
1896.	Hollerith osniva Tabulating Machine Company
1911.	spajanjem nastaje Computing-Tabulating-Recording Company (C-T-R)
1924.	Thomas J. Watson mijenja ime u **International Business Machines (IBM)**
1928.	IBM-ova kartica s **80 stupaca i 12 redaka**, u upotrebi više od 50 godina ("Hollerithove kartice")

Napomena o "digitalnom svijetu"

Knjiga se usredotočuje na digitalne izume (telegraf, Braille, Babbageovi strojevi, Hollerithova kartica). Ali 19. stoljeće obilježili su i mnogi izumi koji **nisu** digitalni. I većina prirodnog svijeta koji opažamo osjetilima nije digitalna, nego **kontinuum** koji se ne može lako izraziti brojevima.

Elektromehanička računala (releji)

Iako je Hollerith koristio releje, računala od releja (**elektromehanička računala**) počela su se graditi tek sredinom 1930-ih. Koristili su se releji iz **telefonskog** sustava (za usmjeravanje poziva), ne telegrafski.

Razlike prema našem računalu iz poglavlja 17:

- nisu sva koristila **binarne** brojeve (danas se to podrazumijeva, tada nije),
- nitko nije gradio 524.288 bitova memorije od releja; mala memorija služila je samo za **međurezultate**,
- programi su bili na **fizičkom mediju**, npr. papirnatoj vrpci s rupama. Unos koda i podataka u istu memoriju je moderniji pojam.

Tko	Kada	Što
Konrad Zuse	od 1935.	stroj u Berlinu; binarni brojevi, ali u ranim verzijama mehanička memorija; program na bušenoj 35-mm filmskoj vrpci
George Stibitz (Bell Labs)	1937.	1-bitno zbrajalo od telefonskih releja na kuhinjskom stolu ("K stroj", K od *kitchen*); iz toga 1939. Complex Number Computer

Tko	Kada	Što
Howard Aiken, Harvard + IBM	1943.	**Harvard Mark I** (ASCC): prvo digitalno računalo koje je ispisivalo tablice; Mark II: najveći relejni stroj, **13.000 releja**

Prvi "bug"

Releji su mehanički: savijaju komadiće metala, pa se nakon dugog rada mogu slomiti, a zaprljanje ili nešto zaglavljeno među kontaktima može izazvati kvar. Godine 1947. iz releja računala Harvard Mark II izvađen je **moljac**. Grace Murray Hopper zalijepila ga je u dnevnik računala uz bilješku "first actual case of bug being found" (prvi stvarni slučaj pronađenog *buga*).

Elektronske cijevi

Elektronsku (vakuumsku) cijev razvili su John Ambrose Fleming i Lee de Forest za radio. Do 1940-ih koristile su se za pojačanje u telefoniji i u radio-prijemnicima.

> Cijevi se mogu spojiti u vrata AND, OR, NAND i NOR, kao i releji. Zato **sve** što smo naučili o relejnim sklopovima vrijedi i za cijevi: vrata se uvijek mogu složiti u zbrajala, selektore, dekodere, bistabile i brojila.

	Relej	Elektronska cijev
Vrijeme preklapanja	oko **1 ms** (tisućinka sekunde)	oko **1 µs** (milijuntinka sekunde)
Prednost	pouzdan princip	**1000 puta brži**
Mane	mehaničko trošenje, prljavština	skupe, troše mnogo struje, griju se, **pregorijevaju**

Problem pregorijevanja: računalo ima toliko cijevi da statistički neka pregori svakih nekoliko minuta, a kvar se ne mora odmah primijetiti. (Telefonski sustav imao je mnogo redundancije, pa ondje to nije bilo važno.)

Zanimljivo: u ranim računalima brzina cijevi nije bila presudna, jer je ukupnu brzinu ograničavalo **čitanje programa s papirnate ili filmske vrpce**.

Od ranih 1940-ih cijevi zamjenjuju releje; do 1945. prijelaz je završen. Relejni strojevi su **elektromehanička** računala, a strojevi s cijevima prva **elektronička** računala.

Računalo / rad	Kada	Bitno
Colossus (Velika Britanija)	1943.	razbijanje njemačkog šifrirnog stroja Enigma
Alan Turing: članak o izračunljivosti	1937.	što računala mogu, a što ne mogu izračunati; apstraktni model danas zvan **Turingov stroj**
Turing: članak o umjetnoj inteligenciji		test strojne inteligencije, danas **Turingov test**
ENIAC (Eckert i Mauchly)	kraj 1945.	18.000 cijevi, oko 30 tona; najveće računalo po masi

Von Neumannova arhitektura

John von Neumann sudjelovao je u dizajnu nasljednika ENIAC-a, računala **EDVAC**. U radu iz 1946. (s Burksom i Goldstineom) opisane su značajke koje su EDVAC učinile velikim napretkom:

1. Računalo interno koristi **binarne** brojeve (ENIAC je koristio dekadske).
2. Računalo ima što više **memorije**, a ta memorija čuva **i kod programa i podatke** dok se program izvodi. (ENIAC se programirao prekidačima i kabelima.)
3. Instrukcije su **slijedno** smještene u memoriji i adresiraju se **programskim brojilom**, ali su mogući i **uvjetni skokovi**.

> Ovaj dizajn zove se **koncept pohranjenog programa** (*stored-program concept*), a danas **von Neumannova arhitektura**.

Računalo iz poglavlja 17 je klasičan von Neumannov stroj.

Von Neumannovo usko grlo (*bottleneck*): takav stroj troši mnogo vremena samo na **dohvat instrukcija** iz memorije. U računalu iz poglavlja 17 dohvat je zauzimao **tri četvrtine** vremena svake instrukcije.

Analogija: kuhar koji za svaki korak recepta mora otići u drugu sobu pročitati sljedeći redak. Većinu vremena hoda, a ne kuha.

Rane vrste memorije

Vrsta	Kako radi	Kapacitet
Memorija sa živinom linijom kašnjenja (*mercury delay line*)	cijev sa živom duga oko 1,5 m (5 stopa); impulsi razmaknuti oko 1 µs putuju kroz živu oko 1 ms, na drugom kraju se detektiraju (kao zvučni valovi) i vraćaju na početak	oko 1024 bita po cijevi
Memorija s magnetskim jezgrama (*core*), sredina 1950-ih	veliki nizovi malih magnetiziranih metalnih prstenova nanizanih na žice; svaki prsten čuva 1 bit	—

Stariji programeri dugo su memoriju procesora zvali "*core*", i nakon što su jezgre zamijenjene drugim tehnologijama.

Ostali važni mislioci 1940-ih

Tko	Doprinos
Claude Shannon	1938.: veza prekidača, releja i Booleove algebre (poglavlje 11); 1948.: "Matematička teorija komunikacije", prva tiskana upotreba riječi **bit** i temelj **teorije informacije** (prijenos digitalne informacije uz šum i kako ga nadoknaditi)
Norbert Wiener	1948.: **kibernetika** (grč. kormilar): veza bioloških procesa i mehanike računala i robota

Prva komercijalno dostupna računala: **UNIVAC** (završen 1951., prvi isporučen Uredu za popis stanovništva; 1952. na televiziji predviđao rezultate predsjedničkih izbora) i IBM 701 (1952.).

Tranzistor

Bell Labs (osnovan 1925.) imao je zadatak razvijati tehnologije za telefonski sustav. Jedan vječni cilj bio je **pojačanje** glasovnih signala bez izobličenja. Cijevi su za to bile jedino rješenje, ali velike, gladne struje i pregorijevale su.

Dana **16. prosinca 1947.** John Bardeen i Walter Brattain napravili su novo pojačalo od germanija (poluvodiča) i trake zlatne folije, i tjedan dana kasnije pokazali ga šefu Williamu Shockleyju. To je bio prvi **tranzistor**. Shockley, Bardeen i Brattain dobili su 1956. Nobelovu nagradu za fiziku.

Vodiči, izolatori, poluvodiči

Vrsta	Građa	Primjer
Vodič	**jedan** elektron u vanjskoj ljusci atoma; lako se otkine i slobodno kreće kao struja	bakar, srebro, zlato (isti stupac periodnog sustava)
Izolator	struju gotovo ne vodi	guma, plastika
Poluvodič	**četiri** elektrona u vanjskoj ljusci (pola najvećeg broja); čist poluvodič stvara vrlo stabilne veze, kristalnu strukturu sličnu dijamantu, i **ne vodi dobro**	germanij, silicij

Naziv "poluvodič" ne znači da vodi upola dobro, nego da se njegova **vodljivost može mijenjati**.

Dopiranje (*doping*): dodavanje određenih nečistoća poluvodiču.

- **N-tip** (N od *negative*): nečistoća dodaje **višak elektrona** u odnosu na one potrebne za veze.
- **P-tip**: druga vrsta nečistoće.

NPN tranzistor

Ako se P-tip poluvodiča stavi **između dva N-tipa**, nastaje **NPN tranzistor**. Tri dijela zovu se:

Dio	Uloga
Kolektor (*collector*)	prema napajanju
Baza (*base*)	upravljački ulaz
Emiter (*emitter*)	izlaz struje

> **Mali napon na bazi upravlja mnogo većim naponom (strujom) od kolektora prema emiteru.** Bez napona na bazi tranzistor je praktički isključen.

Analogija: tranzistor je slavina. Mali pokret ručice (baza) upravlja jakim mlazom vode (kolektor → emiter). To je isto ono što radi relej, ali bez pokretnih dijelova.

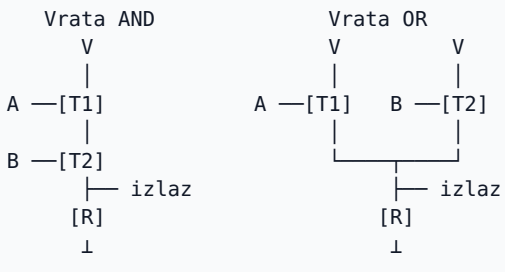
Tranzistor je započeo **poluvodičku elektroniku** (*solid-state*): nema vakuuma, sve je u čvrstom materijalu, danas najčešće siliciju. Prednosti pred cijevima: **mnogo manji**, troše mnogo **manje struje**, stvaraju mnogo **manje topline**, **traju dulje**.

Godina	Primjena tranzistora
prva komercijalna	slušni aparat
1954.	džepni tranzistorski radio (tranzistori Texas Instruments)
1956.	prva tranzistorska računala; Shockley osniva tvrtku u Palo Altu, začetak **Silicijske doline**
1960.	prvi tranzistorski televizor

Logička vrata od tranzistora

Kao i releji, tranzistori se mogu spojiti u vrata:

- **Vrata AND:** dva tranzistora u **seriji** (emiter gornjeg na kolektor donjeg). Izlaz je 1 samo kad oba vode, tj. kad su A i B oba 1. Otpornik sprječava kratki spoj.
- **Vrata OR:** kolektori oba tranzistora spojeni na napajanje, emiteri međusobno spojeni (**paralelno**).



Sve što smo naučili o relejnim vratima vrijedi i za tranzistore.

Tranzistori ne olakšavaju gradnju

Tranzistori stanu u manji prostor, ali **spojeve** među njima i dalje treba ožičiti. Kad bi se računalo iz poglavlja 17 s 64 KB RAM-a gradilo od pojedinačnih tranzistora, većina posla bilo bi spajanje milijuna veza.

Integrirani krug (čip)

Neke kombinacije tranzistora stalno se ponavljaju: parovi čine vrata, vrata čine bistabile, zbrajala, selektore, dekodere; bistabili čine zasune i RAM. Bilo bi lakše kad bi tranzistori **unaprijed bili spojeni** u te česte sklopove.

Tko	Kada	Što
Geoffrey Dummer	svibanj 1952.	predviđa elektroničku opremu "u čvrstom bloku bez spojnih žica"
Jack Kilby (Texas Instruments)	srpanj 1958.	više tranzistora, otpornika i drugih dijelova na jednom komadu silicija
Robert Noyce (Fairchild)	siječanj 1959.	ista ideja, neovisno; prvi dobiva patent

Kilby i Noyce smatraju se suizumiteljima **integriranog kruga** (*integrated circuit*, **IC**), popularno **čipa**.

- Izrada: tanke silicijske pločice slažu se u slojeve, precizno dopiraju i jetkaju na određenim mjestima da nastanu mikroskopski dijelovi.
- Razvoj novog čipa je skup, ali **masovna proizvodnja** ga pojeftinjuje.

Kućište DIP

Silicijski čip je tanak i osjetljiv, pa se pakira u kućište. Najčešće je **DIP** (*dual inline package*): pravokutno plastično kućište s 14, 16 ili do 40 nožica sa strane.

Numeriranje nožica (gledano odozgo, s udubljenjem lijevo): počinje **dolje lijevo** s 1 i ide **u krug** suprotno od kazaljke sata, završava gore lijevo (npr. 16 kod 16-nožičnog čipa). Razmak nožica je točno 1/10 inča.

Mooreov zakon

Gordon Moore je 1965. primijetio da se broj tranzistora na čipu **udvostručuje svake godine** od 1959. Stvarni trend bio je nešto sporiji, pa je **Mooreov zakon** izmijenjen u:

> **Broj tranzistora na čipu udvostručuje se otprilike svakih 18 mjeseci.**

Primjer računa: za 18 godina ima 12 razdoblja od 18 mjeseci, pa je porast $2^{12} \approx$ **4000 puta**.

Razine integracije

Naziv	Kratica	Broj logičkih vrata na čipu
Small-scale integration	SSI	manje od 10
Medium-scale integration	MSI	10 – 100
Large-scale integration	LSI	100 – 5.000
Very-large-scale integration	VLSI	5.000 – 50.000
Super-large-scale integration	SLSI	50.000 – 100.000
Ultra-large-scale integration	ULSI	više od 100.000

Obitelji čipova: TTL i CMOS

Tehnologija izrade čipova zove se **obitelj** (*family*). Sredinom 1970-ih prevladavale su dvije: **TTL** i **CMOS**.

TTL i serija 7400

TTL = *transistor-transistor logic*. Standardni priručnik bio je *The TTL Data Book for Design Engineers* (Texas Instruments, 1973.), s opisom **serije 7400**: svaki čip ima oznaku koja počinje s 74.

Svaki čip serije 7400 sadrži logička vrata **unaprijed spojena** u određenu konfiguraciju: neki jednostavna vrata, drugi bistabile, zbrajala, selektore, dekodere.

Čip 7400 ("četverostruka vrata NAND s 2 ulaza, pozitivna logika"): četiri neovisna vrata NAND s po 2 ulaza i 1 izlazom, 14 nožica.

Nožica	Oznaka	Značenje
14	**V_CC**	napajanje (isto što i naš simbol V); dvostruko slovo u indeksu označava izvor napajanja, C se odnosi na kolektor tranzistora
7	**GND**	uzemljenje (*ground*)
ostale	1A, 1B, 1Y…	ulazi A, B i izlaz Y svakih vrata

Svaki čip mora biti spojen na napajanje i zajedničko uzemljenje.

Napajanje i logičke razine

- **V_CC = 5 V ± 5 %**, tj. između **4,75 V i 5,25 V**. Ispod 4,75 V čip možda neće raditi; iznad 5,25 V može se oštetiti. Baterije obično nisu dovoljno točne; TTL treba napajanje iz zidne utičnice.

Ulaz u stvarnosti može biti bilo koji napon od 0 do 5 V. TTL ga tumači ovako:

Napon na ulazu	Tumačenje
0 – 0,8 V	logička **0** (*low*, L)
0,8 – 2 V	**izbjegavati** (neodređeno)
2 – 5 V	logička **1** (*high*, H)

Izlaz TTL vrata	Tipičan napon
logička 0	oko 0,2 V
logička 1	oko 3,4 V

Otpornost na šum: izlaz 1 (3,4 V) može izgubiti oko 1,4 V i još biti iznad 2 V, dakle 1. Izlaz 0 (0,2 V) može dobiti 0,6 V i još biti ispod 0,8 V, dakle 0. Ta rezerva štiti od **šuma**.

Pozitivna i negativna logika: zbog odstupanja napona govori se o razinama *low* i *high*, ne 0 i 1. U **pozitivnoj** logici visoki napon je 1; u **negativnoj** logici niski napon je 1, a visoki 0. Riječ *positive* u nazivu čipa 7400 znači pozitivnu logiku.

Vrijeme propagacije

> **Vrijeme propagacije** (*propagation time*) je vrijeme potrebno da se promjena na ulazu pokaže na izlazu. To je vjerojatno **najvažniji** podatak o čipu.

Mjeri se u **nanosekundama**:

Jedinica	Kratica	Dio sekunde
milisekunda	ms	1/1.000
mikrosekunda	µs	1/1.000.000
nanosekunda	ns	1/1.000.000.000

Za vrata NAND čipa 7400 jamči se vrijeme propagacije **manje od 22 ns** (0,000000022 s).

Nanosekunda je izvan ljudskog iskustva. Primjer iz knjige: ako držiš knjigu 30 cm od lica, nanosekunda je otprilike vrijeme koje svjetlo treba da stigne od stranice do očiju.

Upravo nanosekunda omogućuje računala. Procesor radi "glupo jednostavne" stvari (premjesti bajt, zbroji dva bajta, vrati rezultat), ali ih radi **izuzetno brzo**. Kako je rekao Robert Noyce: kad se pomiriš s nanosekundom, operacije računala su pojmovno prilično jednostavne.

Neki čipovi serije 7400

Čip	Sadržaj
7400	četvora vrata NAND s 2 ulaza
7402	četvora vrata NOR s 2 ulaza
7404	šest invertora
7408	četvora vrata AND s 2 ulaza
7430	jedna vrata NAND s 8 ulaza (neke nožice: NC = *no connection*, nije spojeno)
7432	četvora vrata OR s 2 ulaza
7474	dva D-bistabila okidana pozitivnim bridom, s ulazima Preset i Clear
7483	4-bitno binarno potpuno zbrajalo
74151	selektor podataka 8 na 1
74154	dekoder 4 na 16
74161	sinkrono 4-bitno binarno brojilo
74175	četiri D-bistabila s Clear; dva takva čipa daju 8-bitni zasun

Dakle, sve komponente koje knjiga koristi od poglavlja 11 postoje kao gotovi TTL čipovi.

Tablica čipa 7474 (H = visoko, L = nisko; X = nije važno; ↑ = prijelaz iz L u H):

Pre	Clr	Clk	D	Q	$\bar{Q}$
L	H	X	X	H	L
H	L	X	X	L	H
L	L	X	X	H*	H*
H	H	↑	H	H	L
H	H	↑	L	L	H
H	H	L	X	Q_o	$\bar{Q}_o$

Razlika prema bistabilu iz poglavlja 14: tamo su Preset i Clear normalno **0**, a ovdje su normalno **1 (H)**; aktivni su kad su L. (Oznaka * označava nestabilno stanje kad su oba L. Q_o znači prethodno stanje.)

Gradnja računala od TTL čipova

- Spajanje čipova je mnogo lakše od spajanja pojedinačnih tranzistora.
- Ali za **memoriju** TTL nije bio dobar: najveći RAM čip u priručniku iz 1973. bio je **256 × 1** bit. Za 64 KB trebalo bi **2048** takvih čipova.

Oscilator: umjesto invertora s povratnom vezom bolje je koristiti **kvarcni kristal**, koji titra vrlo točnom frekvencijom, obično barem milijun puta u sekundi.

> 1 milijun ciklusa u sekundi = **1 megaherc (MHz)**.

Računalo iz poglavlja 17 od TTL čipova moglo bi raditi na **10 MHz**. Jedan ciklus traje 100 ns, a instrukcija traje 4 ciklusa, dakle **400 ns**.

CMOS

CMOS = *complementary metal-oxide semiconductor*. Serija **4000**.

Svojstvo	TTL (7400)	CMOS (4000)
Napajanje	4,75 – 5,25 V	**3 – 18 V**
Potrošnja	veća	**mnogo manja** (može raditi na baterije)
Brzina	**brz**	spor
Primjer: 4-bitno potpuno zbrajalo	24 ns	4008: 750 ns na 5 V, 250 ns na 10 V, 190 ns na 15 V

Sredinom 1970-ih izbor je bio jasan: brzina (TTL) ili mala potrošnja (CMOS). Danas postoje i TTL niske potrošnje i brzi CMOS.

Spajanje čipova u praksi

Način	Opis
Eksperimentalna pločica (*breadboard*)	plastična ploča s rupicama; svaki kratki red od 5 rupica spojen je ispod; čip se stavi preko središnjeg utora, pa je svaka nožica spojena s još 4 rupice; spaja se komadićima žice
Wire-wrap	čip u podnožju s dugim četvrtastim stupićima; posebnim pištoljem tanka izolirana žica čvrsto se omota oko stupića, a oštri rubovi probiju izolaciju
Tiskana pločica (*printed circuit board*)	pločica s rupama i bakrenim slojem; bakar koji treba ostati zaštiti se, ostatak se otopi kiselinom; komercijalne pločice imaju **više slojeva** veza

Mikroprocesor

Početkom 1970-ih cijeli procesor mogao se smjestiti na jednu pločicu s čipovima. Sljedeći korak bio je cijeli procesor na **jednom čipu**.

- Texas Instruments 1971. prijavljuje patent za računalo na jednom čipu.
- **Intel** (osnovan 1968., Noyce i Moore) prvi ga je stvarno napravio. Prvi veliki Intelov proizvod (1970.) bio je memorijski čip od **1024 bita**, tada najveći.
- Za japansku tvrtku Busicom Intel je razvijao čipove za programabilni kalkulator. Inženjer Ted Hoff predložio je drukčiji pristup: napraviti **računalo opće namjene** programirano da bude kalkulator.

Tako je nastao **Intel 4004**, prvo "računalo na čipu", **mikroprocesor** (studeni 1971.), s **2300 tranzistora**. (Po Mooreovu zakonu, 18 godina kasnije trebalo bi biti oko 4000 puta više, oko 10 milijuna, što je bila prilično točna procjena.)

Tri mjere mikroprocesora

Mjera	Značenje	Intel 4004	Za usporedbu
Širina podataka	koliko bitova procesor obrađuje odjednom	**4 bita**	poglavlje 17: 8 bitova; 16-bitni od kasnih 1970-ih; 32-bitni od sredine 1980-ih
Najveća brzina sata	najveća frekvencija oscilatora s kojom procesor ispravno radi	**108 kHz**	1999.: 500 MHz, oko 5000 puta brže
Adresabilna memorija	koliko memorije procesor može adresirati	**640 bajtova**	poglavlje 17: 64 KB; Intel 1999.: 64 TB

(kHz = kiloherc, tisuću ciklusa u sekundi.)

Mogućnosti i brzina

> Ove tri mjere **ne određuju što računalo može**, nego **koliko brzo** to može.

- 4-bitni procesor može zbrojiti 32-bitne brojeve, radeći to u 4-bitnim dijelovima.
- Ako hardver jednog procesora može nešto što drugi ne može, drugi to može **softverom**. Sva digitalna računala u tom su smislu **jednaka**. To je jedna od posljedica Turingova članka iz 1937. o izračunljivosti.
- Razlikuju se u **brzini**:
- brzina sata određuje koliko brzo se izvodi svaka instrukcija,
- širina podataka: 4-bitni procesor zbraja 32-bitne brojeve, ali ne toliko brzo kao 32-bitni,
- adresabilna memorija na prvi pogled nema veze s brzinom, ali procesor s malo memorije mora neke adrese koristiti za upravljanje drugim medijem (npr. bušiti bajtove na papirnatu vrpcu), a to sve usporava.

Tri broja samo grubo govore o brzini. Ništa ne govore o unutarnjoj arhitekturi ni o učinkovitosti strojnih instrukcija. Kako procesori napreduju, sve više poslova koji su se radili softverom ugrađuje se u sam procesor.

Brzina je ipak presudna za korisnost: računalo sporije od ljudskog mozga beskorisno je za računanje, a film se ne može gledati ako procesor treba minutu za jednu sliku.

Nakon 4004

- **Intel 8008** (travanj 1972.): 8-bitni, 200 kHz, adresira 16 KB.
- 1. u razmaku od pet mjeseci Intel i Motorola izbacuju mikroprocesore koji poboljšavaju 8008. Ta dva čipa tema su sljedećeg poglavlja.

Ključne definicije

Pojam	Značenje
Logaritam	broj čiji zbroj zamjenjuje množenje (log umnoška = zbroj logaritama)
Elektromehaničko računalo	računalo od releja
Elektroničko računalo	računalo od elektronskih cijevi (kasnije tranzistora)
Von Neumannova arhitektura	binarni brojevi, kod i podaci u istoj memoriji, slijedne instrukcije s programskim brojilom i uvjetnim skokovima
Von Neumannovo usko grlo	veliki udio vremena troši se na dohvat instrukcija
Poluvodič	materijal (germanij, silicij) čija se vodljivost može mijenjati dopiranjem
Tranzistor (NPN)	kolektor, baza, emiter; mali napon na bazi upravlja većom strujom
Integrirani krug (čip)	mnogo tranzistora i drugih dijelova na komadiću silicija
Mooreov zakon	broj tranzistora na čipu udvostručuje se oko svakih 18 mjeseci
TTL / CMOS	obitelji čipova: brza (TTL) / štedljiva (CMOS)
Vrijeme propagacije	kašnjenje od promjene ulaza do promjene izlaza
MHz	milijun ciklusa u sekundi
Mikroprocesor	cijeli procesor na jednom čipu

Ključno za zapamtiti

1. Prijenos je središnji problem strojeva za zbrajanje.
2. Babbageov analitički stroj imao je memoriju (skladište), aritmetičku jedinicu (mlin) i programiranje karticama.

3. Releji, cijevi i tranzistori mogu tvoriti ista logička vrata; logika sklopova se ne mijenja, mijenjaju se brzina, veličina i pouzdanost.
4. Cijev je oko 1000 puta brža od releja; tranzistor je manji, štedljiviji i trajniji od cijevi.
5. Von Neumannova arhitektura: binarni brojevi + kod i podaci u memoriji + programsko brojilo i uvjetni skokovi.
6. Integrirani krug rješava problem ožičenja: česti sklopovi dolaze unaprijed spojeni.
7. TTL: 5 V ± 5 %, 0 = 0–0,8 V, 1 = 2–5 V; razlika između ulaznih i izlaznih razina štiti od šuma.
8. Vrijeme propagacije (ns) je najvažniji podatak o čipu.
9. Mikroprocesor se grubo opisuje s tri broja: širina podataka, brzina sata, adresabilna memorija.
10. Sva digitalna računala mogu isto; razlikuju se u brzini.

Najčešće zabune

- **"Poluvodič vodi upola dobro."** Naziv govori da se njegova vodljivost može **mijenjati** dopiranjem.
- **"Tranzistori su učinili računala jednostavnijima za gradnju."** Manjima i pouzdanijima da, ali ožičenje je ostalo problem; riješio ga je tek integrirani krug.
- **"8-bitni procesor ne može raditi s 32-bitnim brojevima."** Može, softverski, u dijelovima; samo sporije.
- **"Kod TTL-a svaki napon iznad 0 je 1."** Između 0,8 i 2 V stanje je neodređeno i treba ga izbjegavati.
- **"Preset i Clear na čipu 7474 aktivni su kad su 1."** Na 7474 normalno su H, a aktivni su kad su L.

Brza provjera razumijevanja

1. Kako se pomoću tablice logaritama pomnože dva broja?
2. Navedi tri značajke von Neumannove arhitekture i objasni što je von Neumannovo usko grlo.
3. Ulaz TTL vrata ima 1,2 V. Kako će ga čip protumačiti? A 2,6 V?
4. Ako jedan ciklus sata traje 50 ns, kolika je frekvencija u MHz i koliko traje instrukcija od 4 ciklusa?
5. Zašto Intel 4004 može zbrojiti 32-bitne brojeve iako je 4-bitni procesor?

19. Dva klasična mikroprocesora

Što ćeš naučiti u ovom poglavlju

- Osnovne podatke o mikroprocesorima **Intel 8080** i **Motorola 6800** (1974.).
- Što znače nožice mikroprocesora: adresne, podatkovne i upravljačke.
- **Registre** 8080 (A, B, C, D, E, H, L) i **parove registara** (BC, DE, HL).
- Tri načina adresiranja: **izravno**, **indeksno** i **neposredno**.
- Glavne skupine instrukcija: prijenos (MOV, MVI, LDA, STA), aritmetika i logika, **zastavice**, pomak (rotacija), povećanje i smanjenje.
- Kako radi **stog** (PUSH, POP, pokazivač stoga).
- Kako rade **potprogrami** (CALL, RET) i **prekidi**.
- Razliku **little-endian** i **big-endian** zapisa.
- Kratko: što je slijedilo nakon 8080 i 6800 (x86, 68000, RISC, cjevovod, priručna memorija).

Mikroprocesor promatramo kao **crnu kutiju**: ne zanima nas njegova unutarnja shema (to smo vidjeli u poglavlju 17), nego kako komunicira s vanjskim svijetom i koje **instrukcije** zna izvoditi (**skup instrukcija**, *instruction set*).

Dva čipa iz 1974.

	Intel 8080	Motorola 6800
Predstavljen	travanj 1974.	kolovoz 1974.
Širina podataka	8 bitova	8 bitova
Tranzistora	oko 6000	oko 4000
Brzina sata	2 MHz	1 MHz (do 1977. i 1,5 i 2 MHz)
Adresabilna memorija	64 KB	64 KB
Kućište	40 nožica (DIP)	40 nožica (DIP)

Iste godine pojavili su se i 4-bitni TMS 1000 (Texas Instruments; kalkulatori, igračke, uređaji) i PACE (National Semiconductor), prvi 16-bitni mikroprocesor. Povijesno su ipak najvažniji 8080 i 6800.

Ovi čipovi zovu se **jednočipni mikroprocesori**, a netočno i "računala na čipu". Procesor je samo **dio** računala. Računalo treba barem i **RAM**, **ulazni uređaj**, **izlazni uređaj** i nekoliko čipova koji sve povezuju (poglavlje 21).

Silicijska pločica unutar kućišta mnogo je manja od samog kućišta: kod ranih 8-bitnih procesora manja od 6 × 6 mm. Kućište je štiti i preko nožica daje pristup njezinim ulazima i izlazima.

Nožice Intela 8080

Napajanje i sat

- 8080 traži **tri** napona napajanja: +5 V (nožica 20), −5 V (nožica 11) i +12 V (nožica 28). Nožica 2 je uzemljenje. (Intel 8085 iz 1976. pojednostavio je napajanje.)
- Treba dva usklađena signala sata od 2 MHz, φ_1 i φ_2 (nožice 22 i 15). Najlakše ih daje Intelov čip **8224** (generator signala sata) na koji se spoji kvarcni kristal od 18 MHz.

Strelica **iz** čipa na shemi znači **izlazni** signal (procesor ga postavlja, drugi čipovi reagiraju). Strelica **u** čip znači **ulazni** signal. Neke nožice su oboje.

Adresne i podatkovne nožice

Nožice	Broj	Smjer	Uloga
A_0 – A_{15}	16	izlaz	adresa memorije; 2^{16} = 65.536 bajtova
D_0 – D_7	8	**ulaz i izlaz**	pri čitanju bajta su ulazi, pri pisanju izlazi

Broj adresnih nožica izravno određuje koliko memorije procesor može adresirati.

Upravljački signali

Preostalih deset nožica su **upravljački signali**. Primjeri:

- **RESET** (ulaz): ponovno pokretanje procesora.
- **WR̄** (izlaz): procesor želi upisati bajt u RAM; odgovara ulazu *Write* RAM polja.

Dodatni upravljački signali pojavljuju se na nožicama D_0–D_7 u određenom trenutku dok čip čita instrukciju. Sustavi s 8080 obično koriste čip **8228** (sistemski kontroler) koji te signale zadržava. Upravljački signali 8080 poznati su kao vrlo nezgrapni; detalji nisu potrebni ako ne projektiraš računalo.

Što radi nakon pokretanja

1. Nakon **RESET**-a 8080 postavlja 16 nula na A_0–A_{15} i čita bajt s adrese **0000h**. To je **dohvat instrukcije**.
2. Instrukcije su duge **1, 2 ili 3 bajta** (u poglavlju 17 sve osim HLT imale su 3).
3. Neke instrukcije čitaju bajt iz memorije, neke pišu bajt u memoriju, a neke rade nešto **unutar** procesora bez RAM-a.
4. Nakon prve instrukcije procesor dohvaća drugu, i tako dalje. Zajedno one čine **program**.

Vrijeme: pri 2 MHz jedan ciklus sata traje 1 / 2.000.000 s = **500 ns**. Svaka instrukcija 8080 traje **4 do 18 ciklusa**, dakle **2 do 9 µs**.

Skup instrukcija 8080

Računalo iz poglavlja 17 imalo je 12 instrukcija. 8-bitni procesor može imati do 256 operacijskih kodova (i više, ako neki imaju 2 bajta). 8080 ima **244**. Ipak ne radi mnogo više od računala iz poglavlja 17: na primjer, za **množenje i dijeljenje** i dalje treba napisati vlastiti mali program.

Mnemonici su samo pomoć ljudima; procesor čita samo bajtove. (Knjiga je mnemonike radi jasnoće malo prilagodila u odnosu na Intelovu dokumentaciju; ovdje koristimo zapis knjige.) U zapisu `aaaa` je 16-bitna adresa, a `xx` jedan bajt podatka.

Load i Store akumulatora

8-bitni akumulator u 8080 zove se **A**, kao u poglavlju 17. Instrukcije Load i Store postoje i ovdje:

Kod	Instrukcija	Značenje
32h	STA [aaaa],A	spremi akumulator na adresu aaaa (*Store Accumulator*)
3Ah	LDA A,[aaaa]	učitaj akumulator s adrese aaaa (*Load Accumulator*)

Kod je praćen dvobajtnom adresom.

Registri

Osim akumulatora, 8080 ima još **šest 8-bitnih registara**: **B, C, D, E, H, L**.

> **Registar** je zasun unutar procesora koji čuva 8-bitnu vrijednost. Akumulator je posebna vrsta registra.

- Podaci se mogu prenositi iz memorije u registre i iz registara u memoriju.
- Ostali registri nisu svestrani kao akumulator: npr. rezultat zbrajanja uvijek ide **u akumulator**.
- Zašto H i L? H je *high*, L je *low*: dva 8-bitna registra zajedno čine 16-bitni **par registara HL** (H je viši bajt, L niži). HL se često koristi za **adresiranje memorije**.
- Isto tako B i C čine par **BC**, a D i E par **DE**.

Zašto registri? U teoriji nisu nužni (poglavlje 17 je radilo bez njih). Ali programi često istovremeno barataju s više brojeva, a to je lakše i **brže** ako su brojevi u registrima. Što rjeđe program pristupa memoriji, to je u pravilu brži.

Analogija: registri su stvari na tvom radnom stolu, a memorija je arhiva u podrumu. Ono što stalno koristiš, držiš na stolu.

MOV: prijenos između registara

Instrukcija **MOV** (*move*, premjesti) ima **63** operacijska koda, svaki od jednog bajta. Kopira sadržaj jednog registra u drugi (ili isti). **Odredište je lijevo, izvor desno.**

Sve MOV instrukcije imaju isti uzorak bitova:

> **01 ddd sss**: ddd = odredište, sss = izvor

Kod (3 bita)	Registar
000	B
001	C
010	D
011	E
100	H
101	L
110	[HL]: memorija na adresi iz HL
111	A (akumulator)

Primjer: MOV L,E → 01 101 011 = 0110 1011 = **6Bh**.

Primjer: MOV A,[HL] → 01 111 110 = **7Eh**. MOV [HL],C → 01 110 001 = **71h**.

Iz uzorka se lako dobije cijela tablica od 40h do 7Fh. Iznimke:

- Kombinacija 01 110 110 (bila bi MOV [HL],[HL]) **ne postoji**; kod **76h** je **HLT** (zaustavi).
- Neke instrukcije, poput MOV A,A, ne rade ništa korisno.

Vjerojatno se u procesoru tri bita sss koriste u **selektoru 8 na 1**, a tri bita ddd upravljaju **dekoderom 3 na 8** koji određuje koji registar sprema vrijednost (poglavlje 16).

Tri načina adresiranja memorije

Način	Primjer	Kako se dolazi do podatka
Izravno (*direct*)	LDA A,[aaaa]	adresa je zapisana u samoj instrukciji
Indeksno (*indexed*)	MOV B,[HL]	adresa je u paru registara HL (mogla je biti izračunata)
Neposredno (*immediate*)	MVI E,37h	podatak je zapisan u samoj instrukciji

Za indeksno adresiranje mogu se koristiti i parovi BC i DE, ali samo s akumulatorom:

Kod	Instrukcija	Kod	Instrukcija
02h	STAX [BC],A	0Ah	LDAX A,[BC]
12h	STAX [DE],A	1Ah	LDAX A,[DE]

MVI: neposredno punjenje

MVI (*Move Immediate*) ima 2 bajta: kod i bajt podatka koji se upiše u registar ili u memoriju na adresi HL. Uzorak: **00 ddd 110**.

Kod	Instrukcija	Kod	Instrukcija
06h	MVI B,xx	26h	MVI H,xx
0Eh	MVI C,xx	2Eh	MVI L,xx
16h	MVI D,xx	36h	MVI [HL],xx
1Eh	MVI E,xx	3Eh	MVI A,xx

Nakon `MVI E,37h` registar E sadrži 37h.

Aritmetika i logika

32 koda izvode četiri aritmetičke operacije iz poglavlja 17, a još 32 logičke operacije i usporedbu. **Akumulator je uvijek jedan od operanada i odredište rezultata.** Uzorak:

> **10 ooo sss**: ooo = operacija, sss = drugi operand (registar ili [HL], po istoj tablici kao gore)

ooo	Operacija	Kodovi	Značenje
000	ADD A,r	80h – 87h	zbroji
001	ADC A,r	88h – 8Fh	zbroji s prijenosom
010	SUB A,r	90h – 97h	oduzmi
011	SBB A,r	98h – 9Fh	oduzmi s posudbom
100	AND A,r	A0h – A7h	I po bitovima
101	XOR A,r	A8h – AFh	isključivo ILI po bitovima
110	OR A,r	B0h – B7h	ILI po bitovima
111	CMP A,r	B8h – BFh	usporedi

Primjer: SUB A,B = 10 010 000 = 90h; ADD A,[HL] = 10 000 110 = 86h.

Primjer (vlastiti): A = 6Ch, B = 2Ah. Nakon `SUB A,B` u A je **42h**.

Primjer (vlastiti): A = 40h, H = 20h, L = 3Ah, a na adresi 203Ah je 15h. `ADD A,[HL]` zbraja 40h i bajt na adresi HL (15h): u A je **55h**.

Zbrajanje 16-bitnih brojeva

ADC i SBB omogućuju rad s 16-, 24-, 32-bitnim i većim brojevima. Ako BC i DE sadrže 16-bitne brojeve, a zbroj treba u BC:

```
MOV A,C     ; niži bajt
ADD A,E
MOV C,A
MOV A,B     ; viši bajt
ADC A,D
MOV B,A
```

Niži bajtovi zbrajaju se s ADD, viši s ADC. Budući da se zbrajati može samo u akumulatoru, trebaju čak četiri MOV instrukcije. U kodu za 8080 MOV instrukcija ima mnogo.

Zastavice

Računalo iz poglavlja 17 imalo je zastavice prijenosa i nule. 8080 ima **pet** zastavica, spremljenih u 8-bitni registar **PSW** (*Program Status Word*, statusna riječ programa).

Zastavica	Postavlja se na 1 kad...
Predznak (*Sign*, S)	je najznačajniji bit rezultata 1 (rezultat je negativan)
Nula (*Zero*, Z)	je rezultat 0
Paritet (*Parity*, P)	rezultat ima **paran** broj jedinica (parni paritet); 0 za neparan. Koristi se kao gruba provjera pogrešaka, rijetko
Prijenos (*Carry*, CY)	ADD ili ADC daju prijenos, **ili** SUB ili SBB **ne** daju prijenos (drukčije nego u poglavlju 17)
Pomoćni prijenos (*Auxiliary Carry*, AC)	nastane prijenos iz nižeg u viši nibl; koristi ga samo instrukcija DAA

- **LDA, STA i MOV ne mijenjaju zastavice.** ADD, SUB, ADC i SBB ih mijenjaju.
- Zastavicu prijenosa izravno mijenjaju:

Kod	Instrukcija	Značenje
37h	STC	postavi zastavicu prijenosa na 1
3Fh	CMC	komplementiraj (obrni) zastavicu prijenosa

Logičke operacije po bitovima

AND, OR i XOR rade **po bitovima** (*bitwise*): operacija se izvodi na svakom paru bitova zasebno. To radi **ALU** (aritmetičko-logička jedinica).

Primjer (vlastiti): A = 3Ch, B = 0Fh

```
A = 0011 1100
B = 0000 1111
AND → 0000 1100 = 0Ch
OR  → 0011 1111 = 3Fh
XOR → 0011 0011 = 33h
```

CMP: usporedba

CMP (*compare*) radi isto što i SUB, ali **rezultat ne sprema**. Čemu onda? Zbog **zastavica**: one pokazuju odnos dvaju bajtova.

Nakon `CMP A,B`:

- zastavica nule = 1 ako je **A jednako** B,
- zastavica prijenosa = 1 ako je **A manje** od B.

Sadržaj A ostaje nepromijenjen.

Neposredne aritmetičke i logičke instrukcije

Svih osam operacija ima i inačicu s **neposrednim bajtom** (drugi operand je u instrukciji):

Kod	Instrukcija	Kod	Instrukcija
C6h	ADI A,xx	E6h	ANI A,xx

Kod	Instrukcija	Kod	Instrukcija
CEh	ACI A,xx	EEh	XRI A,xx
D6h	SUI A,xx	F6h	ORI A,xx
DEh	SBI A,xx	FEh	CPI A,xx

Na primjer `MVI B,40h` i `CMP A,B` mogu se zamijeniti jednom instrukcijom `CPI A,40h`.

CMA i DAA

Kod	Instrukcija	Značenje
2Fh	CMA	komplementiraj akumulator (komplement jedinice: svaka 0 postaje 1 i obrnuto)
27h	DAA	dekadska prilagodba akumulatora (*Decimal Adjust Accumulator*)

CMA primjer: 1100 0011 → 0011 1100. Isto se dobije s `XRI A,FFh`.

DAA je vjerojatno najsloženija pojedinačna instrukcija 8080; za nju postoji poseban mali dio procesora. Pomaže kod aritmetike u **BCD** zapisu (*binary-coded decimal*): svaki nibl ima vrijednost od 0000 do 1001, tj. jednu dekadsku znamenku 0–9, pa bajt sadrži dvije dekadske znamenke.

Primjer (vlastiti): zbroji BCD brojeve 58 i 67.

```
MVI A,58h   ; BCD 58
MVI B,67h   ; BCD 67
ADD A,B     ; A = BFh (nije ispravan BCD: nibl veći od 9)
DAA         ; A = 25h, zastavica prijenosa = 1
```

Rezultat se čita kao **125** (prijenos 1 i BCD 25), što je točno: 58 + 67 = 125.

Povećanje i smanjenje za 1

U poglavlju 17 oduzimali smo 1 dodavanjem FFh. 8080 ima posebne instrukcije: **povećanje** (*increment*) i **smanjenje** (*decrement*) za 1.

Registar	INR (+1)	DCR (−1)
B	04h	05h
C	0Ch	0Dh
D	14h	15h
E	1Ch	1Dh
H	24h	25h
L	2Ch	2Dh
[HL]	34h	35h
A	3Ch	3Dh

INR i DCR mijenjaju sve zastavice **osim zastavice prijenosa**.

Rotacija

Četiri instrukcije pomiču sadržaj akumulatora za **1 bit** lijevo ili desno. Mijenjaju **samo zastavicu prijenosa**.

Kod	Instrukcija	Značenje
07h	RLC	rotiraj lijevo: najviši bit ide na najniže mjesto i u zastavicu prijenosa
0Fh	RRC	rotiraj desno: najniži bit ide na najviše mjesto i u zastavicu prijenosa
17h	RAL	rotiraj lijevo **kroz** prijenos: najniži bit dobiva staru vrijednost prijenosa, prijenos dobiva najviši bit

Kod	Instrukcija	Značenje
1Fh	RAR	rotiraj desno **kroz** prijenos: najviši bit dobiva staru vrijednost prijenosa, prijenos dobiva najniži bit

Primjer (vlastiti): A = 1101 0011 (D3h), zastavica prijenosa = 0.

Instrukcija	Novi A	Prijenos
RLC	1010 0111 (A7h)	1
RRC	1110 1001 (E9h)	1
RAL	1010 0110 (A6h)	1
RAR	0110 1001 (69h)	1

Pomak ulijevo **množi** broj s 2, a pomak udesno ga **dijeli** s 2.

Stog

Nasumičan, slijedni i LIFO pristup

RAM se tako zove jer procesor može pristupiti **bilo kojoj** adresi samo navođenjem adrese, kao knjiga koju otvoriš na bilo kojoj stranici. Mikrofilm ili kazeta imaju **slijedni pristup**: do traženog mjesta moraš proći sve prije.

Ponekad je korisno memoriju koristiti na treći način. **Analogija:** hrpa tanjura. Novi tanjur stavljaš **na vrh**, a uzimaš uvijek **s vrha**. Zadnji stavljen tanjur prvi je uzet.

> **Stog** (*stack*) je memorija u kojoj se stvari slažu odozdo prema gore i uzimaju odozgo. Zove se i **LIFO** (*last in, first out*: zadnje unutra, prvo van).

Pojam	Značenje
PUSH	stavi na stog
POP	uzmi sa stoga

Čemu služi

Program koristi registre A, B i C, a mora usput obaviti drugi izračun koji također treba A, B i C. Umjesto da sam pamti gdje je u memoriji spremio vrijednosti, stavi ih na stog:

```
PUSH A
PUSH B
PUSH C
... drugi izračun slobodno koristi A, B, C ...
POP C
POP B
POP A
```

POP se mora izvesti **obrnutim redoslijedom**. Zamjena redoslijeda je bug.

Prednost: više dijelova programa može koristiti stog bez međusobnih smetnji. Ako drugi dio programa unutra spremi C, D i E, mora ih vratiti (POP E, POP D, POP C) prije nego prvi dio vrati svoje.

Kako je stog izveden

Stog je običan dio RAM-a koji se ne koristi za ništa drugo. 8080 ima poseban 16-bitni registar koji adresira taj dio: **pokazivač stoga** (*Stack Pointer*, SP).

Stvarni 8080 na stog stavlja **16-bitne** vrijednosti (parove registara):

Kod	Instrukcija	Kod	Instrukcija
C5h	PUSH BC	C1h	POP BC
D5h	PUSH DE	D1h	POP DE
E5h	PUSH HL	E1h	POP HL
F5h	PUSH PSW	F1h	POP PSW

PUSH PSW i POP PSW spremaju i vraćaju **akumulator i registar zastavica** zajedno. Za spremanje svih registara i zastavica: PUSH PSW, PUSH BC, PUSH DE, PUSH HL, a vraćanje obrnuto: POP HL, POP DE, POP BC, POP PSW.

Primjer (vlastiti): SP = 9000h, izvodi se `PUSH DE`:

1. SP se smanji na **8FFFh**.
2. Sadržaj D upiše se na adresu 8FFFh.
3. SP se smanji na **8FFEh**.
4. Sadržaj E upiše se na adresu 8FFEh.

`POP DE` (dok je SP = 8FFEh) poništava sve: E se učita s 8FFEh, SP → 8FFFh, D se učita s 8FFFh, SP → 9000h.

Stog dakle **raste prema nižim adresama**: svaki PUSH ga poveća za 2 bajta.

Problem	Opis
Prekoračenje stoga (*stack overflow*)	stog naraste toliko (npr. zbog buga) da prebriše kod ili podatke
Podbačaj stoga (*stack underflow*)	previše POP instrukcija isprazni stog prerano

S 64 KB memorije SP se može postaviti na **0000h**: prvi PUSH smanji ga na FFFFh, pa stog zauzima najviše adrese, daleko od programa koji je obično pri adresi 0000h.

16-bitne instrukcije

LXI (*Load Extended Immediate*) puni par registara ili SP dvama bajtovima iza koda:

Kod	Instrukcija
01h	LXI BC,xxxx
11h	LXI DE,xxxx
21h	LXI HL,xxxx
31h	LXI SP,xxxx

`LXI DE,1F40h` isto je što i `MVI D,1Fh` i `MVI E,40h`, ali štedi jedan bajt. Jedna od prvih instrukcija nakon pokretanja često je `0000h: LXI SP,0000h`.

Kod	Instrukcija	Kod	Instrukcija
03h	INX BC	0Bh	DCX BC
13h	INX DE	1Bh	DCX DE
23h	INX HL	2Bh	DCX HL
33h	INX SP	3Bh	DCX SP

INX/DCX povećavaju i smanjuju par registara ili SP kao 16-bitnu vrijednost.

DAD zbraja 16-bitni par registara s HL (od zastavica mijenja samo prijenos). Obično služi za računanje adresa:

Kod	Instrukcija
09h	DAD HL,BC

Kod	Instrukcija
19h	DAD HL,DE
29h	DAD HL,HL
39h	DAD HL,SP

`DAD HL,BC` zamjenjuje šest instrukcija (MOV A,L / ADD A,C / MOV L,A / MOV A,H / ADC A,B / MOV H,A).

Ostale korisne instrukcije:

Kod	Instrukcija	Značenje
22h	SHLD [aaaa],HL	spremi HL: L na aaaa, H na aaaa + 1
2Ah	LHLD HL,[aaaa]	učitaj HL s adrese aaaa
E9h	PCHL PC,HL	učitaj programsko brojilo iz HL (vrsta skoka)
F9h	SPHL SP,HL	učitaj pokazivač stoga iz HL
E3h	XTHL HL,[SP]	zamijeni HL i dva bajta na vrhu stoga
EBh	XCHG HL,DE	zamijeni DE i HL

Skokovi, potprogrami i prekidi

Skokovi

Programsko brojilo (PC) sadrži adresu sljedeće instrukcije. Normalno instrukcije idu redom; **Jump** (ili *Branch*, *Goto*) puni PC drugom vrijednošću. **Uvjetni** skok skače ovisno o zastavici. Upravo uvjetni skok pretvorio je zbrajalo iz poglavlja 17 u računalo opće namjene.

8080 ima **devet** instrukcija skoka: bezuvjetni i uvjetni prema zastavicama nule, prijenosa, pariteta i predznaka (pomoćni prijenos se ne koristi za skokove).

CALL i RET: potprogrami

> **CALL** je kao Jump, ali prije punjenja PC-a novom adresom **sprema staru adresu na stog**. **RET** (*Return*) uzima 2 bajta sa stoga i vraća ih u PC.

Spremljena adresa je adresa **instrukcije iza CALL**. Tako se program nakon potprograma nastavlja tamo gdje je stao.

> **Potprogram** (*subroutine*) je dio koda koji se koristi više puta. Potprogrami su glavni organizacijski elementi programa u asemblerskom jeziku.

Zašto ne običan Jump? Ako množenje treba na dva mjesta u programu, ne želimo kod pisati dvaput (gubitak vremena i memorije). Običnim skokom možemo doći do koda za množenje, ali se **ne možemo vratiti** na pravo mjesto. CALL i RET to omogućuju.

Primjer potprograma za množenje: množi bajt u B bajtom u C, a 16-bitni umnožak stavlja u HL.

```
Multiply:  PUSH PSW        ; spremi registre koje mijenjamo
           PUSH BC

           MVI H,00h       ; HL (rezultat) = 0000h
           MVI L,00h

           MOV A,B         ; množitelj u A
           CPI A,00h       ; ako je 0, gotovo
           JZ  AllDone

           MVI B,00h       ; viši bajt BC = 0, pa je BC = 16-bitni C

MultLoop:  DAD HL,BC       ; HL = HL + BC
           DCR A           ; smanji množitelj
           JNZ MultLoop    ; ponavljaj dok nije 0

AllDone:   POP BC          ; vrati spremljene registre
           POP PSW
           RET             ; povratak
```

Kako radi:

1. Oznaka **Multiply** je adresa početka potprograma.
2. Potprogram najprije sprema registre koje će mijenjati (PUSH), a na kraju ih vraća (POP). Dobra praksa za svaki potprogram.
3. HL se postavi na 0.
4. Ako je množitelj 0, umnožak je 0 i JZ preskače na kraj.
5. Inače se BC pretvori u 16-bitni broj (B = 0), i petlja ga dodaje u HL onoliko puta koliko je bio množitelj.

(Knjiga napominje da se učinkovitiji potprogram za množenje može napisati pomoću instrukcija pomaka.)

Korištenje (vlastiti primjer): 0Ch × 15h

```
MVI B,0Ch
MVI C,15h
CALL Multiply      ; nakon povratka HL = 00FCh (12 × 21 = 252)
```

Tablica skokova, poziva i povrataka

8080 ima i uvjetne CALL i RET, ali se rjeđe koriste:

Uvjet	RET	JMP aaaa	CALL aaaa
bez uvjeta	C9h RET	C3h JMP	CDh CALL
Z = 0 (nije nula)	C0h RNZ	C2h JNZ	C4h CNZ
Z = 1 (nula)	C8h RZ	CAh JZ	CCh CZ
CY = 0	D0h RNC	D2h JNC	D4h CNC
CY = 1	D8h RC	DAh JC	DCh CC
neparan paritet	E0h RPO	E2h JPO	E4h CPO
paran paritet	E8h RPE	EAh JPE	ECh CPE
S = 0 (pozitivno)	F0h RP	F2h JP	F4h CP
S = 1 (negativno)	F8h RM	FAh JM	FCh CM

Ulaz i izlaz

Računalo treba i ulazne i izlazne (**U/I**, *I/O*) uređaje, npr. tipkovnicu i zaslon. Sve što je spojeno na procesor, a nije memorija, zove se **periferija**.

Dva pristupa:

Pristup	Opis
Memorijski mapiran U/I (*memory-mapped I/O*)	periferija zauzima neke adrese koje bi inače bile memorija; procesor joj piše i čita kao memoriji
U/I portovi (8080)	uz 65.536 adresa memorije postoji još **256** posebnih adresa za U/I uređaje; koriste nožice A_0–A_7, a od memorije se razlikuju upravljačkim signalima (čip 8228)

Kod	Instrukcija	Značenje
D3h	OUT pp	pošalji akumulator na port pp
DBh	IN pp	učitaj bajt s porta pp u akumulator

Prekidi

Kad pritisneš tipku, korisno je da procesor to **odmah** sazna. Za to služi **prekid** (*interrupt*): signal s periferije na ulaz **INT** procesora.

- Nakon RESET-a 8080 **ne reagira** na prekide. Program ih mora uključiti instrukcijom **EI** (FBh, *Enable Interrupts*), a može ih isključiti s **DI** (F3h, *Disable Interrupts*).
- Izlaz **INTE** pokazuje jesu li prekidi uključeni.
- Kad periferija postavi INT na 1, procesor dohvaća instrukciju, ali upravljački signali javljaju da je riječ o prekidu. Periferija tada procesoru obično pošalje jednu od instrukcija **RST** (*Restart*):

Kod	Instrukcija	Skače na
C7h	RST 0	0000h
CFh	RST 1	0008h
D7h	RST 2	0010h
DFh	RST 3	0018h
E7h	RST 4	0020h
EFh	RST 5	0028h
F7h	RST 6	0030h
FFh	RST 7	0038h

RST je kao CALL: sprema PC na stog i skače na zadanu adresu, gdje je kod koji obrađuje prekid. Na primjer, prekid tipkovnice može izazvati RST 4, a na 0020h je kod koji čita bajt s tipkovnice (detaljnije u poglavlju 21).

NOP i neiskorišteni kodovi

Kod	Instrukcija	Značenje
00h	NOP	*no operation*: ne radi ništa; služi za popunjavanje prostora

Opisano je 243 + NOP = **244** koda. Dvanaest kodova nije dodijeljeno ničemu: 08h, 10h, 18h, 20h, 28h, 30h, 38h, CBh, D9h, DDh, EDh i FDh.

Motorola 6800

Mnogo toga je slično kao kod 8080, pa samo razlike:

Svojstvo	6800
Napajanje	V_SS = uzemljenje, V_CC = 5 V

Svojstvo	6800
Adrese / podaci	16 adresnih izlaza, 8 podatkovnih ulaza/izlaza
Upravljanje	RESET, R/$\bar{W}$ (čitanje/pisanje), IR$\bar{Q}$ (*interrupt request*, zahtjev za prekid); vremenski odnosi signala mnogo jednostavniji nego kod 8080
U/I	**nema U/I portova**: svi U/I uređaji su dio memorijskog prostora
Registri	16-bitno programsko brojilo, 16-bitni pokazivač stoga, 8-bitni statusni registar (zastavice), **dva akumulatora A i B** (sve što se može s A, može i s B); nema drugih 8-bitnih registara
Indeksni registar	16-bitni; drži adresu (slično HL kod 8080); za mnoge instrukcije adresa = indeksni registar + bajt iza koda
Zastavice	nema zastavicu pariteta, ali ima zastavicu **preljeva** (*Overflow*), koju 8080 nema

6800 radi gotovo iste operacije (punjenje, spremanje, zbrajanje, oduzimanje, pomak, skokove, pozive), ali su **kodovi i mnemonici potpuno drukčiji**. Na primjer, instrukcije grananja:

Kod	Mnemonik	Grananje ako je…
20h	BRA	uvijek (*Branch*)
22h	BHI	veće (*Higher*)
23h	BLS	manje ili jednako (*Lower or Same*)
24h	BCC	prijenos obrisan (*Carry Clear*)
25h	BCS	prijenos postavljen (*Carry Set*)
26h	BNE	nije jednako (*Not Equal*)
27h	BEQ	jednako (*Equal*)
28h	BVC	preljev obrisan (*Overflow Clear*)
29h	BVS	preljev postavljen (*Overflow Set*)
2Ah	BPL	pozitivno (*Plus*)
2Bh	BMI	negativno (*Minus*)
2Ch	BGE	veće ili jednako nuli
2Dh	BLT	manje od nule
2Eh	BGT	veće od nule
2Fh	BLE	manje ili jednako nuli

Neka grananja ovise o **kombinaciji** zastavica.

Nekompatibilnost

Čipove su u isto vrijeme dizajnirale dvije skupine inženjera u dvije tvrtke. Zato:

- nijedan čip ne može izvoditi strojni kod drugoga,
- program u asemblerskom jeziku za jedan čip ne može se izravno prevesti u kodove drugoga.

Pisanje programa koji rade na više procesora tema je poglavlja 24.

Little-endian i big-endian

U oba procesora LDA puni akumulator s navedene adrese. Usporedi:

Procesor	Bajtovi	Učitava se s adrese
8080	3Ah 7Bh 34h	**347Bh**: prvo niži bajt, pa viši
6800 (prošireno adresiranje)	B6h 7Bh 34h	**7B34h**: prvo viši bajt, pa niži

Način	Tko	Što je na nižoj adresi
Little-endian	Intel	najmanje značajan bajt
Big-endian	Motorola	najznačajniji bajt

Ta razlika **nikad nije razriješena** i stvara nekompatibilnost pri dijeljenju podataka između sustava. Nijedan način nije "ispravan". (Nazivi dolaze iz *Gulliverovih putovanja*: rat oko toga s kojeg kraja treba razbiti jaje.) Računalo iz poglavlja 17 koristilo je big-endian.

Analogija: isti datum može se pisati 10.10.2026. ili 2026-10-10. Oba su točna, ali tko očekuje jedan zapis, a dobije drugi, krivo će pročitati.

Što je bilo poslije

Godina	Događaj
siječanj 1975.	**Altair 8800** (s 8080) na naslovnici časopisa *Popular Electronics*: prvo "kućno računalo"; prednja ploča sa žaruljicama i prekidačima, kao upravljačka ploča iz poglavlja 16
1976.	Intel 8085
—	**Z-80** (Zilog, osnovao Federico Faggin): potpuno kompatibilan s 8080, uz mnogo novih instrukcija; 1977. u Radio Shack TRS-80 Model 1
1977.	**Apple II** (Jobs i Wozniak) s jeftinijim čipom **6502** (MOS Technology), poboljšanjem 6800
lipanj 1978.	**Intel 8086**: 16-bitni, adresira **1 MB**; nije kompatibilan s 8080, ali ima množenje i dijeljenje
1979.	**8088**: iznutra 8086, ali vanjski pristup memoriji po bajt, pa koristi 8-bitne pomoćne čipove
jesen 1981.	**IBM PC** (model 5150) s 8088; mnogi proizvođači rade kompatibilna računala
1982. – 1993.	obitelj **x86**: 186 i 286 (1982.), 32-bitni 386 (1985.), 486 (1989.), Pentium (od 1993.); svi podržavaju kodove svih prethodnika od 8086
1984.	Apple Macintosh s **Motorolom 68000** (16-bitni potomak 6800; obitelj "68K")
od 1994.	Macintosh koristi **PowerPC** (Motorola, IBM, Apple)

RISC

PowerPC je **RISC** procesor (*Reduced Instruction Set Computing*): brzinu povećava pojednostavljenjem.

- sve instrukcije iste duljine (kod PowerPC 32 bita),
- pristup memoriji samo instrukcijama Load i Store,
- instrukcije rade jednostavne operacije,
- mnogo registara, da bi se izbjegao čest pristup memoriji.

PowerPC ne može izvoditi kod za 68K, ali ga **emulira**: program emulator ispituje kod 68K, instrukciju po instrukciju, i izvodi odgovarajuću radnju. Sporije je, ali radi.

Na što odlaze novi tranzistori

Prema Mooreovu zakonu broj tranzistora udvostručuje se svakih 18 mjeseci. Koriste se za:

- **veću širinu podataka** (4 → 8 → 16 → 32 bita),
- **nove instrukcije**: aritmetika s pomičnim zarezom (poglavlje 23), instrukcije za ponavljajuće izračune pri prikazu slika i filmova,
- **cjevovod** (*pipelining*): dok izvodi jednu instrukciju, procesor već čita sljedeće, pa i donekle predviđa kako će skokovi promijeniti tijek,
- **priručnu memoriju** (*cache*): niz vrlo brzog RAM-a u procesoru koji čuva nedavno izvršene instrukcije. Programi često vrte male petlje, pa se instrukcije ne moraju stalno ponovno učitavati.

Sve to traži više logike i više tranzistora.

Ključne definicije

Pojam	Značenje
Skup instrukcija	sve instrukcije koje procesor zna izvesti
Registar	zasun u procesoru za jednu vrijednost (A, B, C, D, E, H, L)
Par registara	dva 8-bitna registra kao jedna 16-bitna vrijednost (BC, DE, HL)
Izravno / indeksno / neposredno adresiranje	adresa u instrukciji / adresa u registru / podatak u instrukciji
PSW	registar zastavica 8080 (S, Z, P, CY, AC)
Po bitovima (*bitwise*)	logička operacija na svakom paru bitova zasebno
BCD	dekadska znamenka u svakom niblu
Stog (LIFO)	memorija u kojoj se zadnje stavljeno prvo uzima
Pokazivač stoga (SP)	16-bitni registar s adresom vrha stoga
Potprogram	dio koda pozvan s CALL, završava s RET
Prekid	signal kojim periferija traži pozornost procesora
Periferija	uređaj spojen na procesor koji nije memorija
Little-endian / big-endian	niži / viši bajt na nižoj adresi
RISC	arhitektura s jednostavnim instrukcijama jednake duljine
Cjevovod / priručna memorija	preklapanje dohvata i izvršavanja / brzi RAM u procesoru

Ključno za zapamtiti

1. 8080: 8-bitni, 16 adresnih linija (64 KB), 8 dvosmjernih podatkovnih linija; nakon RESET-a počinje na 0000h.
2. Instrukcije 8080 imaju 1–3 bajta i traju 4–18 ciklusa sata.
3. MOV ima uzorak 01 ddd sss; aritmetika i logika 10 ooo sss; akumulator je uvijek odredište.
4. MOV, LDA i STA ne mijenjaju zastavice; aritmetičke i logičke instrukcije ih mijenjaju.
5. CMP oduzima samo radi zastavica; rezultat se ne sprema.
6. Stog radi po načelu LIFO; PUSH smanjuje SP, POP ga povećava; vraćati se mora obrnutim redom.
7. CALL sprema adresu povratka na stog, RET je vraća: to omogućuje potprograme.
8. Prekidi omogućuju periferiji da odmah dobije pozornost procesora.
9. 8080 i 6800 rade slične stvari, ali s potpuno različitim kodovima; nisu kompatibilni.
10. Intel je little-endian, Motorola big-endian.

Najčešće zabune

- **"MOV A,B premješta A u B."** Odredište je **lijevo**: MOV A,B kopira B **u** A.
- **"MOV briše izvor."** Ne: MOV **kopira**; izvor ostaje isti.
- **"Zastavica prijenosa kod oduzimanja na 8080 radi kao u poglavlju 17."** Na 8080 je CY = 1 kad SUB/SBB **ne** daju prijenos (dakle kad je bilo posudbe).
- **"Stog raste prema višim adresama."** Na 8080 PUSH **smanjuje** SP; stog raste prema nižim adresama.
- **"[HL] je registar."** [HL] je **bajt u memoriji** na adresi koju čuva HL.

- **"Little-endian je pogrešan način."** Oba su načina jednako ispravna; problem je samo nekompatibilnost.

Brza provjera razumijevanja

1. Koji je kod instrukcije MOV D,[HL]? A MOV [HL],A? Izračunaj iz uzorka bitova.
2. A = 9Ah, B = 5Ch. Koliko je A nakon AND A,B, OR A,B i XOR A,B?
3. SP = 4000h. Opiši što se događa pri PUSH BC i na kojim adresama završavaju B i C.
4. Zašto se za ponovno korištenje koda koristi CALL, a ne JMP?
5. Bajtovi u memoriji su 12h 34h. Koju 16-bitnu vrijednost vidi 8080, a koju 6800?

20. ASCII i skup znakova

Što ćeš naučiti u ovom poglavlju

- Kako se **tekst** prikazuje bitovima: **kodovi znakova**, **skup kodiranih znakova**, **niz znakova** (*string*).
- Kako radi 5-bitni kod **Baudot** i zašto su **kodovi pomaka** (*shift*) problematični.
- Zašto za tekst treba **7 bitova**.
- Cijelu tablicu **ASCII**: grafičke i **upravljačke** znakove.
- Kako se u ASCII-ju mala slova pretvaraju u velika (razlika 20h).
- Kako je građen IBM-ov kod **EBCDIC** i zašto ima "rupe".
- Koliko memorije zauzima tekst.
- Ograničenja ASCII-ja: **prošireni ASCII** (Latin-1), dvobajtni skupovi i **Unicode**.

Tekst kao niz znakova

Memorija čuva samo bitove, pa i tekst mora postati bitovi. Treba sustav u kojem svaki znak ima **jedinstveni kod**.

Pojam	Značenje
Alfanumerički znakovi	slova, brojke, interpunkcija
Skup kodiranih znakova (*coded character set*)	sustav koji svakom znaku dodjeljuje kod
Kod znaka (*character code*)	broj dodijeljen jednom znaku
Niz znakova (*string*)	uzastopni kodovi znakova koji čine tekst

Dva pojednostavljenja:

1. Tekst ne gledamo kao dvodimenzionalne stupce na stranici, nego kao **jednodimenzionalni niz** slova, brojki i interpunkcije, uz eventualni poseban kod za kraj odlomka. Priča se ne mijenja ako je u knjizi stupac širi nego u časopisu.
2. Za početak radimo s **čistim tekstom** (*plain text*): bez kurziva, podebljanja, podcrtavanja, boja, indeksa i bez naglasnih znakova. Samo osnovna latinična abeceda. Promjena fonta mijenja izgled, ali ne i sadržaj.

Primjer: rečenica "Imam 27 sestara." je niz od 16 znakova (uključujući razmake i točku). Znamenke **2** i **7** ovdje su **znakovi** kao i slova. Njihovi kodovi ne moraju imati veze s brojčanim vrijednostima 2 i 7 (binarno 10 i 111).

Što smo već vidjeli

Kod	Širina	Problem za računala
Morse	**promjenjiva**: kraći kodovi za česta slova	nespretno za računala; ne razlikuje velika i mala slova
Braille	**stalna**: 6 bitova po znaku (bolje za računala)	velika slova traže poseban **escape** kod ispred (dva koda po slovu); brojke traže **shift** kod

Escape kod mijenja značenje samo **sljedećeg** znaka. **Shift kod** mijenja značenje **svih** sljedećih znakova, dok ne dođe drugi shift kod.

Baudot: 5-bitni kod

Najekonomičniji kod za tekst vjerojatno je **5-bitni** kod iz telegrafa za ispis koji je 1874. razvio Émile Baudot. Kasnije ga je izmijenio Donald Murray, a 1931. standardiziran je kao **Međunarodna telegrafska abeceda br. 2** (ITA-2). U SAD-u se zove Baudot, a točnije je Murrayev kod.

U 20. stoljeću koristio se u **teleprinterima**: tipkovnica sa samo 30 tipki i razmaknicom. Tipke su prekidači koji stvaraju binarni kod i šalju ga kabelom bit po bit. Teleprinter ima i mehanizam za ispis: kodovi koji stignu pokreću elektromagnete koji otiskuju znakove.

5 bitova daje samo **32 koda** (00h – 1Fh). Zato se isti kodovi koriste dvaput: jednom za **slova**, jednom za **brojke i interpunkciju**.

Kod	Slovo	Brojka	Kod	Slovo	Brojka
00	—	—	10	E	3
01	T	5	11	Z	+
02	*Carriage Return*	*Carriage Return*	12	D	*Who Are You?*
03	O	9	13	B	?
04	*Space*	*Space*	14	S	'
05	H	#	15	Y	6
06	N	,	16	F	$
07	M	.	17	X	/
08	*Line Feed*	*Line Feed*	18	A	-
09	L	)	19	W	2
0A	R	4	1A	J	*Bell*
0B	G	&	1B	*Figure Shift*	*Figure Shift*
0C	I	8	1C	U	7
0D	P	0	1D	Q	1
0E	C	:	1E	K	(
0F	V	=	1F	*Letter Shift*	*Letter Shift*

- **04h** je razmak.
- **02h** *Carriage Return* (povratak valjka): ispis se vraća na lijevi rub. **08h** *Line Feed* (novi red): papir se pomakne jedan red. Nazivi dolaze od pisaćeg stroja, gdje jedna poluga radi obje stvari; u Baudotu su to dva odvojena koda.
- **1Bh** *Figure Shift*: svi sljedeći kodovi su brojke i interpunkcija, sve do **1Fh** *Letter Shift*.
- *Bell* zvoni zvoncem na teleprinteru; *Who Are You?* pokreće mehanizam kojim se teleprinter identificira.
- Standard ITU ne definira kodove 05h, 0Bh i 16h; rezervirani su "za nacionalnu upotrebu" (tablica pokazuje kako su se koristili u SAD-u).
- Kao i Morse, Baudot ne razlikuje velika i mala slova.

Primjer (vlastiti): "PAY 40 NOW."

```
P  A  Y  ␣  [FS] 4  0  [LS] ␣  N  O  W  [FS] .  CR LF
0D 18 15 04 1B   0A 0D 1F   04 06 03 19 1B   07 02 08
```

Problem kodova pomaka

Ako se taj niz pošalje teleprinteru **dvaput zaredom**, druga linija ispadne ovako:

```
PAY 40 NOW.
0-6 40 NOW.
```

Zadnji kod pomaka prije druge linije bio je *Figure Shift* (ispred točke), pa su kodovi 0D, 18, 15 protumačeni kao brojke: 0, -, 6.

> Kodovi pomaka su ekonomični, ali stvaraju ovakve neugodne pogreške. Bolje je imati **zasebne** kodove za brojke i interpunkciju, i zasebne kodove za velika i mala slova.

Koliko bitova treba

Skupina	Broj znakova
Velika slova	26
Mala slova	26
Znamenke 0–9	10
Ukupno bez interpunkcije	**62**

Uz nešto interpunkcije prelazi se 64 (6 bitova nije dovoljno), ali ostaje mnogo mjesta do 128. Dakle:

> Za engleski tekst s velikim i malim slovima, bez pomaka, treba **7 bitova**.

Kodovi mogu biti bilo kakvi. Ali korisno je:

- da se **svi** dogovore oko istih kodova (računala mogu razmjenjivati tekst),
- da slova imaju **uzastopne** kodove (lakše abecedno razvrstavanje).

ASCII

ASCII (*American Standard Code for Information Interchange*), formaliziran 1967., najvažniji je standard u računalnoj industriji. Uz jednu veliku iznimku (EBCDIC, niže), kad god se na računalu radi s tekstom, negdje je uključen ASCII.

ASCII je **7-bitni** kod: od 0000000 do 1111111, tj. **00h do 7Fh**.

Grafički znakovi (20h – 7Eh)

Tablica: redak je viša heks znamenka, stupac niža. Npr. "A" je u retku 4, stupcu 1: **41h**.

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
2-	razmak	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3-	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4-	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5-	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6-	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7-	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	(DEL)

- **20h** je razmak.
- Znamenke 0–9 su **30h–39h**: kod znamenke = 30h + vrijednost znamenke.
- Velika slova su **41h–5Ah**, mala **61h–7Ah**.

- Osim @ i podvlake, interpunkcija u retku 5 i 7 nije bila na pisaćim strojevima, ali je danas standard na tipkovnicama.
- Ima **95** grafičkih znakova (imaju vidljiv oblik). Do 128 nedostaje 33 koda: to su upravljački znakovi.

Primjeri (vlastiti):

```
"Bok, Ana!"  →  42 6F 6B 2C 20 41 6E 61 21
"Soba 34."   →  53 6F 62 61 20 33 34 2E
```

Broj 34 u tekstu zapisan je kodovima **33h i 34h** (znakovi "3" i "4"). **Nije** zapisan kao 03h 04h, ni kao BCD 34h, ni kao heks 22h (34 dekadski). Ti kodovi u ASCII-ju znače nešto sasvim drugo.

Velika i mala slova: razlika 20h

Veliko slovo i odgovarajuće malo razlikuju se točno za **20h**: "A" = 41h, "a" = 61h. U binarnom zapisu razlika je samo u **jednom bitu** (trećem slijeva, bitu 5):

```
A = 0100 0001
a = 0110 0001
```

To olakšava pretvorbu. Potprogram za 8080 koji tekst pretvara u velika slova (HL = adresa prvog znaka, C = broj znakova):

```
Capitalize: MOV A,C        ; C = broj preostalih znakova
            CPI A,00h      ; usporedi s 0
            JZ  AllDone    ; ako je 0, gotovo

            MOV A,[HL]     ; sljedeći znak
            CPI A,61h      ; manji od 'a'?
            JC  SkipIt     ; ako da, preskoči

            CPI A,7Bh      ; veći od 'z'?
            JNC SkipIt     ; ako da, preskoči

            SUI A,20h      ; malo slovo: oduzmi 20h
            MOV [HL],A     ; spremi znak

SkipIt:     INX HL         ; sljedeća adresa
            DCR C          ; smanji brojač
            JMP Capitalize ; na početak

AllDone:    RET
```

Oduzimanje 20h može se zamijeniti instrukcijom:

```
ANI A,DFh
```

DFh = 1101 1111. Operacija I po bitovima čuva sve bitove osim trećeg slijeva, koji postaje 0. Time malo slovo postaje veliko.

(Knjiga ovdje koristi instrukciju SBI; ovdje je zamijenjena sa SUI, oduzimanjem bez posudbe, jer prethodna usporedba ostavlja postavljenu zastavicu prijenosa.)

Upravljački znakovi

ASCII ima i **33 upravljačka znaka** (*control characters*): nemaju vidljiv oblik, nego izvode neku radnju. Nastali su uglavnom za teleprintere i mnogi su danas nejasni.

Kod	Kratica	Naziv	Kod	Kratica	Naziv
00	NUL	Null (ništa)	10	DLE	Data Link Escape
01	SOH	Start of Heading	11	DC1	Device Control 1
02	STX	Start of Text	12	DC2	Device Control 2
03	ETX	End of Text	13	DC3	Device Control 3
04	EOT	End of Transmission	14	DC4	Device Control 4
05	ENQ	Enquiry	15	NAK	Negative Acknowledge
06	ACK	Acknowledge	16	SYN	Synchronous Idle
07	BEL	Bell (zvono)	17	ETB	End of Transmission Block
08	BS	Backspace (korak natrag)	18	CAN	Cancel
09	HT	Horizontal Tabulation (tabulator)	19	EM	End of Medium
0A	LF	Line Feed (novi red)	1A	SUB	Substitute Character
0B	VT	Vertical Tabulation	1B	ESC	Escape
0C	FF	Form Feed (nova stranica)	1C	FS	File Separator
0D	CR	Carriage Return (povratak na početak reda)	1D	GS	Group Separator
0E	SO	Shift-Out	1E	RS	Record Separator
0F	SI	Shift-In	1F	US	Unit Separator
			7F	DEL	Delete (brisanje)

Ideja: upravljački znakovi miješaju se s grafičkima i daju **osnovno oblikovanje**. Zamisli teleprinter ili jednostavan pisač: normalno ispiše znak i pomakne se jedno mjesto udesno. Upravljački znakovi to mijenjaju.

Znak	Učinak na pisaču
HT (09h), tabulator	ispis se nastavlja na sljedećem položaju koji je višekratnik od 8 (položaji se broje od 0); za poravnanje u stupce
FF (0Ch)	izbaci trenutnu stranicu i počni novu
BS (08h)	vrati se jedno mjesto; na starim pisačima za složene znakove
CR (0Dh)	glava pisača ide na lijevi rub
LF (0Ah)	glava pisača ide jedan red niže

Primjeri:

- `41 09 42 09 43 09` ispisuje A, B i C poravnane u stupce na položajima 0, 8 i 16.
- `65 08 60` ispisuje "e", vraća se i preko njega ispisuje "`": dobije se **è**.
- Za novi red obično trebaju **oba**: CR i LF. Sam CR omogućuje pisanje preko postojećeg reda; sam LF prelazi u novi red bez povratka na lijevi rub.

EBCDIC: IBM-ova iznimka

ASCII se ne koristi na mnogim većim IBM-ovim sustavima. Uz System/360 IBM je razvio 8-bitni kod **EBCDIC** (*Extended BCD Interchange Code*), proširenje starijeg 6-bitnog koda BCDIC, koji potječe od kodova IBM-ovih bušenih kartica. Kodovi su se razvijali desetljećima kroz razne tehnologije, pa u njima nema mnogo logike ni dosljednosti.

Bušena kartica

IBM-ova kartica (od 1928.) ima **80 stupaca**; svaki stupac je jedan znak, kodiran kombinacijom jedne ili više rupa.

Red	Naziv
0 – 9	brojčani redovi (*digit punches*)
11 (iznad reda 0)	zonski red (*zone punch*)
12 (najviši)	zonski red

Reda 10 nema.

Kako je građen EBCDIC kod

> Bajt EBCDIC = **viši nibl** (kod zonskih rupa, prilično proizvoljan) + **niži nibl** (BCD vrijednost brojčane rupe).

Zonske rupe	Viši nibl	Znakovi	Kodovi
nema	1111	znamenke 0–9	F0h – F9h
12	1100	A – I	C1h – C9h
11	1101	J – R	D1h – D9h
0	1110	S – Z	E2h – E9h
12 i 0	1000	a – i	81h – 89h
12 i 11	1001	j – r	91h – 99h
11 i 0	1010	s – z	A2h – A9h

U nizu slova postoje **rupe** (npr. iza I = C9h slijedi J = D1h). Kod programiranja s EBCDIC tekstom te rupe znaju biti vrlo neugodne.

Zašto ne 12 bitova po stupcu? Svaki stupac kartice mogao bi teoretski nositi 12 bitova, pa i 7-bitni ASCII. U praksi bi bilo **previše rupa** i kartica bi izgubila čvrstoću.

Zašto ASCII ima 7 bitova

Mnogi 8-bitni kodovi u EBCDIC-u nisu definirani, što pokazuje da 7 bitova ima smisla. Kad je ASCII nastajao, memorija je bila vrlo skupa:

- neki su predlagali **6-bitni** kod s pomakom za mala i velika slova (odbijeno),
- drugi **8-bitni**, jer je bilo vjerojatnije da će računala biti 8-bitna.

Danas su 8-bitni bajtovi standard, pa se ASCII, iako 7-bitni, gotovo uvijek **sprema kao 8-bitne vrijednosti**.

Koliko memorije zauzima tekst

Jedan znak = jedan bajt. Zato se veličina teksta lako procjenjuje brojanjem znakova:

Tekst	Procjena	Bajtova
Strojem pisana stranica, dvostruki prored	27 redaka × oko 65 znakova	oko 1750
Ista stranica, jednostruki prored	dvostruko više	oko 3,5 KB
Stranica časopisa *The New Yorker*	3 stupca × 60 redaka × 40 znakova	7200
Stranica *The New York Timesa* (samo tekst)	6 stupaca × 155 redaka × 35 znakova	32.550 ≈ 32 KB
Prosječna knjiga	500 riječi × 6 znakova = 3000 znakova po stranici × 333 stranice	oko **1 MB**
Knjižnica Kongresa SAD-a	oko 20 milijuna knjiga	oko **20 TB**

Prosječna riječ ima oko 5 slova, a s razmakom 6 znakova. Knjige jako variraju: od oko 300 KB (*Veliki Gatsby*) do oko 3,9 MB (*Rat i mir*) i 7,7 MB (*U potrazi za izgubljenim vremenom*).

Ograničenja ASCII-ja

ASCII je "previše američki":

- nema znak britanske funte (£),
- nema naglašena slova mnogih zapadnoeuropskih jezika,
- nema nelatinična pisma: grčko, arapsko, hebrejsko, ćirilicu, indijska pisma (devanagari, bengalsko...), tajlandsko, tibetsko,
- nikako ne može obuhvatiti **desetke tisuća** kineskih, japanskih i korejskih ideograma, ni oko deset tisuća korejskih slogova hangul.

Standard ASCII ipak je predvidio malo prostora: deset kodova (**40h, 5Bh, 5Ch, 5Dh, 5Eh, 60h, 7Bh, 7Ch, 7Dh, 7Eh**) mogu se redefinirati za nacionalne potrebe. # se može zamijeniti s £, a $ općim znakom valute ¤. To radi samo ako **svi** koji koriste dokument znaju za promjenu.

Prošireni ASCII

Budući da se znakovi spremaju kao 8-bitne vrijednosti, može se napraviti **prošireni skup znakova** od 256 znakova:

- 00h – 7Fh: isto kao ASCII,
- 80h – FFh: nešto drugo, npr. naglašena slova i nelatinična pisma.

Primjer je **Latin Alphabet No. 1** (Latin-1), koji definira znakove A0h – FFh. Redak je niža heks znamenka, stupac viša:

	A-	B-	C-	D-	E-	F-
-0	(NBSP)	°	À	Ð	à	ð
-1	¡	±	Á	Ñ	á	ñ
-2	¢	²	Â	Ò	â	ò
-3	£	³	Ã	Ó	ã	ó
-4	¤	´	Ä	Ô	ä	ô
-5	¥	µ	Å	Õ	å	õ
-6	¦	¶	Æ	Ö	æ	ö
-7	§	·	Ç	×	ç	÷
-8	¨	¸	È	Ø	è	ø
-9	©	¹	É	Ù	é	ù
-A	ª	º	Ê	Ú	ê	ú
-B	«	»	Ë	Û	ë	û
-C	¬	¼	Ì	Ü	ì	ü
-D	(SHY)	½	Í	Ý	í	ý
-E	®	¾	Î	Þ	î	þ
-F	¯	¿	Ï	ß	ï	ÿ

- **A0h, razmak bez prijeloma** (*no-break space*, NBSP): prikazuje se kao razmak, ali program ondje ne smije prelomiti red (npr. u "WW II"). Obični razmak 20h je mjesto gdje se red smije prelomiti.

- **ADh, meka crtica** (*soft hyphen*, SHY): crtica za rastavljanje riječi na slogove; na papiru se pojavljuje samo kad se riječ mora prelomiti između dvaju redaka.

Primijeti da u ovoj tablici nema hrvatskih slova č, ć, š, ž.

Problem: tijekom desetljeća nastalo je **mnogo različitih** proširenja ASCII-ja, što stvara zbrku i nekompatibilnost.

Dvobajtni skupovi znakova

Za kineske, japanske i korejske ideograme ASCII je proširen radikalnije. U kodu **Shift-JIS** (japanski industrijski standard) kodovi **81h – 9Fh** su zapravo **prvi bajt dvobajtnog koda**. Tako se dobiva oko 6000 dodatnih znakova. U Aziji su popularna još tri standardna **dvobajtna skupa znakova** (DBCS).

Problemi:

- postoji više **nekompatibilnih** dvobajtnih skupova,
- obični ASCII znakovi imaju 1 bajt, a ideogrami 2 bajta: miješanje duljina otežava rad s tekstom.

Unicode

Godine 1988. nekoliko velikih računalnih tvrtki počelo je razvijati alternativu ASCII-ju: **Unicode**, jedinstven i nedvosmislen sustav kodiranja za sve jezike svijeta.

	ASCII	Unicode (kako ga opisuje knjiga)
Širina koda	7 bitova	**16 bitova** (2 bajta za svaki znak)
Raspon	00h – 7Fh	0000h – FFFFh
Broj znakova	128	**65.536**

- Prvih 128 znakova Unicodea (0000h – 007Fh) jednako je ASCII-ju.
- 00A0h – 00FFh jednako je Latin-1.
- U Unicode su uključeni i drugi svjetski standardi.

Prepreke: ASCII i njegova mnoga proširenja duboko su ukorijenjeni. Stvarni problem Unicodea: više ne vrijedi "jedan znak = jedan bajt". Knjiga *Plodovi gnjeva* u ASCII-ju ima oko 1 MB, a u Unicodeu oko 2 MB. To je mala cijena za univerzalan i nedvosmislen sustav.

Ključne definicije

Pojam	Značenje
Kod znaka	broj dodijeljen jednom znaku
Niz znakova (string)	uzastopni kodovi znakova
Kod promjenjive / stalne širine	različit / jednak broj bitova za svaki znak
Kod pomaka (shift)	mijenja značenje svih sljedećih kodova
Escape kod	mijenja značenje samo sljedećeg koda
Baudot (ITA-2)	5-bitni kod teleprintera s kodovima pomaka
ASCII	7-bitni standardni kod (1967.), 00h – 7Fh
Grafički znak	znak s vidljivim oblikom (95 u ASCII-ju)
Upravljački znak	znak koji izvodi radnju (33 u ASCII-ju)
CR / LF	povratak na početak reda / novi red
EBCDIC	IBM-ov 8-bitni kod izveden iz bušenih kartica
Prošireni ASCII	8-bitni skup; 80h – FFh za dodatne znakove
Latin-1	proširenje s naglašenim slovima zapadne Europe (A0h – FFh)

Pojam	Značenje
DBCS	dvobajtni skup znakova (npr. Shift-JIS)
Unicode	prema knjizi 16-bitni kod za sve jezike; prvih 128 = ASCII

Ključno za zapamtiti

1. Tekst se sprema kao niz kodova znakova; znamenke u tekstu su znakovi, ne brojevi.
2. Kodovi pomaka štede bitove, ali stvaraju pogreške ako se izgubi trenutno stanje.
3. Engleski tekst bez pomaka traži 7 bitova.
4. ASCII: razmak 20h, znamenke 30h–39h, velika slova 41h–5Ah, mala 61h–7Ah.
5. Veliko i malo slovo razlikuju se za 20h (jedan bit).
6. 00h–1Fh i 7Fh su upravljački znakovi; najvažniji su CR (0Dh) i LF (0Ah).
7. EBCDIC je izveden iz bušenih kartica i u nizu slova ima rupe.
8. Jedan ASCII znak je jedan bajt; prosječna knjiga je oko 1 MB.
9. Proširenja ASCII-ja su brojna i međusobno nekompatibilna.
10. Unicode (prema knjizi): 2 bajta po znaku, 65.536 znakova, prvih 128 = ASCII.

Najčešće zabune

- **"Broj 12 u tekstu sprema se kao 0Ch."** U tekstu se sprema kao dva znaka: 31h i 32h.
- **"Kod znaka '0' je 00h."** Znak "0" je **30h**; 00h je upravljački znak NUL.
- **"CR i LF su isto."** CR vraća na početak reda, LF spušta jedan red; za novi red obično trebaju oba.
- **"ASCII je 8-bitni kod."** ASCII je 7-bitni; samo se obično sprema u 8-bitne bajtove.
- **"Prošireni ASCII je jedan standard."** Postoji mnogo različitih, nekompatibilnih proširenja.

Brza provjera razumijevanja

1. Zapiši u ASCII heks kodovima: "Kat 5!" Koliko bajtova zauzima?
2. Koji znak ima ASCII kod 4Dh, a koji 6Dh? Kako se razlikuju u binarnom zapisu?
3. Zašto bi niz Baudot kodova poslan dvaput zaredom mogao biti krivo ispisan?
4. Koji je EBCDIC kod za slovo "K", a koji za znamenku 7? Kako su ti kodovi izgrađeni?
5. Procijeni koliko bajtova zauzima knjiga od 200 stranica s oko 400 riječi po stranici.

21. Sabirnica: kako se dijelovi računala povezuju

Što ćeš naučiti u ovom poglavlju

- Što je **sabirnica** (*bus*) i koje četiri vrste signala prenosi.
- Što su **otvorena** i **zatvorena arhitektura** i zašto je IBM PC postao standard.
- Kako se gradi **memorijska ploča**: čip 2102, **odabir čipa** (CS), **banke**, **komparator** adrese.
- Što je izlaz s **tri stanja** i zašto je ključan za sabirnicu.
- Razliku **SRAM** i **DRAM** memorije.
- Kako radi **video zaslon**: linije, slike, **pojasna širina**, **pikseli**, **razlučivost**, generator znakova u **ROM-u**, boje.
- Kako **tipkovnica** stvara kod tipke (**scan code**).
- Kako rade **trajne pohrane**: magnetska vrpca, disketa, tvrdi disk; **staze** i **sektori**.
- Razliku između **memorije** i **pohrane**.

Sabirnica

Procesor je najvažniji dio računala, ali nije jedini. Računalo treba i:

- **RAM** sa strojnim kodom i podacima,
- **ulazni uređaj** (da instrukcije dođu u RAM),
- **izlazni uređaj** (da se vide rezultati),
- **trajnu pohranu**, jer je RAM nepostojan (gubi sadržaj bez struje).

Čipovi se montiraju na **tiskane pločice**. Kod manjih strojeva sve stane na jednu pločicu, ali obično su dijelovi raspoređeni na više njih. Pločice međusobno komuniciraju preko **sabirnice**.

> **Sabirnica** (*bus*) je skup digitalnih signala koji su dovedeni do svake pločice u računalu. Sabirnica pločicama dovodi i napajanje.

Vrsta signala	Tko ga daje	Čemu služi
Adresni signali	procesor	uglavnom adresiranje RAM-a, ali i drugih uređaja
Izlazni podatkovni signali	procesor	pisanje u RAM i druge uređaje
Ulazni podatkovni signali	drugi dijelovi (najčešće izlaz RAM-a)	procesor ih čita
Upravljački signali	procesor ili drugi uređaji	razni; obično odgovaraju upravljačkim signalima procesora (npr. signal za pisanje u memoriju)

Pazi na smjer: izlazni podatkovni signal **procesora** je ulazni podatkovni signal **RAM-a**. "Ulaz" i "izlaz" uvijek se gledaju s nečijeg stajališta.

Analogija: sabirnica je gradska ulica uz koju su sve kuće (pločice). Svaka kuća ima priključak na ulicu i vidi sav promet, ali se javlja samo kad je pošiljka za nju.

S-100 sabirnica

Jedna od prvih popularnih sabirnica za kućna računala bila je **S-100**, uvedena 1975. u računalu MITS Altair.

- Temeljena na 8080, kasnije prilagođena i drugim procesorima (npr. 6800).
- Pločica 5,3 × 10 inča; jedan rub ulazi u utor sa **100 kontakata** (otuda ime).
- **Matična ploča** (*motherboard*, *main board*) ima više utora (npr. 12) međusobno spojenih; utori se zovu **utori za proširenje** (*expansion slots*), a pločice **pločice za proširenje** (*expansion boards*).
- Procesor 8080 i pomoćni čipovi su na jednoj pločici; RAM na jednoj ili više drugih.

Signali S-100: **16 adresnih**, **8 ulaznih** i **8 izlaznih** podatkovnih (8080 ima zajedničke podatkovne nožice; razdvajaju ih drugi čipovi na pločici s procesorom) i **8 signala prekida**.

Prekidi na S-100: npr. tipkovnica pri pritisku tipke šalje prekid. Pločica s 8080 obično ima čip **Intel 8214** (jedinica za upravljanje prioritetnim prekidima). Kad stigne prekid, on šalje signal prekida procesoru, a kad ga 8080 prihvati, čip mu da instrukciju **RST**. RST sprema programsko brojilo i skače na 0000h, 0008h, 0010h... ili 0038h, ovisno o prekidu.

Otvorena i zatvorena arhitektura

	Otvorena arhitektura	Zatvorena arhitektura
Specifikacija sabirnice	javno objavljena	tajna (vlasnička, *proprietary*)
Pločice za proširenje	mogu ih raditi i **drugi proizvođači** (*third-party*)	samo proizvođač
Posljedica	više pločica → korisnije računalo → više prodaje → veće tržište za pločice; sabirnica može postati **industrijski standard**	manje proširenja

- **IBM PC** (jesen 1981.) najpoznatije je računalo otvorene arhitekture. IBM je objavio priručnik *Technical Reference* s potpunim shemama računala i svih pločica. Zahvaljujući tome nastale su pločice drugih proizvođača i čitavi **klonovi** PC-a, gotovo jednaki IBM-ovu i s istim softverom. Potomci PC-a činili su tada oko **90 %** tržišta stolnih računala.
- **Apple Macintosh** prvotno je imao zatvorenu arhitekturu; to vjerojatno objašnjava njegov udio manji od 10 %.
- Otvorenost ili zatvorenost arhitekture **ne utječe** na mogućnost da drugi pišu **softver** za računalo.

Sabirnice IBM PC-a i nasljednici

ISA (*Industry Standard Architecture*), sabirnica izvornog PC-a (procesor 8088, 1 MB memorije; iznutra 16-bitni, prema van pristupa memoriji po 8 bitova). Pločice imaju **62 kontakta**:

Signali ISA	Broj
Adresni	20
Podatkovni (zajednički ulaz/izlaz)	8
Zahtjevi za prekid	6
Zahtjevi za DMA	3

> **DMA** (*direct memory access*, izravni pristup memoriji): normalno procesor obavlja svako čitanje i pisanje memorije. S DMA neki drugi uređaj (npr. disk) **preuzme sabirnicu** i izravno čita ili piše memoriju, zaobilazeći procesor. Tako je prijenos brži.

Kod PC-a su procesor, pomoćni čipovi i dio RAM-a na **sistemskoj ploči** (matičnoj ploči); kod S-100 sve je na pločicama za proširenje.

Godina	Sabirnica / računalo	Bitno
1984.	PC AT (80286, 16 MB)	stara sabirnica + dodatni utor s 36 kontakata: 7 adresnih (trebala su samo 4), 8 podatkovnih, 5 prekida, 4 DMA
1987.	MCA (*Micro Channel Architecture*, IBM)	dijelom patentirana, IBM naplaćuje licence; zato **nije** postala standard
1988.	EISA (konzorcij devet tvrtki bez IBM-a)	32-bitna
poslije	PCI (Intel)	uobičajena u PC-kompatibilnim računalima

Sabirnice se mijenjaju kad ih procesori **prerastu**: po širini podataka (8 → 16 → 32 bita), broju adresnih signala ili **brzini**. Rane sabirnice bile su za procesore od nekoliko MHz. Sabirnica koja nije projektirana za velike brzine može stvarati **radiofrekvencijske smetnje** (RFI) u obližnjim radijima i televizorima.

U nastavku se vraćamo u sredinu 1970-ih i promatramo **sučelja** (*interfaces*) za memoriju, zaslon, tipkovnicu i pohranu.

Memorijska ploča

Čip 2102

Popularan memorijski čip sredinom 1970-ih: **2102**.

Svojstvo	2102
Tehnologija	MOS (kao 8080 i 6800); lako se spaja s TTL-om, gušći od TTL-a, ali sporiji
Kapacitet	**1024 bita** (adrese A_0–A_9, jedan DI, jedan DO): 1024 × 1
Vrijeme pristupa za čitanje	350 – 1000 ns, ovisno o inačici
R/$\bar{W}$	normalno 1 (čitanje); za pisanje 0 barem 170 – 550 ns
C$\bar{S}$ (*chip select*, odabir čipa)	kad je 1, čip je **neodabran** i ne reagira na R/$\bar{W}$

Vrijeme pristupa za čitanje je vrijeme od postavljanja adrese do trenutka kad je izlaz podatka ispravan.

Za 8-bitni procesor treba **osam** čipova 2102 s istim adresnim signalima, istim R/$\bar{W}$ i istim C$\bar{S}$. Dobije se polje **1024 × 8 = 1 KB**.

> Skup čipova spojenih tako da zajedno spremaju cijeli bajt zove se **banka** (*bank*).

Na S-100 pločicu stane do 64 takva čipa (8 KB), ali uzmimo skromnije: **4 KB = 32 čipa = 4 banke po 8 čipova**.

Kako ploča dijeli adresu

8080 i 6800 imaju 16-bitne adrese (64 KB). Na ploči od 4 KB 16 adresnih signala dijeli se ovako:

```
A15 A14 A13 A12 | A11 A10 | A9 A8 A7 A6 A5 A4 A3 A2 A1 A0
  izbor ploče   | izbor   |      adresa unutar RAM čipa
                | banke   |
```

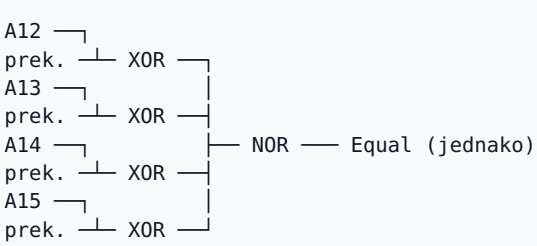
- **A_0–A_9** idu izravno na RAM čipove.
- **A_{10} i A_{11}** biraju jednu od četiri banke.
- **A_{12}–A_{15}** određuju **na koje adrese ploča odgovara**: jedan od 16 raspona od 4 KB (0000h–0FFFh, 1000h–1FFFh, ..., F000h–FFFFh).

Primjer (vlastiti): ploča odgovara na **6000h – 6FFFh** (A_{15}–A_{12} = 0110).

Banka	Adrese
1.	6000h – 63FFh
2.	6400h – 67FFh
3.	6800h – 6BFFh
4.	6C00h – 6FFFh

Komparator i DIP prekidač

Raspon adresa obično se može naknadno odabrati pomoću **DIP prekidača**: niza sićušnih prekidača (2 do 12) u DIP kućištu koje ide u obično podnožje za čip. Četiri prekidača uspoređuju se s A_{12}–A_{15} u sklopu koji se zove **komparator** (usporednik):



- XOR daje **0 kad su ulazi jednaki** (oba 0 ili oba 1).
- Ako su sva četiri adresna bita jednaka položajima prekidača, sva XOR vrata daju 0, pa NOR daje **Equal = 1**.

Za naš primjer (0110) zatvore se prekidači za A_{14} i A_{13}.

Signal **Equal** zajedno s A_{10} i A_{11} ide u **dekoder 2 na 4** koji daje signale **$\overline{\text{CS}}$** za četiri banke. $\overline{\text{CS}}$ je aktivan kad je **0** (crta iznad imena to označava). Npr. kad je A_{10} = 0, a A_{11} = 1, odabrana je **treća** banka.

Izlaz s tri stanja

Prema poglavlju 16 očekivali bismo da trebaju i **selektori 4 na 1** da se izabere izlaz jedne od četiri banke. Ne trebaju. Evo zašto.

Normalno se izlazi čipova **ne smiju spajati**. Izlazi TTL-kompatibilnih čipova su iznad 2,2 V (1) ili ispod 0,4 V (0). Ako je jedan izlaz 1, a drugi 0, a spojeni su, ne zna se što će biti.

> Izlaz čipa 2102 je izlaz s **tri stanja** (*3-state*, *tri-state*). Osim 0 i 1 ima i treće stanje: **ništa**, kao da na nožicu ništa nije spojeno. U treće stanje prelazi kad je $\overline{\text{CS}}$ = 1.

Zato se odgovarajući izlazi svih četiriju banaka **smiju spojiti** i voditi na 8 ulaznih podatkovnih signala sabirnice.

Važno za sabirnicu: sve što je spojeno na sabirnicu koristi ulazne podatkovne signale. U svakom trenutku **samo jedna** pločica smije određivati njihovu vrijednost; sve ostale moraju biti spojene preko **neodabranih** izlaza s tri stanja.

Analogija: sastanak na kojem govori samo onaj kome je predsjedavajući dao riječ. Ostali ne šute tako da kažu "ništa", nego su potpuno isključeni iz razgovora.

SRAM i DRAM

	SRAM (statički RAM)	**DRAM** (dinamički RAM)
Tranzistora po bitu	oko 4	**1**
Pomoćni sklopovi	jednostavni	složeniji

	SRAM (statički RAM)	DRAM (dinamički RAM)
Bez napajanja	gubi sadržaj	gubi sadržaj
Osvježavanje	ne treba	**treba**: sadržaj se mora periodički čitati, i kad nije potreban, **nekoliko stotina puta u sekundi**

2102 je SRAM. **Osvježavanje** (*refresh*) kod DRAM-a je kao povremeno drmanje nekoga da ne zaspi.

Unatoč tome, DRAM je postao **standard** zbog sve većeg kapaciteta:

- Intel je 1975. predstavio DRAM čip od **16.384 bita**,
- kapacitet DRAM čipova (u skladu s Mooreovim zakonom) otprilike se **učetverostručuje svake tri godine**,
- memorija se danas stavlja u utore na sistemskoj ploči kao male pločice **SIMM** ili **DIMM** s više DRAM čipova (1999.: DIMM od 128 MB za manje od 300 dolara).

Ne treba sav adresni prostor popuniti memorijom: dio treba ostaviti za **izlazni uređaj**.

Video zaslon

Katodna cijev

Najčešći izlazni uređaj računala postala je **katodna cijev** (CRT, *cathode-ray tube*), poznata iz televizora. Spojena na računalo zove se **video zaslon** ili **monitor**. Elektronika koja daje signal zaslonu je **video adapter**, obično na zasebnoj **video pločici**.

Slika je zapravo **jedna neprekidna zraka svjetla** koja vrlo brzo prelazi zaslonom:

1. Zraka kreće u **gornjem lijevom** kutu i ide udesno.
2. Na kraju se brzo vrati lijevo za sljedeći red. Svaki vodoravni red je **linija** (*scan line*), a povratak je **vodoravni povratak** (*horizontal retrace*).
3. Kad završi zadnju liniju, zraka se iz donjeg desnog kuta vrati u gornji lijevi (**okomiti povratak**, *vertical retrace*).

Pojam	Vrijednost (američka TV)
Frekvencija polja (*field rate*)	**60 polja u sekundi**: dovoljno brzo da slika ne treperi
Isprepleteni prikaz (*interlaced*)	jedna potpuna **slika** (*frame*) = 2 **polja**: jedno s parnim, drugo s neparnim linijama
Vodoravna frekvencija	**15.750 Hz**
Linija po polju	15.750 / 60 = **262,5**
Linija po slici	**525**

Video signal (crno-bijeli)

Dio signala	Napon	Trajanje / učestalost
Okomiti sinkronizacijski impuls (početak polja)	0 V	oko 400 µs, 60 puta u sekundi
Vodoravni sinkronizacijski impuls (početak linije)	0 V	5 µs, 15.750 puta u sekundi
Slika između impulsa	0,5 V (crno) do 2 V (bijelo)	naponi između daju nijanse sive

Televizijska slika je dakle **djelomično digitalna** (525 odvojenih linija), a **djelomično analogna** (unutar linije napon se mijenja neprekidno).

Pojasna širina i pikseli

Napon se ne može mijenjati proizvoljno brzo. Gornja granica brzine kojom se signal može promijeniti od crnog do bijelog i natrag zove se **pojasna širina** (*bandwidth*). Za američku televiziju to je oko **4,2 MHz**. Pojasna širina je vrlo važan pojam u komunikacijama: povezana je s količinom informacije koju neki medij može prenijeti.

Računalu je lakše zaslon smatrati **potpuno digitalnim**: slika je pravokutna mreža odvojenih točaka, **piksela** (*picture element*, element slike).

Koliko piksela po liniji?

- 4,2 MHz omogućuje dva piksela 4,2 milijuna puta u sekundi.
- 2 × 4.200.000 / 15.750 = **533** piksela po liniji.
- Oko trećine nije vidljivo (na rubovima slike ili tijekom povratka): ostaje oko **320** korisnih piksela.

Koliko okomito? Ne 525: dio se izgubi na vrhu, dnu i u okomitom povratku, a računalu je zgodnije **ne** koristiti isprepleteni prikaz. Razuman broj je **200**.

> **Razlučivost** (*resolution*) jednostavnog video adaptera spojenog na televizor: **320 × 200** piksela = 64.000 piksela.

Tekst na zaslonu

Ako svaki znak zauzima mrežu **8 × 8** (64 piksela), na zaslon 320 × 200 stane **25 redaka po 40 znakova**. Svaki znak ima 7-bitni ASCII kod, ali i 64 bita koji određuju njegov **izgled** (i to se može smatrati kodom). Kodovi 00h–1Fh nemaju vidljiv oblik.

Koliko RAM-a treba video adapteru? Između 1 KB i 192 KB, ovisno o vrsti. RAM adaptera najzgodnije je dio **običnog memorijskog prostora** procesora, da procesor u njega može pisati.

Samo tekst: 25 × 40 = 1000 znakova; svaki 7-bitni ASCII kod; oko **1 KB**.

Takav adapter mora imati i **generator znakova** s uzorcima piksela svih znakova, obično u **ROM-u**.

> **ROM** (*read-only memory*, memorija samo za čitanje) je čip proizveden tako da određena adresa **uvijek** daje određeni izlaz. Za razliku od RAM-a nema ulaznih podatkovnih signala. ROM se može shvatiti kao sklop koji **jedan kod pretvara u drugi**.

ROM za 128 znakova sa 7 adresnih i 64 izlazna signala bio bi prevelik. Praktičnije: **10 adresnih** i **8 izlaznih** signala.

- 7 adresnih bitova = ASCII kod (dolazi iz RAM-a video pločice),
- 3 adresna bita = redak znaka (000 gornji, 111 donji),
- 8 izlaznih bitova = 8 piksela tog retka.

Primjer (vlastiti uzorak): slovo **H**, ASCII 48h = 1001000.

Adresa (ASCII + redak)	Izlaz
1001000 000	11001100
1001000 001	11001100
1001000 010	11001100
1001000 011	11111100
1001000 100	11001100
1001000 101	11001100
1001000 110	11001100

Adresa (ASCII + redak)	Izlaz
1001000 111	00000000

Jedinice "nacrtaju" slovo H na pozadini nula.

Tekstualni adapter treba i logiku za **kursor** (crtica koja pokazuje gdje će se pojaviti sljedeći znak). Redak i stupac kursora čuvaju se u dva 8-bitna registra u koje procesor može pisati.

Grafički adapter i boje

Adapter koji nije ograničen na tekst je **grafički**. Procesor upisom u njegov RAM može crtati slike i tekst u raznim veličinama i stilovima. Treba više memorije:

Način	Po pikselu	RAM za 320 × 200
Dvije boje (npr. crno i bijelo)	1 bit	64.000 bitova = **8000 bajtova**
256 nijansi sive (00h crno, FFh bijelo)	1 bajt	**64.000 bajtova** (gotovo cijeli adresni prostor 8-bitnog procesora)
Puna boja	3 bajta (crvena, zelena, plava)	**192.000 bajtova**

Pod povećalom se vidi da je svaka boja na zaslonu kombinacija osnovnih boja: **crvene, zelene i plave**. Za puni raspon boja treba po jedan bajt za intenzitet svake od njih.

> **Broj boja = 2^(broj bitova po pikselu)**

Monitori za računala

Razlučivost 320 × 200 je otprilike najbolje što daje običan televizor. Zato monitori za računala imaju mnogo veću pojasnu širinu.

- Prvi monitori uz IBM PC (1981.): **25 redaka po 80 znakova**, kao na IBM-ovim terminalima velikih računala. Zašto 80? Jer toliko znakova stane na **IBM-ovu bušenu karticu**. Retci tekstualnog zaslona ponekad se i zovu "kartice".
- 1987.: IBM PS/2 i Apple Macintosh II uvode **640 × 480**, odonda minimalni standard.

Omjer stranica (*aspect ratio*): omjer širine i visine slike. Oko 1889. Edison i Dickson za filmsku kameru odabrali su sliku za trećinu širu nego višu: **4:3** (1,33:1). Taj omjer koristio se za filmove više od 60 godina i za televiziju; tek su 1950-ih uvedeni širokokutni formati.

Monitori su također 4:3, a i 640 × 480 je 4:3. Zato je vodoravna crta od 100 piksela jednako duga kao okomita od 100 piksela: **kvadratni pikseli**, poželjno svojstvo za grafiku.

Današnji adapteri (1999.) gotovo uvijek podržavaju 640 × 480, ali i druge **načine rada**: 800 × 600, 1024 × 768, 1280 × 960, 1600 × 1200.

Tipkovnica

Zaslon i tipkovnica fizički su odvojeni, iako ono što tipkaš vidiš na zaslonu.

- Svaka tipka je jednostavan **prekidač**, zatvoren dok je tipka pritisnuta.
- Tipkovnica slična pisaćem stroju može imati samo 48 tipki; tipkovnice PC-a često imaju više od 100.

Hardver tipkovnice mora dati **jedinstveni kod** za svaku tipku. To **nije** ASCII kod:

- tipka A može značiti 41h ili 61h, ovisno o tome je li pritisnut Shift,
- mnoge tipke nemaju ASCII znak.

Hardver daje **kod tipke** (*scan code*), a kratak program odredi koji ASCII znak (ako ijedan) odgovara pritisnutoj tipki.

Sklop za 16 tipki

Za 16 tipki treba 4-bitni kod (0000 – 1111). Sklop koristi poznate dijelove:

- **oscilator** i **4-bitno brojilo** koje vrlo brzo i neprestano prolazi kroz 16 kodova (brže nego što čovjek može pritisnuti i pustiti tipku),
- tipke složene u **matricu 4 × 4**,
- **dekoder 2 na 4** (pokreće retke matrice) i **selektor 4 na 1** (čita stupce); izborne ulaze obaju daju izlazi brojila,
- **4-bitni zasun**.

Rad:

1. Ako nijedna tipka nije pritisnuta, izlaz selektora nikad nije 1.
2. Ako je tipka pritisnuta, izlaz selektora postaje 1 točno kad brojilo pokaže **njezin kod**. Nijedan drugi kod ne daje 1 za tu tipku.
3. Taj izlaz **upisuje** trenutnu vrijednost brojila u zasun: to je kod tipke.
4. Isti izlaz može poslužiti kao signal **prekida** procesoru, koji tada izvede RST i skoči na program koji obrađuje tipku.

Broj tipki	Kod	Matrica	Dekoder + selektor
16	4 bita	4 × 4	2 na 4 + 4 na 1
64	6 bitova	8 × 8	3 na 8 + 8 na 1
65 – 128	7 bitova	8 × 16	4 na 16 + 8 na 1 (ili 3 na 8 + 16 na 1)

Napredniji sklop može imati **1 bit RAM-a po tipki** (adresiran brojilom): 0 ako je tipka gore, 1 ako je dolje. Procesor može čitati taj RAM i znati stanje svake tipke.

Trajna pohrana

RAM (od releja, cijevi ili tranzistora) gubi sadržaj kad se isključi struja. Računalo zato treba i nešto za **dugotrajno** spremanje.

Bušeni papir

Stari način: rupe u papiru ili kartonu (IBM-ove kartice, papirnata vrpca u ranim malim računalima). Problemi:

- medij nije ponovno upotrebljiv (rupa se ne može "zatvoriti"),
- nije učinkovit: ako bit možeš **vidjeti**, zauzima previše prostora.

Magnetska pohrana

Tko, kada	Što
Oberlin Smith, 1878.	opisao načela
Valdemar Poulsen, 1898.	prvi uređaj koji radi, **telegrafon**: snimao telefonske poruke na **čeličnu žicu** pomoću elektromagneta
Fritz Pfleumer, 1928.	patent za snimanje na dugu papirnatu vrpcu presvučenu česticama željeza; papir zamijenjen čvršćom podlogom od celuloznog acetata: **magnetska vrpca**
Remington Rand, 1950.	prvi komercijalni sustav vrpce za **digitalne** podatke; kolut vrpce od pola inča sprema nekoliko MB

Princip: elektromagnet magnetizira žicu (vrpcu) razmjerno valnom obliku zvuka. Magnetizirana žica, kad prolazi pokraj zavojnice elektromagneta, inducira struju istog oblika. Elektromagnet za spremanje i čitanje zove se **glava** (*head*), bez obzira na vrstu medija.

U ranim kućnim računalima koristili su se obični **kazetofoni**: program je spremio sadržaj dijela memorije na vrpcu i kasnije ga vratio. Prvi IBM PC imao je priključak za kazetu. Vrpca je i danas popularna za **dugotrajno arhiviranje**, ali nije idealna: nema brzog pristupa proizvoljnom mjestu, treba premotavati.

Disk

Disk je geometrijski pogodniji za brz pristup. Vrti se oko središta, a jedna ili više glava na ručicama pomiče se od ruba prema sredini. **Bilo kojem dijelu** diska može se brzo pristupiti.

Prvi disk za računalne podatke: IBM, 1956., **RAMAC**: 50 metalnih diskova promjera 2 stope (oko 60 cm), **5 MB**. (Za zvuk je magnetski disk čak stariji od vrpce.)

	Disketa (***floppy disk***, ***diskette***)	**Tvrdi disk** (***hard disk***, ***fixed disk***)
Građa	jedan list presvučene plastike u zaštitnom omotu od kartona, kasnije tvrde plastike	više metalnih diskova trajno ugrađenih u pogon
Pogon	čovjek je umeće u **disketni pogon** (*drive*)	disk i pogon su jedno
Prednost	može se izvaditi: prijenos podataka među računalima, distribucija softvera	**brži** i veći kapacitet
Veličine	8 inča (rane), 5¼ inča (prvi IBM PC), 3,5 inča (danas najčešće)	—

Staze i sektori

Površina diska podijeljena je na:

- **staze** (*tracks*): koncentrične prstenove,
- **sektore** (*sectors*): dijelove staze, kao kriške torte. Sektor sadrži određen broj bajtova, obično **512**.

> **Kapacitet = strane × staze po strani × sektori po stazi × bajtovi po sektoru**

Disk	Račun	Kapacitet
Disketa prvog IBM PC-a (5¼", jedna strana)	1 × 40 × 8 × 512	163.840 B = **160 KB**
Disketa 3,5" (PC-kompatibilna)	2 × 80 × 18 × 512	1.474.560 B = **1440 KB**

Prvi IBM-ov tvrdi disk za PC (PC-XT, 1983.) imao je **10 MB**; 1999. disk od **20 GB** stoji manje od 400 dolara.

Sučelja diska

Disketa i tvrdi disk imaju vlastito električno sučelje i trebaju još jedno sučelje prema procesoru. Popularni standardi za tvrde diskove: **SCSI**, **ESDI** i **IDE**. Svi koriste **DMA**: preuzmu sabirnicu i prenose podatke izravno između RAM-a i diska, zaobilazeći procesor. Prijenos se odvija u komadima veličine sektora (obično 512 bajtova).

Memorija i pohrana

Novi korisnici često brkaju RAM i diskove. Pravilo nazivlja:

Riječ	Koristi se za
Memorija (*memory*)	samo poluvodički RAM
Pohrana (*storage*)	sve ostalo: diskete, tvrde diskove, vrpce

(Knjiga se drži tog pravila, iako postoje strojne instrukcije Store koje spremaju bajtove u RAM.)

	Memorija	Pohrana
Bez napajanja	**nepostojana**: gubi sadržaj	**postojana**: podaci ostaju dok se namjerno ne obrišu ili prepišu
Pristup procesora	procesor je **adresira izravno**	procesor je **ne adresira**; podatke prvo treba prenijeti u memoriju

Kad procesor postavi adresu, uvijek adresira **memoriju**, ne pohranu. Da bi podatak s diska bio dostupan, procesor mora izvesti kratak program koji naredi pogonu da prenese podatke s diska u memoriju.

Analogija (iz knjige): memorija je **površina radnog stola**: s onim što je na stolu radiš izravno. Pohrana je **ormar s fasciklima**: da nešto upotrijebiš, moraš ustati, otići do ormara, izvaditi fascikl i donijeti ga na stol. Kad je stol pretrpan, nešto vratiš u ormar.

Analogija je posebno prikladna jer se podaci na disku spremaju u cjeline koje se zovu **datoteke** (*files*). Spremanje i dohvat datoteka posao je vrlo važnog softvera: **operacijskog sustava** (poglavlje 22).

Ključne definicije

Pojam	Značenje
Sabirnica	skup signala (adresa, podaci, upravljanje, napajanje) zajednički svim pločicama
Matična ploča / utor za proširenje	ploča s utorima / utor za pločicu
Otvorena arhitektura	javna specifikacija sabirnice; drugi mogu raditi proširenja
DMA	uređaj preuzima sabirnicu i izravno pristupa memoriji
Banka	čipovi koji zajedno spremaju bajt
$\overline{CS}$ (odabir čipa)	signal koji aktivira čip (aktivan kad je 0)
Komparator	sklop koji provjerava jesu li dvije vrijednosti jednake
Izlaz s tri stanja	izlaz koji može biti 0, 1 ili "odspojen"
SRAM / DRAM	statički (bez osvježavanja) / dinamički (1 tranzistor po bitu, treba osvježavanje)
Linija, polje, slika	vodoravni red / pola slike (parne ili neparne linije) / cijela slika
Pojasna širina	najveća brzina promjene signala; mjera kapaciteta medija
Piksel	točka slike
Razlučivost	broj piksela vodoravno × okomito
ROM	memorija samo za čitanje; adresa uvijek daje isti izlaz
Generator znakova	ROM s uzorcima piksela znakova
Kod tipke (scan code)	kod koji hardver tipkovnice daje za tipku
Staza / sektor	prsten na disku / dio staze (obično 512 B)
Memorija / pohrana	poluvodički RAM / diskovi i vrpce

Ključno za zapamtiti

1. Sabirnica nosi adresne, ulazne i izlazne podatkovne i upravljačke signale te napajanje.
2. Otvorena arhitektura IBM PC-a omogućila je pločice drugih proizvođača i klonove.
3. Memorijska ploča dijeli adresu: viši bitovi biraju ploču, srednji banku, niži adresu u čipu.
4. Komparator od vrata XOR i NOR provjerava odgovara li adresa postavkama DIP prekidača.
5. Izlazi s tri stanja omogućuju da više uređaja dijeli iste podatkovne linije; samo jedan smije biti aktivan.
6. DRAM ima 1 tranzistor po bitu i treba osvježavanje; zbog kapaciteta je postao standard.
7. TV slika: 60 polja u sekundi, 525 linija; pojasna širina 4,2 MHz ograničava na oko 320 × 200 piksela.

8. Tekstualni zaslon treba malo RAM-a i generator znakova u ROM-u; grafički mnogo više (do 3 bajta po pikselu).
9. Tipkovnica daje kod tipke, a ne ASCII; softver ga pretvara.
10. Kapacitet diska = strane × staze × sektori × bajtovi po sektoru; procesor adresira samo memoriju, ne pohranu.

Najčešće zabune

- **"Izlazi čipova mogu se slobodno spajati."** Samo izlazi s tri stanja, i to samo ako je u svakom trenutku najviše jedan aktivan.
- **"$\overline{CS}$ = 1 znači da je čip odabran."** Crta iznad imena znači da je signal aktivan kad je **0**.
- **"Tipkovnica šalje ASCII kod."** Šalje kod tipke; ASCII odredi softver (npr. ovisno o Shiftu).
- **"Disk je također memorija."** Prema dogovoru, disk je **pohrana**. Procesor ga ne može izravno adresirati.
- **"Grafički zaslon 320 × 200 uvijek treba 8000 bajtova."** Samo s 1 bitom po pikselu; za 256 nijansi sive treba 64.000, a za punu boju 192.000 bajtova.

Brza provjera razumijevanja

1. Memorijska ploča od 4 KB treba odgovarati na adrese 3000h – 3FFFh. Koje su vrijednosti A_{15}–A_{12} i koje adrese pripadaju drugoj banci?
2. Zašto se izlazi četiri banke memorije smiju spojiti bez selektora?
3. Koliko RAM-a treba grafički adapter 640 × 480 s 256 boja? A s 2 boje?
4. Izračunaj kapacitet diska s 2 strane, 80 staza i 9 sektora po stazi od 512 bajtova.
5. Objasni razliku između memorije i pohrane s dva svojstva.

22. Operacijski sustav

Što ćeš naučiti u ovom poglavlju

- Zašto novo računalo bez softvera ne radi ništa korisno.
- Kako se prvi programi unose **upravljačkom pločom** i kako se broj prikazuje na zaslonu (pretvorba u ASCII).
- Što su **inicijalizacijski kod**, **obrađivač tipkovnice** i **procesor naredbi**.
- Što su **PROM** i **EPROM** i zašto se osnovni softver stavlja u ROM.
- Što je **datotečni sustav** i kako radi u **CP/M**-u (blokovi, direktorij, ime 8.3).
- Razliku između **tekstualnih** i **binarnih** datoteka.
- Što je **podizanje sustava** (*booting*) i kako izgleda memorija nakon učitavanja CP/M-a.
- Tri glavne funkcije operacijskog sustava, uključujući **API**.
- Osnove sustava **MS-DOS** (FAT, hijerarhijski datotečni sustav) i **UNIX** (prenosivost, višezadaćnost, virtualna memorija).

Računalo bez softvera

Zamislimo da smo sastavili računalo: mikroprocesor, RAM, tipkovnicu, video zaslon i disketni pogon. Uključimo ga. Što vidimo?

Zaslon pokazuje niz savršeno oblikovanih, ali **nasumičnih** ASCII znakova. To je očekivano: poluvodička memorija kad dobije struju počinje u nasumičnom stanju. I RAM procesora sadrži nasumične bajtove, a procesor ih počinje izvoditi kao strojni kod. Ništa loše se neće dogoditi (računalo neće eksplodirati), ali neće biti ni ničeg korisnog.

> Nedostaje **softver**. Nakon uključivanja ili RESET-a procesor počinje izvoditi strojni kod na određenoj adresi (kod 8080 to je **0000h**). Na toj adresi mora biti smislena instrukcija.

Prvi korak: upravljačka ploča

Kako strojni kod dospije u memoriju? Jedan način je upravljačka ploča slična onoj iz poglavlja 16, uz jedan dodatni prekidač: **Reset**, spojen na ulaz RESET procesora.

Element	Uloga
Reset	dok je uključen, procesor ne radi ništa; kad se isključi, procesor počinje izvoditi kod
Takeover	ploča preuzima adresne i podatkovne signale sabirnice
A_0–A_{15}	16-bitna adresa
D_0–D_7 + žaruljice	bajt za upis / prikaz sadržaja adrese
Write	upis bajta

Postupak:

1. Uključi **Reset** (procesor stoji) i **Takeover**.
2. Prekidačima postavi adresu; žaruljice pokazuju sadržaj.
3. Za upis: postavi bajt na D_0–D_7, uključi i isključi **Write**.
4. Kad su svi bajtovi upisani, isključi **Takeover** i **Reset**. Procesor izvodi program.

Mukotrpno je i lako se griješi, ali tako se unose prvi programi u potpuno novo računalo. Programi se pišu na papiru u asemblerskom jeziku i **ručno asembliraju** (to ćemo raditi do poglavlja 24).

Prikaz broja na zaslonu

Tekstualni zaslon iz poglavlja 21 ima 1 KB RAM-a s ASCII kodovima 25 redaka po 40 znakova. Program piše u tu memoriju kao u svaku drugu.

Ali ako je rezultat izračuna npr. **9Eh**, ne smijemo samo upisati 9Eh, jer bi se pojavio znak s tim ASCII kodom. Treba upisati **dva** ASCII znaka: "9" (**39h**) i "E" (**45h**). Svaki nibl rezultata je jedna heks znamenka i prikazuje se ASCII kodom te znamenke.

Potprogram koji nibl (00h – 0Fh) u akumulatoru pretvara u ASCII:

```
NibbleToAscii: CPI A,0Ah   ; slovo ili znamenka?
               JC  Number
               ADD A,37h   ; A–F (0Ah–0Fh) postaju 41h–46h
               RET
Number:        ADD A,30h   ; 0–9 postaju 30h–39h
               RET
```

Zašto 37h? Jer 0Ah + 37h = 41h, a to je "A".

Potprogram koji bajt iz A pretvara u dvije ASCII znamenke u registrima B i C:

```
ByteToAscii:   PUSH PSW            ; spremi akumulator
               RRC                 ; rotiraj A udesno 4 puta...
               RRC
               RRC
               RRC                 ; ...viši nibl je sada dolje
               ANI A,0Fh           ; ostavi samo donji nibl
               CALL NibbleToAscii  ; u ASCII
               MOV B,A             ; rezultat u B
               POP PSW             ; vrati izvorni A
               ANI A,0Fh           ; niži nibl
               CALL NibbleToAscii  ; u ASCII
               MOV C,A             ; rezultat u C
               RET
```

(U ovoj inačici dodan je `ANI A,0Fh` nakon rotacija: četiri rotacije premjeste niži nibl na gornje mjesto, pa ga treba ukloniti prije pretvorbe.)

Za prikaz u **dekadskom** zapisu posla je više: postupak je kao kad čovjek pretvara heks u dekadski, nekoliko dijeljenja s 10.

Tipkovnica umjesto ploče

Upravljačka ploča je najgori mogući oblik ulaza i izlaza. Prvi prioritet je riješiti je se. Ključ je **tipkovnica**: svaki pritisak tipke izaziva **prekid**. Neka kontroler prekida tada pošalje instrukciju **RST 1**: procesor spremi programsko brojilo na stog i skoči na **0008h**. Od te adrese upišemo (pločom) kod koji zovemo **obrađivač tipkovnice** (*keyboard handler*).

Inicijalizacijski kod

Kod koji se izvodi nakon RESET-a zove se **inicijalizacija**:

1. postavi **pokazivač stoga** na ispravno područje memorije,
2. u svaki bajt video memorije upiše **20h** (razmak), da nestanu nasumični znakovi,
3. instrukcijom **OUT** postavi **kursor** u prvi stupac prvog retka,
4. instrukcijom **EI** uključi prekide,

5. izvede **HLT** i zaustavi procesor.

Od tada je računalo uglavnom zaustavljeno. Iz tog stanja ga mogu probuditi samo **Reset** s ploče ili **prekid** s tipkovnice.

Obrađivač tipkovnice

Kad se pritisne tipka, prekid prebaci procesor s HLT-a na obrađivač tipkovnice. On instrukcijom **IN** pročita koja je tipka pritisnuta, **obradi** je i instrukcijom **RET** vrati se na HLT da čeka sljedeću.

Tipka	Što obrađivač radi
slovo, broj, interpunkcija	iz koda tipke (uz stanje Shifta) odredi ASCII kod, upiše ga u video memoriju na mjesto kursora i pomakne kursor za jedno mjesto. To je **jeka** (*echoing*) tipke na zaslon
Backspace (ASCII 08h)	izbriše zadnji znak (upiše 20h na to mjesto) i vrati kursor jedno mjesto
Enter / Return (ASCII 0Dh)	redak teksta na zaslonu tumači kao **naredbu**

Procesor naredbi

Dio obrađivača tipkovnice koji tumači naredbe je **procesor naredbi** (*command processor*). Neka razumije tri naredbe:

Naredba	Primjer (vlastiti)	Značenje
W (*Write*)	`W 2000 3E 2A 32 00 30 76`	upiši bajtove 3Eh, 2Ah… od adrese 2000h. Obrađivač mora ASCII znakove pretvoriti u bajtove (obrnuto od pretvorbe gore)
D (*Display*)	`D 3000`	prikaži 11 bajtova od adrese 3000h (toliko stane u redak od 40 znakova iza adrese)
R (*Run*)	`R 2000`	pokreni program od 2000h: adresa se stavi u HL i izvede **PCHL** (skok na adresu iz HL)

Upisani bajtovi u primjeru su program `MVI A,2Ah` / `STA [3000h],A` / `HLT`. Nakon `R 2000` naredba `D 3000` pokazala bi 2Ah na prvom mjestu.

Ovo je velik korak: unos bajtova tipkovnicom je lakši i brži, a računalo je postalo **interaktivno**: kad nešto utipkaš, ono odgovori na zaslonu.

ROM, PROM, EPROM

Problem: sav taj kod nestaje kad se isključi struja. Zato ga želimo spremiti u **ROM**.

Vrsta	Programiranje
ROM	sadržaj se upisuje pri proizvodnji
PROM (*programmable ROM*)	može se programirati **jednom**
EPROM (*erasable PROM*)	može se programirati više puta; prije toga se potpuno briše izlaganjem **ultraljubičastom svjetlu**

Na sustavu s 8080 jedna RAM ploča isprva je bila na adresi 0000h. Kad napravimo ROM, **ROM zauzima 0000h** (tamo procesor počinje), a RAM ploča se DIP prekidačem premjesti na više adrese.

Disk i potreba za datotečnim sustavom

S procesorom naredbi u ROM-u možemo eksperimentirati s **diskom**: pisati blokove memorije na disk (u komadima veličine sektora) i čitati ih natrag. Programi i podaci na disku su sigurniji nego u RAM-u (gdje nestaju bez struje) i fleksibilniji nego u ROM-u.

Nove naredbe (vlastiti primjer):

```
S 4000 1 20 5     ; spremi blok memorije od 4000h na stranu 1, stazu 20, sektor 5
L 4000 1 20 5     ; učitaj taj sektor natrag u memoriju od 4000h
```

Problemi ručnog pristupa:

- moraš na papiru pamtiti što je gdje spremljeno,
- kod spremljen s jedne adrese ne radi ako se učita na **drugu**: sve adrese u instrukcijama Jump i Call su krive,
- program dulji od sektora treba više sektora,
- slobodni sektori ne moraju biti **uzastopni**, jer su neki zauzeti drugim programima.

Kad ručna evidencija postane prevelik posao, vrijeme je za **datotečni sustav**.

> **Datotečni sustav** (*file system*) je način spremanja na disk u kojem su podaci organizirani u **datoteke**.
> **Datoteka** (*file*) je skup povezanih podataka u jednom ili više sektora, a označena je **imenom**.

Analogija: disk je ormar s fasciklima, a svaki fascikl ima mali jezičac s imenom.

Datotečni sustav gotovo je uvijek dio većeg skupa softvera: **operacijskog sustava** (*operating system*). Obrađivač tipkovnice i procesor naredbi mogli bi postupno izrasti u operacijski sustav. Umjesto toga pogledajmo stvarni.

CP/M

Povijesno najvažniji operacijski sustav za 8-bitne mikroprocesore bio je **CP/M** (*Control Program for Micros*), koji je sredinom 1970-ih za Intel 8080 napisao Gary Kildall (kasnije osnivač tvrtke Digital Research).

Disk u CP/M-u

Najčešći medij: jednostrana disketa od **8 inča**:

> 77 staza × 26 sektora × 128 bajtova = **256.256 bajtova**

Dio diska	Sadržaj
prve 2 staze	sam CP/M
preostalih 75 staza	datoteke

Datotečni sustav CP/M-a je jednostavan, ali ispunjava dva glavna zahtjeva:

1. svaka datoteka ima **ime**, spremljeno na disku zajedno sa svim podacima potrebnima za čitanje datoteke,
2. datoteka **ne mora** zauzimati uzastopne sektore. Kako se datoteke stvaraju i brišu, slobodni prostor postaje **rascjepkan** (fragmentiran), pa je ta mogućnost vrlo korisna.

Blokovi i direktorij

- Sektori na 75 staza grupirani su u **alokacijske blokove** (*allocation blocks*) od **8 sektora = 1024 bajta**. Ima ih **243** (od 0 do 242).
- Prva dva bloka (2048 bajtova) su **direktorij** (*directory*): područje s imenima i ključnim podacima o svakoj datoteci.
- Svaka datoteka ima **zapis u direktoriju** (*directory entry*) od **32 bajta**. Zato disketa može imati najviše 2048 / 32 = **64 datoteke**.

Bajtovi zapisa	Značenje
0	obično 0 (koristi se kad datotečni sustav dijeli više korisnika istovremeno)

Bajtovi zapisa	Značenje
1 – 8	ime datoteke
9 – 11	vrsta datoteke
12	dio datoteke (*extent*)
13 – 14	rezervirano (0)
15	broj sektora u zadnjem bloku
16 – 31	mapa diska (*disk map*)

Ime datoteke 8.3

Ime ima dva dijela odvojena točkom:

- **ime** (*filename*): do 8 znakova,
- **vrsta** (*file type*): do 3 znaka.

Primjeri: `PISMO.TXT`, `RACUN.COM`. Taj dogovor zove se **8.3** ("osam točka tri").

Vrsta	Značenje
TXT	tekstualna datoteka (samo ASCII kodovi)
COM	(*command*) datoteka sa strojnim kodom 8080: **program**

Mapa diska

Mapa diska navodi alokacijske blokove u kojima je datoteka. **Primjer (vlastiti):** ako su prva četiri broja u mapi 31h, 32h, 0Ah i 7Bh, a ostali nule, datoteka zauzima **četiri bloka** (4 KB). Može biti i nešto kraća: bajt 15 kaže koliko je sektora od 128 bajtova stvarno iskorišteno u zadnjem bloku.

Mapa ima 16 bajtova, pa jedan zapis pokriva datoteku do **16 KB** (16.384 bajta). Dulja datoteka ima više zapisa u direktoriju, tzv. **dijelova** (*extents*): bajt 12 je 0 u prvom zapisu, 1 u drugom i tako dalje.

Tekstualne i binarne datoteke

Vrsta	Sadržaj
Tekstualna (ASCII) datoteka	ASCII kodovi (uključujući CR i LF) koji čine tekst čitljiv ljudima
Binarna datoteka	sve ostalo; npr. COM datoteka sa strojnim kodom

Primjer (vlastiti): datoteka treba sadržavati dva 16-bitna broja, **1A2Bh** i **3C4Dh**.

Način spremanja	Bajtovi
Binarno, Intelov poredak (niži bajt prvi)	`2B 1A 4D 3C` (4 bajta)
Binarno, Motorolin poredak	`1A 2B 3C 4D`
Tekst, heks zapis	`31 41 32 42 68 0D 0A 33 43 34 44 68 0D 0A`
Tekst, dekadski zapis	`36 36 39 39 0D 0A 31 35 34 33 37 0D 0A`

Tekstualna datoteka s heks zapisom prikazuje se kao:

```
1A2Bh
3C4Dh
```

a s dekadskim kao:

```
6699
15437
```

Svaki broj završava s CR (0Dh) i LF (0Ah). Budući da su tekstualne datoteke namijenjene ljudima, nema razloga ne koristiti dekadski zapis.

Podizanje sustava

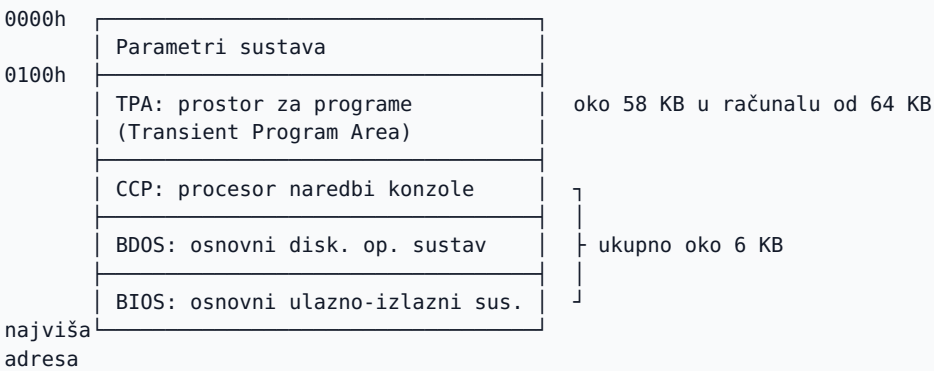
CP/M je spremljen na prve dvije staze diska. Da bi radio, mora se učitati u memoriju.

- ROM računala s CP/M-om ne mora biti velik. Dovoljan je mali program, **punjač za podizanje** (*bootstrap loader*; "vuče ostatak sustava za vezice na čizmama").
- Punjač učita **prvi sektor** (128 bajtova) s diskete u memoriju i pokrene ga.
- Taj sektor sadrži kod koji učita **ostatak** CP/M-a.

> Cijeli postupak zove se **podizanje** operacijskog sustava (*booting*).

Memorija nakon učitavanja CP/M-a

CP/M se smješta u RAM s **najvišim** adresama:



(Shema nije u mjerilu.)

Dio	Uloga
CCP (*Console Command Processor*)	procesor naredbi; **konzola** = tipkovnica + zaslon
BDOS (*Basic Disk Operating System*)	uglavnom održava datotečni sustav
BIOS (*Basic Input/Output System*)	stvarno pristupa hardveru: tipkovnici, zaslonu, pogonima
TPA (*Transient Program Area*)	prostor u koji se učitavaju programi; na početku prazan

Naredbe CCP-a

CCP prikazuje **odzivni znak** (*prompt*):

```
A>
```

To je znak da možeš tipkati. Slovo A označava prvi disketni pogon, s kojeg je CP/M učitan. Utipkaš naredbu i pritisneš Enter; CCP je izvede (obično nešto ispiše) i ponovno prikaže odzivni znak.

Naredba	Primjer	Što radi
DIR	`DIR`, `DIR *.TXT`, `DIR A???B.*`	prikazuje direktorij; `*` i `?` ograničavaju popis. `*.TXT` su sve tekstualne datoteke, a `A???B.*` sve datoteke s imenom od 5 znakova koje počinje s A i završava s B

Naredba	Primjer	Što radi
ERA (*erase*)	`ERA PISMO.TXT`, `ERA *.TXT`	briše datoteku: oslobađa zapis u direktoriju i prostor na disku
REN (*rename*)		mijenja ime datoteke
TYPE	`TYPE PISMO.TXT`	ispisuje sadržaj tekstualne datoteke na zaslon
SAVE		sprema jedan ili više blokova od 256 bajtova iz TPA u datoteku

Ako naredba nije poznata, CP/M pretpostavlja da je to **ime programa** na disku (vrste **COM**). Npr. `RACUN` → CCP potraži `RACUN.COM`, učita ga u memoriju od **0100h** i skoči na 0100h. Zato programi za CP/M moraju biti napisani za učitavanje na 0100h.

Uz CP/M dolaze korisni programi:

- **PIP** (*Peripheral Interchange Program*): kopiranje datoteka,
- **ED**: uređivač teksta.

Mali programi za jednostavne poslove zovu se **uslužni programi** (*utilities*). Veći programi, poput uređivača teksta i proračunskih tablica, su **aplikacije** (*application programs*). I jedni i drugi su COM datoteke.

Tri funkcije operacijskog sustava

Funkcija	U CP/M-u
1. Naredbe i uslužni programi za osnovno održavanje datoteka	DIR, ERA, REN, TYPE, PIP…
2. Učitavanje programa u memoriju i pokretanje	ime programa → učitaj na 0100h → skoči
3. Jednostavan pristup hardveru za programe: **API**	CALL 5

API

Program obično **ne** piše izravno u video memoriju, **ne** čita hardver tipkovnice i nikako **ne** pristupa izravno hardveru disketnog pogona. Umjesto toga koristi zbirku potprograma ugrađenih u CP/M. Program ne mora znati ništa o sektorima i stazama; može spremati i čitati **cijele datoteke**.

> Pristup koji operacijski sustav daje programima zove se **sučelje za programiranje aplikacija** (*application programming interface*, **API**).

Program pod CP/M-om koristi API tako da u registar **C** stavi broj **funkcije** i izvede **CALL 5**:

Funkcija	Priprema	Rezultat
`MVI C,01h` / `CALL 5`		u A je ASCII kod utipkane tipke
`MVI C,02h` / `CALL 5`	znak u A	znak se ispiše na mjestu kursora, kursor se pomakne
`MVI C,16h` / `CALL 5`	DE pokazuje na područje s imenom datoteke	stvara se prazna datoteka

Nakon stvaranja program može pisati u datoteku i na kraju je **zatvoriti** (završio je s njom). Isti ili drugi program može je kasnije **otvoriti** i čitati.

Kako to radi: CP/M na adresu **0005h** stavlja instrukciju **JMP** prema BDOS-u, gdje su potprogrami za sve funkcije. BDOS za pristup hardveru koristi potprograme BIOS-a.

> **BIOS je jedini dio CP/M-a koji mora poznavati hardver računala.** CCP i uslužni programi sve rade preko funkcija BDOS-a.

API je **neovisan o uređajima** (*device-independent*): program ne mora znati kako na određenom stroju rade tipkovnica, zaslon ili disk. Zato isti CP/M program radi na **različitim računalima** s različitim hardverom, ako imaju procesor koji izvodi instrukcije 8080 (8080, 8085, Z-80) i CP/M. Bez standardnog API-ja program bi trebalo prilagoditi svakom računalu.

Analogija: API je kao šalter u banci. Ne moraš znati gdje je trezor i kako se otvara; predaš zahtjev na propisanom obrascu, a službenik (BIOS) obavi ostalo.

MS-DOS

CP/M je snažno utjecao na 16-bitni sustav **QDOS** (*Quick and Dirty Operating System*) koji je Tim Paterson (Seattle Computer Products) napisao za Intelove 8086 i 8088.

- QDOS je preimenovan u 86-DOS i licenciran Microsoftu.
- Kao **MS-DOS** (*Microsoft Disk Operating System*) licenciran je IBM-u za prvi IBM PC (1981.); na IBM-ovim računalima zvao se PC-DOS. Licenciran je i proizvođačima kompatibilnih računala.
- Postojao je i CP/M-86, ali je MS-DOS brzo postao standard.

Datotečni sustav FAT

MS-DOS nije preuzeo datotečni sustav CP/M-a, nego **FAT** (*File Allocation Table*, tablica dodjele datoteka; Microsoft, 1977.):

- prostor diska dijeli se na **klastere** (*clusters*) od 512 do 16.384 bajta, ovisno o veličini diska,
- datoteka je skup klastera,
- zapis u direktoriju navodi samo **početni** klaster,
- **FAT** za svaki klaster na disku kaže koji je **sljedeći** klaster datoteke.

Analogija: potraga za blagom u kojoj svaki trag kaže gdje je sljedeći.

Zapisi direktorija imaju 32 bajta i ime 8.3, ali zadnja tri slova zovu se **nastavak** (*extension*). Zapis ne sadrži popis blokova, nego **datum i vrijeme** zadnje promjene i **veličinu** datoteke.

Ostale osobine

- BIOS nije bio dio MS-DOS-a, jer je IBM PC imao potpuni **BIOS u ROM-u**.
- Procesor naredbi je datoteka **COMMAND.COM**.
- Programi s nastavkom **COM** ograničeni su na 64 KB; veći imaju nastavak **EXE** (*executable*).
- API: isprva i CALL 5, a zatim novije sučelje preko **softverskog prekida**: program poziva funkciju instrukcijom **INT 21h**, a ne mora znati stvarnu adresu potprograma.

Zaobilaženje sustava

U teoriji aplikacije pristupaju hardveru **samo** preko operacijskog sustava. U praksi su mnogi programeri 1970-ih i ranih 1980-ih zaobilazili sustav, posebno kod **video zaslona**: izravno pisanje u video memoriju bilo je brže, a za grafiku je operacijski sustav bio potpuno neprikladan. Programerima se kod MS-DOS-a sviđalo što im "ne smeta".

Posljedica: popularni programi oslanjali su se na posebnosti hardvera IBM PC-a, pa su ih proizvođači kompatibilnih računala morali **oponašati**. Softver je često tražio "IBM Personal Computer ili 100 % kompatibilno".

Hijerarhijski datotečni sustav

MS-DOS 2.0 (ožujak 1983.) podržava **tvrde diskove**. Na većem disku je više datoteka, pa ih je teže pronaći i organizirati. Rješenje je **hijerarhijski datotečni sustav**:

- direktorij je popis datoteka; u hijerarhijskom sustavu neke od tih datoteka **same su direktoriji**, a mogu sadržavati nove direktorije,
- glavni direktorij diska je **korijenski direktorij** (*root directory*),
- direktoriji unutar drugih direktorija su **poddirektoriji**; zovu se i **mape** (*folders*),
- tako se grupiraju povezane datoteke.

Analogija: ladica u ormaru sadrži fascikle, a fascikl može sadržavati manje fascikle.

UNIX

Hijerarhijski datotečni sustav i druge osobine MS-DOS-a 2.0 posuđene su iz **UNIX-a**, koji su ranih 1970-ih u Bell Labsu razvili uglavnom Ken Thompson i Dennis Ritchie. Ime je igra riječi s **Multics**, starijim sustavom koji je Bell Labs razvijao s MIT-om i GE-om.

Osobina	Opis
Prenosivost (*portable*)	dok je većina sustava pisana za određena računala, UNIX se može prilagoditi za rad na mnogim računalima
Licenciranje	AT&T zbog sudskih odluka o telefonskom monopolu isprva nije smio prodavati UNIX, nego ga je od 1973. licencirao sveučilištima, tvrtkama i vladi; 1983. izdao je vlastitu verziju
Mnogo inačica	ne postoji jedan UNIX, nego mnoge verzije pod raznim imenima
"UNIX filozofija"	**tekstualne datoteke** kao zajednički nazivnik: uslužni programi čitaju tekst, obrade ga i zapišu novi tekst; mogu se **povezati u lance**

Dijeljenje vremena i višezadaćnost

UNIX je izvorno pisan za računala prevelika i preskupa za jednu osobu. Takva računala omogućuju da više korisnika radi istovremeno tehnikom **dijeljenja vremena** (*time-sharing*):

- računalo je spojeno na više zaslona i tipkovnica, **terminala**,
- operacijski sustav brzo prebacuje pozornost s terminala na terminal, pa se čini da služi sve **u isto vrijeme**.

Pojam	Značenje
Višezadaćni (*multitasking*) sustav	izvodi više programa istovremeno; složeniji od jednozadaćnih (CP/M, MS-DOS)
Problem datoteka	više korisnika može htjeti koristiti iste datoteke istovremeno
Upravljanje memorijom (*memory management*)	sustav dodjeljuje memoriju različitim programima
Virtualna memorija (*virtual memory*)	kad memorije nema dovoljno, blokovi memorije koji trenutno nisu potrebni spremaju se u privremene datoteke i vraćaju kad zatrebaju

GNU i Linux

- **Free Software Foundation (FSF)** i projekt **GNU** (osnivač Richard Stallman). GNU znači "GNU's Not UNIX": kompatibilan s UNIX-om, ali distribuiran tako da softver **ne može postati vlasnički**.
- Projekt je dao mnoge uslužne programe i alate, te **Linux**: **jezgru** (*kernel*) operacijskog sustava kompatibilnog s UNIX-om, koju je uglavnom napisao Linus Torvalds.

Najvažniji trend od sredine 1980-ih su veliki sustavi s **grafikom** i bogatim prikazom (Apple Macintosh, Microsoft Windows), koji olakšavaju korištenje aplikacija. To je tema poglavlja 25.

Ključne definicije

Pojam	Značenje
Inicijalizacija	kod koji se izvodi nakon RESET-a i priprema računalo
Obrađivač tipkovnice	kod koji se pokreće prekidom tipkovnice i obrađuje tipku
Jeka (echo)	prikaz utipkanog znaka na zaslonu
Procesor naredbi	dio softvera koji tumači utipkane naredbe
PROM / EPROM	ROM koji se programira jednom / koji se briše UV svjetlom i ponovno programira
Datoteka	imenovan skup povezanih podataka na disku
Datotečni sustav	način organiziranja diska u datoteke
Direktorij	područje diska s imenima i podacima o datotekama
Alokacijski blok / klaster	jedinica prostora koja se dodjeljuje datoteci
Tekstualna / binarna datoteka	samo ASCII tekst / bilo koji bajtovi
Podizanje sustava (booting)	učitavanje operacijskog sustava pomoću malog punjača iz ROM-a
BIOS	dio sustava koji izravno pristupa hardveru
API	sučelje preko kojeg programi koriste usluge operacijskog sustava
FAT	tablica koja za svaki klaster kaže koji je sljedeći klaster datoteke
Hijerarhijski datotečni sustav	direktoriji mogu sadržavati poddirektorije
Višezadaćnost	istovremeno izvođenje više programa
Virtualna memorija	privremeno spremanje dijelova memorije na disk

Ključno za zapamtiti

1. Bez softvera procesor izvodi nasumične bajtove; na adresi pokretanja mora biti smislen kod.
2. Prvi programi unose se pločom; zatim se pišu obrađivač tipkovnice i procesor naredbi, pa se spremaju u ROM.
3. Broj se na zaslonu prikazuje pretvorbom svakog nibla u ASCII znak.
4. Datotečni sustav daje datotekama imena i dopušta da zauzimaju neuzastopne dijelove diska.
5. CP/M: blokovi od 1 KB, direktorij od 2 KB, zapisi od 32 bajta, imena 8.3.
6. Podizanje: mali punjač iz ROM-a učita prvi sektor, a taj ostatak sustava.
7. CP/M se sastoji od CCP-a, BDOS-a i BIOS-a; programi se učitavaju na 0100h.
8. API (CALL 5 u CP/M-u, INT 21h u MS-DOS-u) daje programima pristup hardveru neovisno o uređaju.
9. Samo BIOS mora poznavati hardver; zato isti program radi na različitim računalima.
10. MS-DOS koristi FAT i od verzije 2.0 hijerarhijske direktorije; UNIX je prenosiv i višezadaćan.

Najčešće zabune

- **"Operacijski sustav je samo ono što korisnik vidi."** Uz naredbe, sustav učitava programe i daje im API za pristup hardveru.
- **"Datoteka uvijek zauzima uzastopne sektore."** Ne mora; zato postoje mapa diska (CP/M) i FAT (MS-DOS).
- **"Program se može učitati na bilo koju adresu."** Adrese u Jump i Call instrukcijama vrijede samo za adresu za koju je program napisan (u CP/M-u 0100h).

- **"Tekstualna i binarna datoteka s istim brojevima su jednake."** Binarna sprema bajtove broja; tekstualna sprema ASCII znakove znamenki.
- **"BIOS je dio svakog programa."** BIOS je dio operacijskog sustava (ili ROM-a); programi ga koriste preko API-ja.

Brza provjera razumijevanja

1. Koje ASCII kodove treba upisati u video memoriju da bi se prikazao bajt C7h? Kako ih dobiva NibbleToAscii?
2. Koliko alokacijskih blokova i koliko zapisa u direktoriju treba datoteka od 20.000 bajtova u CP/M-u?
3. Zapiši bajtove tekstualne datoteke koja sadrži dekadske brojeve 25 i 300, svaki u svom retku.
4. Opiši korake podizanja CP/M-a i objasni ulogu punjača.
5. Zašto program koji koristi API može raditi na različitim računalima, a program koji izravno piše u video memoriju možda ne može?

23. Nepomični i pomični zarez

Što ćeš naučiti u ovom poglavlju

- Vrste brojeva: **cijeli, racionalni, iracionalni, transcendentni, realni, imaginarni, kompleksni**.
- Zašto računalo može spremiti samo **diskretne** vrijednosti.
- Kako se novčani iznosi spremaju u **BCD** zapisu s **nepomičnim zarezom** (*fixed point*).
- Što je **znanstveni zapis**: **signifikand** i **eksponent**, normalizirani oblik.
- Kako izgledaju **binarni razlomci** (znamenke iza binarne točke).
- Kako radi standard **IEEE 754**: jednostruka (4 bajta) i dvostruka (8 bajtova) preciznost, posebni slučajevi (0, beskonačno, NaN).
- Što znači **preciznost** i zašto 16.777.216 i 16.777.217 mogu biti "isti" broj.
- Kako se zbrajaju i množe brojevi s pomičnim zarezom i kako se računaju funkcije poput sinusa.
- Što je **matematički koprocesor** (FPU) i **mikrokod**.

Napomena o nazivlju: engleski *point* u hrvatskom je **zarez** (decimalni zarez). Za binarne brojeve govorimo o **binarnoj točki**, jer u zapisu služi isto kao decimalni zarez.

Vrste brojeva

Do sada smo bitovima prikazivali **pozitivne cijele brojeve** (*integers*) i, komplementom dvojke, **negativne cijele brojeve**.

Bitova	Pozitivni cijeli	Komplement dvojke
8	0 do 255	−128 do 127
16	0 do 65.535	−32.768 do 32.767
32	0 do 4.294.967.295	−2.147.483.648 do 2.147.483.647

Racionalni brojevi

Racionalni broj može se prikazati kao **omjer** (razlomak) dvaju cijelih brojeva. Primjer: ¾ je omjer 3 i 4. Isti broj u **decimalnom** zapisu je 0,75, što zapravo znači 75/100.

Znamenke lijevo od decimalnog zareza su višekratnici **pozitivnih** potencija broja 10, a desno višekratnici **negativnih** potencija (poglavlje 7). Primjer (vlastiti):

```
3,625 = 3 × 10⁰ + 6 × 10⁻¹ + 2 × 10⁻² + 5 × 10⁻³
      = 3 × 1   + 6 × 0,1  + 2 × 0,01  + 5 × 0,001
```

Neki racionalni brojevi nemaju konačan decimalni zapis, nego se znamenke **ponavljaju** u nedogled:

- 1/3 = 0,3333… (piše se 0,3 s crticom iznad trojke),
- 1/7 = 0,142857142857… (ponavlja se skupina 142857).

I dalje su racionalni, jer su omjer dvaju cijelih brojeva.

Iracionalni i transcendentni brojevi

Vrsta	Definicija	Primjeri
Iracionalni	ne može se prikazati kao omjer dvaju cijelih brojeva; decimale se nastavljaju **bez ponavljanja**	$\sqrt{2} \approx 1{,}41421356\ldots$, rješenje jednadžbe $x^2 - 2 = 0$
Transcendentni	nije rješenje **nijedne** algebarske jednadžbe s cjelobrojnim koeficijentima	$\pi \approx 3{,}14159265\ldots$ (omjer opsega i promjera kruga); $e \approx 2{,}71828182\ldots$ (vrijednost kojoj teži $(1 + 1/n)^n$ kad n raste)

Svi transcendentni brojevi su iracionalni, ali nisu svi iracionalni brojevi transcendentni ($\sqrt{2}$ nije).

Realni, imaginarni, kompleksni

- Svi navedeni brojevi (racionalni i iracionalni) su **realni** brojevi.
- **Imaginarni** brojevi su drugi korijeni negativnih brojeva.
- **Kompleksni** brojevi su kombinacije realnih i imaginarnih. Unatoč imenu, imaginarni brojevi pojavljuju se u stvarnom svijetu, npr. u rješavanju nekih problema u elektronici.

Kontinuirano i diskretno

Brojeve doživljavamo kao **kontinuirane**: između svaka dva racionalna broja uvijek postoji treći (npr. njihov prosjek). Bitovi su samo 0 ili 1, bez ičega između. Zato digitalna računala rade s **diskretnim** vrijednostima. Broj mogućih vrijednosti ovisi o broju bitova: s 32 bita za pozitivne cijele brojeve možeš spremiti 0 do 4.294.967.295, ali ne i 4,5. Za razlomke treba drukčiji pristup.

BCD i nepomični zarez

Najjednostavniji način za razlomke vjerojatno je **BCD** (binarno kodirani dekadski zapis, poglavlje 19): svaka dekadska znamenka u 4 bita.

Znamenka	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

BCD je posebno koristan za programe koji rade s **novcem** (kune i lipe, dolari i centi): banke, osiguranja. Mnogi njihovi razlomci trebaju točno **dvije decimale**.

- **Pakirani BCD** (*packed BCD*): dvije znamenke u jednom bajtu.
- BCD ne koristi komplement dvojke, pa treba poseban **bit predznaka**. Budući da je zgodno da broj zauzme cijele bajtove, taj jedan bit obično "pojede" 4 ili 8 bitova.

Primjer (vlastiti): iznosi nikad ne prelaze 10 milijuna, dakle raspon je −9.999.999,99 do 9.999.999,99. Treba 9 znamenki + predznak = 10 niblova = **5 bajtova**. Iznos **−1.284.509,75**:

```
prvi nibl = predznak (1), zatim znamenke 1 2 8 4 5 0 9 | 7 5

0001 0001   0010 1000   0100 0101   0000 1001   0111 0101
   11h         28h         45h         09h         75h
```

- Krajnji lijevi nibl **1** znači negativan broj (0 bi značilo pozitivan).
- Svaka znamenka zauzima 4 bita, pa se iznos čita izravno iz heks zapisa.
- Za raspon do 99.999.999,99 trebalo bi 6 bajtova: 5 za 10 znamenki i cijeli bajt za predznak.

> Ovo je zapis s **nepomičnim zarezom** (*fixed-point*): decimalni zarez je uvijek na istom mjestu (ovdje: dvije decimale). Položaj zareza **nije spremljen** uz broj. Program mora **znati** gdje je zarez.

Brojevi s nepomičnim zarezom mogu imati bilo koji broj decimala, a u istom programu mogu se miješati različiti formati. Ali svaki dio programa koji računa s njima mora znati gdje su zarezi.

Kad nepomični zarez ne radi

Nepomični zarez radi dobro samo ako znaš da brojevi neće biti preveliki za predviđenu memoriju i da ne trebaju više decimala. Potpuno zakazuje kad brojevi mogu biti **vrlo veliki i vrlo mali**. Primjer iz knjige (udaljenosti u stopama):

- udaljenost Zemlje od Sunca: 490.000.000.000 stopa,
- polumjer atoma vodika: 0,00000000026 stopa.

Za oba bi trebalo **12 bajtova** nepomičnog zareza.

Znanstveni zapis

Znanstvenici i inženjeri koriste **znanstveni zapis**: potencija broja 10 zamjenjuje dugačke nizove nula.

Broj	Znanstveni zapis
490.000.000.000	$4{,}9 \times 10^{11}$
0,00000000026	$2{,}6 \times 10^{-10}$
3.800.000.000 (vlastiti)	$3{,}8 \times 10^{9}$
0,000047 (vlastiti)	$4{,}7 \times 10^{-5}$

Dio	Naziv	Značenje
4,9	**signifikand** (zove se i razlomljeni dio, karakteristika, mantisa)	znamenke broja
11	**eksponent**	potencija na koju se diže 10; kaže za koliko je mjesta pomaknut zarez

Normalizirani oblik: signifikand je **≥ 1 i < 10**. Ovi zapisi su jednaki, ali se prvi preferira:

> $4{,}9 \times 10^{11} = 49 \times 10^{10} = 490 \times 10^{9} = 0{,}49 \times 10^{12} = 0{,}049 \times 10^{13}$

Predznak eksponenta govori samo o **veličini** broja, a ne o tome je li broj negativan. Primjeri (vlastiti):

- $-2{,}5 \times 10^{3} = -2500$
- $-2{,}5 \times 10^{-3} = -0{,}0025$

U računalu je alternativa nepomičnom zarezu **pomični zarez** (*floating-point*). Idealan je za vrlo male i vrlo velike brojeve jer se temelji na znanstvenom zapisu, ali s **binarnim** brojevima.

Binarni razlomci

Desno od decimalnog zareza su negativne potencije broja 10. Desno od **binarne točke** su negativne potencije broja **2**.

Primjer (vlastiti): $110{,}011_2$

Znamenka	1	1	0	,	0	1	1
Potencija	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}
Vrijednost	4	2	1		0,5	0,25	0,125
Doprinos	4	2	0		0	0,25	0,125

$110{,}011_2 = 4 + 2 + 0{,}25 + 0{,}125 = \mathbf{6{,}375}$

Negativne potencije broja 2 dobiju se uzastopnim dijeljenjem s 2: 1, 0,5, 0,25, 0,125, 0,0625...

Binarni znanstveni zapis

U binarnom zapisu normalizirani signifikand je **≥ 1 i < 10_2** (tj. manji od 2). Primjer:

```
110,011₂ = 1,10011₂ × 2²
```

> Važna posljedica: normalizirani binarni signifikand **uvijek** ima **1** lijevo od binarne točke i ništa drugo.

Standard IEEE 754

Većina računala i programa koristi standard organizacije **IEEE** iz 1985., koji je priznao i ANSI: *ANSI/IEEE Std 754-1985, IEEE Standard for Binary Floating-Point Arithmetic*. Dugačak je samo 18 stranica. Definira dva osnovna formata:

Format	Veličina
Jednostruka preciznost (*single precision*)	4 bajta (32 bita)
Dvostruka preciznost (*double precision*)	8 bajtova (64 bita)

Jednostruka preciznost

```
┌─────────┬──────────────┬──────────────────────────────┐
│ s       │ e            │ f                            │
│ 1 bit   │ 8 bitova     │ 23 bita                      │
│predzn.  │ eksponent    │ razlomljeni dio signifikanda │
└─────────┴──────────────┴──────────────────────────────┘
```

- **s**: predznak; 0 pozitivno, 1 negativno.
- **f**: samo dio signifikanda **iza** binarne točke. Jedinica ispred točke uvijek postoji, pa se **ne sprema**. Zato se kaže da je **preciznost 24 bita**, iako su spremljena 23.
- **e**: 8-bitni eksponent (0 do 255) s **pomakom** (*biased exponent*). Od spremljene vrijednosti oduzima se **pomak** (*bias*) da se dobije stvarni eksponent. Za jednostruku preciznost pomak je **127**.

Za e od 1 do 254 vrijednost broja je:

> **$(-1)^s \times 1{,}f \times 2^{(e-127)}$**

$(-1)^s$ je matematički način da se kaže: ako je s = 0, broj je pozitivan (jer je sve na nultu potenciju 1); ako je s = 1, broj je negativan.

Primjer (vlastiti): kodiraj **6,375**.

1. $6{,}375 = 110{,}011_2 = 1{,}10011_2 \times 2^2$.
2. s = 0 (pozitivno).
3. e = 2 + 127 = 129 = 1000 0001.
4. f = 10011 i zatim nule do 23 bita.

```
s  e         f
0  10000001  10011000000000000000000

0100 0000 1100 1100 0000 0000 0000 0000  =  40CC0000h
```

Primjer (vlastiti): $-0{,}75 = -0{,}11_2 = -1{,}1_2 \times 2^{-1}$ → s = 1, e = −1 + 127 = 126 = 0111 1110, f = 1000…0 → **BF400000h**.

Posebni slučajevi

Eksponenti 0 i 255 imaju posebno značenje. Tako se prikazuje i **nula**, koju opća formula ne može dati.

e	f	Značenje
0	0	**nula**; obično su svih 32 bita 0. Ako je s = 1, to je **negativna nula**: vrlo mali broj manji od 0 koji se ne može prikazati
0	≠ 0	ispravan, ali **nenormaliziran** broj: $(-1)^s \times 0{,}f \times 2^{-126}$ (signifikand ima 0 ispred točke)
255	0	**pozitivna ili negativna beskonačnost**, ovisno o s
255	≠ 0	**NaN** (*not a number*, nije broj): nepoznat broj ili rezultat nedopuštene operacije

(U knjizi je za nenormalizirani broj navedeno 2^{-127}; standard IEEE 754 koristi 2^{-126}.)

Raspon jednostruke preciznosti

	Binarno	Dekadski približno
Najmanji normalizirani	$1{,}000\ldots0_2 \times 2^{-126}$ (23 nule)	$1{,}175494351 \times 10^{-38}$
Najveći normalizirani	$1{,}111\ldots1_2 \times 2^{127}$ (23 jedinice)	$3{,}402823466 \times 10^{38}$

Preciznost

10 binarnih znamenki približno odgovara 3 dekadske znamenke: 10 jedinica (3FFh = 1023) je otprilike 999. Dakle **$2^{10} \approx 10^3$**. Zato 24-bitni signifikand odgovara otprilike **7 dekadskih znamenki**.

> Jednostruka preciznost: **24 bita**, oko **7 dekadskih znamenki**.

Kod nepomičnog zareza točnost je očita (dvije decimale = točno do lipe). Kod pomičnog zareza **ovisi o eksponentu**: ponekad je broj točan do sićušnog dijela lipe, a ponekad ni do cijele kune. Točnije je reći: broj jednostruke preciznosti točan je na **1 dio od 2^{24}**, tj. 1 dio od 16.777.216.

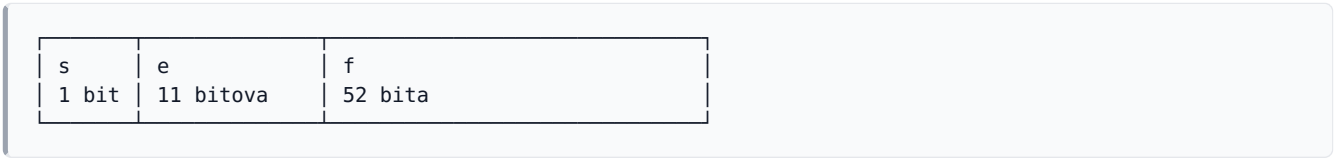
Što to stvarno znači

- 16.777.216 i 16.777.217 kao brojevi jednostruke preciznosti postaju **isti broj**. Isto vrijedi za sve između, npr. 16.777.216,5. Sva tri spremaju se kao **4B800000h**:

```
0 10010111 00000000000000000000000   →   1,000...0₂ × 2²⁴
```

- Sljedeći mogući signifikand, $1{,}000\ldots01_2 \times 2^{24}$, predstavlja **16.777.218**.
- Za banku: 262.144,00 i 262.144,01 su **isti broj** ($1{,}0 \times 2^{18}$). Zato se za novac radije koristi **nepomični zarez**.
- Pomični zarez ima i druge "čudnovatosti": izračun koji bi trebao dati 3,50 može dati 3,499999999999. To se u izračunima s pomičnim zarezom događa i ne može se mnogo učiniti.

Dvostruka preciznost



- Pomak eksponenta je **1023** (3FFh): vrijednost = **(−1)ˢ × 1,f × 2^(e − 1023)**.
- Za 0, beskonačnost i NaN vrijede slična pravila.

	Binarno	Dekadski približno
Najmanji	$1{,}000\ldots0_2 \times 2^{-1022}$ (52 nule)	$2{,}2250738585072014 \times 10^{-308}$
Najveći	$1{,}111\ldots1_2 \times 2^{1023}$ (52 jedinice)	$1{,}7976931348623158 \times 10^{308}$

10^{308} je 1 iza kojeg slijedi 308 nula.

- 53 bita signifikanda (s nespremljenom jedinicom) daju oko **16 dekadskih znamenki**. Mnogo bolje, ali i ovdje će neki brojevi biti jednaki: npr. 140.737.488.355.328,00 i 140.737.488.355.328,01 spremaju se kao isti broj **42E0000000000000h** ($1{,}0 \times 2^{47}$).

	Jednostruka	Dvostruka
Bajtova	4	8
Bitovi s / e / f	1 / 8 / 23	1 / 11 / 52
Pomak eksponenta	127	1023
Preciznost	24 bita ≈ 7 znamenki	53 bita ≈ 16 znamenki
Raspon (približno)	10^{-38} do 10^{38}	10^{-308} do 10^{308}

Računanje s pomičnim zarezom

Format je tek mali dio posla. Treba napisati funkcije za zbrajanje, oduzimanje, množenje i dijeljenje brojeva s pomičnim zarezom. Srećom, svaki od tih poslova može se razbiti na manje poslove s **cijelim** brojevima, koje već znamo.

Zbrajanje

Zbrajaju se signifikandi, ali najprije ih treba **poravnati** prema eksponentima: signifikand broja s manjim eksponentom pomakne se udesno za razliku eksponenata.

Primjer (vlastiti): $(1{,}101 \times 2^4) + (1{,}01 \times 2^1)$

```
Razlika eksponenata: 4 − 1 = 3
1,01 × 2¹ = 0,00101 × 2⁴      (pomak za 3 mjesta)

  1,10100 × 2⁴
+ 0,00101 × 2⁴
--------------
  1,11001 × 2⁴
```

Provjera: 26 + 2,5 = 28,5, a $1{,}11001_2 \times 16 = 1{,}78125 \times 16 = 28{,}5$. ✓

Ponekad su eksponenti toliko udaljeni da manji broj **uopće ne utječe** na zbroj, npr. udaljenost do Sunca + polumjer atoma vodika.

Množenje

Signifikandi se množe kao cijeli brojevi, a eksponenti se **zbrajaju**. Normalizacija rezultata može tražiti da se novi eksponent promijeni za jedan ili dva.

Funkcije

Korijeni, eksponencijalne funkcije, logaritmi i trigonometrijske funkcije mogu se izračunati pomoću četiri osnovne operacije. Na primjer, **sinus** se računa razvojem u red:

> **$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$**

- x mora biti u **radijanima**: 360° = 2π radijana.
- **!** je **faktorijel**: umnožak cijelih brojeva od 1 do zadanog broja; npr. 5! = 1 × 2 × 3 × 4 × 5 = 120.
- Potencija je množenje; ostalo su dijeljenje, zbrajanje i oduzimanje.
- Tri točke znače da se računa **zauvijek**. U praksi, ako se ograničiš na raspon 0 do π/2 (iz kojeg se dobiju sve ostale vrijednosti sinusa), nakon **desetak članova** rezultat je točan na 53 bita dvostruke preciznosti.

Ljepota softvera: kad netko jednom napiše rutine s pomičnim zarezom za neki stroj, mogu ih koristiti svi. U ranim danima računala pisanje tih rutina bio je jedan od **prvih** softverskih poslova za svako novo računalo.

Pomični zarez u hardveru

Ima smisla ugraditi instrukcije s pomičnim zarezom **izravno u hardver**, kao što su 16-bitni procesori dobili instrukcije za množenje i dijeljenje. Tada su svi izračuni brži.

Godina	Hardver	Bitno
1954.	**IBM 704**	prvo komercijalno računalo s hardverom za pomični zarez (kao opcija); brojevi od 36 bitova: 27-bitni signifikand, 8-bitni eksponent, bit predznaka; hardver zbraja, oduzima, množi i dijeli, ostale funkcije su u softveru
1980.	**Intel 8087**	matematički **koprocesor** (*math coprocessor*), danas **FPU** (*floating-point unit*)
—	Intel 287, 387	koprocesori za 286 i 386
1989.	**486DX**	FPU **ugrađen** u procesor
1991.	486SX	jeftiniji, bez FPU-a; dodatni 487SX kao opcija
1993.	**Pentium**	ugrađeni FPU opet standard
1990.	Motorola 68040	ugrađeni FPU (prije toga koprocesori 68881 i 68882)
—	PowerPC	ugrađen hardver za pomični zarez

Intel 8087

- Zove se **ko**procesor jer ne može raditi sam, nego samo uz 8086 ili 8088.
- 40 nožica; koristi mnoge iste signale kao 8086/8088, preko kojih je povezan s procesorom.
- Kad procesor naiđe na posebnu instrukciju **ESC** (*Escape*), koprocesor preuzme i izvede sljedeći strojni kod: jednu od **68 instrukcija**, uključujući trigonometrijske, eksponencijalne i logaritamske.
- Tipovi podataka temelje se na standardu IEEE.
- U svoje vrijeme smatran je najsloženijim integriranim krugom ikad napravljenim.

Mikrokod

Koprocesor je kao malo samostalno računalo. Za određenu instrukciju (npr. **FSQRT**, drugi korijen) on interno izvodi vlastiti niz instrukcija spremljenih u ROM-u. Te unutarnje instrukcije zovu se **mikrokod** (*microcode*). Obično se vrte u petljama, pa rezultat nije odmah dostupan, ali koprocesor je ipak obično barem **10 puta brži** od istih rutina u softveru.

Koprocesor u praksi

- Matična ploča izvornog IBM PC-a imala je podnožje za 8087 pokraj 8088, ali **prazno**. Tko je trebao brzinu, kupio ga je i ugradio sam.
- Ni s koprocesorom nisu sve aplikacije bile brže: uređivači teksta jedva trebaju pomični zarez; proračunske tablice ga trebaju mnogo, ali ni one nisu sve koristile koprocesor.
- Programeri su morali pisati **poseban kod** za instrukcije koprocesora. Budući da većina korisnika nije imala koprocesor, ionako su morali pisati vlastite rutine, pa je podrška za 8087 bila **dodatni** posao.
- S vremenom su programi naučili koristiti koprocesor **ako postoji**, a **oponašati** ga (emulirati) ako ne postoji.

Hardver za pomični zarez lijep je dar programerima u asembleru, ali mali povijesni korak u usporedbi s onim što je počelo ranih 1950-ih: **programskim jezicima** (poglavlje 24).

Ključne definicije

Pojam	Značenje
Racionalni broj	omjer dvaju cijelih brojeva
Iracionalni broj	nije omjer cijelih brojeva; decimale se ne ponavljaju
Transcendentni broj	nije rješenje algebarske jednadžbe s cjelobrojnim koeficijentima (π, e)
Pakirani BCD	dvije dekadske znamenke u bajtu; predznak u zasebnom niblu ili bajtu
Nepomični zarez	zarez je na unaprijed dogovorenom mjestu; ne sprema se
Pomični zarez	broj se sprema kao predznak, eksponent i signifikand
Signifikand	znamenke broja u znanstvenom zapisu (mantisa)
Eksponent	potencija baze (10 ili 2)
Normalizirani oblik	signifikand ≥ 1 i < baze
Eksponent s pomakom	spremljeni eksponent = stvarni + pomak (127 ili 1023)
NaN	"nije broj": rezultat nedopuštene operacije
Preciznost	broj značajnih bitova signifikanda (24 ili 53)
FPU / koprocesor	hardver za računanje s pomičnim zarezom
Mikrokod	unutarnje instrukcije procesora spremljene u ROM-u

Ključno za zapamtiti

1. Računalo sprema samo diskretne vrijednosti; razlomci traže poseban zapis.
2. Za novac je pogodan BCD s nepomičnim zarezom: točan je do zadnje decimale.
3. Nepomični zarez ne odgovara kad su brojevi vrlo veliki i vrlo mali.
4. Pomični zarez je binarni znanstveni zapis: predznak, eksponent, signifikand.
5. Normalizirani binarni signifikand uvijek počinje s 1, pa se ta jedinica ne sprema.
6. Jednostruka preciznost: 1 + 8 + 23 bita, pomak 127, oko 7 dekadskih znamenki.
7. Dvostruka preciznost: 1 + 11 + 52 bita, pomak 1023, oko 16 dekadskih znamenki.
8. e = 0 i e = maksimum su posebni slučajevi: nula, nenormalizirani brojevi, beskonačnost, NaN.

9. Zbrajanje: poravnaj signifikande prema eksponentima pa zbroji; množenje: pomnoži signifikande, zbroji eksponente.
10. Matematički koprocesor (FPU) izvodi pomični zarez u hardveru, obično barem 10 puta brže.

Najčešće zabune

- **"Pomični zarez je uvijek točniji."** Ima veći raspon, ali ograničenu preciznost; 262.144,00 i 262.144,01 mogu biti isti broj.
- **"Negativan eksponent znači negativan broj."** Eksponent govori o veličini; predznak broja je zasebni bit.
- **"Svi 24 bita signifikanda su spremljeni."** Spremljena su 23; vodeća 1 se podrazumijeva.
- **"Izračun s pomičnim zarezom uvijek daje točan dekadski rezultat."** Ne: izračun koji bi trebao dati 3,50 može dati 3,499999999999.
- **"Koprocesor ubrzava svaki program."** Samo programe koji ga koriste i koji uopće računaju s pomičnim zarezom.

Brza provjera razumijevanja

1. Pretvori $101{,}101_2$ u dekadski broj i zapiši ga u normaliziranom binarnom znanstvenom zapisu.
2. Kodiraj broj 10,5 u formatu jednostruke preciznosti (heks).
3. Koliko bajtova pakiranog BCD-a treba za iznose od −999.999,99 do 999.999,99? Zapiši −52.310,08.
4. Zašto 16.777.217 nije moguće točno spremiti u jednostrukoj preciznosti?
5. Što znači kad je e = 255, a f ≠ 0?

24. Jezici visoke i niske razine

Što ćeš naučiti u ovom poglavlju

- Kako **asembler** (program) pretvara asemblerski jezik u strojni kod; što su **izvorni kod**, **izvršna datoteka** i **križni asembler**.
- Dva glavna problema asemblerskog jezika.
- Što je **jezik visoke razine**, **sintaksa** i **prevoditelj** (*compiler*).
- Prednosti i nedostatke jezika visoke razine.
- Osnovne pojmove programiranja na primjeru jezika **ALGOL**: **varijabla, deklaracija, pridruživanje, izraz, petlja, blok, uvjet, polje**, Booleove varijable.
- Algoritam **Eratostenovo sito**.
- Razliku između **sintaksne pogreške** i **pogreške pri izvođenju**, te između **prevoditelja** i **interpretera**.
- Kratak pregled jezika: FORTRAN, COBOL, PL/I, BASIC, Pascal, Ada, C, LISP, APL i objektno orijentirani jezici.

Analogija iz knjige: programiranje u strojnom kodu je kao jesti čačkalicom. Zalogaji su tako mali da večera traje vječno. Svaki bajt strojnog koda radi nešto sićušno (učitaj broj, dodaj drugi, spremi rezultat), pa je teško vidjeti kako to tvori cijeli "obrok".

Asembler: računalo samo asemblira

Do sada smo program pisali na papiru i **ručno asemblirali**. Bolje je da pretvorbu radi **računalo**. Ako na 8080 radi CP/M, svi alati već postoje.

Korak 1: uređivačem teksta (npr. ED.COM) napiši tekstualnu datoteku, npr. **BOK.ASM** (vrsta ASM znači asemblerski program). Primjer (vlastita inačica primjera iz knjige):

```
        ORG 0100h
        LXI DE,Text
        MVI C,9
        CALL 5
        RET
Text:   DB 'Bok!$'
        END
```

Naredba	Značenje
ORG (*origin*)	nije instrukcija 8080; kaže da sljedeća naredba počinje na adresi 0100h (tamo CP/M učitava programe)
LXI DE,Text	u par DE stavi 16-bitnu adresu oznake Text
DB (*Data Byte*)	bajtovi podataka; mogu biti brojevi odvojeni zarezima ili tekst u jednostrukim navodnicima
MVI C,9	u C stavi 9
CALL 5	poziv funkcije CP/M-a; funkcija 9 ispisuje niz znakova od adrese u DE sve do znaka **$** (neobičan, ali takav je CP/M)
RET	završava program i vraća upravljanje CP/M-u
END	kraj asemblerske datoteke

Korak 2: asembliraj programom **ASM.COM**, koji dolazi uz CP/M:

```
ASM BOK.ASM
```

ASM čita BOK.ASM i stvara **BOK.COM** sa strojnim kodom. (Zapravo postoji još jedan međukorak, nevažan za ovaj opis.)

Korak 3: pokreni `BOK`. Program ispiše "Bok!" i završi.

BOK.COM sadrži 14 bajtova:

```
11 09 01   0E 09   CD 05 00   C9   42 6F 6B 21 24
LXI DE     MVI C   CALL 5     RET  B  o  k  !  $
```

Asembler je sam izračunao da LXI mora u DE staviti adresu **0109h**: program počinje na 0100h, instrukcije zauzimaju 9 bajtova, pa tekst počinje na 0109h. Programer koji koristi asembler ne mora brinuti o točnim adresama dijelova programa.

Pojmovi

Pojam	Značenje
Asembler (program)	čita asemblerski program i piše strojni kod
Izvorni kod (*source code*)	datoteka s programom koji je napisao čovjek
Izvršna datoteka (*executable*)	datoteka sa strojnim kodom koji se može pokrenuti

Asembleri su prilično **jednostavni** programi jer između mnemonika i strojnog koda postoji odnos **jedan na jedan**. Asembler dijeli svaki redak na mnemonik i argumente i uspoređuje ih s popisom svih mogućih mnemonika i argumenata.

Kako nastaje prvi asembler? Prvi asembler za neko računalo morao je biti ručno asembliran. Novi (bolji) asembler može se napisati u asemblerskom jeziku i asemblirati prvim. Kad je novi asembler gotov, može asemblirati i sam sebe.

Križni asembler (*cross-assembler*): za novi mikroprocesor asembler se može napisati na **postojećem** računalu. Asembler radi na računalu A, a stvara kod za računalo B.

Dva problema asemblerskog jezika

1. **Zamoran je.** Radiš na razini čipa i moraš brinuti o svakoj sitnici.
2. **Nije prenosiv** (*portable*). Program za Intel 8080 ne radi na Motoroli 6800; mora se ponovno napisati u asembleru za 6800. To je lakše od prvog pisanja (organizacija i algoritmi su već riješeni), ali je i dalje velik posao.

Jezici visoke razine

Strojne instrukcije za pomični zarez su korisne, ali još bolje bi bilo potpuno napustiti instrukcije ovisne o procesoru i matematičke operacije pisati uobičajenim **algebarskim zapisom**:

```
A × Sin (2 × PI + B) / C
```

gdje su A, B i C brojevi, a PI je 3,14159. Ako je takav izraz u tekstualnoj datoteci, može se napisati program (u asembleru) koji čita tekst i pretvara izraz u strojni kod.

Izraz se obično računa za **mnogo** različitih vrijednosti A, B i C (inače bi ga izračunao ručno), pa mu treba i **kontekst** koji to omogućuje.

> Takav jezik zove se **jezik visoke razine** (*high-level language*). Asemblerski jezik je jezik **niske razine** (*low-level*) jer je vrlo blizu hardveru.

"Visoka razina" označava svaki jezik osim asemblerskog, ali neki su "viši" od drugih. Kad bi direktor tvrtke mogao reći računalu "izračunaj ovogodišnju dobit, napiši godišnje izvješće, ispiši par tisuća primjeraka i pošalji ih dioničarima", to bi bio jezik vrlo visoke razine. Stvarni jezici nisu ni blizu tome.

Ljudski jezici nastaju stoljećima kroz složene utjecaje. Programski jezici su **namjerno osmišljeni**. Procjena iz 1993.: od početka 1950-ih izmišljeno je i ostvareno više od **1000** jezika visoke razine.

Pojam	Značenje
Sintaksa (*syntax*)	pravila kako se u jeziku izražava sve što se njime može reći
Prevoditelj (*compiler*)	program koji naredbe jezika visoke razine pretvara u strojni kod

Prevoditelj, kao i asembler, čita izvorni kod znak po znak i rastavlja ga na riječi, simbole i brojeve. Ali je **mnogo složeniji**: jedna naredba visoke razine obično postaje **mnogo** strojnih instrukcija. Prevoditelje nije lako napisati; o njihovoj izradi pišu se cijele knjige.

Prednosti i nedostaci

Prednosti	Nedostaci
lakše se uče i u njima se lakše programira	dobar programer u asembleru gotovo uvijek napiše **bolji** kod: izvršna datoteka iz jezika visoke razine je **veća i sporija** (sve manje izraženo jer prevoditelji bolje optimiziraju)
programi su jasniji i sažetiji	jezik visoke razine **ne čini procesor moćnijim**: može samo smanjiti njegove mogućnosti
često su **prenosivi**: ne ovise o procesoru; programer ne mora poznavati građu stroja	prenosiv jezik ne može koristiti posebnosti pojedinih procesora
za više procesora treba samo prevoditelj za svaki; izvršne datoteke ostaju vezane uz procesor	

Primjer ograničenja: mnogi procesori imaju instrukcije **pomaka bitova**, ali gotovo nijedan jezik visoke razine ih nema. Treba ih oponašati množenjem ili dijeljenjem s 2. (Nije loše: mnogi prevoditelji upravo instrukcije pomaka koriste za množenje i dijeljenje potencijama broja 2.) Mnogi jezici nemaju ni Booleove operacije nad bitovima.

U ranim danima kućnih računala većina aplikacija pisana je u asembleru. Danas se asembler koristi rijetko, za posebne namjene:

- uz **cjevovod** (istovremeno postupno izvođenje više instrukcija) asembler je postao teži,
- prevoditelji su postali napredniji,
- s više memorije i pohrane programeri ne moraju štedjeti svaki bajt.

Rana povijest

Jezik	Kada, tko	Bitno
A-0	1952., Grace Murray Hopper, Remington-Rand, za UNIVAC	obično se smatra prvim pravim prevoditeljem
FORTRAN (*FORmula TRANslation*)	sredina 1950-ih, IBM, za računala 704	najstariji jezik visoke razine koji se još koristi; jezik znanstvenika i inženjera; jaka podrška za pomični zarez i kompleksne brojeve
ALGOL (*ALGOrithmic Language*)	ALGOL 58 (međunarodni odbor, 1957.–1958.), ALGOL 60 (1960.; "Revised Report" 1962./1963.), ALGOL 68	izravni predak mnogih jezika opće namjene; i danas se govori o jezicima "sličnima ALGOL-u"

Mnogi jezici imaju imena velikim slovima jer su skraćenice.

Programiranje na primjeru ALGOL-a

Knjiga koristi ALGOL 60 kao "neutralan" primjer, jer ga danas gotovo nitko ne koristi, a temeljan je za mnoge kasnije jezike. Pretpostavimo prevoditelj ALGOL.COM za CP/M ili MS-DOS i izvorne datoteke vrste **ALG**.

Prvi program i pogreške

Program u ALGOL-u mora biti između riječi **begin** i **end**:

```
begin
    print ('Ovo je moj prvi ALGOL progam!');
ende
```

Prevođenje: `ALGOL PRVI.ALG`. Prevoditelj javi:

```
Line 3: Unrecognized keyword 'ende'.
```

To je **sintaksna pogreška** (*syntax error*): prevoditelj je očekivao **ključnu riječ** (riječ koju prepoznaje), a našao "ende". Prevoditelj je pedantniji od najstrožeg učitelja pravopisa.

Nakon ispravka prevoditelj stvori izvršnu datoteku (PRVI.COM ili PRVI.EXE; ponekad treba još jedan korak). Pokretanje `PRVI` ispiše:

```
Ovo je moj prvi ALGOL progam!
```

Opet pogreška ("progam"), ali takvu prevoditelj **ne može** otkriti. To je **pogreška pri izvođenju** (*run-time error*): vidi se tek kad se program pokrene.

Vrsta pogreške	Kada se otkrije	Primjer
Sintaksna	pri prevođenju	krivo napisana ključna riječ
Pri izvođenju	tek kad program radi	krivo napisan tekst, krivi rezultat

- **print** nije dio službene specifikacije ALGOL-a, nego **ugrađena funkcija** (*built-in function*) našeg prevoditelja.
- Većina naredbi (osim begin i end) završava **točkom sa zarezom**.
- Uvlačenje redaka nije obavezno, ali pokazuje strukturu programa.

Varijable, deklaracija, pridruživanje

> **Varijabla** je ime (slovo, nekoliko slova ili kratka riječ) koje u programu označava mjesto u memoriji. Program se na nju poziva imenom, a ne brojčanom adresom.

Primjer (vlastiti):

```
begin
    real a, b, c;

    a := 12.5;
    b := 3.2;
    c := a × b;

    print ('Umnozak ', a, ' i ', b, ' je ', c);
end
```

Naredba	Vrsta	Značenje
`real a, b, c;`	**deklaracija**	najavljuje varijable a, b, c kao realne brojeve (pomični zarez). Za cijele brojeve postoji ključna riječ **integer**
`a := 12.5;`	**pridruživanje**	varijabla lijevo dobiva vrijednost **izraza** desno
`c := a × b;`	pridruživanje	c dobiva umnožak a i b

- Ime varijable obično mora počinjati **slovom**; smije sadržavati i brojke, ali ne razmake i većinu drugih znakova. Prevoditelji često ograničavaju duljinu imena.
- Ako prevoditelj koristi IEEE standard, svaka realna varijabla zauzima 4 bajta (jednostruka preciznost) ili 8 bajtova (dvostruka).
- U ALGOL-u se pridruživanje prepoznaje po **:=** (u većini jezika samo **=**).

Operatori: znak × obično nije dopušten, jer ga nema u ASCII-ju ni EBCDIC-u; većina jezika za množenje koristi **zvjezdicu ***. ALGOL za dijeljenje koristi **/**, za cjelobrojno dijeljenje **÷** (koliko puta djelitelj stane u djeljenik), a za potenciranje strelicu **↑** (također izvan ASCII-ja).

print spaja tekst i varijable odvojene zarezima. Mora i pretvoriti brojeve s pomičnim zarezom u ASCII:

```
Umnozak 12.5 i 3.2 je 40
```

Zatim program završi i vrati upravljanje operacijskom sustavu.

Da ne bi trebalo mijenjati program i ponovno ga prevoditi za druge brojeve, postoji ugrađena funkcija **read**: čita ASCII znakove utipkane na tipkovnici i pretvara ih u broj s pomičnim zarezom.

```
print ('Upisi prvi broj: ');
read (a);
```

Petlja i blok

> **Petlja** (*loop*) omogućuje da program isto radi za mnogo različitih vrijednosti varijable.

Primjer (vlastiti): kvadrati brojeva 2, 4, 6 i 8.

```
begin
    real a, b;

    for a := 2, 4, 6, 8 do
    begin
        b := a × a;
        print ('Kvadrat od ', a, ' je ', b);
    end
end
```

- **for** postavi a na 2 i izvede naredbu iza **do**.
- Ako treba izvesti više naredbi, stavljaju se između **begin** i **end**. Te dvije ključne riječi čine **blok** naredbi.
- Zatim se blok ponavlja za a = 4, 6 i 8.

Druga inačica **for** naredbe:

```
for a := 1 step 3 until 100 do
```

a počinje od 1, nakon svakog izvođenja bloka povećava se za **step** (3): 1, 4, 7, ..., 100. Kad prijeđe 100, petlja završava.

Programski jezici imaju vrlo **strogu sintaksu**. U ALGOL-u 60 iza **for** smije doći samo jedna vrsta stvari: ime varijable. U ljudskom jeziku iza neke riječi može doći gotovo bilo što. Zato su prevoditelji, iako složeni, mnogo jednostavniji od programa koji bi razumjeli ljudski jezik.

Uvjet

> **Uvjetna naredba** (*conditional*) izvodi drugu naredbu samo ako je neki uvjet ispunjen.

Primjer (vlastiti, s ugrađenom funkcijom **sqrt** za drugi korijen, koja ne radi za negativne brojeve):

```
begin
    real a, b;

    print ('Upisi broj: ');
    read (a);

    if a < 0 then
        print ('Broj je negativan.');
    else
    begin
        b := sqrt(a);
        print ('Drugi korijen od ', a, ' je ', b);
    end
end
```

< znači "manje od". Ako je a < 0, izvodi se prvi print; inače (a ≥ 0) izvodi se blok iza **else**.

Polja

Do sada je svaka varijabla čuvala jednu vrijednost. Često je zgodno da jedna varijabla čuva mnogo vrijednosti.

> **Polje** (*array*) je varijabla s mnogo vrijednosti, **elemenata**. Element se bira **indeksom** u uglatim zagradama.

```
real array a[1:100];
```

Polje od 100 realnih vrijednosti: a[1], a[2], ..., a[100].

```
begin
    real array a[1:100];
    integer i;

    for i := 1 step 1 until 100 do
        a[i] := sqrt(i);

    for i := 1 step 1 until 100 do
        print ('Drugi korijen od ', i, ' je ', a[i]);
end
```

Varijabla **i** je **integer** (tradicionalno ime za cjelobrojnu varijablu jer je prvo slovo riječi *integer*). Prva petlja svakom elementu pridruži drugi korijen njegova indeksa, druga ih ispiše.

Booleove varijable i Eratostenovo sito

Varijable mogu biti i **Boolean** (poglavlje 10): imaju samo dvije vrijednosti, **true** i **false**.

Prosti brojevi su cijeli brojevi djeljivi bez ostatka samo sa sobom i s 1: 2 (jedini paran), 3, 5, 7, 11, 13, 17...

Eratostenovo sito (Eratosten, oko 276.–196. pr. Kr., knjižničar u Aleksandriji, poznat po izračunu opsega Zemlje):

1. Napiši popis cijelih brojeva od 2 nadalje.

2. 2 je prost; prekriži sve višekratnike od 2 (osim samog 2).
3. Sljedeći neprekriženi broj je 3; prekriži sve višekratnike od 3.
4. 4 je već prekrižen. Sljedeći je 5; prekriži njegove višekratnike.
5. Nastavi tako. Što ostane neprekriženo, prosti su brojevi.

Primjer do 30: nakon prekrižavanja višekratnika od 2, 3 i 5 ostaju **2, 3, 5, 7, 11, 13, 17, 19, 23, 29**. (Dovoljno je ići do 5, jer je $5 \times 5 = 25 \leq 30$, a $7 \times 7 = 49 > 30$.)

Program za proste brojeve do 100 (ista struktura kao program iz knjige, koji ide do 10.000):

```
begin
    Boolean array a[2:100];
    integer i, j;

    for i := 2 step 1 until 100 do
        a[i] := true;

    for i := 2 step 1 until 10 do
        if a[i] then
            for j := 2 step 1 until 100 ÷ i do
                a[i × j] := false;

    for i := 2 step 1 until 100 do
        if a[i] then
            print (i);
end
```

1. Prva petlja postavi sve elemente na **true**: pretpostavka je da su svi brojevi prosti.
2. Druga petlja ide od 2 do 10 (drugi korijen od 100). Ako je i prost (a[i] je true), unutarnja petlja postavlja sve višekratnike od i na **false**.
3. Treća petlja ispiše sve i za koje je a[i] true.

Umjetnost ili znanost

Ljudi raspravljaju je li programiranje umjetnost ili znanost: postoje studiji računarske *znanosti*, a i poznata serija knjiga Donalda Knutha *The Art of Computer Programming* ("umjetnost"). Fizičar Richard Feynman rekao je da je računarstvo više kao inženjerstvo: riječ je o tome da se nešto napravi.

Ako 100 ljudi napiše program za proste brojeve, dobit ćeš 100 različitih rješenja; čak ni oni koji koriste Eratostenovo sito neće ga napisati jednako. Da je programiranje čista znanost, ne bi bilo toliko mogućih rješenja, a kriva bi bila očitija. Ponekad problem izazove bljesak kreativnosti (to je "umjetnost"), ali programiranje je ponajviše **projektiranje i gradnja**, kao kod mosta.

Jezici za širi krug ljudi

Rani programeri bili su znanstvenici i inženjeri koji su probleme znali izraziti matematičkim algoritmima, kakve traže FORTRAN i ALGOL. Kasnije su nastajali jezici za širi krug ljudi.

COBOL

COBOL (*COmmon Business Oriented Language*), od 1959.; razvio ga je odbor američke industrije i ministarstva obrane, pod utjecajem prevoditelja Grace Hopper.

- Namjera: da menadžeri mogu barem **čitati** kod i provjeriti radi li ono što treba (u praksi se to rijetko događa).
- Jaka podrška za čitanje **zapisa** (*records*) i izradu **izvješća** (*reports*). Zapis je skup podataka organiziranih na dosljedan način; npr. osiguravajuće društvo ima zapis za svaku policu s imenom osobe, datumom rođenja i drugim podacima.

- Mnogi rani COBOL programi radili su sa zapisima od 80 stupaca na IBM-ovim karticama. Da bi se štedio prostor, godine su se pisale s **dvije** znamenke umjesto četiri. To je najčešći uzrok poznatog **"milenijskog buga"** pred 2000. godinu.

PL/I

PL/I (IBM, sredina 1960-ih, uz System/360): "I" je rimski broj jedan, dakle *Programming Language One*. Trebao je spojiti blokovsku strukturu ALGOL-a, znanstvene funkcije FORTRAN-a i zapise i izvješća COBOL-a. Nikad nije dosegnuo popularnost FORTRAN-a i COBOL-a.

BASIC

Na kućnim računalima nijedan od tih jezika nije imao utjecaj kakav je imao **BASIC** (*Beginner's All-purpose Symbolic Instruction Code*), koji su 1964. razvili John Kemeny i Thomas Kurtz na Dartmouthu, za njihov sustav s dijeljenjem vremena.

- Studenti uglavnom nisu bili matematičari ni inženjeri i nisu htjeli bušene kartice ni teško sintaksu.
- Student za terminalom piše naredbe s **brojevima redaka**; brojevi određuju redoslijed.
- Naredbe bez broja su naredbe sustavu: **SAVE** (spremi program na disk), **LIST** (prikaži retke redom), **RUN** (prevedi i pokreni).

Primjer (vlastiti):

```
10 LET X = (9 + 6) * 2
20 PRINT X
30 END
```

Ispisuje 30.

Za razliku od ALGOL-a, BASIC nije tražio da programer navede je li varijabla cijeli broj ili broj s pomičnim zarezom; većina brojeva spremala se s pomičnim zarezom.

Prevoditelj i interpreter

	Prevoditelj (*compiler*)	Interpreter
Što radi	čita izvorni kod i **stvara izvršnu datoteku**	čita izvorni kod i **izvodi ga izravno** dok ga čita, bez izvršne datoteke
Izrada	teže ga je napisati	lakše ga je napisati
Brzina programa	brže	obično sporije

Mnoge kasnije inačice BASIC-a bile su interpreteri. Na kućnim računalima BASIC je rano krenuo kad su Bill Gates i Paul Allen 1975. napisali BASIC interpreter za Altair 8800 i time pokrenuli tvrtku **Microsoft**.

Analogija: prevoditelj je prevoditelj knjige: jednom prevede cijelu knjigu i čitatelji je zatim brzo čitaju. Interpreter je simultani prevoditelj: prevodi rečenicu po rečenicu dok govornik govori, svaki put iznova.

Pascal i Ada

- **Pascal** (kasne 1960-e, Niklaus Wirth): struktura od ALGOL-a, zapisi od COBOL-a. Popularan među programerima za IBM PC, posebno kao **Turbo Pascal** (Borland, 1983., 49,95 dolara; napisao Anders Hejlsberg). Turbo Pascal je imao **integrirano razvojno okruženje** (IDE): uređivač teksta i prevoditelj u jednom programu, za vrlo brzo programiranje. IDE-ovi su postojali na velikim računalima, a Turbo Pascal ih je donio na mala.
- **Ada**: jezik za američko ministarstvo obrane, pod snažnim utjecajem Pascala; ime po Augusti Adi Byron (poglavlje 18).

C

C (1969.–1973., uglavnom Dennis Ritchie, Bell Labs). Ime: izveden je iz jezika **B**, pojednostavljene inačice **BCPL**-a, koji potječe od **CPL**-a.

- **UNIX** je 1973. napisan (prepisan) u C-u. Većina sustava tada je pisana u asembleru za određeni procesor; UNIX je trebao biti prenosiv. Od tada su UNIX i C usko povezani.
- C je vrlo **sažet**: blokove označava vitičastim zagradama { } umjesto begin i end.

```
i = i + 5;     /* povećaj i za 5 */
i += 5;        /* isto, kraće */
i++;           /* povećaj i za 1 */
```

Na 16- i 32-bitnim procesorima `i++` se može izvesti **jednom** strojnom instrukcijom.

- C je iznimka od pravila: ima operacije **pomaka bitova** i **Booleove operacije nad bitovima**.
- Podržava **pokazivače** (*pointers*): to su zapravo brojčane adrese memorije.
- Zato se C ponekad zove **"asembler visoke razine"**: bolje od drugih jezika sličnih ALGOL-u oponaša tipične skupove instrukcija procesora.

Izvan von Neumanna

Svi jezici slični ALGOL-u (a to je većina jezika u upotrebi) osmišljeni su za računala **von Neumannove arhitekture**. Teško je izaći iz takvog načina razmišljanja, a još teže uvjeriti druge da koriste takav jezik.

Jezik	Tko, kada	Bitno
LISP (*List Processing*)	John McCarthy, kasne 1950-e	koristan za **umjetnu inteligenciju**
APL (*A Programming Language*)	Kenneth Iverson, kasne 1950-e	zbirka neobičnih simbola koji rade operacije nad **cijelim poljima** brojeva odjednom

Objektno orijentirani jezici

Jezici slični ALGOL-u zadržali su prevlast, ali su posljednjih godina dobili poboljšanja zbog kojih se zovu **objektno orijentirani** jezici. Korisni su za rad s grafičkim operacijskim sustavima (poglavlje 25). Detalji objektno orijentiranog programiranja: **U knjizi nije detaljno objašnjeno.**

Ključne definicije

Pojam	Značenje
Asembler	program koji asemblerski jezik pretvara u strojni kod
Izvorni kod	program kako ga je napisao čovjek
Izvršna datoteka	datoteka sa strojnim kodom spremnim za pokretanje
Križni asembler	asembler koji radi na računalu A, a stvara kod za računalo B
Jezik niske / visoke razine	blizu hardvera / neovisan o procesoru, bliži ljudskom zapisu
Sintaksa	pravila zapisa naredbi u jeziku
Prevoditelj	pretvara cijeli program visoke razine u strojni kod
Interpreter	izvodi izvorni kod izravno, naredbu po naredbu
Ključna riječ	riječ koju jezik prepoznaje (begin, end, for, if...)
Sintaksna pogreška / pogreška pri izvođenju	otkriva je prevoditelj / vidi se tek pri radu programa
Varijabla	ime koje označava mjesto u memoriji
Deklaracija	naredba koja najavljuje varijable i njihov tip

Pojam	Značenje
Pridruživanje	varijabla dobiva vrijednost izraza
Petlja	ponavljanje naredbi za razne vrijednosti
Blok	skupina naredbi između begin i end (u C-u { })
Uvjetna naredba	izvodi se samo ako je uvjet ispunjen (if-then-else)
Polje, element, indeks	varijabla s mnogo vrijednosti / jedna vrijednost / njezin redni broj
Booleova varijabla	ima vrijednost true ili false
Pokazivač	brojčana adresa memorije (u C-u)
IDE	integrirano razvojno okruženje: uređivač + prevoditelj

Ključno za zapamtiti

1. Asembler pretvara asemblerski jezik u strojni kod odnosom jedan na jedan i sam računa adrese oznaka.
2. Asemblerski jezik je zamoran i nije prenosiv.
3. Jezik visoke razine omogućuje algebarski zapis; prevoditelj jednu naredbu pretvara u mnogo instrukcija.
4. Programi visoke razine su prenosivi, ali za svaki procesor treba prevoditelj, a izvršne datoteke ostaju vezane uz procesor.
5. Jezik visoke razine ne može učiniti procesor moćnijim; može samo ograničiti njegove mogućnosti.
6. Osnovni gradivni elementi: varijable, pridruživanje, petlje, uvjeti, polja.
7. Sintaksne pogreške otkriva prevoditelj; pogreške pri izvođenju vide se tek kad program radi.
8. Interpreter je lakše napisati, ali programi su sporiji nego prevedeni.
9. C je sažet, ima operacije nad bitovima i pokazivače; zovu ga "asembler visoke razine".
10. Većina jezika (slični ALGOL-u) osmišljena je za von Neumannova računala; LISP i APL su iznimke.

Najčešće zabune

- **"Asembler i prevoditelj su isto."** Asembler prevodi jedan na jedan; prevoditelj jednu naredbu pretvara u mnogo instrukcija i mnogo je složeniji.
- **"Program u jeziku visoke razine radi na svakom procesoru."** Radi izvorni kod, ako postoji prevoditelj; izvršna datoteka je i dalje vezana uz procesor.
- **"Ako se program prevede bez pogreške, ispravan je."** Prevoditelj otkriva samo sintaksne pogreške.
- **"a := b znači da su a i b jednaki."** To je **pridruživanje**: a dobiva vrijednost b.
- **"Jezik visoke razine može sve što i asembler."** Ne može koristiti posebnosti procesora (npr. pomak bitova) koje jezik ne predviđa.

Brza provjera razumijevanja

1. Koji bi bajtovi nastali asembliranjem programa koji ispisuje "Hej$" (isti oblik kao primjer BOK)? Na kojoj adresi počinje tekst?
2. Navedi dvije prednosti i dva nedostatka jezika visoke razine.
3. Napiši u stilu ALGOL-a petlju koja ispisuje kubove brojeva od 1 do 10.
4. Provedi Eratostenovo sito do 50. Koje višekratnike treba prekrižiti i do kojeg broja?
5. U čemu se razlikuju prevoditelj i interpreter?

25. Grafička revolucija

Što ćeš naučiti u ovom poglavlju

- Što je **korisničko sučelje** i kako se razvijalo: od skupne obrade i teleprintera do **grafičkog sučelja** (GUI).
- Kako rade **escape sekvence** za upravljanje zaslonom.
- Zašto su mala računala s video memorijom u adresnom prostoru omogućila nove programe (VisiCalc).
- **Rasterski** i **vektorski** zasloni; **boje** (RGB), **dubina boje**.
- Glavne korake razvoja GUI-ja: Sketchpad, miš, Xerox Alto, Macintosh, Windows.
- Što nudi **grafički operacijski sustav**: grafički API, WYSIWYG, zajedničke komponente sučelja.
- Osnovnu ideju **objektno orijentiranog programiranja**.
- **Vektorsku** i **rastersku** grafiku, **sažimanje** podataka (RLE, LZW, JPEG; bez gubitaka i s gubicima).
- Kako se **zvuk** digitalizira (ADC, PCM, frekvencija uzorkovanja, CD) i što je **MIDI**.
- Koliko podataka traži **video** i čemu služi MPEG.
- Kako računala komuniciraju: **modem**, **Internet**, **TCP/IP**, **HTML**, **Java** i **optička vlakna**.

Ideja: informacija na dohvat ruke

U članku "As We May Think" (1945.) Vannevar Bush, koji je ranije projektirao jedno od najvažnijih **analognih** računala (diferencijalni analizator, 1927.–1931.), opisao je zamišljeni uređaj **Memex**: stol s mikrofilmovima koji čuva knjige, članke, zapise i slike, i omogućuje **tematske veze** među njima, onako kako ih ljudski um povezuje. Članak nije o kućanskim aparatima ni robotima, nego o **informaciji** i o tome kako nam tehnologija može pomoći da se s njom nosimo.

U šest i pol desetljeća od prvih relejnih računala računala su postala **manja, brža i jeftinija** u isto vrijeme. Jeftinija znači da svatko može imati svoje; manja i brža znači da softver može biti složeniji i preuzeti više posla.

Dio te snage troši se na najvažniji dio računalnog sustava:

> **Korisničko sučelje** (*user interface*) je mjesto gdje se susreću čovjek i računalo.

Ljudi i računala su vrlo različiti, a lakše je ljude nagovoriti da se prilagode računalu nego obratno.

Razvoj interakcije

Razdoblje	Način rada
Rana računala	**nisu bila interaktivna**: programiranje prekidačima i kabelima, ili bušenom papirnatom vrpcom i filmom
1950-e – 1970-e	**skupna obrada** (*batch processing*): programi i podaci bušeni na kartice, učitani, obrađeni, rezultati ispisani na papir
Prva interaktivna računala	**teleprinteri**, npr. Dartmouthov sustav s dijeljenjem vremena (početak 1960-ih): korisnik utipka redak, računalo odgovori jednim ili više redaka; razmjena je niz ASCII znakova i nekoliko upravljačkih kodova (CR, LF); tekst ide samo u jednom smjeru niz rolu papira
1970-e	**katodna cijev**: zaslon bi se mogao koristiti kao dvodimenzionalna ploha, ali mnogi programi koristili su ga kao "**stakleni teleprinter**": ispis red po red i pomicanje teksta prema gore kad dođe do dna

Svi uslužni programi CP/M-a i većina u MS-DOS-u radili su kao teleprinter. Arhetipski "teleprinterski" sustav je UNIX, koji tu tradiciju i dalje čuva.

Escape sekvence

ASCII nije potpuno neprikladan za zaslone. Kod **1Bh (Escape)** bio je namijenjen proširenjima skupa znakova. ANSI je 1979. objavio standard "Additional Controls for Use with ASCII" za upravljanje dvodimenzionalnim uređajima za prikaz znakova (zaslonima i pisačima).

Escape je samo jedan bajt, pa radi tako da stoji **ispred niza znakova promjenjive duljine** koji opisuje funkciju:

Bajtovi	Znakovi	Značenje
1Bh 5Bh 32h 4Ah	ESC [2J	obriši cijeli zaslon i stavi kursor u gornji lijevi kut
1Bh 5Bh 35h 3Bh 32h 39h 48h	ESC [5;29H	stavi kursor u redak 5, stupac 29
1Bh 5Bh 31h 32h 3Bh 34h 30h 48h (vlastiti)	ESC [12;40H	stavi kursor u redak 12, stupac 40

Na teleprinteru to nije moguće.

> **"Glupi terminal"** (*dumb terminal*): tipkovnica i zaslon koji reagiraju na ASCII kodove (i escape sekvence) s udaljenog računala. Brži i fleksibilniji od teleprintera, ali ne dovoljno brz za prave novosti u sučelju.

Video memorija u adresnom prostoru

Prave novosti donijela su mala računala 1970-ih kojima je **video memorija dio adresnog prostora procesora** (kao računalo iz poglavlja 21).

VisiCalc (Dan Bricklin i Bob Frankston, 1979., za Apple II) bio je prvi znak da će kućna računala biti nešto sasvim drugo:

- **proračunska tablica** (*spreadsheet*): prije je to bio papir s retcima i stupcima za nizove izračuna,
- zaslon daje **dvodimenzionalni** pogled na tablicu; korisnik se kreće po njoj, unosi brojeve i formule, a nakon promjene se sve ponovno izračuna,
- VisiCalc se **nije mogao napraviti na većim računalima**: mora vrlo brzo osvježavati zaslon, pa piše **izravno** u video memoriju Apple II. Veza između velikog računala s dijeljenjem vremena i glupog terminala preslaba je za to.

> Što brže računalo reagira na tipkovnicu i mijenja zaslon, to je čvršća interakcija korisnika i računala.

Većina softvera za IBM PC u 1980-ima pisala je izravno u video memoriju. Budući da je IBM postavio hardverski standard kojeg su se drugi proizvođači držali, programi su mogli zaobići operacijski sustav bez straha da neće raditi na nekim strojevima.

Rane aplikacije za PC uglavnom su koristile samo **tekst**: tekstualni zaslon prikazuje znak upisom **jednog** bajta (ASCII koda), a grafički mora upisati **8 ili više** bajtova da nacrta sliku znaka. Tekst je bio brži.

Računalna grafika: od SAGE-a do miša

Prijelaz s prikaza znakova na grafiku bio je vrlo važan korak, ali se razvijao sporo.

Kada	Tko, što
1945.	John von Neumann zamišlja zaslon sličan osciloskopu za grafički prikaz
rane 1950-e	**SAGE** (*Semi-Automatic Ground Environment*): MIT-ov Lincoln Laboratory (uz IBM) gradi računala za protuzračnu obranu, s grafičkim zaslonima za analizu velikih količina podataka

Rasterski i vektorski zasloni

	Rasterski zaslon (današnji PC)	Vektorski zaslon (SAGE)
Princip	kao televizor: slika od vodoravnih linija koje elektronski top iscrtava amo-tamo	više kao osciloskop: elektronski top može se električki usmjeriti **bilo kamo** i izravno crtati crte i krivulje
Memorija	blok memorije s 1 ili više bitova za svaki **piksel**	mnogo manje
Napomena	danas prirodan izbor; u ranim danima nepraktičan jer je tražio mnogo (tada skupe) memorije	slika se zadržava na zaslonu dovoljno dugo da se crte slože u jednostavne slike

Primjer: 640 × 480 = 307.200 piksela. S 1 bitom po pikselu (npr. 0 = crno, 1 = bijelo) treba 307.200 **bitova** = **38.400 bajtova**. Za nijanse sive obično je po pikselu jedan bajt: 00h crno, FFh bijelo, između nijanse sive.

Boja

Katodna cijev u boji ima **tri elektronska topa**, po jedan za svaku **aditivnu** osnovnu boju: **crvenu, zelenu i plavu** (RGB). (Tisak koristi drugi skup osnovnih boja.)

Kombinacija	Rezultat
crvena + zelena	žuta
crvena + plava	magenta
zelena + plava	cijan
sve tri	bijela

Najjednostavniji adapter u boji: 3 bita po pikselu, po 1 bit za svaku osnovnu boju:

Bitovi (R G B)	Boja
000	crna
001	plava
010	zelena
011	cijan
100	crvena
101	magenta
110	žuta
111	bijela

To je dobro samo za jednostavne crtane slike. Stvarne boje su kombinacije različitih **razina** osnovnih boja:

Način	Bitova po pikselu	Razina po boji	Ukupno boja
High color ("tisuće boja")	16 (5 + 5 + 5, 1 bit višak)	32	32.768
Full color ("milijuni boja")	24 (po 1 bajt)	256	16.777.216

> **Dubina boje** (*color depth*, razlučivost boje) = broj bitova po pikselu. **Broj boja = 2^(dubina boje)**.

Memorija adaptera ograničava kombinacije razlučivosti i dubine boje. Adapter s **1 MB** (1.048.576 bajtova):

Razlučivost	3 bajta po pikselu	2 bajta po pikselu
640 × 480	921.600 B: **stane**	—
800 × 600	1.440.000 B: **ne stane**	960.000 B: stane

Svjetlosna olovka

SAGE je podržavao **svjetlosne olovke** (*light pens*): pisaljke s žicom kojima operater mijenja sliku na zaslonu. Trik: olovka svjetlo **ne odašilje**, nego ga **opaža**. Sklop koji upravlja elektronskim topom zna gdje je zraka u svakom trenutku, pa iz trenutka kad svjetlo pogodi olovku zna i gdje olovka pokazuje.

Vizionari interaktivnog računanja

Tko	Što
Ivan Sutherland, 1963.	**Sketchpad** na računalima SAGE: opisi slika u memoriji, prikaz na zaslonu, crtanje i mijenjanje slika svjetlosnom olovkom; računalo sve prati
Douglas Engelbart	pročitao Bushov članak 1945.; sredinom 1960-ih (Stanford Research Institute) iznova osmislio ulazne uređaje: tipkovnicu s pet tipki za naredbe (nije zaživjela) i manji uređaj s kotačićima i tipkom: **miš** (danas univerzalno prihvaćen za pomicanje pokazivača i odabir objekata na zaslonu); rad iz 1963. "za uvećanje ljudskog intelekta"
Xerox PARC (osnovan 1970.)	okupio entuzijaste grafičkog računanja u vrijeme kad su rasterski zasloni postali isplativi
Alan Kay (PARC)	zamislio prijenosno računalo **Dynabook**

Xerox Alto (1972.–1973.)

Svojstvo	Alto
Obrada	16-bitna; procesor od oko 200 integriranih krugova (16-bitni jednočipni procesori tada još nisu postojali)
Memorija	128 KB (proširivo na 512 KB)
Pohrana	dva disk pogona od 3 MB
Miš	s tri tipke
Zaslon	veličine lista papira (8 × 10 inča), rasterski, **606 × 808** = 489.648 piksela, 1 bit po pikselu
Video memorija	64 KB, dio adresnog prostora procesora

Upisom u video memoriju softver je crtao slike i prikazivao tekst u raznim **fontovima** i veličinama. Mišem se pokazivač pomiče po zaslonu i radi s objektima. Zaslon nije više teleprinter, nego dvodimenzionalno polje informacija velike gustoće i izravniji izvor korisničkog unosa.

Programi za Alto razvili su:

- više programa u **prozorima** (*windows*) na zaslonu istovremeno,
- grafičke objekte: **gumbe**, **izbornike** i male sličice, **ikone** (*icons*),
- miš za odabir prozora i pokretanje funkcija.

> To su počeci **grafičkog korisničkog sučelja**, **GUI** (*graphical user interface*).

Xerox Alto nije prodavao (koštao bi više od 30.000 dolara). Prošlo je više od desetljeća prije uspješnog proizvoda s tim idejama.

Macintosh

- 1979.: Steve Jobs i ekipa iz Applea posjetili PARC.
- Siječanj 1983.: **Apple Lisa**, neuspješna.
- 1984.: **Macintosh**:

Svojstvo	Izvorni Macintosh
Procesor	Motorola 68000
ROM / RAM	64 KB / 128 KB

Svojstvo	Izvorni Macintosh
Pohrana	disketa 3½ inča (400 KB)
Zaslon	512 × 342 = 175.104 piksela (katodna cijev od 9 inča), 1 bit po pikselu, oko **22 KB** video memorije
Ulaz	tipkovnica, miš

Hardver je bio elegantan, ali ne revolucionaran. Revolucionaran je bio **operacijski sustav** (sistemski softver, kasnije **Mac OS**).

Grafički operacijski sustav

	Tekstualni sustav (CP/M, MS-DOS)	Grafički sustav (Mac OS)
Veličina	malen	mnogo veći
API	mali; uglavnom datotečni sustav i ispis teksta kao na teleprinteru	**stotine** funkcija, svaka s imenom koje opisuje što radi

U teoriji bi za grafiku bila dovoljna jedna API funkcija koja postavlja boju piksela na zadanim koordinatama. U praksi je to **neučinkovito** i grafika bi bila vrlo spora. Zato operacijski sustav ima potpun **grafički programski sustav**:

- API funkcije za **crte, pravokutnike i elipse** (i krugove); crte pune ili isprekidane, likovi ispunjeni uzorcima,
- **tekst** u raznim fontovima i veličinama, podebljan, podcrtan,
- sustav sam određuje kako te objekte prikazati kao skup točaka.

WYSIWYG (*What you see is what you get*, "što vidiš, to i dobiješ"): programi koriste **iste** API funkcije za crtanje na zaslonu i na pisaču. Uređivač teksta može prikazati dokument gotovo točno kako će izgledati ispisan.

Dosljednost: različite aplikacije rade otprilike jednako, pa korisnik prenosi iskustvo s jedne na drugu. Zato sustav ima i API funkcije za **komponente sučelja**: gumbe, izbornike. GUI je jednostavno okruženje za korisnike, ali jednako važno **za programere**: suvremeno sučelje mogu napraviti bez "ponovnog izmišljanja kotača".

Windows

Grafički sustav za IBM PC bio je teži nego za Macintosh:

- Apple je projektirao hardver i softver zajedno: jedna vrsta disketnog pogona, jedan zaslon, dva pisača; PC je trebao podržati **mnogo različitog hardvera**,
- korisnici PC-a nisu htjeli odustati od svojih **MS-DOS aplikacija**, pa je grafički sustav morao pokretati i njih. (Macintosh nije pokretao programe za Apple II, uglavnom jer je imao drugi procesor.)

Godina	Događaj
1985.	GEM (Digital Research), VisiOn (VisiCorp), **Windows 1.0** (Microsoft)
svibanj 1990.	**Windows 3.0**: početak velike popularnosti; danas na oko 90 % malih računala

Unatoč sličnom izgledu, API-ji Macintosha i Windowsa vrlo su različiti.

Hardver: u teoriji grafički sustav ne traži mnogo više od tekstualnog (osim zaslona). Izvorni Macintosh nije imao tvrdi disk; Windows 1.0 nije zahtijevao ni tvrdi disk ni miš (iako je s mišem bilo mnogo lakše). Ali kako su procesori postali brži, a memorija i pohrana obilnije, sustavi su narasli: tada (1999.) traže nekoliko stotina MB diska i više od 32 MB memorije.

Objektno orijentirano programiranje

Aplikacije za grafičke sustave gotovo se nikad ne pišu u asembleru. Za rani Macintosh popularan je bio **Pascal**, za Windows **C**.

PARC je pokazao i drugi pristup: od oko 1972. razvijao je jezik **Smalltalk**, koji ostvaruje **objektno orijentirano programiranje** (OOP).

	Tradicionalni jezici	Objektno orijentirani
Osnova	razlikuju **kod** (naredbe s ključnim riječima poput *set*, *for*, *if*) i **podatke** (brojeve u varijablama); to potječe od von Neumannove arhitekture	**objekt** = kod + podaci zajedno
Podaci	dostupni cijelom programu	način spremanja podataka u objektu zna samo **kod tog objekta**
Suradnja	pozivi potprograma	objekti razmjenjuju **poruke** (*messages*): upute objektu ili traženje informacije od njega

Primjer: gumb. Gumb ima veličinu, položaj na zaslonu i tekst ili sličicu: to su podaci objekta. Kod objekta određuje kada je korisnik "pritisnuo" gumb (tipkovnicom ili mišem) i šalje **poruku** da je gumb aktiviran. Programer objekte na zaslonu tretira onako kako ih vidi korisnik.

- Najpopularniji objektno orijentirani jezici za mala računala su proširenja jezika sličnih ALGOL-u (C, Pascal). Najpoznatiji je **C++** (Bjarne Stroustrup, Bell Labs; ++ je operator povećanja u C-u). C++ je isprva ostvaren kao prevoditelj iz C++-a u (vrlo ružan i nečitljiv) C, koji se zatim normalno prevodi.
- OOP ne može ništa **više** od tradicionalnih jezika. Ali programiranje je rješavanje problema, a OOP omogućuje rješenja koja su često **strukturno bolja**.
- Moguće je (ne lako) napisati jedan program koji se prevodi i za Macintosh i za Windows: program ne poziva API izravno, nego koristi objekte koji pozivaju API; za svaki sustav koristi se druga definicija objekata.
- Programeri danas rade u **integriranom razvojnom okruženju** (IDE), koje je i samo grafička aplikacija, i koriste **vizualno programiranje**: prozori se slažu mišem, postavljanjem gumba i drugih komponenti.

Tekst s oblikovanjem

Tekstualne datoteke (poglavlje 22) bile su idealne za razmjenu podataka, a lako se **pretražuju**. Ali kad sustav može prikazivati fontove, kurziv i podebljanje, obična tekstualna datoteka nije dovoljna, a ni za slike nije prikladna. Većina uređivača teksta zato sprema dokumente u vlastitom **binarnom** formatu.

Može se i u tekstualnu datoteku upisati oblikovanje, ako se odabere **escape znak** koji ga označava:

Format	Opis
RTF (*Rich Text Format*, Microsoft)	za razmjenu oblikovanog teksta; vitičaste zagrade { } i obrnuta kosa crta \ označavaju upute za oblikovanje
PostScript (John Warnock, Adobe)	tekstualni format doveden do krajnosti: cijeli **programski jezik opće namjene za grafiku**, uglavnom za crtanje teksta i slika na kvalitetnim pisačima

Vektorska i rasterska grafika

	Vektorska grafika	Rasterska (bitmap) grafika
Kako nastaje	**algoritamski**: crte, krivulje, ispunjena područja	pravokutno polje bitova koji odgovaraju pikselima izlaznog uređaja
Primjena	**CAD** (*computer-assisted drawing*), inženjerstvo i arhitektura	fotografije i slike stvarnog svijeta
Spremanje	**metadatoteka** (*metafile*): zbirka naredbi za crtanje, obično binarno	**bitmapa**: prostorna dimenzija (razlučivost, širina i visina u pikselima) i dimenzija boje (dubina boje)

Primjer iz knjige: za projektiranje mosta vektorska grafika je savršena, ali za prikaz kako izgrađeni most stvarno izgleda potpuno je nedovoljna; ta slika je presložena.

Iako je bitmapa dvodimenzionalna, sama je **jedan niz bajtova**: obično gornji red piksela, zatim drugi, treći i tako dalje.

Slike iz stvarnog svijeta

Bitmape nastaju ručno (programom za crtanje), algoritamski, ili najčešće iz stvarnog svijeta. Za to se koristi **CCD** (*charge-coupled device*): poluvodič koji oslobađa električni naboj kad je izložen svjetlu. Za svaki piksel treba jedna CCD ćelija.

Uređaj	Kako radi
Skener (najstariji)	kao fotokopirni stroj: red CCD-ova prelazi po slici; softver naboje pretvara u bitmapu u datoteci
Videokamera	dvodimenzionalni niz CCD ćelija; slike obično na videovrpci
Video frame grabber	pločica koja analogni video signal (npr. iz videorekordera) pretvara u polje piksela
Digitalni fotoaparat	polje CCD-ova umjesto filma; slika se sprema u memoriju aparata i prenosi u računalo

Formati bitmapa: na Macintoshu **Paint** (po programu MacPaint) i **PICT** (bitmape i vektorska grafika zajedno); u Windowsu **BMP**.

Sažimanje podataka

Bitmape mogu biti vrlo velike, pa ih je korisno smanjiti. To je područje **sažimanja podataka** (*data compression*).

RLE (*run-length encoding*, kodiranje duljine niza). Primjer (vlastiti): na slici neba, kuće i travnjaka s 3 bita po pikselu jedan red počinje s **50 plavih** piksela, a zatim slijedi **14 zelenih**. Umjesto 64 jednaka zapisa, spremi se "50 × plava, 14 × zelena".

- Faks koristi RLE: slika je crno-bijela, s dugim nizovima bijelih piksela.

GIF (*Graphics Interchange Format*, CompuServe, 1987.) koristi **LZW** (Lempel, Ziv, Welch). LZW je jači od RLE-a jer prepoznaje **uzorke** piksela različitih vrijednosti, a ne samo nizove istih.

	Bez gubitaka (*lossless*)	**S gubicima (*lossy*)**
Povrat originala	izvorna datoteka može se **potpuno** obnoviti; sažimanje je **reverzibilno**	dio podataka se odbacuje; nije reverzibilno
Primjeri	RLE, LZW	**JPEG** (*Joint Photographic Experts Group*; zapravo više tehnika, nekih bez gubitaka i nekih s gubicima)
Prikladno za	sve; obavezno za tablice i dokumente (svaki broj i riječ su važni)	slike, ako odbačeni podaci ne mijenjaju bitno ukupnu sliku; temelji se na **psihovizualnim** istraživanjima ljudskog vida

Lako je dokazati da sažimanje bez gubitaka ne radi za **svaku** datoteku: ponekad je "sažeta" datoteka **veća** od izvorne.

Pretvorbe

- **Metadatoteka → bitmapa:** lako. Video memorija i bitmapa su pojmovno isto: tko zna nacrtati metadatoteku u video memoriju, zna je nacrtati i u bitmapu.
- **Bitmapa → metadatoteka:** teško, a za složene slike možda nemoguće. Srodna tehnika je **OCR** (*optical character recognition*, optičko prepoznavanje znakova): bitmapa teksta (s faksa ili skenirane stranice) pretvara se u ASCII kodove. Softver analizira uzorke bitova; zbog složenosti obično nije 100 % točan. Prepoznavanje rukopisa još je manje točno.

Zvuk

Bitmape i metadatoteke su digitalni prikaz vizualne informacije. I zvuk se može pretvoriti u bitove i bajtove.

CD (Philips i Sony, 1983.) bio je najveći uspjeh potrošačke elektronike: **74 minute** digitaliziranog zvuka na jednoj strani diska promjera **12 cm**. (Duljina je odabrana tako da stane Beethovenova Deveta simfonija.)

Zvuk je titranje

Zvuk je **titranje**: glasnice, tuba ili drvo koje pada pokreću molekule zraka. Zrak se naizmjence zgušnjava i razrjeđuje stotine ili tisuće puta u sekundi, a to titra naše bubnjiće.

- Edisonov fonograf (1877.): zvučni valovi kao brežuljci i dolinice na staniolu. Tehnika se do CD-a jedva mijenjala (valjci → ploče; staniol → vosak → plastika); kasnije je dodano električno pojačanje.
- **Mikrofon** (promjenjivi otpornik) pretvara zvuk u električnu struju, a **zvučnik** (elektromagnet) struju natrag u zvuk.
- Ta struja **nije** digitalni signal "uključeno-isključeno": napon se mijenja **kontinuirano**, kao zvučni val. Struja je **analogija** (analogni prikaz) zvučnog vala.

Pretvorba analognog u digitalno

Pojam	Značenje
ADC (*analog-to-digital converter*)	čip koji analogni napon pretvara u binarni broj od obično 8, 12 ili 16 bitova. 12-bitni ADC daje broj od 000h do FFFh: razlikuje **4096** razina napona
DAC (*digital-to-analog converter*)	pretvara brojeve natrag u električnu struju. Koristi se i u grafičkim pločicama u boji: vrijednost piksela pretvara u analogne signale boje za monitor
PCM (*pulse code modulation*)	napon zvučnog vala pretvara se u digitalne vrijednosti **stalnom brzinom**
Frekvencija uzorkovanja (*sampling rate*)	koliko puta u sekundi se napon pretvara u broj

Na CD-u se brojevi spremaju kao sitne udubine u površini diska i čitaju laserskim svjetlom koje se odbija od površine.

> **Nyquist** (Bell Labs, 1928.): frekvencija uzorkovanja mora biti **barem dvostruko veća** od najveće frekvencije koju treba snimiti i reproducirati.

Smatra se da čovjek čuje od 20 Hz do 20.000 Hz. CD koristi nešto više od dvostrukog maksimuma: **44.100 uzoraka u sekundi**.

Dinamički raspon

Broj bitova po uzorku određuje **dinamički raspon**: razliku između najglasnijeg i najtišeg zvuka koji se može snimiti. Glasnoća se mjeri u belima: **bel** je deseterostruko povećanje intenziteta zvuka (ime po Alexanderu Grahamu Bellu), a **decibel** je desetina bela. Jedan decibel je otprilike najmanje povećanje glasnoće koje čovjek primijeti.

16 bitova po uzorku daje dinamički raspon od oko **96 decibela**, otprilike razliku između praga čujnosti (ispod kojeg ništa ne čujemo) i praga boli. CD koristi 16 bitova po uzorku.

Koliko podataka

Veličina	Račun	Rezultat
Mono, 1 s	44.100 × 2 bajta	88.200 B
Stereo, 1 s	× 2 kanala	**176.400 B**
Stereo, 1 min	× 60	10.584.000 B
CD, 74 min	× 74	**783.216.000 B**

Zato digitalno snimanje zvuka nije bilo uobičajeno prije 1980-ih.

Prednosti digitalnog zvuka: svako kopiranje analognog zvuka gubi kvalitetu, a brojevi se uvijek mogu vjerno prepisati i kopirati. Nekad je telefonski signal bio lošiji što je dulje putovao žicom; danas je većina telefonskog sustava digitalna, pa poziv s drugog kraja zemlje zvuči jednako jasno kao iz susjedstva.

CD-ROM i multimedija

- CD koji čuva samo podatke je **CD-ROM** (*CD Read-Only Memory*), obično do oko **660 MB**. Mnogo softvera i igara distribuira se na CD-ROM-u.
- Zvuk, glazba i video u računalu zvali su se **multimedija**; danas je to toliko uobičajeno da ne treba posebno ime.
- **Zvučna kartica** ima ADC (snimanje preko mikrofona) i DAC (reprodukcija preko zvučnika). Zvuk se sprema u **valne datoteke** (*waveform files*).
- Kad ne treba kvaliteta CD-a, Macintosh i Windows nude niže frekvencije (22.050 Hz, 11.025 Hz, 8000 Hz), 8 bitova po uzorku i mono. Minimalno: **8000 bajtova u sekundi** = 480.000 bajtova u minuti.

Govor

- **Sinteza govora:** (1) čovjek snimi dijelove rečenica, riječi i brojeve koji se spajaju (npr. telefonski informacijski sustavi; dobro kad je kombinacija malo); (2) općenitije, pretvorba proizvoljnog ASCII teksta u valni oblik. Pravopis nije uvijek dosljedan, pa softver koristi rječnik ili algoritme za izgovor; osnovni glasovi (**fonemi**) spajaju se u riječi, uz prilagodbe (npr. kod pitanja se zadnjoj riječi podiže ton).
- **Prepoznavanje govora** (valni oblik → ASCII) mnogo je teže; i ljudi teško razumiju regionalne razlike. Programi za diktiranje obično trebaju vježbu s određenim govornikom. Da računalo zaista "razumije" što je rečeno, problem je **umjetne inteligencije**.

MIDI

Zvučne kartice imaju i male elektroničke **sintetizatore** koji oponašaju **128** glazbenih instrumenata i **47** udaraljki: **MIDI** sintetizatore (*Musical Instrument Digital Interface*; specifikacija s početka 1980-ih, konzorcij proizvođača, za povezivanje elektroničkih instrumenata međusobno i s računalima).

- Sintetizator reagira na kratke **poruke** od 1, 2 ili 3 bajta: koji instrument, početak tona, kraj tona.
- **MIDI datoteka** je zbirka poruka s vremenskim podacima, obično cijela skladba. Mnogo je **manja** od valne datoteke iste glazbe.
- **Analogija iz knjige:** valna datoteka je kao bitmapa, a MIDI datoteka kao vektorska metadatoteka.
- Mana: ista MIDI datoteka može zvučati izvrsno na jednom sintetizatoru, a grozno na drugom. Kvaliteta zvuka nije dio MIDI specifikacije.

Video

Prividno kretanje u filmu i na televiziji postiže se brzim prikazom niza mirnih slika, **sličica** (*frames*):

Medij	Sličica u sekundi
Film	24
Televizija u Sjevernoj Americi	30
Televizija u većini drugih zemalja	25

Film u računalu je niz bitmapa sa zvukom. Bez sažimanja (640 × 480, 24-bitna boja):

Veličina	Bajtova
Jedna sličica	921.600
1 sekunda (30 sličica)	27.648.000
1 minuta	1.658.880.000

Veličina	Bajtova
Film od 2 sata	199.065.600.000 ≈ **200 GB**

Zato su filmovi na računalu (tada) kratki, mali i trzaju.

MPEG (*Moving Picture Experts Group*) za filmove radi ono što JPEG radi za slike. Sažimanje pokretnih slika koristi činjenicu da sličica obično sadrži mnogo informacija koje se **ponavljaju iz prethodne** sličice.

- **MPEG-2** koristi se za televiziju visoke razlučivosti (HDTV) i **DVD** (*digital video* ili *versatile disc*).
- DVD je veličine CD-a, ali se može snimati na **obje strane** i u **dva sloja** po strani. Video je sažet oko **50 puta**, pa film od 2 sata treba samo oko **4 GB** (jedan sloj jedne strane). S oba sloja i obje strane kapacitet je oko **16 GB**, oko 25 puta više od CD-a.

Jesu li CD-ROM i DVD-ROM današnji Memex? Elektronički mediji imaju prednost da se lako **pretražuju**. Ali najbliže Bushovoj ideji nije sve informacije držati na stolu, nego **povezati računala** da dijele informacije i pohranu.

Komunikacija među računalima

Prvi udaljeni rad s računalom: George Stibitz, 1940., na demonstraciji na Dartmouthu upravljao je relejnim računalom na daljinu.

Modem

Telefonski sustav prenosi **zvuk**, ne bitove. Za slanje bitova telefonom bitovi se pretvaraju u zvuk i natrag.

- Stalni zvučni val jedne frekvencije i amplitude, **nositelj** (*carrier*), ne nosi nikakvu informaciju.
- Ako se val **modulira**, tj. mijenja između dva stanja, može predstavljati 0 i 1.
- Pretvorbu radi **modem** (*modulator/demodulator*).

Modem je **serijsko** sučelje: bitovi bajta šalju se **jedan za drugim**. (Pisači se često spajaju **paralelno**: osam žica prenosi cijeli bajt istovremeno.)

FSK (*frequency-shift keying*) u ranim modemima. Modem od **300 bita u sekundi**:

Bit	Frekvencija
0	1070 Hz
1	1270 Hz

Ispred svakog bajta je **početni bit**, a iza **završni bit**, pa jedan bajt traži **10 bitova**. Brzina je zato 300 / 10 = **30 bajtova u sekundi**. (Vlastiti primjer istog računa: 1200 bita/s → 120 bajtova/s.) Moderniji modemi koriste složenije tehnike i više su od 100 puta brži.

BBS (*bulletin board system*): entuzijast postavi računalo s modemom, a drugi se spajaju i **preuzimaju** (*download*) datoteke s udaljenog računala na svoje. Iz toga su izrasli veliki informacijski servisi poput CompuServea. Komunikacija je uglavnom bila ASCII tekst.

Internet

Internet je kvalitativno drukčiji jer je **decentraliziran**. To je zapravo skup **protokola** (pravila) po kojima računala razgovaraju.

- **TCP/IP** (*Transmission Control Protocol* i *Internet Protocol*): umjesto slanja niza ASCII znakova, odašiljač veće blokove podataka dijeli na manje **pakete** (*packets*), koji putuju zasebno (često telefonskom linijom) i na drugom kraju se ponovno slažu.
- **World Wide Web** je popularni grafički dio Interneta. Koristi **HTTP** (*Hypertext Transfer Protocol*).

- Sadržaj web stranica opisan je tekstualnim formatom **HTML** (*Hypertext Markup Language*). **Hipertekst** znači povezivanje srodnih informacija, slično kao kod Bushova Memexa: HTML datoteka može sadržavati **poveznice** na druge stranice.

O HTML-u:

- sličan je RTF-u: ASCII tekst s podacima o oblikovanju,
- može se pozivati na slike u formatima **GIF**, **PNG** (*Portable Network Graphics*) i **JFIF** (JPEG),
- tekstualni format znači da se datoteke mogu pogledati i lako **pretraživati**,
- unatoč imenu, HTML **nije programski jezik**: preglednik čita HTML i prema njemu oblikuje tekst i slike.

Kod na webu

Ponekad treba izvesti i program dok se radi sa stranicom.

Strana	Što radi	Primjer
Poslužitelj (*server*): računalo na kojem su stranice	obrađuje npr. obrasce koje korisnik ispuni	**CGI** skripte (*Common Gateway Interface*)
Klijent (*client*): tvoje računalo	jednostavan program unutar HTML-a	**JavaScript**; preglednik ga interpretira kao i HTML

Zašto web stranica ne bi jednostavno dala **izvršni program**?

1. **Koje je tvoje računalo?** Macintosh treba strojni kod za PowerPC i Mac OS API; PC treba kod za Pentium i Windows API; postoje i drugi.
2. **Sigurnost:** izvršne datoteke iz nepouzdanog izvora mogu biti **zlonamjerne**.

Java (Sun Microsystems; ne brkati s JavaScriptom) jedan je odgovor. To je potpun objektno orijentirani jezik, sličan C++-u, i nalazi se **između** prevedenih i interpretiranih jezika:

- Java program se **prevodi**, ali ne u strojni kod nego u **Java bajtkod** (*byte codes*): po građi sličan strojnom kodu, ali za **zamišljeno** računalo, **Java virtualni stroj** (JVM),
- računalo koje izvodi program **oponaša** JVM interpretiranjem bajtkoda,
- program koristi grafički sustav koji postoji na stroju,
- to omogućuje programiranje **neovisno o platformi**.

Optička vlakna

Knjiga se bavila uglavnom slanjem signala **strujom** kroz žicu. Učinkovitiji medij je **svjetlo** kroz **optička vlakna**: tanke cjevčice od stakla ili polimera koje vode svjetlo i oko zavoja. Brzine prijenosa dosežu područje **gigaherca**: milijarde bitova u sekundi.

Informacije budućnosti donosit će **fotoni**, a ne elektroni: kao brže točkice i crtice Morseove abecede i ono pažljivo treptanje svjetiljke kojim smo kasno navečer razgovarali s prijateljem preko puta, s čime je knjiga i počela.

Ključne definicije

Pojam	Značenje
Korisničko sučelje	mjesto susreta čovjeka i računala
Skupna obrada	programi i podaci na karticama, rezultati na papiru, bez interakcije
Escape sekvenca	ESC + niz znakova koji opisuje funkciju zaslona
Glupi terminal	tipkovnica i zaslon koji samo prikazuju ASCII s udaljenog računala
Rasterski / vektorski zaslon	slika od linija piksela / crte koje top crta izravno

Pojam	Značenje
Dubina boje	broj bitova po pikselu; boja = 2^dubina
GUI	grafičko korisničko sučelje: prozori, ikone, izbornici, miš
WYSIWYG	prikaz na zaslonu jednak ispisu
Objekt (OOP)	kod + podaci; komunicira porukama
Metadatoteka	vektorska slika: niz naredbi za crtanje
Bitmapa	rasterska slika: polje piksela
CCD	poluvodič koji pri osvjetljenju oslobađa naboj
Sažimanje bez gubitaka / s gubicima	reverzibilno / nereverzibilno
OCR	pretvorba slike teksta u ASCII
ADC / DAC	analogno → digitalno / digitalno → analogno
PCM	uzorkovanje signala stalnom frekvencijom
Frekvencija uzorkovanja	uzoraka u sekundi; barem 2 × najveća frekvencija
MIDI	poruke za glazbeni sintetizator
MPEG	sažimanje videa koje koristi sličnost uzastopnih sličica
Modem	pretvara bitove u zvuk i obratno (modulacija)
TCP/IP	protokoli Interneta; podaci u paketima
HTML / HTTP	tekstualni format web stranica / protokol weba
Java bajtkod / JVM	kod za zamišljeni stroj / njegov oponašatelj

Ključno za zapamtiti

1. Napredak hardvera troši se velikim dijelom na bolje korisničko sučelje.
2. Interakcija se razvijala: skupna obrada → teleprinter → "stakleni teleprinter" → grafičko sučelje.
3. Video memorija u adresnom prostoru procesora omogućila je brze interaktivne programe (VisiCalc).
4. Rasterski zaslon treba memoriju za svaki piksel: širina × visina × bajtova po pikselu.
5. Broj boja = 2^(bitova po pikselu); 24 bita daju 16.777.216 boja.
6. GUI (Alto, Macintosh, Windows) donosi prozore, ikone, izbornike i miš; grafički OS ima velik API za crtanje i komponente sučelja.
7. OOP spaja kod i podatke u objekte koji komuniciraju porukama.
8. Vektorska grafika je opis crtanja, rasterska je polje piksela; sažimanje može biti bez gubitaka ili s gubicima.
9. Digitalni zvuk: ADC, uzorkovanje barem dvostruko brže od najviše frekvencije; CD = 44.100 × 16 bitova × 2 kanala.
10. Računala komuniciraju modemima i Internetom (TCP/IP, paketi); web koristi HTML i HTTP; Java koristi bajtkod za neovisnost o platformi.

Najčešće zabune

- **"Svjetlosna olovka svijetli."** Ne: ona **opaža** svjetlo zrake na zaslonu.
- **"Sažimanje uvijek smanjuje datoteku."** Sažimanje bez gubitaka ne može smanjiti svaku datoteku; ponekad je poveća.
- **"JPEG je uvijek sažimanje s gubicima."** JPEG obuhvaća više tehnika, neke bez gubitaka, neke s gubicima.

- **"HTML je programski jezik."** HTML je format teksta s oznakama oblikovanja; programi na webu su npr. JavaScript i CGI skripte.
- **"Java i JavaScript su isto."** Java je potpun objektno orijentirani jezik koji se prevodi u bajtkod; JavaScript je jednostavan jezik koji preglednik interpretira unutar HTML-a.
- **"Modem od 300 bita/s prenosi 300/8 bajtova u sekundi."** Zbog početnog i završnog bita prenosi 30 bajtova u sekundi.

Brza provjera razumijevanja

1. Koliko memorije treba zaslon 1024 × 768 s 16 bitova po pikselu? Koliko boja može prikazati?
2. Objasni razliku između vektorske i rasterske grafike na primjeru projekta mosta.
3. Kolika je minimalna frekvencija uzorkovanja za zvuk do 10.000 Hz? Koliko bajtova u minuti zauzima taj zvuk s 8 bitova, mono?
4. Zašto je VisiCalc mogao raditi na Apple II, a ne na velikom računalu s glupim terminalom?
5. Kako Java postiže neovisnost o platformi?

Završni dio

Najvažnije lekcije iz knjige

1. **Kod je dogovor.** Svaki sustav za prijenos informacije (Morse, Braille, ASCII, strojni kod) je kod: skup znakova i pravila. Isti bitovi mogu značiti broj, slovo, boju ili instrukciju; značenje daje kontekst.
2. **Dva znaka su dovoljna.** S n binarnih znakova dobiva se 2^n kombinacija. Zato je bit (0 ili 1) temelj svega.
3. **Prekidač je sve što treba.** Relej, elektronska cijev i tranzistor rade isto: jedan signal upravlja drugim. Od takvih prekidača grade se logička vrata, a od vrata sve ostalo.
4. **Logika i aritmetika su isto.** Booleova algebra opisuje i prekidače i zbrajanje: zbroj dvaju bitova je XOR, a prijenos AND. Oduzimanje je zbrajanje komplementa dvojke.
5. **Povratna veza daje pamćenje.** Izlaz vraćen na ulaz daje oscilator (sat) i bistabil (1 bit memorije). Od bistabila nastaju zasuni, brojila i RAM.
6. **Adresa bira, podatak se čuva.** Memorija je mnogo zasuna; dekoder bira gdje se piše, selektor odakle se čita. Broj vrijednosti = 2^(broj adresnih linija).
7. **Računalo = memorija + brojilo + akumulator + instrukcije + uvjetni skok.** Uvjetni skok (petlja) razlikuje računalo od kalkulatora.
8. **Von Neumannova arhitektura:** binarni brojevi, kod i podaci u istoj memoriji, programsko brojilo i uvjetni skokovi. Gotovo sva računala i jezici građeni su po tom modelu.
9. **Sva digitalna računala mogu isto; razlikuju se brzinom.** Što hardver ne zna, softver može nadomjestiti.
10. **Slojevi apstrakcije.** Hardver → strojni kod → asembler → operacijski sustav i API → jezici visoke razine → grafička sučelja. Svaki sloj skriva detalje sloja ispod.
11. **Sve se svodi na bitove:** tekst (ASCII, Unicode), razlomci (nepomični i pomični zarez), slike (bitmape, vektori), zvuk (PCM) i video.
12. **Svaki zapis ima ograničenja:** raspon i preciznost brojeva, nekompatibilni kodovi znakova, poredak bajtova (little-endian i big-endian), sažimanje s gubicima.

Kako su poglavlja povezana

Knjiga gradi znanje kao kuću: svaki kat stoji na prethodnom.

```
KODOVI               1 Morse — 2 kombinacije — 3 Braille (shift, escape)
  |                                 |
STRUJA               4 struja — 5 uzemljenje — 6 relej
  |                                 |
BROJEVI              7 pozicijski — 8 binarni — 9 bit, paritet
  |                                 |
LOGIKA               10 Booleova algebra — 11 vrata (od releja)
  |                                 |
ARITMETIKA           12 zbrajalo — 13 oduzimanje (komplementi)
  |                                 |
PAMĆENJE             14 bistabili, brojila — 15 bajt, heks — 16 RAM
  |                                 |
RAČUNALO             17 automatizacija: instrukcije, skokovi, asembler
  |                                 |
STVARNI HARDVER      18 tranzistor, čip — 19 Intel 8080, Motorola 6800
  |                                 |
SUSTAV               20 ASCII — 21 sabirnica, zaslon, tipkovnica, disk
  |                                 |
SOFTVER              22 operacijski sustav — 23 pomični zarez — 24 jezici
  |                                 |
SUVREMENO RAČUNALO   25 grafika, zvuk, video, Internet
```

Najvažnije veze, opisane riječima:

- **Kod i bitovi (1 – 3, 8 – 9).** Morse i Braille pokazuju da se informacija može prikazati dvama znakovima. Poglavlja o brojevima pokazuju da je binarni sustav prirodan za takve znakove, a bit je jedinica informacije. Shift i escape kodovi iz Braillea vraćaju se u Baudotu, ASCII-ju i escape sekvencama zaslona (20, 25).
- **Relej → vrata → zbrajalo (6, 10 – 12).** Relej je prekidač kojim upravlja struja. Serijski i paralelni spoj prekidača ostvaruju AND i OR iz Booleove algebre. Kombinacija vrata daje poluzbrajalo, potpuno zbrajalo i 8-bitno zbrajalo.
- **Zbrajalo → oduzimanje → predznak (12 – 13).** Komplement dvojke pretvara oduzimanje u zbrajanje. Ta ideja je ugrađena u stroj iz poglavlja 17 (Subtract = invertor + ulazni prijenos) i u procesor 8080 (19).
- **Bistabil → memorija → računalo (14 – 17).** Zasun pamti bit; s dekoderom i selektorom nastaje RAM; s brojilom i akumulatorom nastaje automatsko zbrajalo; s instrukcijama, adresama i uvjetnim skokom nastaje računalo.
- **Model → stvarnost (17 – 19).** Računalo iz poglavlja 17 namjerno je slično mikroprocesorima iz 1970-ih. Zato se u poglavlju 19 prepoznaju akumulator, zastavice, LOD/STO (LDA/STA) i skokovi.
- **Procesor → sustav (19 – 22).** Procesor treba memoriju, ulaz, izlaz i pohranu, povezane sabirnicom. Prekidi (19) omogućuju tipkovnicu (21), a obrađivač tipkovnice i procesor naredbi postaju zameci operacijskog sustava (22).
- **Brojevi → razlomci (7, 13, 15, 23).** Pozicijski sustav s negativnim potencijama daje decimalne i binarne razlomke; BCD i pomični zarez proširuju ideju zapisa broja.
- **Asembler → jezici → objekti (17, 24, 25).** Mnemonici iz poglavlja 17 postaju asembler, asembler vodi do prevoditelja i jezika visoke razine, a grafički sustavi do objektno orijentiranih jezika.
- **Zaslon → grafika (21, 25).** Video memorija u adresnom prostoru procesora omogućuje i jednostavan tekstualni zaslon i cijelu grafičku revoluciju.

Praktični savjeti

1. **Uvježbaj pretvorbe.** Tablicu 16 niblova (0000 = 0 do 1111 = F) nauči napamet. Pretvaraj brojeve između binarnog, heksadecimalnog i dekadskog dok ne postane automatski. To je osnova za sve što slijedi.
2. **Crtaj i prati signale.** Za svaki sklop (vrata, zbrajalo, bistabil, dekoder) napiši tablicu istine i ručno provjeri nekoliko ulaza. Kod bistabila prati i redoslijed promjena u vremenu.
3. **Simuliraj računalo na papiru.** Za program iz poglavlja 17 napravi tablicu s memorijom, brojilom, akumulatorom i zastavicama, pa ga izvedi korak po korak.
4. **Ručno asembliraj.** Uz uzorke kodova 8080 (MOV = 01 ddd sss, aritmetika = 10 ooo sss) prevedi kratke programe u bajtove i obrnuto.
5. **Kod svakog niza bitova pitaj: kako se tumači?** Bez predznaka, s predznakom, BCD, ASCII znak, instrukcija ili adresa? Kod višebajtnih vrijednosti provjeri i poredak bajtova.
6. **Razlikuj memoriju i pohranu.** Procesor izravno adresira samo memoriju (RAM, ROM); podatke s diska prvo treba prenijeti u memoriju.
7. **Računaj potrebnu memoriju formulama.** Slika: širina × visina × bajtova po pikselu. Zvuk: uzoraka u sekundi × bajtova po uzorku × kanala × sekundi. Disk: strane × staze × sektori × bajtova po sektoru.
8. **Za novac koristi nepomični zarez.** Pomični zarez ima velik raspon, ali ograničenu preciznost; dva različita iznosa mogu postati isti broj.
9. **Vrati se temeljima.** Ako nešto u poglavljima 19 – 25 nije jasno, gotovo uvijek pomaže ponoviti poglavlja 8 – 17.
10. **Koristi pitanja za samoprovjeru.** Pokušaj odgovoriti na "Brzu provjeru razumijevanja" bez gledanja u tekst; gdje zapneš, ponovno pročitaj taj dio.

Pojmovnik ključnih pojmova

Pojmovi su poredani abecedno. Broj u zadnjem stupcu je poglavlje u kojem je pojam objašnjen.

Pojam	Kratko objašnjenje	Pogl.
ADC / DAC	pretvornik analognog napona u broj / broja u analogni napon	25
Adresa	broj koji određuje mjesto u memoriji	16
Akumulator	registar (zasun) koji čuva međurezultat računanja	14, 17
ALU	aritmetičko-logička jedinica procesora	17
AND (I)	logička operacija: 1 samo ako su svi ulazi 1	10, 11
API	sučelje kojim programi koriste usluge operacijskog sustava	22
ASCII	7-bitni standardni kod znakova (00h – 7Fh)	20
Asembler	program koji asemblerski jezik pretvara u strojni kod	24
Asemblerski jezik	zapis strojnog koda mnemonicima, argumentima i oznakama	17
Bajt	skupina od 8 bitova; 256 vrijednosti	15
Banka memorije	skup čipova koji zajedno spremaju bajt	21
BCD	zapis u kojem je svaka dekadska znamenka u 4 bita	15, 23
Big-endian / little-endian	najznačajniji / najmanje značajan bajt na nižoj adresi	19
BIOS	dio operacijskog sustava koji izravno pristupa hardveru	22
Bistabil (flip-flop)	sklop s dva stabilna stanja; pamti 1 bit	14
Bit	binarna znamenka; najmanja jedinica informacije	8, 9
Bitmapa	rasterska slika: polje piksela	25
Booleova algebra	algebra s vrijednostima 0 i 1 i operacijama AND, OR, NOT	10
Brojilo	lanac bistabila čiji izlazi broje binarno	14
Bug	pogreška u programu	17
Ciklus instrukcije	dohvat + izvršavanje instrukcije	17
CPU (procesor)	središnja jedinica za obradu	17
CR / LF	povratak na početak reda / prelazak u novi red	20
Datoteka	imenovan skup povezanih podataka na disku	22
Datotečni sustav	način organiziranja diska u datoteke i direktorije	22
De Morganovi zakoni	NOT A AND NOT B = NOT (A OR B); NOT A OR NOT B = NOT (A AND B)	11
Dekoder	sklop koji ulaz šalje na točno jedan od više izlaza	11, 16
DMA	izravni pristup memoriji bez procesora	21
Dohvat instrukcije	čitanje instrukcije iz memorije	17
DRAM / SRAM	dinamički RAM (1 tranzistor po bitu, osvježavanje) / statički RAM	21
Dubina boje	broj bitova po pikselu; broj boja = 2^dubina	25
EBCDIC	IBM-ov 8-bitni kod znakova izveden iz bušenih kartica	20
Eksponent	potencija baze u znanstvenom zapisu	23
Elektromagnet	željezo omotano žicom; magnet dok teče struja	6
Escape kod	kod koji mijenja značenje sljedećeg koda (ili niza)	3, 20, 25
FAT	tablica koja povezuje klastere datoteke na disku	22
FPU (koprocesor)	hardver za računanje s pomičnim zarezom	23
Frekvencija	broj ciklusa u sekundi (Hz); f = 1/T	14
GUI	grafičko korisničko sučelje: prozori, ikone, izbornici, miš	25
Heksadecimalni sustav	brojevni sustav s bazom 16 (0 – 9, A – F)	15
HTML / HTTP	tekstualni format web stranica / protokol weba	25

Pojam	Kratko objašnjenje	Pogl.
Instrukcija	naredba procesoru: operacijski kod i eventualno adresa ili podatak	17
Integrirani krug (čip)	mnogo tranzistora i drugih dijelova na komadiću silicija	18
Interpreter	program koji izvodi izvorni kod izravno, bez izvršne datoteke	24
Invertor (NOT)	sklop koji okreće 0 u 1 i 1 u 0	11
Izlaz s tri stanja	izlaz koji može biti 0, 1 ili "odspojen"	21
Izvorni kod	program kako ga je napisao čovjek	24
Jezik visoke razine	programski jezik neovisan o procesoru (FORTRAN, ALGOL, C...)	24
Kilobajt (KB)	1024 bajta; MB = 1024 KB; GB = 1024 MB	16
Kod	sustav znakova i pravila za prijenos informacije	1
Kod pomaka (shift)	kod koji mijenja značenje svih sljedećih kodova	3, 20
Kod tipke (scan code)	kod koji hardver tipkovnice daje za tipku	21
Komplement dvojke	komplement jedinice + 1; zapis negativnih binarnih brojeva	13
Komplement jedinice	svi bitovi invertirani	13
Kontrolna znamenka	znamenka izračunata iz ostalih radi otkrivanja grešaka	9
Korisničko sučelje	mjesto gdje se susreću čovjek i računalo	25
Križni asembler	asembler koji radi na jednom računalu, a stvara kod za drugo	24
Logička vrata	sklop koji izvodi jednu logičku operaciju	11
Memorija / pohrana	poluvodički RAM (nepostojan) / diskovi i vrpce (postojani)	21
Mikrokod	unutarnje instrukcije procesora ili koprocesora u ROM-u	23
Mikroprocesor	cijeli procesor na jednom čipu	18
Mnemonik	kratka oznaka strojne instrukcije (npr. MOV, ADD, JMP)	17
Modem	uređaj koji bitove pretvara u zvuk i obrnuto	25
Mooreov zakon	broj tranzistora na čipu udvostručuje se oko svakih 18 mjeseci	18
Morseov kod	kod od točaka i crta	1
NAND / NOR	negirani AND / negirani OR	11
NaN	"nije broj": posebna vrijednost pomičnog zareza	23
Napon	potencijal za obavljanje rada; mjeri se u voltima	4
Nepomični zarez	zapis razlomaka s unaprijed dogovorenim mjestom zareza	23
Nibl	4 bita; jedna heks znamenka	15
Objekt	u OOP-u spoj koda i podataka koji komunicira porukama	25
Ohmov zakon	I = E / R	4
Operacijski kod (opcode)	broj koji označava instrukciju	17
Operacijski sustav	softver koji upravlja datotekama, pokreće programe i daje API	22
OR (ILI)	logička operacija: 1 ako je barem jedan ulaz 1	10, 11
Oscilator (sat)	sklop čiji se izlaz sam pravilno mijenja između 0 i 1	14
Otpor	koliko tvar ometa protok struje; mjeri se u omima	4
Paritet	je li broj jedinica paran ili neparan; za provjeru grešaka	9
Periferija	uređaji spojeni na procesor koji nisu memorija	19
Petlja	kontrolirano ponavljanje dijela programa	17, 24
Piksel	točka slike	21
Podizanje sustava (booting)	učitavanje operacijskog sustava malim punjačem iz ROM-a	22

Pojam	Kratko objašnjenje	Pogl.
Pojasna širina	najveća brzina promjene signala; kapacitet medija	21
Pokazivač stoga (SP)	registar s adresom vrha stoga	19
Poluvodič	tvar čija se vodljivost mijenja dopiranjem (silicij, germanij)	18
Poluzbrajalo / potpuno zbrajalo	zbraja 2 bita / 3 bita (s ulaznim prijenosom)	12
Pomični zarez	binarni znanstveni zapis: predznak, eksponent, signifikand	23
Potprogram	dio koda pozvan s CALL, završava s RET	19
Pozicijski sustav	vrijednost znamenke ovisi o njezinu mjestu	7
Prekid	signal kojim periferija traži pozornost procesora	19
Preljev (overflow)	rezultat prevelik za zadani broj bitova	13
Prevoditelj (compiler)	program koji jezik visoke razine pretvara u strojni kod	24
Programsko brojilo (PC)	registar s adresom sljedeće instrukcije	17
RAM	memorija s nasumičnim pristupom, za čitanje i pisanje	16
Razlučivost	broj piksela vodoravno × okomito	21
Registar	zasun u procesoru za jednu vrijednost	19
Relej	prekidač kojim upravlja struja kroz elektromagnet	6
RISC	arhitektura s jednostavnim instrukcijama jednake duljine	19
ROM / PROM / EPROM	memorija samo za čitanje / programabilna jednom / brisiva UV svjetlom	21, 22
Sabirnica	skup signala zajednički svim dijelovima računala	21
Sažimanje	smanjivanje podataka, bez gubitaka (RLE, LZW) ili s gubicima (JPEG)	25
Selektor	sklop koji prosljeđuje jedan od više ulaza	14, 16
Signifikand	znamenke broja u znanstvenom zapisu (mantisa)	23
Skok / uvjetni skok	promjena adrese sljedeće instrukcije, uvijek / ovisno o zastavici	17
Stog (LIFO)	memorija u kojoj se zadnje stavljeno prvo uzima	19
Strojni jezik	brojčani kodovi instrukcija koje procesor izvodi	17
Struja	kretanje elektrona; mjeri se u amperima	4
Šum / zalihost	ometanje prijenosa / dodatna informacija koja pomaže da poruka preživi šum	9
Tranzistor	poluvodički prekidač i pojačalo (kolektor, baza, emiter)	18
TTL / CMOS	obitelji čipova: brža / štedljivija	18
Unicode	prema knjizi 16-bitni kod znakova za sve jezike	20
Uzemljenje	spoj sa Zemljom; zajednički povratni put struje	5
Varijabla	ime koje u programu označava mjesto u memoriji	24
Virtualna memorija	privremeno spremanje dijelova memorije na disk	22
Von Neumannova arhitektura	binarno, kod i podaci u memoriji, programsko brojilo, uvjetni skokovi	18
Vrijeme propagacije	kašnjenje od promjene ulaza do promjene izlaza	18
XOR (isključivo ILI)	1 kad je točno jedan od dva ulaza 1; bit zbroja	12
Zasun (latch)	bistabil koji sprema podatak na zahtjev; 1 ili više bitova	14, 16
Zastavica	bit koji opisuje rezultat zadnje operacije (nula, prijenos, predznak…)	17, 19